ADVANCED JAVA

NATARAJU

SATYA TECHNOLOGIES

MANOJ XEROX
Gayarti nager,Behind-Huda-Ameer pet
SOFT WARE INSTITUTES MATERIAL AVAILABLE
CELL :8978989710

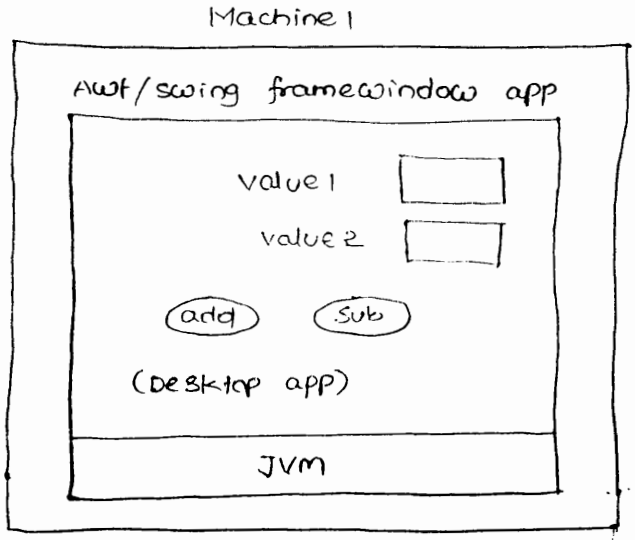
Advanced Java

* Since Java can be used to develop all kinds of software applications so it is called as programming suite.
* Java can be divided into 3 modules. They are
  (i) JSE - Java standard Edition
  (ii) JEE - Java Enterprise Edition
  (iii) JME - Java Mobile / Micro Edition

JSE:

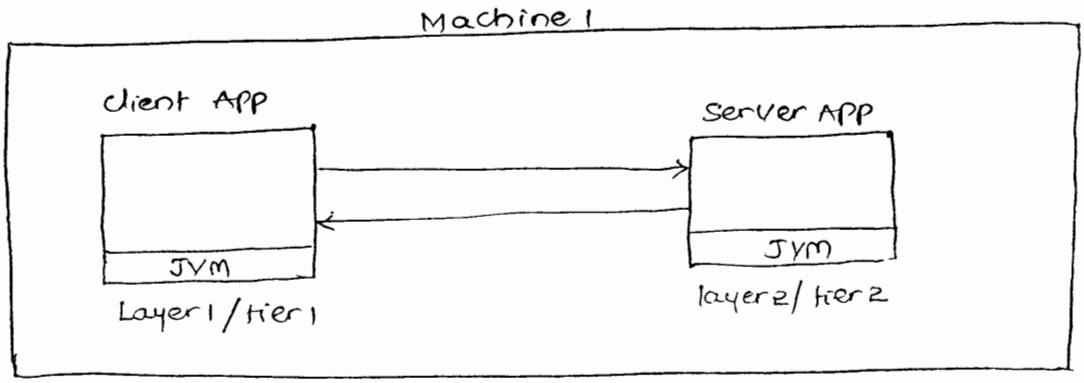
* It is installable software as jdk software.
* Latest version is 7.0 (dolphin)
* Java 6.0 is called as mustang and Java 5.0 is called as tiger.
* This module is given to develop stand-alone applications, desktop applications, two-tier applications....
* The application that is specific to one computer and contains main() is called as standard or) stand-alone application.
* The stand-alone application that contains GUIness is called as desktop application.

  Ex: Awt framewindow application, swing framewindow application.

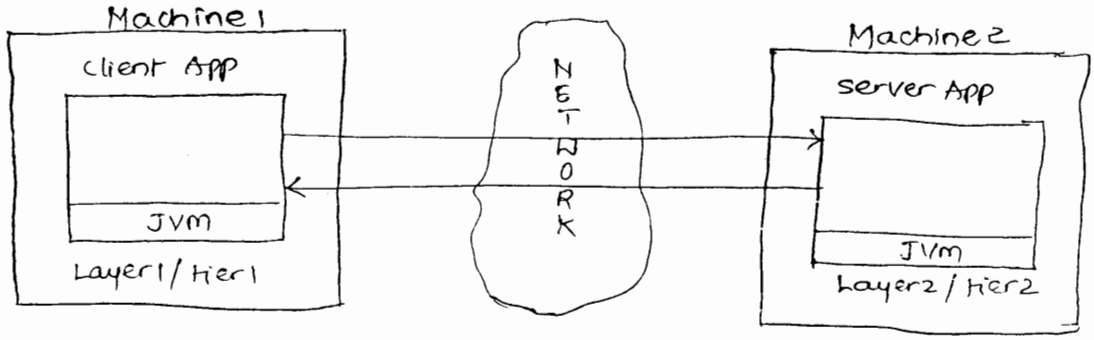


* The application that contains two layers communicating with each other is called as two-tier application.
* ⇔ layer represents logical partition in the application having logics.

* The layers of two-tier application can be there in a single computer (or) can be there in two different computers.



socker programming based client-server application (two-tier app)



socker programming based client-server application (two-tier app)

* To execute multiple Java applications parallelly (or) simultaneously from a single computer, multiple JVMs will be activated simultaneously (or) parallelly.
* Applet is not desktop application. It is a compiled java class that can be sent over the network as webpage giving alternate to the HTML based form pages.

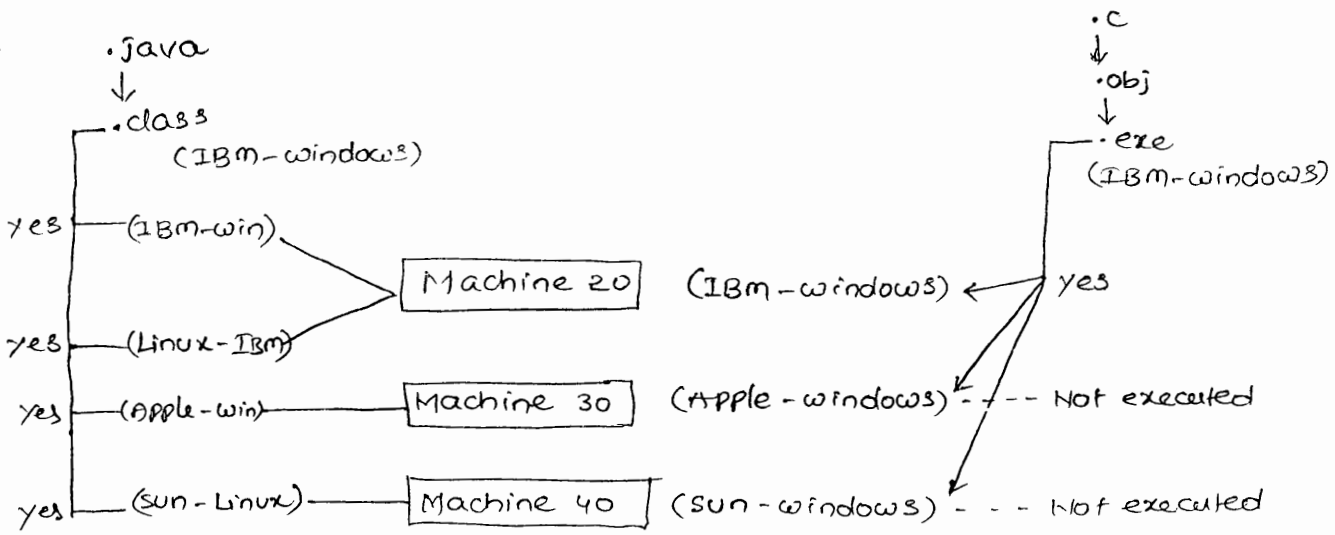
JSE Concepts : awt, swings, collection frame work, JDBC, JNDI (Java Naming and Directory Interface) etc.

* The alternate technologies for JSE module are VB, VB.net, D2K. But these technologies based applications are platform dependent and architectural specific whereas the JSE module applications are architectural neutral and platform independent.

* As of now three popular architectures are there to manifacture computers. Each architecture use methodology and plan of manifacturing

computer. They are

(1) IBM architecture
(2) Apple architecture
(3) Sun architecture.



* The c, c++ generated .exe files not only contains operating system related instructions but they also contain instructions related to underlying computer architecture. Due to this we can say c, c++ languages are platform dependent and architecture specific whereas java programming language is platform independent and architectural neutral because the java compiler generated .class file doesn't contain underlying OS, computer architecture instructions.

JEE:

* JEE means Java Enterprise Edition.
* It is not installable software, it is given as specification.
* JEE specification contains rules and guidelines to develop webserver and application server software like weblogic, tomcat etc.

NOTE: working with JEE is nothing but working with one (or) other web-server (or) app server software to develop the app.

* These webserver, application server softwares are installable softwares.
* For JEE module JSE module is base module.

* Using jee module the following applications can be developed.

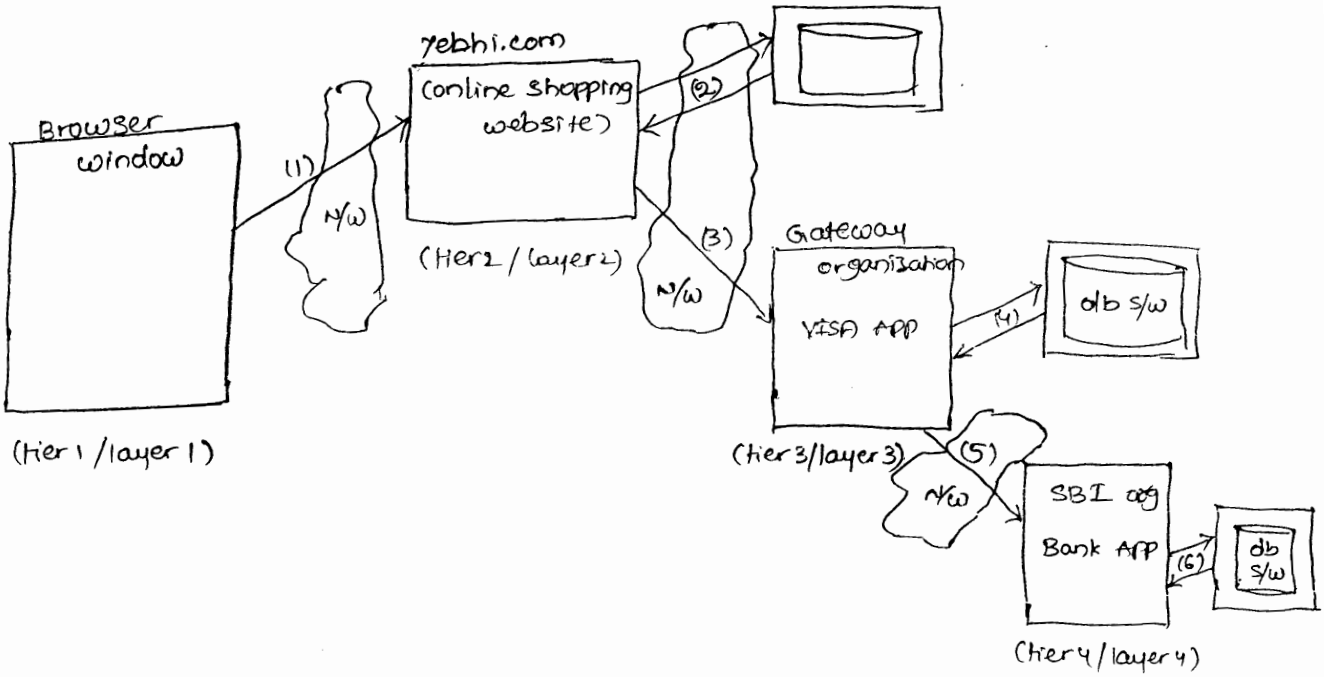
(1) web applications (websites)
(2) Distributed applications
(3) Enterprise applications
(4) n-tier applications

* The applications which can provide 24x7 access to their resources using internet environment are called as websites.
* The client-server applications which provide location transparency are called as distributed applications. If client application is able to reconize the server application location change dynamically are called as location transparency.
* A web application / website can be developed as distributed or non-distributed application.
* All credit card, debit card processing applications will be developed generally as distributed applications.
* A large scale application that deals with complex and heavy weight business logic by having middleware service support is called as enterprise application.
  Ex: Banking application.
* The additional services that are configurable on the applications to make applications running perfectly in all the situations are called as middleware services.
  Ex: security service, Transaction management service, pooling service and etc..

NOTE: Middleware services are not minimum logics of the application, they are additional and optional logics to develop.

* The application that contains multiple layers / tiers (more than two) is called as n-tier applications.
  Ex: online shopping website environment.

online shopping website environment (n-tier APP)



* Generally enterprise applications will be developed as n-tier applications.
* JEE module concepts are servlets, JSP, EJB, JTA (Java Transaction API), JMS (Java messaging service), Java mail, JAAS (Java Authentication and Autherization serive), JMX (Java management console), JCA (Java connector API) and etc...
* struts, spring and hibernate technologies are not part of sunmicro systems JEE module. They are given by third party organizations as alternate and enhancement to JEE concepts.
* while working with all JEE concepts we must take support of webserver (or) application server softwares.
* majority projects of java environment will be developed based on JEE module.
* Every project belongs one domain. This JEE module is suitable for developing banking financial service, insurance, health care domain projects.
* JSE module is suitable to develop gaming, automation and retail domain projects.

* JME module is suitable for developing mobile gaming, telecommunication domain projects.
* The latest version of JEE module is 6.

JME:

* JME means Java mobile/micro edition.
* For JME module JSE module is base module.
* It is installable software.
* Latest version is 2.x
* Given to develop mobile and micro applications in java environment like sim cards, mobile games and etc.
* use wml (wireless markup language), wap in the application development.
* These applications can be embded with different types of devices (electronic, electrical, automobile and etc...).
* JSE, JEE module based applications can not be attached with (or) embedded with different types of devices by bringing the code outside the computer. But this is possible with JME module.
* The above process is called as embedded system programming and this is useful to add artificial intellegence to devices, that means we can make the devices to think and take the decisions.

Ex: A fully automated washing machine, Robot.

* To develop mobile related and artificial intellegence related applications in java environment use JME module.

API (Application programming Interface):

* It is the base for the programmer to develop certain software technology based software applications.
* Every software technology used built-in APIs. Using these APIs the programmers can develop user defined APIs and software applications.

* In "C" language API is set of functions which come in the form of header files.
* In "c++" language API is set of functions, classes which come in the form of header files.
* In "java" language API is set of classes, Interfaces, enums, annotations which come in the form of packages.

Examples of java APIs :

(1) AWT API → working with java.awt, java.awt.event packages.
(2) Reflection API → working with java.lang.reflect package
(3) JDBC API → working with java.sql, javax.sql packages
(4) Utility API → working with java.util package

* We can see three types of APIs in java.

(1) Built-in APIs → given by sun micro system along with jdk softwares.
Ex: AWT, JDBC etc...
(2) User defined APIs → given by programmers
(3) Third party APIs → given by third party organizations (other than sun micro systems and programmers).
Ex: Java Zoom API (given by javaZoom organization, downloadable from www.javazoom.net).

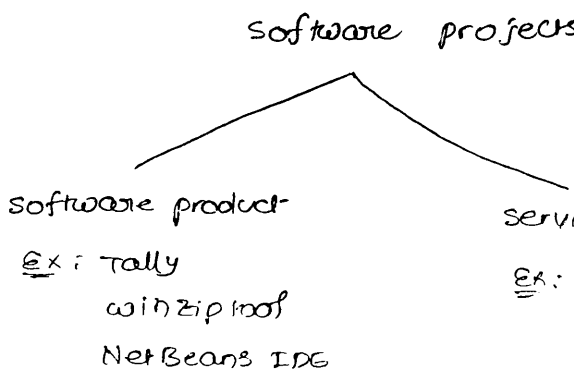
+ Generally the java related APIs come as in the form of jar files. (Java style zip file).
* JAVA_HOME\jre\lib\rt.jar file represents all the built-in APIs of jdk software.
* The company who creates softwares is called as software vendor company.
Ex: IBM, sun micro system, oracle corporation and etc.

* The organization who gives their requirements to software development company is called as client organization.

Ex: city bank, SBI, APSRTC etc..

* The company who takes the support of softwares given by software vendor companies and develop the project for client organizations is called as software development company.

Ex: Infosys, wipro, polaries etc..



* If project is designed and developed exclusively for one client organization by gathering requirements then it is called as service sector project.
* If project is developed based on common requirements and given to multiple clients then it is called as software product.
* For service sector project client organization will be there before development of the project.
* For software products client organizations will be there after the development of product.

What is the difference between static block and constructor?

A) Static block is class level one time execution block. When JVM loads java class the static block of that class executes automatically. This block is useful to initialize static member variables of a class.

Constructor is object level one time execution block when JVM creates object for a class then the constructor of that class executes once automatically. Constructor is use ful to initialize instance variables of java class object.

* Class.forName(-) makes JVM of the current application to load given class into application from the memory. But this method does not create object of the loader class.

Example application to understand above concepts.

```
class Demo
 {
     static
      {
        static System.out.println ("Demo: static block");
      }
     Demo()
      {
        system.out.println ("Demo: Constructor");
      }
 }
class Test
 {
     static
      {
        System.out.println ("Test: static block");
      }
     Test()
      {
        system.out.println ("Test: Constructor");
      }
public class TestApp
  {
     static
      {
        system.out.println ("TestApp: static block");
      }
     public static void main (String args[])
      {
          sop (" TestApp: main(-)");
          Demo d1 = new Demo();
          Demo d2 = new Demo();
```

```
        Class.forName("Test");
        Class.forName("Test");
    }
}
```

O/P

```
TestApp: static block
TestApp: main(-)
Demo: static block
Demo: Constructor
Demo: Constructor
Test: static block
```

* with respect to above application we can see JVM loading java classes in the following situations.

(1) while creating first object for given java class
(2) when Class.forName() method is called.
(3) when java class is given to "java tool" for execution.

```
> java TestApp
```

Understanding Class.forName():

Prototype:

```
public static Class forName(String name) throws ClassNotFoundException
```

* This method makes JVM to load given class or interface into application from memory dynamically at runtime. If they are not available to load this method throws checked exception called ClassNotFoundException.
* It is a static method (can be called with out object)
* available in java.lang.class (Pre-defined)
* It is factory method
* A method of a java class that is capable of constructing and returning its own java class object is called as factory method.

Ex:

```
Thread t = Thread.currentThread();
String s1 = String.valueOf(10);
Calender c1 = Calander.getInstance();
Class c = Class.forName("Test");
```

```
Class c = Class.forName("Test");
```

* The above statement makes JVM to load class "Test" into application dynamically at runtime. and returns object of (c) "java.lang.Class" referring the loaded class "test".
* In the above statement 'c' is not the object of the loaded class "Test", it is object of java.lang.class referring the loaded class "Test". we can use this object 'c' to gather details about the loaded class "Test" to create object of loaded class "Test" and etc.. operations.
* If java method throws checked exception while calling that method we must handle the exception using try/catch blocks (or) we need to declare the exception to be thrown by using throws statement.

Example application based on all the above statements

```
// TestAppl.java
public class TestAppl
{
    public static void main (String args[]) throws Exception
    {
        Class c = Class.forName (args[0]);
        //gathered details and loaded class and interface
        sop (args[0] + " is an interface ?" + c.isInterface());
        sop ("name of " + args[0] + " is :" + c.getName());
        sop ("super class of " + args[0] + " is: " + c.getSuperclass());
    } // main
} // class
```

```
> javac TestAppl.java (Compile time)
> java TestAppl java.lang.String (run time)
```

* Command line argument args[0]
* Inorder to know class name of any java object programatically then call getClass() method on that object.

* This method is available as public method of "java.lang.Object" class

Ex:
```
String s1 = new String("ok");
Sop("class name of s1 obj" + s1.getClass());
                                  → java.lang.String
java.Util.Date d1 = new java.util.Date();
Sop("class name of d1 obj" + d1.getclass());
                                  → java.util.Date
Class c = Class.forName("java.awt.Button");
Sop("class name of obj 'c' is:" + c.getClass());
                                  → java.lang.Class
```

Problem with "new" keyword:

* "new" keyword creates the object of a java class at runtime but it expects the presence of the class (or) availability of the class at compile time.

```
Test t = new Test();
```

* In the above statement object 't' will be created at run time but it expects the presence of class "Test" at compile time.
* This indicates new keyword is not suitable to create the object of a class whose class name comes to application dynamically at runtime. To overcome this problem use "newInstance()" method of java.lang.Class.
* The newInstance() method can create object of given java class at runtime even though class name is supplied to the application dynamically at runtime. This kind of object creation process is required while are performing drag and drop operations in GUI builder applications.

```
        catch (ClassNotFoundException cnf) // to handle known exception
          {
            cnf.printStackTrace();
          }
        catch (Exception e)   // to handle all unknown exceptions.
          {
            e.printStackTrace();
          }
      } // main
    } // class
```

> javac objTest.java (Compile time)

> java objTest Test (runtime)

> java objTest java.awt.Rectangle (runtime)

o/p Test: 0-arg Constructor
object data is: a=0 b=0.0

* when the above newInstance() method is used to create object of abstract class or interface then that method raises "java.lang.InstantiationException"
* It is always recommended to catch and handle the exceptions by keeping multiple catch blocks after try block. This helps the programmer to handle both known and unknown exceptions and to differentiate the logics based on the exception that are raised (refer above).

what is the difference between <exception obj>.toString() and <exception obj>.printStackTrace() ?

* toString() method prints name of the exception class along with description when exception is raised. whereas printStackTrace() prints the route cause and hierarchy in the application execution when exception is raised.
* Every java method is executed in stack memory. This stack memory is filled up with messages when exception is raised during method execution.

* The printStackTrace() call displays these messages on console monitor.

(@) When java does not support pointers why does it throws NullPointerException?

(A) When java method is called on a variable that holds null value this NullPointerException will be raised. To solve this problem make that variable referring to object before calling the method.

Ex: Problem:

```
Date d;
d.toString(); // raise java.lang.NullPointerException
```

Solution:

```
Date d = new Date();
d.toString(); // success
```

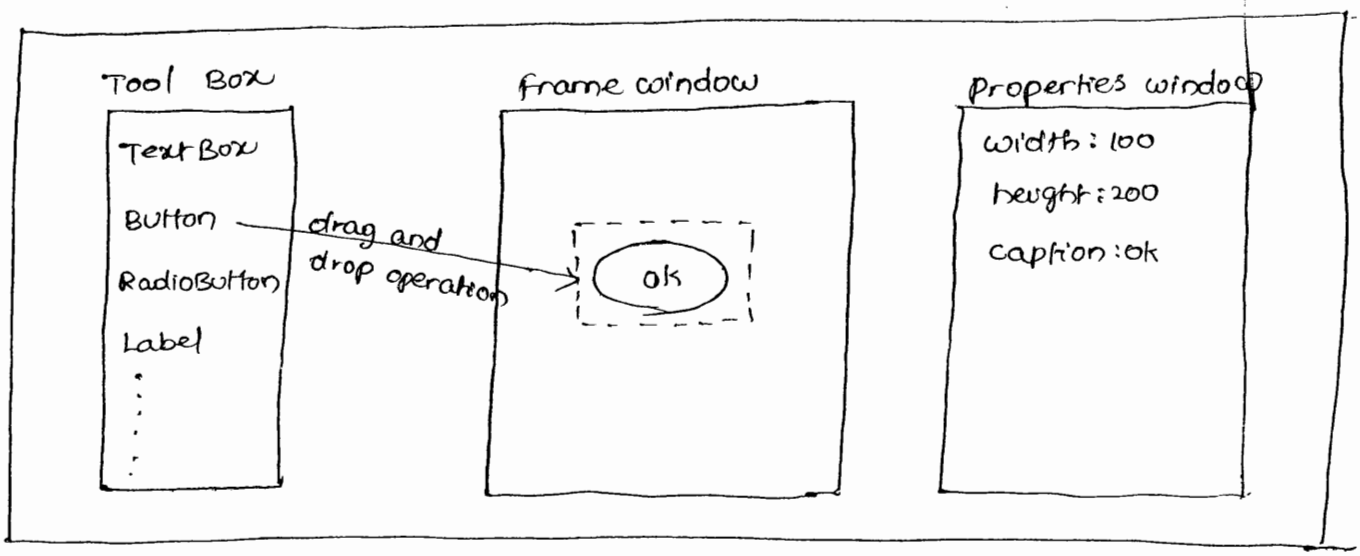
* In java.lang.NullPointerException class name the word pointer is no way related with C, C++ pointers. Look at that word with dictionary meaning which says method is called on a variable that points to null value.

How many ways are there to create object of a java class?

There are multiple ways. They are

(1) By using new keyword
(2) By using static factory method
(3) By using instance factory method
(4) By using factory pattern
(5) By using newInstance()
(6) By using cloning process
(7) By using deserialization process....

GUI Builder Application (Part of IDE s/ws)



String s = new String ("ok");

S(String class obj)



class c = Class.forName("Test");

(The given class/interface to load)

c (obj of java.lang.class)



* In a running java application if we want to refer a java class or interface then we need to use the object of java.lang.Class
* When you call java method catching and using the return value of java method is purely programmer choice.

Class.forName ("Test"); // valid statement

Class c = Class.forName ("Test"); // valid statement

* When java object is created constructor will execute and the instance variables will be assigned with default values.

creating object of java class by using newInstance() method by getting class name into application dynamically at runtime.

```
//Test.java

public class Test
  {
      int a;
      float b;

      public Test()
        {
          System.out.println("Test: 0-arg constructor");
        }
      public String toString()
        {
          return "a="+a+"b="+b;
  } // class
```

>javac Test.java

```
//objTest.java

public class objTest
  {
     public static void main(String args[])
      {
        try
         {
           // load given class into application
            class c = Class.forName(args[0]);
          // "c" object refers to the loaded class
          // create object for loaded class
            Object obj = c.newInstance();
                                  └→ creates object of a java class
                                     that is referred by object "c"
          // print object data
            System.out.println("object data is:"+obj.toString());
         }
        Catch(InstantiationException iae) // to handle unknown exception
          {
             iae.printStackTrace();
          }
```

1) By using new keyword

```
String s1 = new String("Hai");
Date d = new Date();
```

2) By using static factory method

* A method in a java class that is capable of constructing and returning its own java class object is called as static factory method.

```
Thread t = Thread.currentThread();
String s1 = String.valueOf(10);
Class c = Class.forName("Test");
```

} static factory methods

NOTE: when java class contains only private constructors to create object of that class outside the class we take the support of static factory methods.

Ex: java.lang.Class is having only private constructors so we always use Class.forName() method as shown above to create the object.

3) By using instance factory method:

```
String s = new String("hello");
String s1 = s.Concat(" how are u"); // s1 holds "hello how are u"
```

(s.Concat — instance factory method)

* Static factory methods will be called without object. Instance factory methods will be called by using object.

* In order to create a new object for java class by using existing object and its data take the support of instance factory methods.

NOTE: In the above example Concat() method is using object "s" and its data while constructing "s1" object.

4) By using factory pattern

* If method of one java class is returning object of other java class then it is called as factory pattern.

Ex: StringBuffer sb= new StringBuffer("how are you");

String s = sb.subString(6, 3);

↓

The method of StringBuffer class is returning object of String class. (factory pattern).

* If method of one java class wants to return another java class object then these both classes need not be there in inheritance relationship. If they are visible to each other that will be enough.

5) By using newInstance() method

```
class c = Class.forName("java.awt.Button");

Button b = (Button)c.newInstance();
```

* Here "c" represents the loaded class java.awt.Button but it is the object of java.lang.Class.

→ creates and returns the object of java.awt.Button class representing with 'c' object

6) By using Cloning process:

* The process of creating new object for a java class having existing object data is called as cloning process.
* programmer can perform cloning on only clonable objects.
* The object whose class implements marker interface called java.lang.Clonable is called as cloneable object.
* The interface whose implementation makes JVM to provide special run-time behavior for the objects of class is called as marker interface (or) tag interface.
* A marker interface can be there with methods (or) without methods.

Ex: java.lang.Clonable
java.lang.Serializable
java.io.Externalizable
java.rmi.Remote
javax.servlet.SingleThreadModel
java.lang.Runnable.

* When object is created through cloning process constructor will not be executed.
* The clone() method of java.lang.Object class can perform cloning operation.
* When a new object is created through cloning process the new object and existing object allocates memory seperately and acts as two different objects having two different hash codes. So modification done in one object will not reflected the other object, and vice versa.
* Every object will be identified through its unique hash code number.

Example application on cloning

```
// Test.java
class Test implements java.lang.cloneable
 {
    int a;
    float b;
  public Test()
    {
      sop("Test: 0-param Constructor");
    }
  public Test(int x, int y)
    {
       a=x;
       b=y;
       sop("TEST: 2-param constructor");
    }
  public String toString()
    {
      return "a="+a+"b="+b;
    }
  Public Object myclone() throws Exception
    {
       return super.clone();
                 └→ calls clone method of java.lang.Object class
    }
} // Test
Public class MainApp
 {
     P S V M (String args[])
       {
```

```
//original obj
Test t1 = new Test(10,20);
sop("t1 obj data is:"+t1.toString());
//create obj through cloning
Test t2 = (Test) t1.myclone();
sop("t2 obj data is:"+t2.toString());
sop("t1 obj hashCode is:"+t1.hashCode());
sop("t2 obj hashcode is:"+t2.hashCode());
//modify original obj data
t1.a = 30;
sop("t1 obj data is:"+t1.toString());
sop("t2 obj data is:"+t2.toString());
t2.b = 56.4f;
sop("t1 obj data is:"+t1.toString());
sop("t2 obj data is:"+t2.toString());
} // main
} // class
```

```
> javac mainApp.java   (compilation)
> java Main App        (execution)
```

7) By using deserialization process :

* The process of writing data of an object to a file is called as serialization.
* The process of reading data from a file and constructing new object having that data is called as deserialization.
* When object is created deserialization constructor will not execute.

---

* During java application execution the RAM memory will be utilised to maintain application data. (input values and results generated by application), due to this we can not use application data after the execution of the application (or) across the multiple executions of the same application.
* To overcome the above problem store application data in permanent place like files and DB softwares (persistance stores) through persistance operations.

Java app —— uses (I/O streams) (serialization and deserialization) ——→ Files

Java app —— jdbc concepts (oracle, sybase, mysql) ——→ DB software

* The place where we can store and use data for long time is called as persistance store. ex: file, database software.
* A logic that is used to interact with persistance stores and to manipulate the data is called as persistance logic.
* Performing insert, update, delete and read operations on persistance stores is called as performing persistance store operations.
* Persistance operations can be called as CURD operations (or) CRUD operations (or) SCUD operations.

C → create (insert), U → update (modify), R → Read (select)
D → Delete (Remove).

S → select

C → create

U → update

D → delete

Limitations with files as persistance stores:

(1) Can not manage huge amount of data

(2) No security

(3) No query language support to manipulate the data

(4) Delete and update operations are complex operations to perform.

(5) Relationships are not possible.

(6) Merging and comparision of data is complex operation.

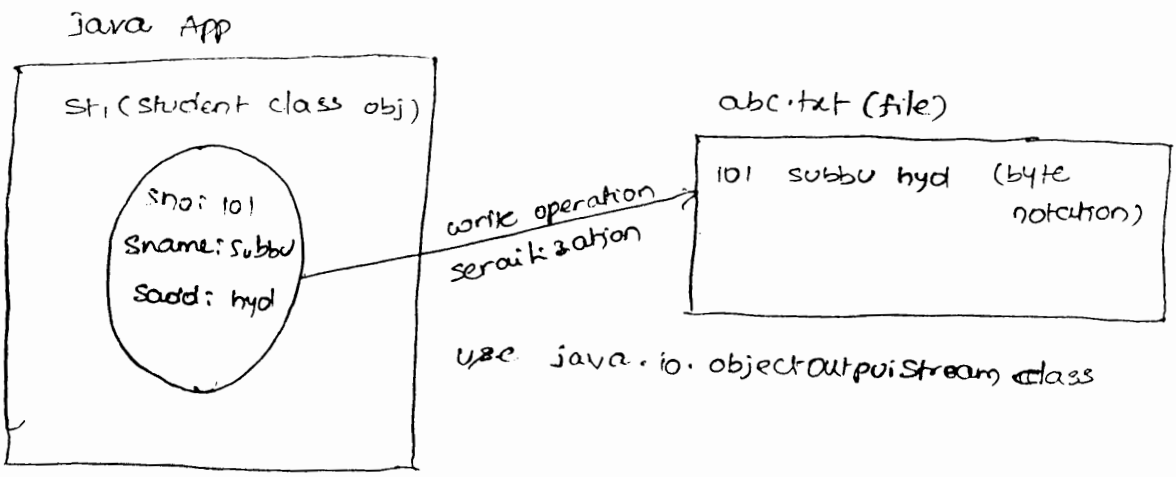
NOTE: To over come above problems use database software as persistance store.

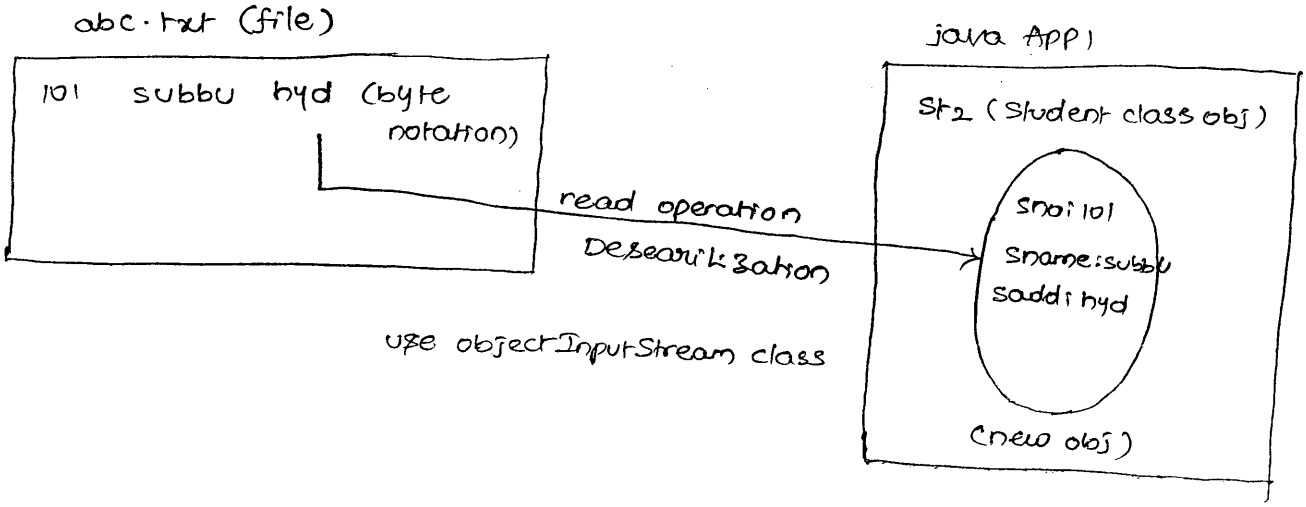
NOTE: In small scale applications like desktop, mobile games use files as persistance store. In medium, large scale applications like bank apps, websites use database software as persistance store.

Serialization and deserialization:

* In small applications the industry generally prefers serialization and deserialization persistance operations on the files.

* The process of capturing object data and writing that data into a file is called as serialization.

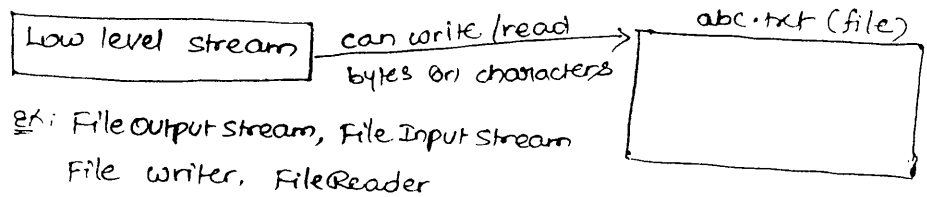




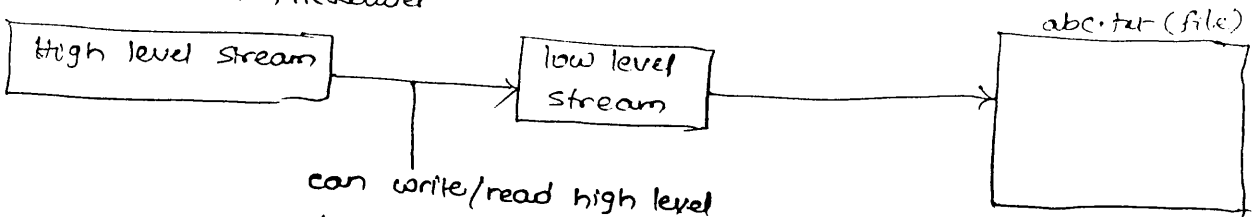
* In serialization process object will not be written to a file but the data of the object will be written to a file.
* The process of reading data from a file and constructing new java object having that data is called as deserialization.
* A stream is a continuous flow of data that represents communication channel between application and destination resource (file).
* Byte streams can write and read bytes.
* Character streams can write and read characters.

what is the difference between highlevel stream and lowlevel stream?

(A)



Ex: FileOutputStream, FileInputStream
FileWriter, FileReader



can write/read high level data like String, long object data values.

Ex: DataOutputStream
DataInputStream
ObjectOutputStream
ObjectInputStream

* High level streams interacts with files through low level streams and can work with high level data like string values, object values and etc.
* Low level streams interacts with files directly and can deal with low level data like characters and bytes.

Sample Application

(1) Read sno, name, m_1, m_2, m_3 values from key board.

(2) calculate total and avg → presentation logic

```
total = m1+m2+m3 /3.0f;
avg = total/3.0f;
```
} Business logic

(3) generate rank for the student — business logic

(4) store student details in db table as record — persistance logic

(5) display student details and results on monitor — presentation logic.

* Presentation logic provides user interface for end user to supply input values and to display results
* Business logic generates results based on given input values (it is main logic of the application)
* persistance logic manipulate application data in persistance stores.
* If entire data of an object is written to a file that is called complete serialization.
* If partial data of an object is written to a file then that is called selective serialization.
* If class is having static or transient variables then objects of that class supports only selective serialization.
* static member variables allocates memory outside the object so they doesn't participate in serialization.
* Transient member variables allocates memory inside the object but they will be marked for not participating in serialization process.

* Programmer can perform persistance operations only on serializable objects.

* Java object becomes serializable only when class of that object implements java.io.serializable interface (marker, empty interface).

```
Public class student implements java.io.serializable
  {
      int sno;
      String name;
      float avg;
      - - - - - -
      - - - - - -
      - - - - - -
  }
```

objects of this class supports only complete serialization.

```
public class student implements java.io.serializable
  {
      int sno;
      String name;
      static / transient float avg;
      - - - - - - - - -
      - - - - - - - - -
  }
```

objects of this class supports only selective serialization.

Sample code for serialization.

```
FileOutputStream fos = new FileOutputStream ("abc.txt"); // low level stream
ObjectOutputStream oos = new ObjectOutputStream (fos); // high level stream
//create object for student class
Student st1 = new Student (101, "raja", 47.65f);
// Perform serialization
oos.writeObject (st1);
```

↳ captures "st1" object data and writes that data to a file.

```
// Close streams
fos.close();
oos.close();
```

Prototype of writeobject() method

public final void writeobject(object obj) throws IOException.

Three important statements.

(1) If java method parameter type is java class name then you can call that method by having that class (or) one of its subclass object as argument value.

(2) If java method parameter type is abstract class name then you can call that method having one subclass object of that abstract class as argument value.

(3) If java method parameter type is interface name then you can call that method with one implementation class object of that interface.

Sample code for deserialization

```
// Create highlevel Stream
ObjectInputStream ois = new ObjectInputStream(new FileInputStream
                                                ("abc.txt"));
// perform deserialization
object obj = ois.readobject();
// type casting
Student st2 = (Student) obj;
// close streams
ois.close();
```

↳ reads data from the abc.txt file and creates and returns java class object having that data.

Prototype of readObject() method

Public final object readobject() throws IOException, ClassNotFound Exception.

(1) If java method return type is Concrete class name then that method can return either that concrete class object or one of its subclass object.

(2) If java method return type is interface name then that method returns one implementation class object of that interface.

(3) If java method return type is abstract class name then that method returns one subclass object of that abstract class.

Application on serialization and deserialization

Resources of the application

student.java

writeTest.java (perform serialization)

readTest.java (perform deserialization)

```
// student.java
import java.io.*;
public class student implements serializable
                                ↳ mandatory
 {
    int sno;
    string sname;
    float avg;
    public student()
      {
        system.out.println("student: 0-param constructor");
      }
    Student(int x, String y, float z)
      {
          sno = x;
          sname = y;
          avg = z;
          sop("student: 3-param constructor");
      }
     public void disp()
       {
          sop("sno="+sno+"sname="+sname+"avg="+avg);
       }
 }
```

```
// writeTest.java
import java.io.*;
public class writeTest
 {
   public static void main(String args[])
    {
      try
       {
         //create stream objects
         FileOutputStream fos = new FileOutputStream("abc.txt");
         ObjectOutputStream oos = new ObjectOutputStream(fos);
         //perform serialization
         Student st1 = new Student(101, "anil", 45.78f);
         oos.writeObject(st1);
        //close streams
          fos.close();
          oos.close(); sop("serialization is completed");
       } //try
      catch (IOException ioe)  //to handle known exception
        {
          ioe.printStackTrace();
        }
      catch(Exception e)        // to handle unknown exception
        {
          e.printStackTrace();
        }
    } //main
 }// class
```

```
> javac student.java
> javac writeTest.java
> java writeTest
```

NOTE: when serialization is done on the object it not only writes object data to a file but it also writes structure of the class to that file.

* If application tries to perform serialization on non-serializable object then that ob the application throws java.io.NotSerializable

Exception.

```
// readTest.java
import java.io.*;
public class readTest
  {
     public static void main(String args[])
       {
          try
           {
             //create stream
             objectInputStream ois = new objectInputStream (new File
                                                    InputStream ("abc.txt");
             // perform deserialization
              Object obj = ois.readObject();
             // type casting
              Student st2 = (Student) obj;
             //display object (st2) data
               st2.disp();
              // close stream
               ois.close();
           }
         catch (IOException ioe)
            {
               ioe.PrintStackTrace();
            }
         catch (classNotFoundException cnf)
            {
               cnf.printStackTrace();
            }
       }// main
   }// class
```

```
> javac readTest.java
> java readTest
```

* There is no terminology called selective deserialization. The object data that is return through serialization process must be retrieved through de serialization process.
* The structure of the class must be same while performing serialization and deserialization operations. otherwise the deserialization

Process raises java.io.InvalidClassException.

* In order to send java object over the network it must be designed as serializable object.
* All wrapper class objects and collection framework data structures are serializable by default.

Elements kept in arraylist are non-serializable java objects. Can you tell me whether we can send these java objects over the network or not?

NO, because arraylist object and its elements must be there as serializable objects in order to send array list object over the network.

* A specification is a document that contains set of rules and guidelines to develop the software or software applications.
* There are two types of specifications.

(1) open specification (Free to every one)

Ex: JDBC specification, servlet specification and etc...

(2) Propraitory specification (specific to one organization)

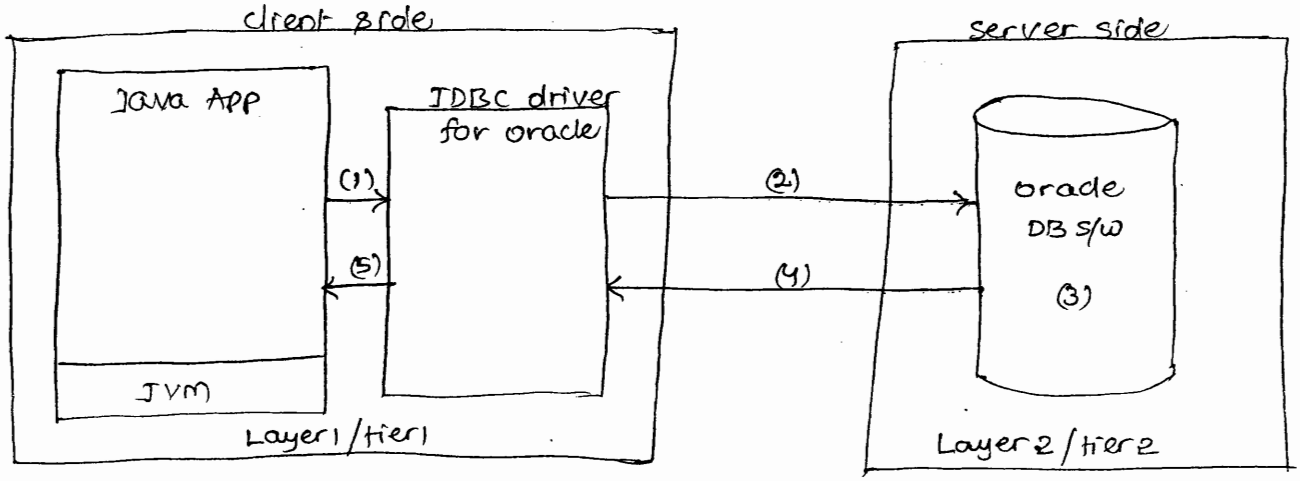
Ex: .net specification

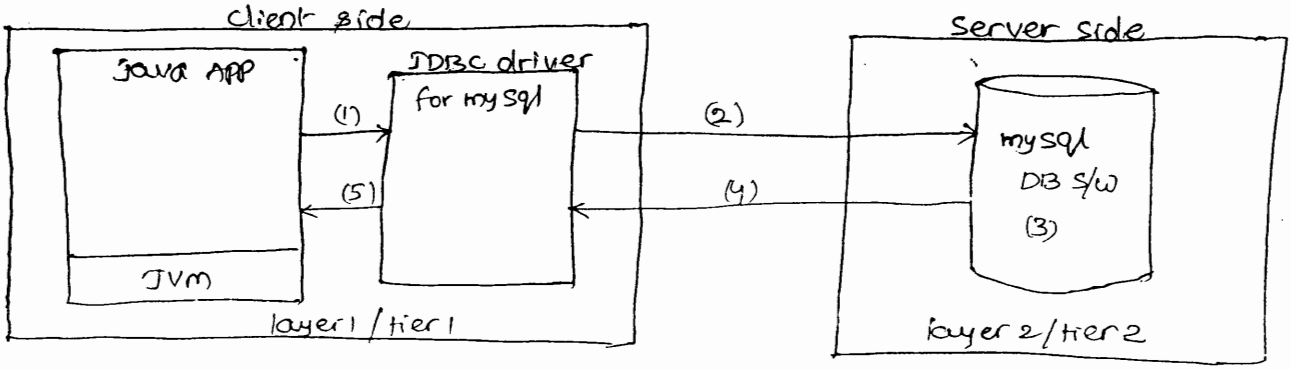
NOTE: Any company can create softwares based on open specification

* Only the creator of specification is allowed to develop software based on propraitory specification.

JDBC

* JDBC is an open specification given by SunMicroSystems having rules and guide lines to develop JDBC drivers.
* JDBC driver is a bridge software between java application and database software. It converts java calls to database calls and viceversa.





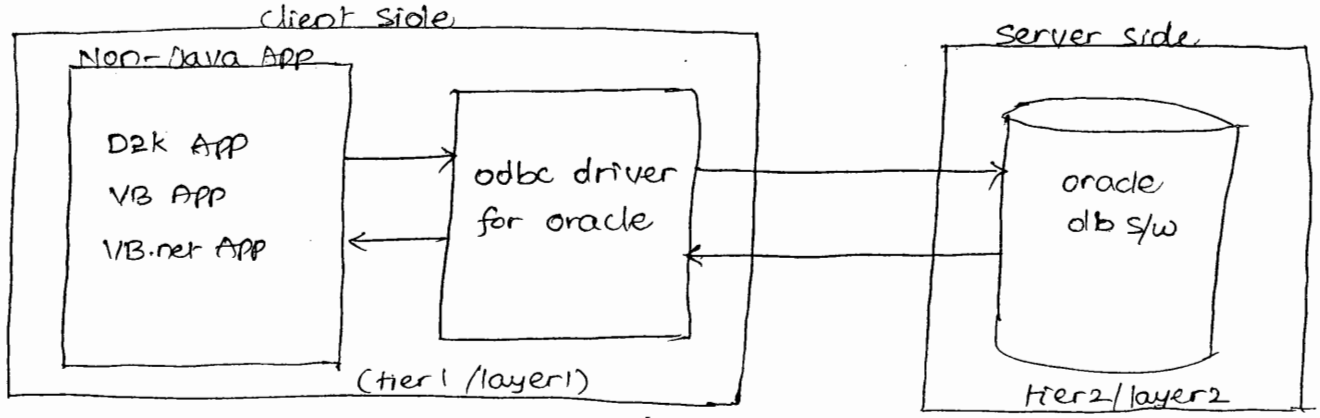
NOTE: The above JDBC applications are two tier applications. The two layers of this two tier application can be there either in same computer or in two different computers.

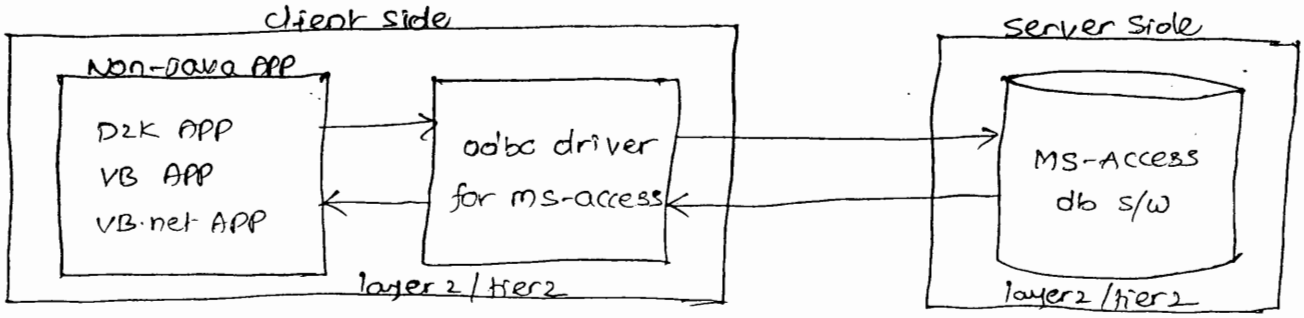
* Each JDBC driver is specific to one database software.
* We can get JDBC drivers from three parties.
  (1) Sun micro systems
  (2) Database vendor
  (3) Third party vendor.
* It is recommended to use Database vendor supplied JDBC drivers.
* Non-java applications use ODBC drivers to interact with database softwares.
* Every ODBC drivers is specific to one database software

What is an ODBC (open database connectivity)?

* ODBC is an open specification ~~that contains~~ given by X-Open company that contains set of rules and guide lines to develop

ODBC drivers for different database softwares.



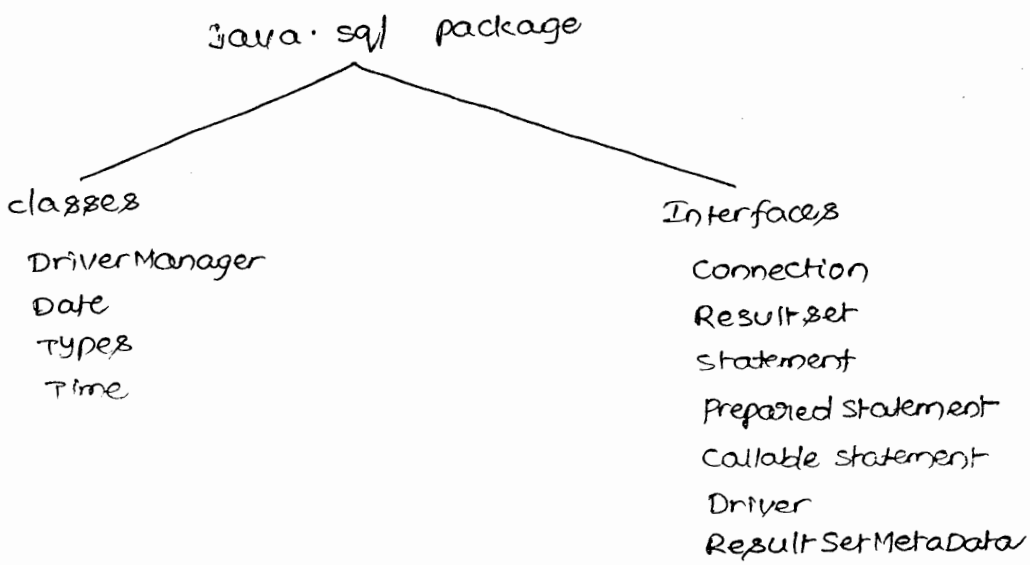


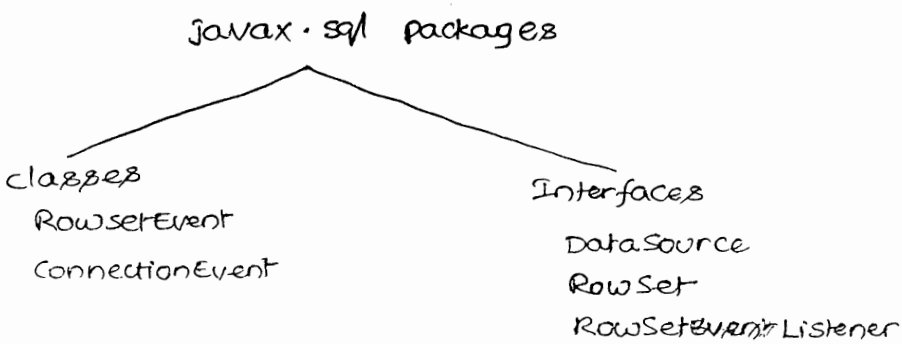
when all other software technologies are using ODBC drives. why java applications need an exclusive JDBC driver to interact with database softwares?

(A) ODBC drivers are given based on ODBC specification. This specification is designed in 'c' language using pointers support. Since java doesn't support pointers the java applications need exclusive java based JDBC drivers to interact with database softwares.

---

* Every specification contains an api representing rules and guidelines.
* In java the abstract methods of API interfaces and abstract classes represent rules whereas the concrete methods of API's abstract classes, concrete classes represent guidelines.
* The JDBC specification related API comes in the form of java.sql, javax.sql packages.





* In the above given JDBC API packages concrete methods of abstract, concrete classes represent guidelines to develop JDBC driver. whereas the abstract methods of interfaces and abstract classes represent rules to develop JDBC driver. The vender company implements these rules and follows these guidelines while developing JDBC driver for each database software.
* Implementation of all the interfaces that are given in the packages of JDBC API is not the responsibility of programmer, it is the responsibility of vendor company who gives JDBC driver to the market.
* Each JDBC driver is set of classes implementing various interfaces of JDBC API packages. while defining methods of these interfaces the classes contains logic to interact with specific database software.
* Since all JDBC drivers are given for different database softwares coming from different vendors are given based on common JDBC specification rules and guidelines given by sunmicrosystems the way we work

with all these JDBC drivers in our java applications to interact with database softwares is going to be much similar.

Sample code to understand JDBC specification, JDBC driver implementation and utilizing JDBC driver from java applications

Assume this is the JDBC specification

MyConnect.java (Interface)

```
public interface MyConnect
  {
    public void getConnection(); //rule
  }
```

Assume this is JDBC driver for oracle (vendor1)

```
public class oracleConnect implements myConnect
   {
      public void getConnection()
        {
          - - - - -
          - - - - - logic to interact with oracle db s/w
        }
   }
```

Assume this is JDBC driver for Mysql (vendor2)

```
public class mysqlConnect implements myConnect
      {
         public void getConnection()
           {
             - - - - -
             - - - - - logic to interact with mysql db s/w
           }
      }
```

Application Development

App1 (Java application)

activates JDBC driver for oracle

call getConnection(); // makes App1 talking with oracle db s/w

App2 (Java application)

activates JDBC driver for mysql

call getConnection(); // makes App2 talking with mysql db s/w

NOTE: In the above sample code App1, App2 applications are using two different JDBC drivers in much similar manner to interact with two different database softwares.

* Generally vendor companies supply their JDBC drivers in the form of jar files.
* To get the source code from compiled code use java decompiler.

  Ex: DJ DeCompiler (seperate installation)
* Even though every JDBC driver software contains lots of classes but the JDBC driver will be identified through its driver class name.
* The java class that implements java.sql.Driver interface directly or indirectly is called as JDBC driver class.
* To activate JDBC driver from our java applications we need to work with the driver class name of that JDBC driver.
* JDK software gives one built-in JDBC driver. Its driver class name is sun.jdbc.odbc.JdbcOdbcDriver
  (sun.jdbc.odbc → package; JdbcOdbcDriver → Driver class implementing java.sql.Driver interface)
* Oracle database software gives one built-in JDBC driver. Its driver class name is oracle.jdbc.driver.OracleDriver
  (oracle.jdbc.driver → package; OracleDriver → Driver class)
* we can expect odbc drivers based on odbc specification from three parties.

  (1) X-open Company

  (2) DB Vendor

  (3) Third party vendor

NOTE: The windows XP OS installation gives us multiple ddbc drivers.

* Every odbc driver is identify through its DSN. There is a possibility of creating three types of dsns.

  (1) User DSN → specific to currently logged in windows user

  (2) System DSN → visible all the windows users of a machine.

(3) File DSN → shariable in the network machines.

Procedure to create Miscrosoft ODBC driver for oracle:

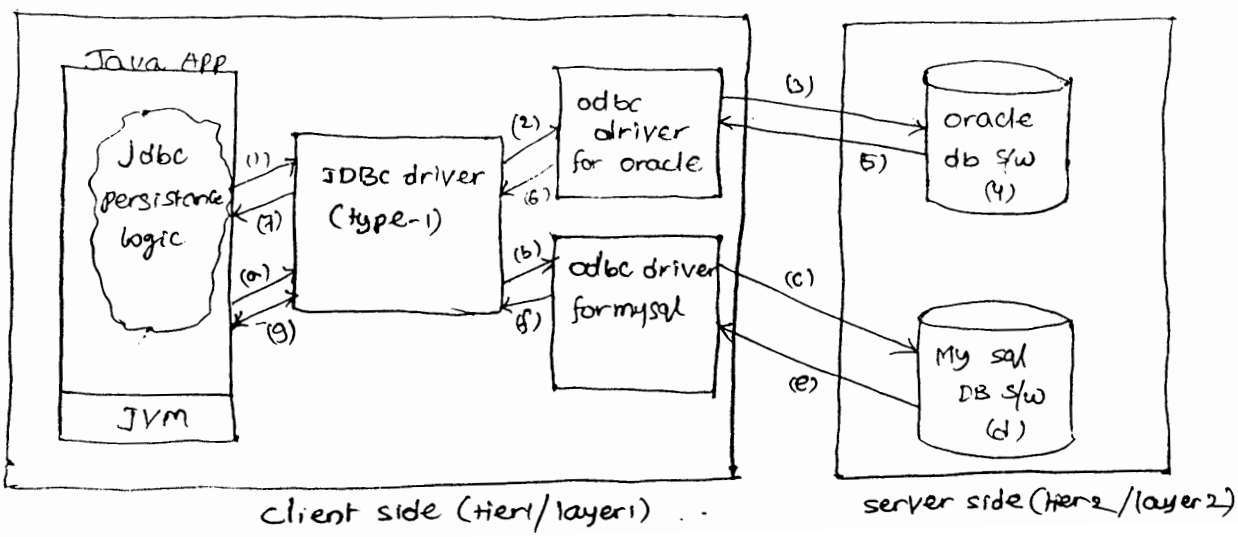
start → settings → control panel → performance and maintanance → Adminstrakive tools → datasources → user dsn → add → microsoft ODBC for oracle → finish → data source name [xyz] username [scott] server [optional for local or personal oracle software installation, manditary for the centralized installation of oracle software (gather dns string)] → ok → ok

* There are four different methodologies or mechanisms to develop Jdbc drivers based on the rules and guidelines of Jdbc specification. They are.

(1) Type 1
(2) Type 2
(3) Type 3
(4) Type 4

* Every mechanism talks about architecture or plan to develop Jdbc drivers.
* The type1 mechanism based drivers are given to take the support of odbc drivers while interacting with database softwares. In this process the type1 driver takes the support of native code to communicate with odbc drivers.

Type1 driver architecture (partial)



* JDBC driver type-1 is not specific to any database software because it won't interact with database software directly. It interacts with database softwares by using the database software specific odbc driver.
* only sun micro system is supplying type-1 mechanism based Jdbc drivers as built-in Jdbc driver of Jdk software.
* Jdk software supplies a basic and built-in service called driver manager service to manage set of Jdbc drivers and to establish the connections with database softwares by using Jdbc drivers.
* Every JDBC driver must be registered with drivermanager service. for this create JDBC driver class object and keep that object in driver manager service.
* our java applications can activate and work with drivermanager service by dealing with java.sql.DriverManager class.

Jdk's built-in driver is Type 1 driver

this driver's class name is: sun.jdbc.odbc.JdbcOdbcDriver

To register the above type1 driver with Drivermanager service

```
//create object for jdbc drivers class
sun.jdbc.odbc.JdbcOdbcDriver obj = new sun.jdbc.odbc.JdbcOdbcDriver();
// register jdbc driver with Drivermanager service
Drivermanager.registerDriver(obj);
```

* The JDBC API of jdbc specification given by sunmicro systems will be used in two different angles.

(1) As rules and guide lines by vendor companies to develop JDBC drivers.

(2) As base for the programmer to activate JDBC driver and to develop JDBC persistance logic from java applications. This persistance logic is responsible to manipulate data base data (CURD operations)

* one JDBC driver type1 can interact with multiple odbc drivers of different db s/ws.

Standard steps to develope jdbc code in our java Application

(1) register jdbc driver with Drivermanager service

(2) Make Drivermanager service to establish Connection with DB s/w.

(3) create statement object.

(4) Use statement object to send sql queries to database software and to execute them in database software.

(5) Use statement object to gather results from database software.

(6) Process the results

(7) close connection with database software

* Statement object acts as courier service or vehicle to send and receive inputs and outputs between java application and database software. This is base for the programmer to manipulate the database data from java applications.
* Connection between java application and database software represents the communication channel between java application and database software.
* To establish this Connection the driver manager service uses an appropriate one of the registered JDBC driver.
* The code that is not written in java but invokable from java applications is called as native code.
* JDBC driver type1 internally uses this native code to interact with ODBC drivers.
* Exp Except type1 mechanism drivers no other mechanism drivers (type-2, type-3, type-4) use ODBC drivers support to interact with database softwares.

Write a java application to establish the Connection with oracle database software by using JDBC driver type-1 (Built-in JDBC driver of Jdk software)

The software setup that is required

Jdk any version (jdk 1.6)

oracle db s/w (any version)

dsn created for microsoft odbc driver for oracle

```
// ConnTest.java
import java.sql.*;
Public class ConnTest
 {
    public static void main (String args[]) throws Exception
      {
         //create JDBC driver class object (Type-1 driver related)
          sun.jdbc.odbc.JdbcOdbcDriver obj= new sun.jdbc.Odbc.Jdbc
                                                OdbcDriver();
        // register jdbc driver with drivermanagerservice
          Drivermanager.registerDriver(obj);
        //Establish connection with database software.   Protocol
          Connection con = Drivermanager.getConnection ("jdbc:odbc:
                                  subname(dsn) oradsn", "scott", "tiger");
                                    ↳db url    ↓db user  ↓db password
        if (con == null)
            system.out.println ("Connection is not established);
        else
            system.out.println ("Connection is established");
       } // main
   } //class
```

```
> javac ConnTest.java
> java ConnTest
```

* Based on the protocol specified in the url of Drivermanger.getConnection() method the Drivermanager service picks up an appropriate JDBC driver even though more JDBC drivers are registered with Drivermanagerservice. The Drivermanager service uses that selected/picked up driver to establish the connection with database software.

* jdbc:odbc protocol makes DriverManager service to pickup the registered type-1 jdbc driver.

* jdbc:odbc is application level protocol

  jdbc → main protocol; odbc → sub protocol

* Protocol is a set of rules followed by two parties who participates in communication.

* Network level protocol contains rules to get communication between two physical computers. Ex: TCP/IP

* Application level protocol contains rules to get communication between two softwares or software applications. Ex: HTTP, jdbc:odbc

* JDBC:ODBC is the application level protocol containing set of rules to get communication between DriverManager service and database software.

* Most of the application level protocols can run on the top of network level protocols.

```
Connection con = DriverManager.getConnection("jdbc:odbc:oradsn", "scott", "tiger");
```

con → jdbc connection object representing connectivity with database software

* In the above statement how can you say "con" is an object when java.sql.Connection is an interface?

A) In the above statement con is object of underlaying JDBC driver supplied java class that implements java.sql.Connection Interface.

To know that class name

```
System.out.println("con obj class name is:" + con.getClass());
```

In type-1 driver con object class name is "sun.jdbc.odbc.JdbcOdbc Connection" and this class implements java.sql.Connection Interface in directly as shown below.

java.sql.Connection (Interface)

↑ Extends

sun.jdbc.odbc.JdbcOdbcConnectionInterface (Interface)

↑ Implements

sun.jdbc.odbc.JdbcOdbcConnection (class)

* class "JdbcOdbcConnection" is implementing java.sql.Connection interface.

NOTE: DriverManager.getConnection() method call returns object of a java class that implements java.sql.Connection Interface.

* when interface reference variable points to its implementation class object then that reference variable can also be called as indirect object of implementation class.

```
Connection con = DriverManager.getConnection("jdbc:odbc:oradsn", );
```

* In the above statement con is the reference variable of java.sql.ConnectionInterface pointing to object of a class that implements java.sql.Connection Interface, so "con" also becomes object of that class.

In our JDBC code why we are not specifying class name of JdbC connection object (con) ?

(A) The class name of Connection object will change based on the Jdbc driver we use so if that class name is specified our Jdbc code becomes specific to one jdbc driver so we never specify that class name to write jdbc code in a common way for all jdbc drivers.

Rewriting ConnTest.java

```
// connTest.java
import java.sql.*;
Public class ConnTest
    {  public static void main(String args[])
        {
```

```
//load jdbc driver class
class.forName("sun.jdbc.odbc.JdbcOdbcDriver");
//here type1 driver will be registered with drivermanager
                                    service internally

//establish connection with db s/w
Connection con = DriverManager.getConnection("jdbc:odbc:oradsn",
                                    "scott", "tiger");
if (con == null)
    system.out.println("Connection is not established");
else
    System.out.println("Connection is established");
} //main
} //class
```

(Q) In the above code class.forName(-) method call just makes JVM to load given jdbc driver class. How can you say jdbc driver is registered with DriverManager service in that process?

(A) class.forName(-) makes JVM to load sun.jdbc.odbc.JdbcOdbc_Driver class → static block of this driver class executes automatically → This static block contains logic to create object of jdbc driver class and to register that object with DriverManager service by calling Drivermanager.registerDriver(-) method. In the static block of sun.jdbc.odbc.JdbcOdbcDriver class we can see the following code registering type-1 driver with Drivermanager service

```
static
{
  - - - - -
  - - - - -
  //creating type-1 jdbc driver class object
  JdbcOdbcDriver jdbcodbcdriver = new JdbcOdbcDriver();
  try
  {
    //registering type1 driver with DriverManager service
    Drivermanager.registerDriver(jdbcodbcdriver)
  }
```

```
catch (SQLException sqlexception)
    {
      - - - - -
      - - - - -
    }
}
```

NOTE: Almost every vendor supplied jdbc driver related jdbc driver class contains the above given static blocks. Registering its respective jdbc driver with drivermanager service.

(Q) How many ways are there to register jdbc driver with Driver-Manager service?

(A) (1)
```
sun.jdbc.odbc.JdbcOdbcDriver obj = new sun.jdbc.odbc.JdbcOdbcDriver();
Drivermanager.registerDriver(obj);
```

The above code registers jdbc driver (same) twice with Drivermanager service.

(i) to static block of driver class when driver class object is created.

(ii) Explicitly when programmer calls Drivermanager.registerDriver().

It is not recommanded process.

(2)
```
sun.jdbc.odbc.JdbcOdbcDriver obj=new Sun.jdbc.odbc.JdbcOdbcDriver();
```

Here jdbc driver will be register Drivermanager service only once but programmer creates one explicit unused object for driver class. It is not recommanded approach.

(3)
```
DriverManager.registerDriver (new sun.jdbc.odbc.JdbcOdbc.Driver());
```
↳ Anonomous object

Limitations are same as approach no:1

(4)
```
class.forName ("sun.jdbc.odbc.JdbcOdbcDriver");
```

Static block of driver class registers jdbc driver with Driver-Manager service only once.

It is recommanded approach.

(5) `Class.forName(args[0]);`

```
// ConnTest.java
import java.sql.*;
public class ConnTest
  {
    public static void main(String args[])
      {
        //load jdbc driver class
        Class.forName(args[0]);
        //establish connection with d/b s/w
        Connection con = Drivermanger.getConnection("jdbc:odbc:oradsn",
                                                    "scott","tiger");
      } //main
  } //class
>javac ConnTest.java
>java ConnTest sun.jdbc.odbc.JdbcOdbcDriver
                 └> args[0] value
```

same as approach no. 4 but allows to pass Jdbc driver class name from outside the application dynamically at runtime. It is recommanded approach.

(6) We can pass input value to java application as System property value by using java -D option. To read this value our application can use System.getProperty(-) method.

```
// ConnTest.java
import java.sql.*;
Public class ConnTest.
  {
    Public static void main(String args[])
      {
        //load jdbc driver class Read system property value
        String cname = System.getProperty("my.driver");
        //load jdbc driver class
        Class.forName(cname);
        //establish connection with db s/w
        Connection con = Drivermanager.getConnection("jdbc:odbc:oradsn",
                                                     "scott","tiger");
```

```
    } //main
} //class
```

```
>javac ConnTest.java
>java -Dmy.Driver = sun.jdbc.odbc.JdbcOdbcDriver ConnTest
```

↳ system property name and value

Recommanded to use same as approach no. 5.

(7) The DriverManager class attempts to load the driver classes automatically which are referenced in the system property called "jdbc.drivers"

```
//ConnTest.java
import java.sql.*;
Public class ConnTest
    {
        public static void main(String args[]) throws Exception
            {
                //establish connection with db s/w
                Connection con = DriverManager.getConnection ("jdbc:odbc:oradsn",
                                                              "scott","tiger");
            } //main
    } //class
```

```
>javac ConnTest.java
>//java -Djdbc/odbc/JdbcOdbc
>java -Djdbc.drivers = sun.jdbc.odbc.JdbcOdbcDriver ConnTest
```

↳ fixed property name

The above process makes current machine remembering registered jdbc driver for long time.

(8) In the process of loading subclass the JVM automatically loads Super class. In this process the static blocks of both subclass and superclass will be executed automatically. The below process is not recommended it makes our class not extending from other useful classes like Thread, Frame, Applet and etc.

```
// ConnTest.java
import java.sql.*;
Public class ConnTest extends sun.jdbc.odbc.JdbcOdbcDriver
 {
    Public static void main (String args[]) throws Exception
        {
          //establish connection with DB s/w
          Connection con = DriverManager.getConnection(
```

* According to DBMS the sql queries are
  (1) DML queries (insert, update, delete queries)
  (2) DRL queries (select queries)
  (3) DDL queries (create table, alter table and etc...)
  (4) TCL queries (Commit, rollback, savepoint and etc...)
* According to jdbc persistance logic there are two types of sql queries
  (1) select sql queries (DRL queries)
  (2) Non-select sql squeries (DDL, DML, TCL queries)
* select sql query execution returns bunch of records that are selected from database table, whereas non-select queries (like DML queries) return a numeric value representing no. of records that are effected because of query execution.
* our jdbc persistance logic uses executeUpdate() to send and execute non-select queries in database software. Similarly it uses executequery() method to send and execute select queries to db s/w

Prototype of executeQuery(-) method

public ResultSet executeQuery(String query) throws SQLException.

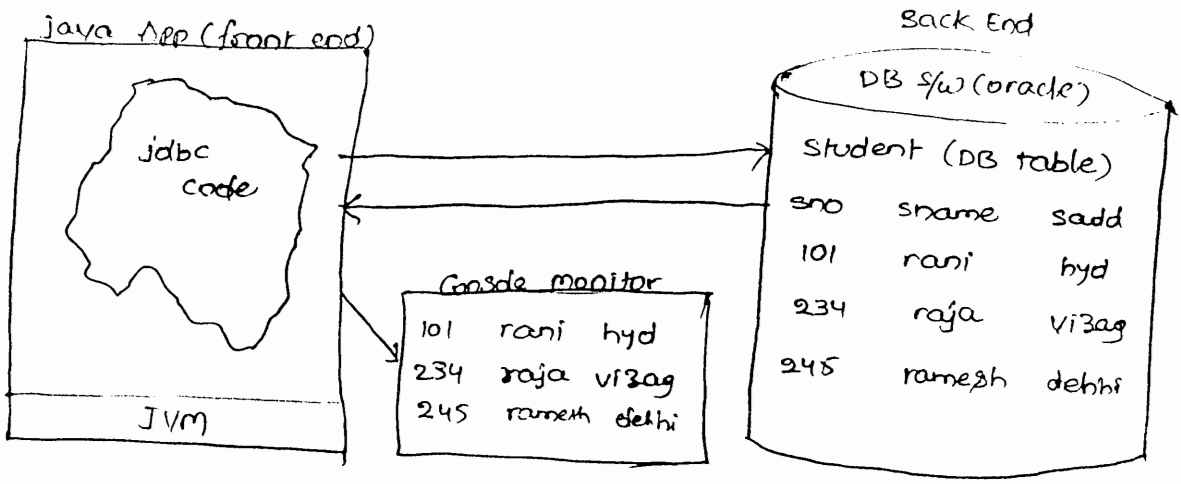
* executeQuery(String query) method returns jdbc resultset object having set of records collected from database software after executing select sql query.

Prototype of executeUpdate(-) method

public int executeUpdate(String query) throws SQLException

* This method returns a numeric value representing no. of records that are effected after executing non-select sql query in database software.
* The above given two methods are declared in java.sql.Statement Interface.
* since the end users like civil engineers, bank employees and etc.. can not work with database software directly by using sql queries. The expect GUI or CUI applications as front end to perform various operations on back end called d/b s/w's.
* while developing these front end applications in java environment we use jdbc support to interact with database software and to manipulate database data.

write a java application to read the records of database table from java application



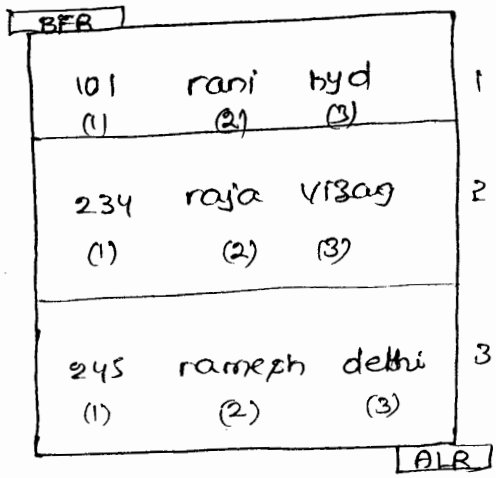
Step(1): Make sure that oradsn is available as dsn for odbc driver for oracle.

Step(2): keep student database table ready in database software (oracle) with records.

Step(3): Develop jdbc code based application using type1 driver as shown below.

```
// selectTest.java
import java.sql.*;
public class selectTest
 {
    public static void main (String args[]) throws Exception
     {
        // load jdbc driver class
        Class.forName ("sun.jdbc.odbc.JdbcOdbcDriver");
       // establish Connection with database software
        Connection con = DriverManager.getConnection ("jdbc:odbc:oradsn",
                                                      "scott","tiger");
      // create jdbc statement object
        Statement st = con.createStatement ();
      // send and execute sql query in database software and
        gather the results.
        ResultSet rs = st.executeQuery ("select * from student");
      // Process the result
        while (rs.next () == true)
           {
             System.out.println ( rs.getInt (1) + " " + rs.getString (2) +
                                  " " + rs.getString (3));
           } // while
      // close jdbc objects
         rs.close() ;
         st.close() :
         con.close();
     }
}
```

rs (Resultset object)



* Every result set object contains two positions
  (1) BFR (Before First record)
  (2) ALR (After Last record)
* By default record pointer / cursor resides in BFR position.
* Records in result set and values in each record of result set will be stored having one based index.
* rs.next() method moves the record pointer from current position to next position. If the next position is valid position (record) then this method returns true. If the next position is ALR (or) BFR position then this method returns false.
* To read each record values from jdbc result set object we can use getXxx(-) methods, with column name (or) column index.

  getXxx(-)

  getString(-), getFloat(-), getInt(-), getDouble(-), getLong(-), getDate(-) and etc.

Prototype of getXxx(-) methods:

Public yyyy getXxx(String columnname / int columnindex) throws SQLException.

* It is always recommanded to close the jdbc objects in the reverse order of their creation.
* If you don't close jdbc objects, at the end of their utilization then they will be closed automatically and abnormally at the end of the application's execution.
* The jdbc applications can not read uncommitted data from the database tables.
* Jdbc statement object means it is the object of a jdbc driver supplied java class implementing java.sql.StatementInterface directly or indirectly.
* Jdbc resultset object means (ors) it is the object of a jdbc driver supplied java class implementing java.sql.resultset interface directly or indirectly.
* All jdbc objects of jdbc programming are objects of jdbc driver supplied java classes implementing various interfaces of jdbc API but programmer never specifies these class names in his jdbc programming because their names will be change based on the jdbc driver.
* When interface reference variable points to its implementation class object then reference variable ~~is~~ obj becomes object of implementation class with limitations.

```
inter-face ABC
  {
      Public void x();
  }
 public class Demo implements ABC
  {
      Public void x()
        {
          sop("Demo: x()");
        }
      public void y()
        { sop("Demo: y()");
        }
```

```
public static void main (String args[])
  {
    ABC ab=new Demo();
     ab.x(); // valid
     ab.y(); // invalid
   sop("class name of ob is" +ab.getClass());
  }                              ↳ gives Demo class
}
```

```
Statement st = con.createStatement();
```

* In the above statement a) createStatement() is the method declared in java.sql.Connection interface (As the rule of jdbc specification).

b) The jdbc driver supplied java class that implements java.sql.Connection interface contains implementation for this createStatement() method (In type-1 driver that class name is sun.jdbc.odbc.JdbcOdbcDriConnection).

c) The definition of createStatement() method contains logic to create and return object of a java class (jdbc driver supplied) that implements java.sql.Statement interface (in type-1 driver that class name is sun.jdbc.odbc.JdbcOdbcStatement).

```
Connection con= DriverManager.getConnection(_,_,_);
```

* getConnection is the static method of DriverManager class. This method definition returns object of jdbc driver supplied java class implementing java.sql.Connection interface as jdbc connection object. (jdbc driver supplied class). (In type-1 driver class is sun.jdbc.odbc.JdbcOdbc Connection).

* SelectTest.java

```
import java.sql.*;
import java.util.*;
```

```
Public class Test1
{
    Public static void main(String args[]) throws Exception
    {
        - - - - - -
        - - - - - - -   } Logic to create jdbc connection object
        - - - - - - -

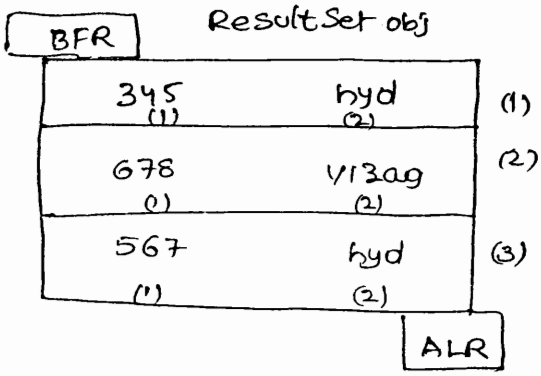
        //read input values from key board.
        Scanner sc = new Scanner(System.in);
        System.out.println("enter Student no:");
        int no = sc.nextInt();
        // create jdbc statement object
        Statement st = con.createStatement();
        //send and execute sql query in db s/w and gather results
        ResultSet rs = st.executeQuery("select * from student
                                        where sno >= "+no);
        //Process the result (ResultSet obj)
        while(rs.next != false)
        {
            s.o.P(rs.getInt(sno)+" "+rs.getString("sname")+" "
                                    + rs.getString("sadd"));
        }
        // close jdbc objects
        rs.close();
        st.close();
        con.close();
    } // main
} // class
```

* while retrieving column values from the records of jdbc ResultSet object use those column indexes in the order column values are stored in the records of jdbc ResultSet object. Don't use those column indexes in the order columns are there in the database table.

Example code:

```
Statement st = Con.createStatement();
ResultSet rs = st.executeQuery("select sno, sadd from student");
                                       ①     ②
// Process the resultset object
while(rs.next()!=false)
{
   sop(rs.getInt(1) + " " + rs.getString(2));
}
```

↳ gives sno column values ↳ gives sadd column values



selecting specific column values from database table with multiple conditions

```
//read input values from keyboard
Scanner sc = new Scanner(System.in);
sop("enter starting range of student no:");
int stno = sc.nextInt();
sop("enter ending ranage of student no:");
int endno = sc.nextInt();
//create jdbc statement obj
Statement st = Con.createStatement();
ResultSet rs = st.executeQuery("select sadd, sname from
                student where sno>="+stno+" and sno<="
                +endno);
```

```
//Process the result set object
while(rs.next()!=false)
{
    sop(rs.getString(1)+" "+rs.getString(2));
}
```

gives sadd values — gives sname values

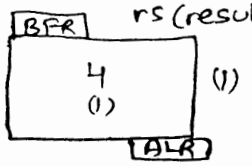
* while framing SQL queries with string input values the programmer needs to be little bit careful

Example Code:

```
//read input values from keyboard
Scanner sc=new Scanner(System.in);
sop("Enter begining char or chars of student name to perform
     search operation");
String cond=sc.next(); // gives "char:r
//Converting input value as required for sql condition
Cond="'"+Cond+"%'"; // r becomes 'r%'
//create jdbc statement object
Statement st=con.createStatement();
ResultSet rs=st.executeQuery("Select * from student where
                              sname like "+Cond);
//Process the resultset object
while(rs.next()!=false)
{
    sop(rs.getString(1)+" "+rs.getString(2)+" "rs.getString(3));
}
```

* Aggrigate functions retrieves data from database table as not specific to any record of the table.

Example to send and execute sql aggrigate function baseol query



```
//create jdbc statement object
 Statement st = con.createStatement();
ResultSet rs= new ResultSet();
ResultSet rs = st.executeQuery("select count(*) from
                                        student");
//process the jdbc ResultSet object
 if (rs.next())
   { int cnt = rs.getInt(1);
     // int cnt = rs.getInt("count(*)");
     sop("count of records:" +cnt);
   }
```

* once ResultSet object is closed we can not retrieve data from ResultSet object.
* when jdbc objects are closed by calling close() method all the database resources that are associated with these objects will be released immediately.
* To solve unsupportedClassVersion Exception problem that raises after installing oracle software make sure that java_home\bin directory is kept as first value of path environment variable.

my computer → properties → advanced tab → env variables

variable name : path

value : D:\java\jdk 1.6.0_11\bin ; D:\oracle\product\10.2.0\db_1\bin;.

D:\java\jdk 1.6.0_11\bin → java_home

D:\oracle\product\10.2.0\db_1\bin → oracle_home

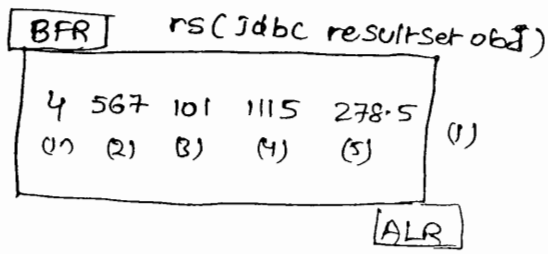
. → current directory

ok → ok → ok

Example code to send and execute multiple aggrigate functions based select sql query

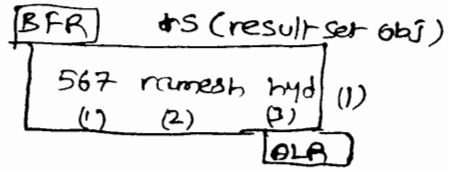
```
//create jdbc statement object
 Statement st = con.createStatement();
```

```
ResultSet rs = st.executeQuery("select count(*), max(sno),
                    min(sno), avg(sno), sum(sno) from student");
//process the jdbc resultset object
if (rs.next())
{
   sop("count of records:" + rs.getInt(1));
   sop("max val of sno" + rs.getInt(2));
   sop("min val of sno" + rs.getInt(3));
   sop("sum of so column" + rs.getInt(4));
   sop("avg of sno col values:" + rs.getFloat(5));
}
```



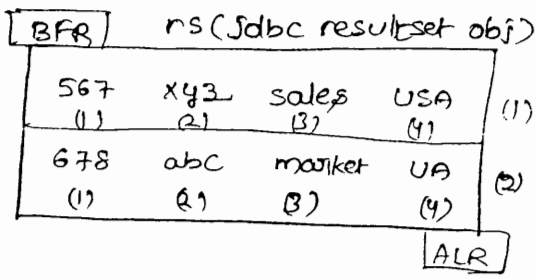
Example code to execute sub query (Gives that student record who is having max sno value)

```
//create jdbc statement object
Statement st = con.createStatement();
ResultSet rs = st.executeQuery("select * from student where
                    sno = (select max(sno) from student)");
//process the jdbc resultset object
if (rs.next())
   {
      sop(rs.nextInt(1)+" "+rs.getString(2)+" "+rs.getString(3));
   }
else
   sop("no record found");
```



Example code that selects the data from multiple columns of multiple database tables.

```
//create jdbc statement object
Statement st = con.createStatement();
ResultSet rs = st.executeQuery("select empno, ename, dname,
                                loc from emp, dept");
//process the resultset object
while(rs.next())
  {
    sop(rs.getInt("empno")+" "+rs.getString("ename")+" "
        +rs.getString("dname")+" "+rs.getString("loc"));
  }
```



write a java application by using jdbc driver type-1 to delete the records of student table based the given student no.

```
// DeleteTest.java (use executeUpdate(-) method)
import java.sql.*;
import java.util.*;
public class DeleteTest
  {
    public static void main(String args[]) throws Exception
     {
       //read input values from keyboard
        Scanner sc = new Scanner(System.in);
        int no = sc.nextInt();
       //register jdbc driver with DriverManager service
        Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");
```

```
        //establish connection with DB software
          Connection con = DriverManager.getConnection("jdbc:odbc:oradsn",
                                                        "scott","tiger");
        //create jdbc statement object
          Statement st = con.createStatement();
        //send and execute query in db software
          int res = st.executeUpdate("delete from student where
                                      sno="+sno);
        //process the result
        if (res==0)
            System.out.println("no record found to delete");
        else
            System.out.println(res+" no.of records are deleted");
        //close jdbc objects
          st.close();
          con.close();
      }
  }
```

* Java application sends and execute non-select queries on database software by enabling autoCommit mode. we can disable this autoCommit mode from our java application by using con.setAutoCommit(false) method.
* After disableing autoCommit mode if java application sends and executes non select sql queries in database software then use con.commit() method to commit the query execution results (or) use con.rollback() method to rollback the query execution results.

Example code

```
        //disable autoCommit mode
          con.setAutoCommit(false);
        //create jdbc statement object
```

```
Statement st = con.createStatement();
//send and execute sql query in db s/w
int res = st.executeUpdate("delete from student where sno="
                           +no);
//rollback or Commit
con.rollback(); //or con.Commit();
```

* The above code demonistrates TCL operations.

(Q) When all other SQL queries are going to database software by using jdbc statement object why commit and rollback instructions should be given to database software by using jdbc Connection object?

A) Statement object deals with single query execution at a time. But Commit and rollback operations should be performed with respect to multiple sql queries execution so we use jdbc connection object which represents whole database to pass the commit and rollback instructions.

The SQL queries whose execution can generate results should be given to database software by using jdbc statement object. The instructions whose execution do not return results like Commit, rollback should be given to database software using jdbc Connection object.

Write a java application to insert the record into database table by gathering values from keyboard.

```
// insertTest.java
import java.sql.*;
import java.util.*;
Public class insertTest
  {
    Public static void main(String args[])
     {
       //read input values from keyboard
        Scanner sc = new Scanner(System.in);
```

```
System.out.println("Enter student no.");
int no = sc.nextInt();
System.out.println("Enter student name");
String name = sc.next();  // raja
System.out.println("Enter student address");
String addr = sc.next();  // hyd
// Convert input values as required for sql query
name = "'" + name + "'";  // raja becomes 'raja'
addr = "'" + addr + "'";  // hyd becomes 'hyd'
// create jdbc connection object
Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");
Connection Con = DriverManager.getConnection("jdbc:odbc:oradsn",
                                             "scott", "tiger");
//create jdbc statement object
Statement st = Con.createStatement();
// frame example sql query with direct values
//insert into student values (101, 'raja', 'hyd');
// frame query with variables
String qry = "insert into student values(" + no + "," + name +
                                          "," + addr + ")";
System.out.println(qry);
// send and execute sql query in db s/w
int result = st.executeUpdate(qry);  // non-select
// process results
if (result == 0)
    System.out.println("Record insertion is failed");
else
    System.out.println("Record insertion is done");
```

```
            // close jdbc objects
               st.close();
               con.close();
            } // main
         } // class
```

Industry coding standards

* Don't declare the exception to be thrown using throws statement. Always catch and handle the exceptions with try and multiple catch blocks.
* write comment for every line.
* To avoid nullPointerException call method on java object after ensuring that object is not holding null value. close jdbc objects (or) any other stream objects in finally block because that block executes irrespective of the exception that is raised and gives the gaurentee of closing stream objects in any situation.

Write a java application to update student details based on student no.

```
// UpdateTest.java (with coding standards)
  /* App to update student details
     * version : 1.0
     * author : Team-S */
  import java.sql.*;
  import java.util.*;
  public class UpdateTest
    {
       public static void main(String args[])
         {
            Connection con = null;
            Statement st = null;
            Scanner sc = null;
```

```
try
 {
  // read input values from keyboard
   sc = new Scanner(System.in);
   int no=0;
   String name=null;
   String addrss=null;
   if(sc!=null)
     {
      System.out.println("Enter existing student number");
      no = sc.nextInt();
      System.out.println("Enter new name");
      name = sc.next();
      system.out.println("Enter new address");
      addrs = sc.next();
     } // if
  //create jdbc con obj
    class.forName("sun.jdbc.odbc.JdbcOdbcDriver");
    con=DriverManager.getConnection("jdbc:odbc:oradsn","scott","tiger");
    // create statement object
    if(con!=null)
       st = con.createStatement();
  // frame example query with direct values
  // String qry = "update student set sname='maharaja', sadd='vizag'
                                        where sno=101;
  String qry="update student set sname='"+name+"', sadd='"+addrs
                                        +"' where sno="+no;
  system.out.println(qry);
  //send and execute the query in database s/w. int res=0;
  if(res!=null)
  res = st.executeUpdate(qry);
  // process results
   if(res==0)
      system.out.println("no record found");
```

```
            else
                System.out.println(res + "no. of records are updated");

        } // try

        Catch (ClassNotFoundException cnf) // handles known exception
          {
            cnf.printStackTrace();
          }
        catch (SQLException se)     //handles known exception
          {
            se.printStackTrace();
          }
        Catch (Exception e)     //handles unknown exception
          {
            e.printStackTrace();
          }
        finally
          {
            //close jdbc objects
             try
               {
                 if (st != null)
                     st.close();
               }
             catch (SQLException se)
               {
                 se.printStackTrace();
               }
             try
               {
                 if (con != null)
                     con.close()
               }
             catch (SQLException se)
               {
                 se.printStackTrace();
               }
          } // finally
    } // main
} // class
```

* In java local variables do not have initial values / default values so when they are declared they must be assigned with initial values based on their data types.

* create table, drop table, alter table etc queries are called as DDL queries. In real time programers are not responsible to do these DDL operations through queries execution.

* Every company contains DBT (Database Team) and that team is responsible to design the tables by satisfying all the six forms of normalisation rules.

NetBeans :

type : IDE for java environment

Version : 6.7.1 (Compatable with jdk 1.6)

vendor : sun microsystems (oracle corporation)

open source software (free software)

To download software : www.netbeans.org

for help : www.netbeans.org

Procedure to develop java application by using netbeans IDE to drop database table

step(1) : Launch netbeans IDE

step(2) : create java project from netbeans IDE

File menu → new project → java → java application → next → Project name → deselect create main class → select main project → finish.

step(3) : Add java class to the project

Right click on the project → new → java class → class name (DropTest) → finish

psvm + tab space

For various shortcuts of netbeans IDE refer help menu keyboard shortcuts.

Ctrl + shift + i → package import

```
//Droptest.java
import java.sql.Connection;
import java.sql.Drivermanager;
import java.sql.Statement;
import java.sql.Scanner;
public class Droptest
    {
      public static void main(String args[]) throws Exception
        {
          //read input values from keyboard
           scanner sc=new scanner(system.in);
           system.out.println("Enter table name to drop");
           String tabname = sc.next();
          //create jdbc connection object
           class.forName("jdbc.sun.jdbc.odbc.JdbcOdbcDriver");
           Connection con= Drivermanager.getConnection("jdbc:odbc:oradsn",
                                                       "scott","tiger");
         //create statement object
          Statement st = con.create statement();
         //send and execute DDL query to db s/w
          int res = st.executeupdate("drop table "+tabname);
         //process the results
          if (res ==0)
               system.out.println(tabname +" is not found");
          else
               system.out.println("table dropped");
         // close jdbc objects
           st.close();
           con.close();
        } //main
    } //class
```

Step(4): Run the application

Right click in the source code of droptest.java → run file → click on it

Procedure to set Command line argument values to java application of netbeans IDE project

Rightclick on project → run → main class [droptest] → arguments

[val1 val2 ---]
↓ ↘
args[0] args[1]

How can we use single method call to execute both select and non-select queries?

(A) Use execute() method call on jdbc statement object for this operation. This method call returns true when select query is executed and this method call returns false when non-select query is executed.

Prototype: public boolean execute(String query) throws SQLException.

This method call makes the programmer to gather the query execution results seperately through other method calls.

if select query is executed (return value is true)

ResultSet rs = st.getResultSet(); → gathers the result

if non-select query is executed (return value is false)

int cnt = st.getUpdateCount();
↓
numeric value representing the no. of records that are effected due to non-select sql query execution.

* working with execute() method is not recommanded process

[illegible] executeQuery() [illegible] for [illegible] select query and

Use executeUpdate() method for executing non-select query.

Example on execute method

```
// create statement object
    Statement st = con.createStatement();
//send and execute query
//boolean flag = st.execute("select * from student");
boolean flag = st.execute("delete from student where sno=100");
if (flag==true)
  {
    System.out.println("select qry executed");
    ResultSet rs= st.getResultSet();
                  st.executeQuery("select * from student");
    while (rs.next())
      {
        system.out.println (rs.getInt(1)+" "+rs.getString(2)+" "
                                          +rs.getString(3));
      }
    rs.close();
  }
else
  {
    System.out.println("non-select query executed");
    int cnt = st.getUpdateCount();
    System.out.println("no. of records that are effected "+cnt);
  }
```

* Any vendor can develop JDBC driver for any database software by using one of the following four methodologies (or) mechanisms. They are

(1) TYPE-1 (Jdbc-Odbc bridge)

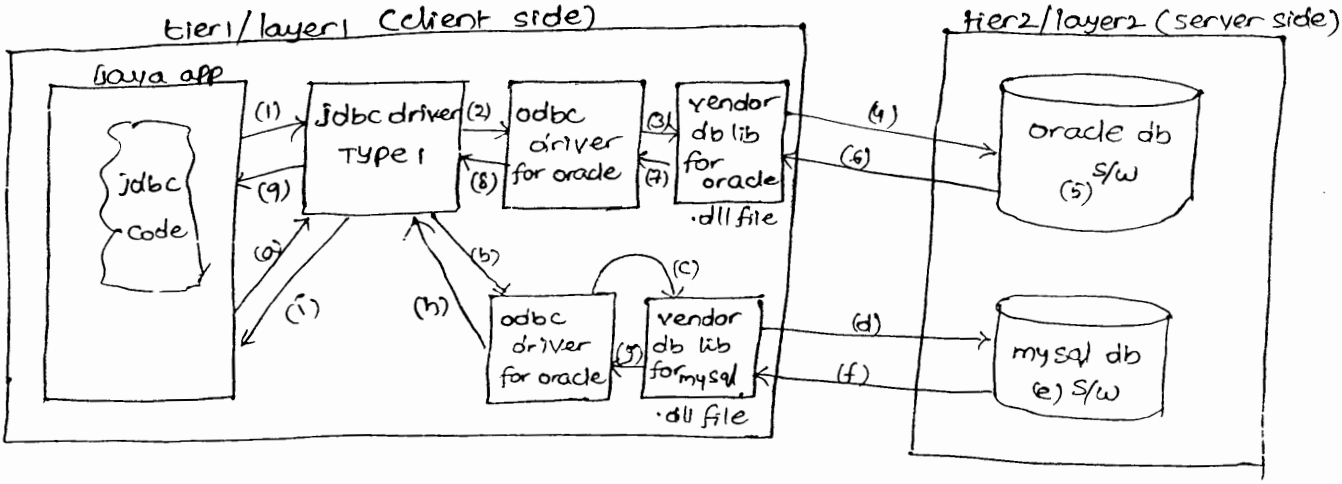
(2) TYPE-2 (Native-API/ partly java Driver)

(3) TYPE-3 (Net protocol /All Java Driver)

(4) TYPE-4 (Native-protocol/all-java Driver)

* Every database software gives one library called vendor db library having code and logic to locate and interact with db software. Jdbc drivers and odbc drivers take the support of these vendor db libraries to locate and communicate with their respective database softwares.
* Generally this vendor database library comes as .dll file. In case of oracle this file name is oci.dll (comes with oracle s/w installation).
* oci - oracle connection interface.

Type1 mechanism/ methodology/ Architecture



* In client server environment in order to work with centralized oracle database software the server oracle software will be installed on the server machine of the network and client oracle software will be installed in the remaining machines of the network. Both client and server oracle software installations give vendor db library for oracle.

Advantages:

* Type-1 driver is not specific to any database software. It uses an appropriate database software specific odbc driver to interact with database softwares. only sun micro systems is supplying jdbc driver based on type-1 mechanism.
* can be used to interact with any db software.

* Comes bas built-in driver of jdk software so no need of arranging this driver seperately.

* Since odbc drivers are available almost for all database softwares we can use type-1 driver to interact with all the database softwares.

Disadvantages

* Because of multiple conversions the performance is poor.

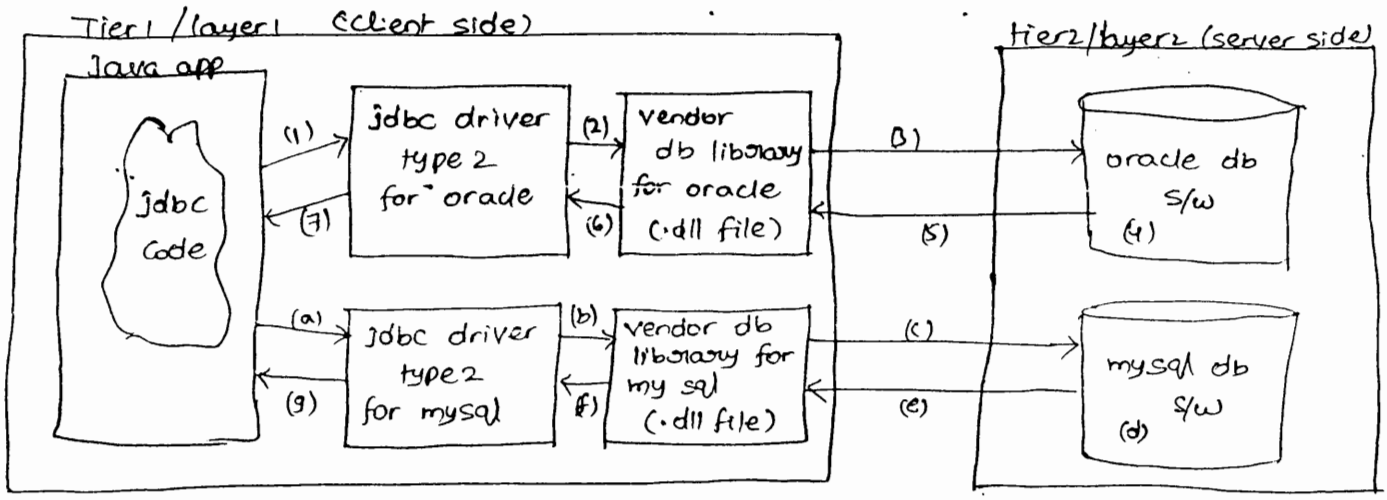
* Since vendor db library and odbc driver are required at client side
  a) Type-1 driver is not suitable for internet based applications.
  b) Type-1 driver is not suitable for untrusted applet to database software communication.

* Type-1 driver is not suitable in large scale projects.

* Trusted applet can interact with files on filesystem to perform read and write operations whereas untrusted applets can not do this work.

Type-2 mechanism :

Type2 mechanism/ methodology / Architecture



* Type-2 jdbc driver uses native code to communicate with Vendor db library. Except type1 mechanism no other mechanism takes the support of odbc drivers.

* Type-2 jdbc driver takes the support of db vendor db library directly to interact with database software.

Advantages

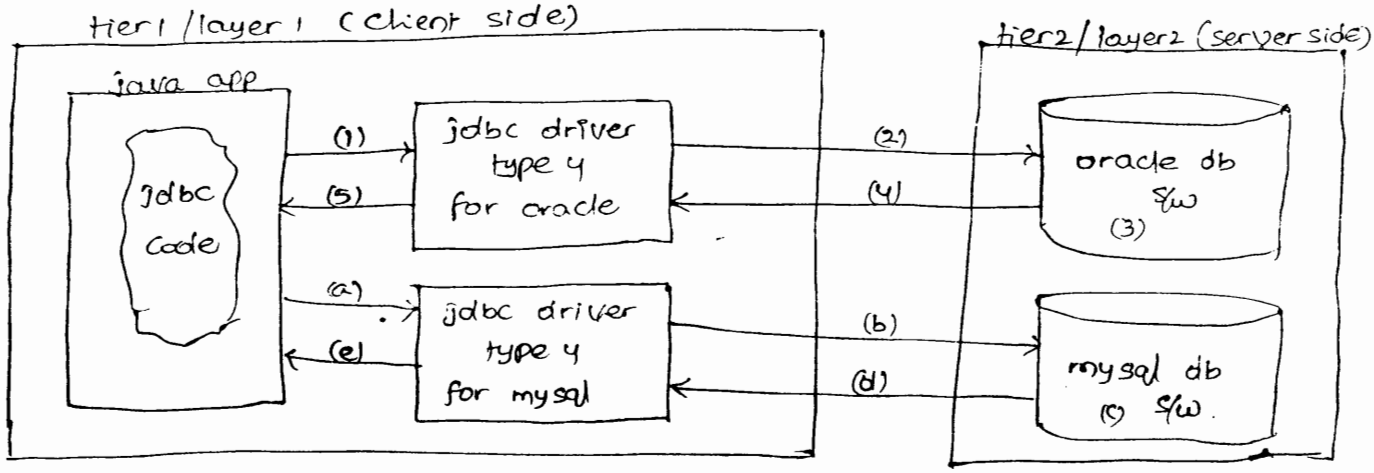
* No need of working with odbc drivers.
* Use significantly better performance compare to type1 driver.

Disadvantages

* For every database software we need one seperate jdbc driver type-2.
* Since odbc driver & vendor db library is required at client side we can not use type-2 driver in internet based applications and for untrusted applet to database software communication.
* Not suitable to large scale applications.
* programmer needs to arrange type-2 jdbc drivers seperately

Type-4 mechanism:

Type 4 mechanism/ methodology/ Architecture



* Type-4 driver is designed to interact with database softwares directly without taking support of vendor db library and odbc driver.

Advantages

* gives better performance compare to type-1 and type-2 drivers.
* NO need of working with odbc drivers and vendor db libraries.
* This driver will be developed purely in java environment so it exhibits platform independency and portability

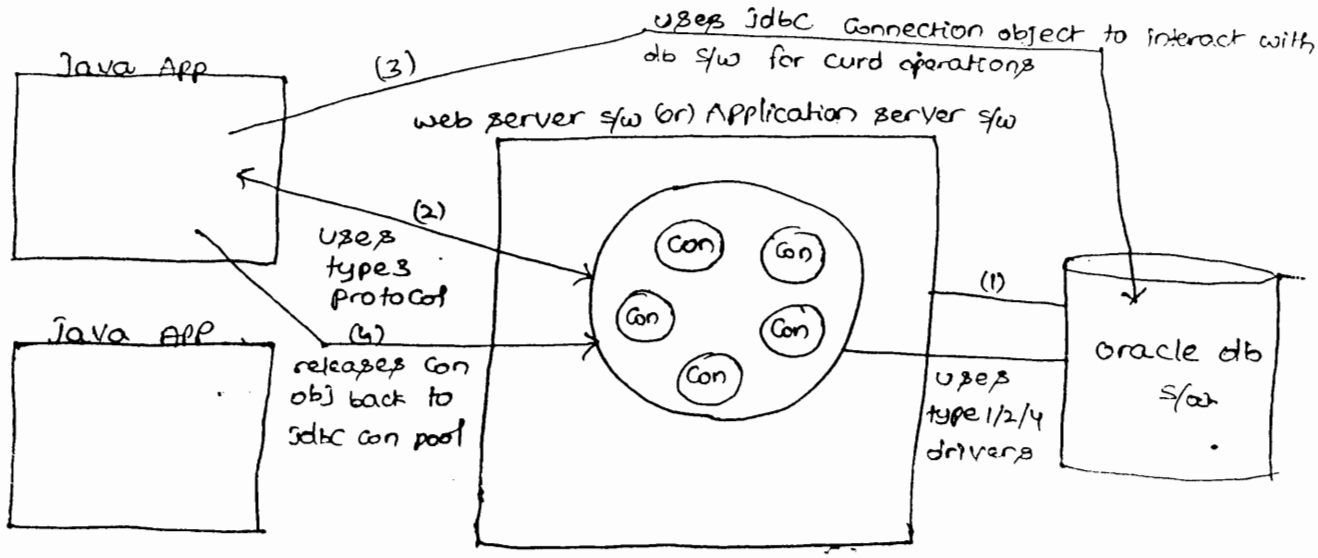
* This driver can be downloaded to client machine dynamically from internet network.
* suitable for internet programming based applications and for untrusted applet to database software communication.
* suitable for large scale projects.

Disadvantages

* For every database software we need one seperate jdbc driver type-4.
* Programmer must arrange jdbc drivers seperately because they don't come along with jdk software.

Type 3 mechanism

* Type 3 is not real jdbc driver it is given as protocol having set of rules to get communication between java application and jdbc connection pool.
* Jdbc connection pool is a factory that contains set of readyly available jdbc connection objects before actually being used.
* The advantage of jdbc connection pooling is we can use less no. of jdbc connection objects to make more java applications interacting with database softwares.
* Jdbc connection pools will be created and managed by web server, application server softwares like tomcat, weblogic and etc.



* All jdbc connection objects of jdbc connection pool represents connectivity with same and single database software.

Ex: Jdbc Connection pool for oracle means all jdbc connection objects in that pool represents connectivity with same database software (oracle).

With respect to diagram

(1) Webserver (or) application server uses type 1 (or) type 2 (or) type 4 driver to interact with database software and to create connection objects in connection pool.

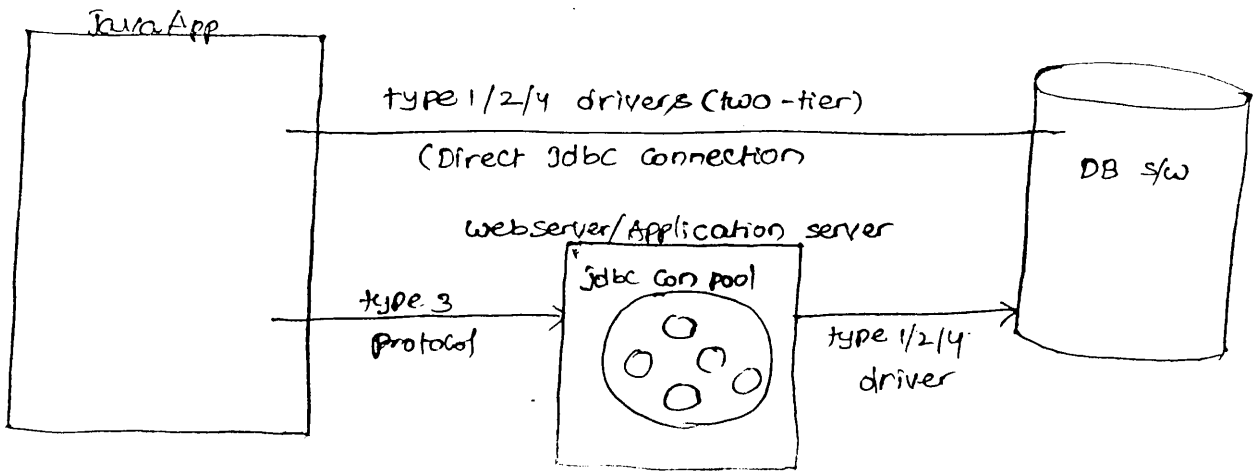
(2) Java application uses type 3 protocol to get one Connection object from connection pool.

(3) Java application uses that Connection object to create other jdbc objects and to perform CURD operations on database.

(4) Java application releases the connection object back to connection pool once task is completed. This Connection object now becomes free to give service to other clients.

* while working with Jdbc Connection pool the programmer is not responsible to create, manage and destroy jdbc connection objects from Java application. All these operations will be taken care by Jdbc connection pool.

summary diagram of jdbc drivers (all)



ronment (Created jdbc Connection)

* The jdbc connection that is created by the programmer directly and manually is called as direct jdbc connection object.

```
Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");
Connection Con = DriverManager.getConnection(-,-,-);
                 ↓
        Direct jdbc Connection object.
```

* The jdbc connection object that is collected from jdbc connection pool is called as pooled connection object.

<> For more information about jdbc drivers and architecture refer page nos. 10-17 of booklet.

* Type 3 is given to work with Jdbc Connection pool environment.

What is the jdbc driver that you have utilized in your project?

(A) If project is small scale application like two-tier application (or) office automation application then use jdbc driver type-4. If project is medium scale (or) large scale project like three-tier application like web application, bank application and etc.. then use type 3 with type-4 jdbc drivers. Here type-4 driver will be used to create connection objects in connection and type-3 protocol will be used to get connection objects from connection pool.

What is the difference between path and class path?

(A) Path: Inorder to make .cmd, .bat and .exe files of one directory executable from any location of computer then add the address location of these files to path environment variable.

* .bat file combines set of DOS commands or windows commands into single unit to execute them in sequence.

Problem:

e:\abc\run.bat

```
date
time
dir
```

e:\abc> run (success)

e:\abc> run.bat (success)

e:\> run (fails)

run is not an external or internal command

d:\demo> run (fails)

run is not an internal or external command.

Solution: work with path environment variables

Take the address location of run.bat file i.e, e:\abc folder and add that to path environment variable.

myComputer → properties → Advanced tab → env variables →

variable name : PATH

value : e:\abc; <other existing values>;.

→ok →ok →ok

e:\abc> run (success)

e:\> run (success)

d:\demo> run (success)

c:\> run (success)

* All our java tools like javac, java, javap are given as .exe files in java_Home\bin directory so to make them executing from any folder of your computer then add java_Home\bin directory to path environment variable.

* Don't set environment variable values (like path and classpath) from command prompt. Always use myComputer properties related environment variables.

* multiple values added to environment variable must be seperated with ";" symbol. we should not use this ";" symbol as beginner of the first value as well as terminator of last value.

* When new values are added to my Computer environment variable they will be reflected only in the new command prompts that are opened after adding new values.

Class path:

* When Java application uses new APIs and/or third party apis (other than Jdk APIs) then the new APIs, third parties APIs related directories are Jar files must be added to class path environment variable to make Java applications, Java tools recognising that new API, third party API.
* classpath is purely related to Java environment.
* API in Java language is nothing but set of classes and interfaces, which come in form of Packages.

Understanding Problem:

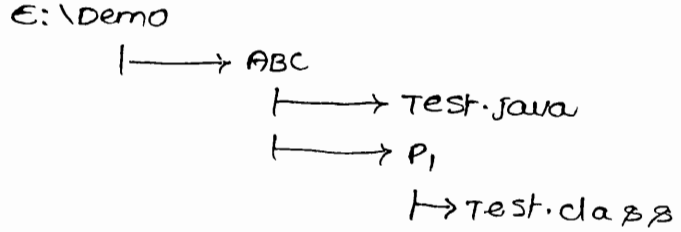
E:\Demo\ABC

Test.java

```
Package P1;
Public class Test
 {
    Public void m1()
      {
        System.out.println("Test:m1()");
      }
 }
```

e:\Demo\ABC > Javac -d . Test.java

gives Test.class in P1 Padcage.



D:\work

|—→ mainApp.java

MainApp.java

```
import p1.Test;
public class mainApp
 {
   public static void main (String args[])
     {
       Test t = new Test();
       t.m1();
     } // main
 } // class
```

D:\work> javac MainApp.java (× → failure)

error: can not find symbols p1, Test, m1()

Applying solution : work with class path

Add the directory location where "p1" package is available (e:\demo\ABC) to CLASSPATH environment variables variable.

my Computer → properties → advanced tab → env variables → new →

variable name: CLASSPATH

Value : e:\demo\ABC;.

→ ok → ok → ok

(or)

my Computer → properties → advanced tab → env variables → edit →

Variable name: CLASSPATH

Value ; e:\demo\ABC; <other existing values>;.

→ ok → ok → ok

D:\work> javac MainApp.java (success)

gives mainApp.class

D:\work> java MainApp

→ gives output.

* To make java tools running any where in your computer then work with path environment variable. To make these java tools to recognising and utilizing third party APIs and new APIs then work with CLASSPATH environment variable.

* Generally programmers and third party vendors keep their APIs in jar files and they will supply them to others. These other programmers uses these new APIs and third party APIs by adding their jar files to classpath.

E:\Demo\ABC> jar cf sathya.jar .

C → create jar file
f → use given name as jar file name.

* To add jar file to classpath

myComputer → properties → Advanced tab → env variables → edit →

variable name: CLASSPATH

value : e:\Demo\ABC\sathya.jar; <other existing values>;.

→ ok → ok → ok

* Oracle corporation supplies its type 4 mechanism, type 2 mechanism related jdbc drivers in the form of "ojdbc14.jar" file along with oracle software installation.

* The location of ojdbc14.jar file is

oracle_home\product\10.2.0\db_1\jdbc\lib (oracle 10g)

* Jar file can represent new APIs (or) third party APIs.

* Jar file can represent jdbc drivers.

* Jar file can represent a package webapplication (web site).

* Jar file can represent an EJB component.

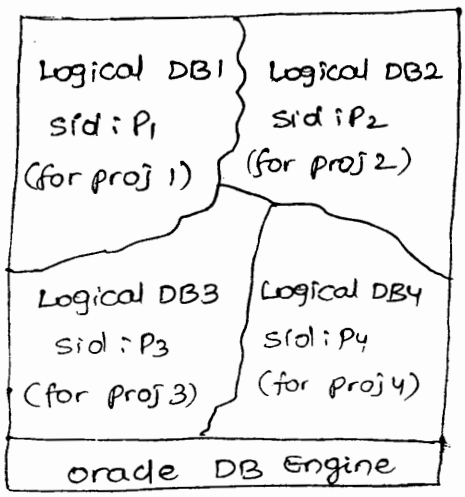
* Jar file can combine multiple resources into single file like zip file.

* In every physically installed database software we can create multiple logical partitions as logical databases.

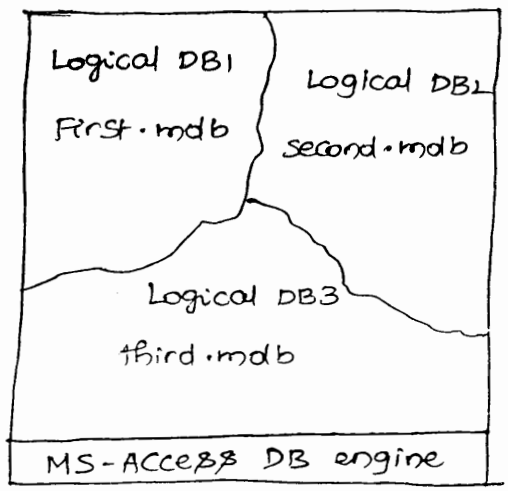
* Every logical database will be identified with it's database name.

* In oracle database software database name of logical database is called as sid.

Ex: If multiple projects of a company are utilizing same oracle database software then the oracle database software should be installed only once in common machine of a company and multiple logical databases will be created in that oracle database software for multiple projects on one per project basis.



(Physical oracle log DB s/w)



(physical MS-Access DB s/w)

* In MS-Access each .mdb file is one logical database and it will be identified with its file name.
* creating new logical databases in physical database software is the responsibility of DBA.
* In each logical database multiple DB tables, DB users and etc. can be there.
* when oracle software is installed it will give one default logical database, to know sid of this logical database:

approach (1):

```
SQL> select * from GLOBAL_NAME;

ORCL.REGRESS.-.-.-.-
```

↳ sid of current logical database (default).

approach (2): open oracle-home\product\10.2.0\db_1\network\admin\tnsnames.ora

file and look for service_name = orcl entry.
↳ sid of logical database.

approach (3):

Control panel → performance and maintance → adminstrative tools → services → Look for oracle service ORCL
↳ sid of logical database.

* If logical database of physical database software is allowing multiple users or client applications concurrently to manipulate the data then it is called as multiuser database software. If only one user (or) client application is allowed to perform the above said operation then it is called single user database software.

Examples of multiuser database softwares

oracle, sybase, mysql, Postgresql and etc...

Examples of singleuser database softwares

Ms-Access, foxpro, foxbase and etc...

Working with jar Command:

E:\temp
⟶ unit1
⟶ a.txt
⟶ b.txt

to create jar file

e:\temp\unit1> jar cf sathya.jar .

C → create jar file
f → with specified name as jar file name.

to see the table of content of jar file

e:\temp\unit1> jar tf sathya.jar

t → table of content in jar file.

* Every jar file contains one manifest.mf file in META-INF folder. This manifest file contain description about various resources of jar file.

to add new file to existing jar file

```
e:\temp
    |--> unit
        |--> c.txt (create new file)
```

e:\temp\unit> jar uf sathya.jar c.txt

Adds c.txt file to sathya.jar file.

u → updates the existing jar file

to extract files from sathya.jar file

to extract specific file

e:\temp\unit1> jar xf sathya.jar a.txt

to extract all files from jar file

e:\temp\unit1> jar xf sathya.jar

x → Extraction of jar file

* we can view and extract the content of jar file either by using win3ip tool (or) winrar tool.

* The oracle corporation supplied type 2 mechanism based Jdbc driver for oracle is called as "oracle oci" driver.

* The oracle corporation supplied type 4 mechanism based Jdbc driver for oracle is called as "oracle thin" driver.

* Every software installed in the computer resides in a logical position called software port and every software port will be identified with its port number.

* Total 65535 software ports can be there in a single computer. In that 1 to 1024 are reserved for operating system services.

oracle software ⟶ default port no: 1521

Tomcat software ⟶ default port no: 8080

weblogic software ⟶ default port no: 7001

InternetExplored ⟶ default port no: 1078

* socket no is: (IP address/host name of machine) + (Port no of software service)
* Port no. of oracle software is: 1521
* socket no. of oracle db software when installed on a machine having IP address 180.45.67.4

180.45.67.4 + 1521

socket no. of oracle db s/w.

* TO communicate with any software installed on any computer we must know its socket number.

Details of oracle Corporation supplied type2 mechanism based jdbc driver for oracle

Vendor : oracle corporation

Commertial jdbc driver

mechanism : type 2 mechanism

technical name: oracle oci driver
↓
oracle Connection Interface

jdbc driver class : oracle.jdbc.driver.OracleDriver
↓ package name
↓ class implementing java.sql.Driver Interface

Database url : jdbc:oracle:oci8:@<sid>
↓ Protocol
↓ Subname
↓ logical database name

Jar file representing this oracle oci driver

oracle 8i ⟶ classes111.jar or classes12.jar

oracle 9i ⟶ classes1111.jar (or) classes12.jar (or) ojdbc14.jar

oracle 10g ⟶ classes12.jar (or) ojdbc14.jar

oracle 11g ⟶ ojdbc6.jar

Procedure to develop java application by using oracle oci driver

step(1): Gather details about oci driver as shown above

step(2): Develop java application by using oracle oci driver as shown below

```
//selectTest.java
import java.sql.*;
import java.util.*;
public class Test
  {
    public static void main(String args[]) throws Exception
     {
       //load jdbc driver class
        Class.forName("oracle.jdbc.driver.OracleDriver");
       //establish connection with DB s/w
        Connection con = DriverManager.getConnection("jdbc:oracle:oci8:
                                                    @orcl", "scott", "tiger");
                                                      ↓
                                                     sid
       //create jdbc statement object
        Statement st = con.createStatement();
       //send and execute sql query in db s/w and gather the values
       ResultSet rs = st.executeQuery("select * from student");
       //process the result-set object
       while(rs.next())
         {
           System.out.println(rs.getInt(1) + " " + rs.getString(2) + " "
                                                 + rs.getString(3));
         }
       //close jdbc objects
        rs.close();
        st.close();
        con.close();
     }
  }
// javac selectTest.java
// java selectTest
```

Step(3): Add ojdbc14.jar file or other related equivalent jar files to class path environment variable.

my Computer → properties → advanced tab → env variables → edit

Variable name: CLASSPATH

Value: oracle\product\10.2.0\db_1\jdbc\lib\ojdbc14.jar; <other existing values>;.

→ok →ok →ok

Step(4): Compile and execute the application.

>javac selectTest.java

>java selectTest

* We can write jdbc code and we can create jdbc objects like statement object, resultset object and etc in a common way irrespective of the jdbc driver that we are using.

* As of now sun micro system is not supplying type2 and type4 mechanism based drivers so the programmer must gather them from database vendors (or) third party vendors.

Oracle corporation supplied type4 mechanism based jdbc driver for oracle

Technical name: oracle thin driver

mechanism: type4

Vendor: oracle corp

Commercial jdbc driver

Jdbc driver class: oracle.jdbc.driver.OracleDriver

(or)

oracle.jdbc.OracleDriver

Jdbc url/db url: jdbc:oracle:thin:@<hostname/IP address>:<portno>:<sid>

jdbc: Protocol; oracle:thin: subname; <hostname/IP address>: of computer where oracle db s/w is installed; <portno>: of oracle db s/w (1521); <sid>: logical db

jar files that represents this driver: same as oracle oci driver

Why the db url of type 4 jdbc drivers is quite lengthy when compare to the db url of type 1, type 2 drivers ?

(A) Type 1 driver is designed to talk with database software through odbc driver so its db url just need to locate odbc driver (jdbc:odbc:<dsn>)

once type 2 driver locates vendor db library of client side that db library takes care of communicating with database software so its db url just need to locate vendor db library (jdbc:oracle:oci8:@<sid>).

Type 4 drivers needs to locate and communicate with database softwares directly without taking support of other resources of client side, so its url contains multiple details. (jdbc:oracle:thin:@<IPaddress/hostname>:<portno>:<sid>).

Procedure to work with oracle thin driver in our java applications.

NOTE: To refer a computer being from current machine we use the word called local host.

Step(1): Gather the details of oracle thin driver as shown above.

Step(2): Develop java application by using oracle thin driver.

//selectTest.java

same as previous class application but the following db url to create jdbc Connection object.

```
//establish connection with DB s/w
Connection con = DriverManager.getConnection("jdbc:oracle:thin:@localhost:1521:orcl","scott","tiger");
```

Step(3): Add ojdbc14.jar file or other equivalent jar files to class path. same as previous class.

Step(4): Compile and execute the java application.

```
>javac selectTest.java
>java selectTest
```

* The values, jar files added to my Computer environment variable class path are not visible to the Projects created in IDE softwares. That means we need to add jar files to the libraries of IDE project seperately.

* If java application of the netbeans IDE project uses oracle thin driver then the oracle thin driver related ojdbc14.jar must be added to the library folder of the project (Right click on libraries folder → add jar → Browse and select ojdbc14.jar).

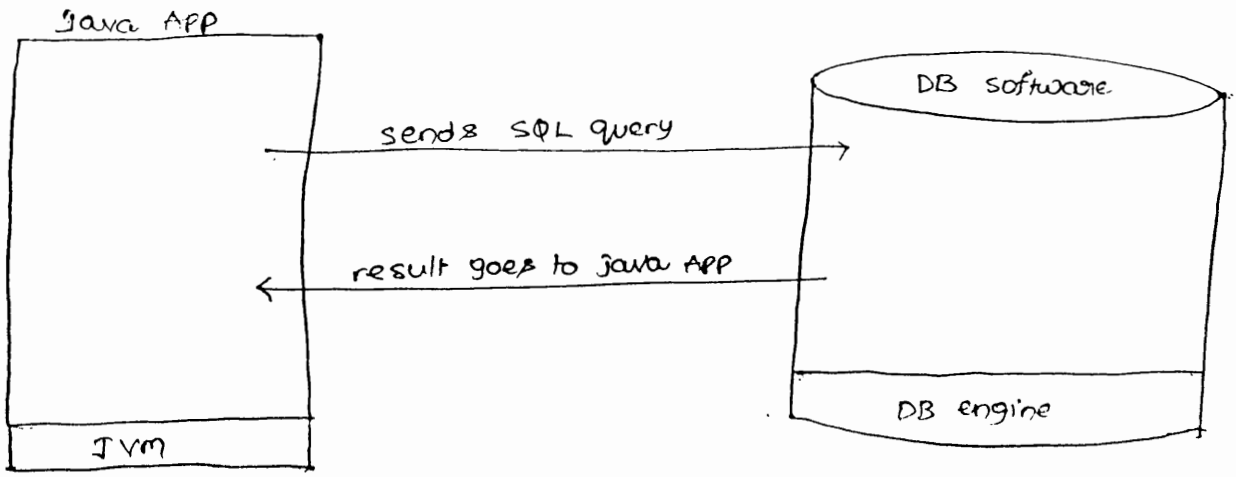
java.sql.Statement
↑ extends
java.sql.PreparedStatement
↑ extends
java.sql.CallableStatement

* There are 3 types of Jdbc statement objects which can be used in our java application to send and execute sql queries in database softwar

(1) Simple statement object (object of a java class that implements java.sql.Statement interface)

(2) PreparedStatement object (object of a java class that implements java.sql.PreparedStatement interface)

(3) CallableStatement object (object of a java class that implements java.sql.CallableStatement interface)



* The database engine of database software performs following operations on SQL query that comes from client application.

(1) parse (2) Execute (3) fetch.

* In parse operation the syntax of the query will be verified by splitting the query into multiple tokens.
* In execute operation the parsed query will be executed and its output will be stored in the buffer.
* In fetch operation results from the buffer will be gathered and results will be delivered to client application.

Limitations with simple statement object :

* Framing sql query with variable for simple statement object is complex process.
* If simple statement is used to execute same query for multiple time with same or different values then
  a) Same query goes to database software for multiple times.
  b) Same query will be parsed in database software for multiple times.
  c) same query will be executed in database software for multiple times with same (or) different values.
  d) The same query output will be fetched out for multiple times.
* In that above operations (a) and (b) are unnecessary operations done for multiple times on single query. But they can not be avoided while working with simple statement object. This makes database software to perform unneccessary operations which may degrade the performance of the application.
* To overcome above limitations use precompiled sql query with the support of prepared statement object.

NOTE: The limitation (2) talks about the simple statement can't execute the query multiple times in db s/w by just sending and parsing that query only for one time in database software.

* Simple Statement object sends raw sql query from java application to database software every time so the query goes through parsing, execution and fetch operations on every query each time.
* The jdbc Prepared statement object deals with pre Compiled sql query.
* The query that comes to database software from java application without values, only once, becomes parsed query in db s/w only once and allows client application (java application) to set the values for the query for multiple times, to execute the query for multiple times, with same or different values to gather the results for multiple times is called as precompiled sql query.
* Jdbc Prepared Statement object resides in java application and represent pre Compiled sql query of database software so the programmer can provide further instructions to precompiled query like assigning values, query execution and etc... by using this prepared statement object from java application.
* To execute sql query only for one time in database software use simple statement object. similarly to execute same sql query for multiple times in database software with different or same values use jdbc prepared statement object.

Procedure to work with jdbc Preparedstatement object

(1) Prepare sql query with parameters / place holders (?)

String query = "insert into student values (?, ?, ?)";
1 2 3

place holders place holders/parameters

(2) Send sql query to database software as pre-compiled query and represent that query with Preparedstatement object.

Preparedstatement PS = Con.Preparestatement(query);

This object represents precompiled query of db s/w being from java application.

This method sends given sql query to db s/w and makes that query as precompiled query in db s/w.

(3) Set values to parameters of precompiled query by using setXXX() methods.

```
PS.setInt(1, 567);
ps.setString(2, "Raja");
PS.setString(3, "hyd");
                 └→ parameter index
```

NOTE: The parameters of sql query will have one based index.

Here "hyd" is parameter value.

(4) Execute the query in database software.

```
int result = PS.executeUpdate();
```

(5) Process the results

```
if (res==0)
    System.out.println("record insertion is failed");
else
    system.out.println("record insertion is successful");
```

(6) to execute same sql query for multiple times with same or different values repeat step3 to step5 for multiple times.

(7) close jdbc Prepared Statement object.

```
PS.close();
```

write a java application to read multiple student details from key board and insert them to database table as records.

NOTE: Here we need to execute same sql query for multiple times for that works with precompiled sql query with support of PreparedStatement object.

```
// PstTest.java (with coding standards)
// Insert multiple student details to DB table
// version : 1.0
// author : team-s          import java.sql.*; import java.util.*;
PUblic class PstTest
 {
   Public static void main(String args[])
     {
```

```
Connection con = null;
PreparedStatement ps = null;
Scanner sc = null;
sc = new Scanner(System.in);
int n = 0;
if (sc != null)
 {
    //read student count from end user
    System.out.println("Enter no. of student");
    n = sc.nextInt();
 } // if
try{
//write jdbc code
    //create jdbc connection object
    Class.forName("oracle.jdbc.driver.OracleDriver");
    con = DriverManager.getConnection("jdbc:oracle:thin:@localhost
                                      :1521:", "scott", "tiger");
                                               ↓
                                              orcl
    //prepare sql query with parameters
    String qry = "insert into student values(?,?,?)";
    //create prepared statement object representing precompiled
                                                  sql query
    if (con != null)
      {
        ps = con.prepareStatement(qry);
      }
    //read each student details from keyboard and insert them
                                        to database table as record.
    if (sc != null && ps != null)
       {
          for (int i=1; i<=n; i++)
             {
               system.out.println("Enter "+i+" student details");
               System.out.println("Enter no: ");
               int no = sc.nextInt();
               System.out.println("Enter name");
               int new String name = sc.next();
```

```
            System.out.println("Enter address:");
            String address = sc.next();
            //set the above values to parameters of query
            ps.setInt(1, no);
            ps.setString(2, name);
            ps.setString(3, address);
            //execute the query
            int res = ps.executeUpdate();
            //process the result
            if(res==0)
              {
                System.out.println(i+" student details are not inserted");
              }
            else
              {
                System.out.println(i+" student details are inserted");
              }
        }//for
    }//if
}//try
catch(ClassNotFoundException cnf)
    {
        cnf.printStackTrace();
    }
catch(SQLException se)
    {
      se.printStackTrace();
    }
catch(Exception e)
    {
      e.printStackTrace();
    }
finally
 {
    try
     {
       if(ps!=null)
            ps.close
     }
     catch(SQLException se)
       {
         se.printStackTrace()
       }
```

```
                try
                {
                    if (con != null)
                        con.close();
                }
                catch (SQLException se)
                {
                    se.printStackTrace()
                }
            }// finally
        }// main
    }// class
```

```
//> javac PstTest.java
//> java PstTest
```

* Compare to simple statement object the jdbc Prepared Statement object reduces network traffic (amount of data) between java application and database software in each round trip.
* The setXXX(-) methods allow the programmer to set values to sql query or query parameters in java notation so working with string type input values for sql query becomes easy. while working with jdbc prepared Statement object.
* By using prepared Statement object we can execute both select and non select sql queries with zero or more parameters.
* Provide one based index sql query parameters from left to right order.

Example to execute select sql query by using jdbc PreparedStatement object

```
//create PreparedStatement object
PreparedStatement ps= con.PrepareStatement("select * from student
                                    where sno>=? and sno<=?");
                                          1        2
//set values to the parameters of query
 ps.setInt(1,100);
 ps.setInt(2,300);
```

```
//execute query
ResultSet rs = PS.executeQuery();
//process results
while(rs.next())
  {
    sop(rs.getInt(1)+" "+rs.getString(2)+" "+rs.getString(3));
  }
```

* The SQL query of jdbc PreparedStatement object can be there with or without parameters.

```
//create jdbc PreparedStatement object
PreparedStatement ps = con.PrepareStatement("select * from student");
//execute query
ResultSet rs = ps.executeQuery();
//process results
while(rs.next())
  {
    sop(rs.getInt(1)+" "+rs.getString(2)+" "+rs.getString(3));
  }
```

query without parameters.

* The parameters of PreparedStatement object related sql query can represent only condition values and input values but they can not represent table names, column name, of the SQL keywords of the query

select * from studen where sname like ? (valid query)

select * from ? where ? like ? (invalid query)

select * ? student where sname like ? (invalid query)

select * from student where sno >= ? and sno<=200 (valid query)

* one java application can have multiple jdbc connection objects, statement objects and ResultSet objects.

* Generally java jdbc applications are two tier applications containing three layers.

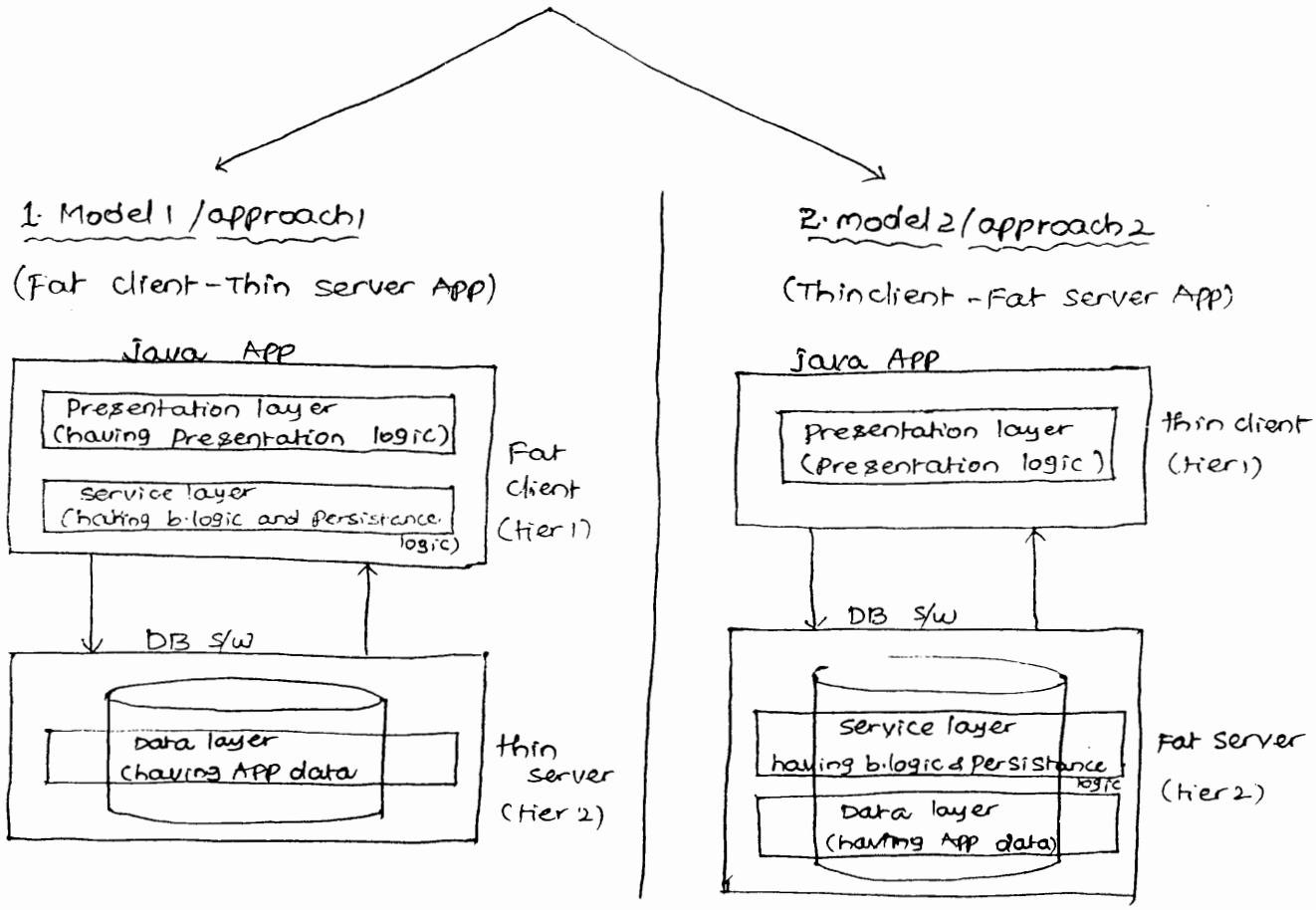
(1) Presentation layer having presentation logic

(2) service layer having business logic and persistance logic.

(3) Data layer having application data.

* Data layer is responsible to maintain, manage application data with the support of persistance stores like business logic, persistance logic and etc.

The two models of Developing Jdbc two-tier applications



* The jdbc applications which deals with simple statement object, Prepared statement object are called as "Fat client - thin server" applications.

* In thin client and Fat server applications the business and persistance logic will be placed in DB S/W as PL/SQL Procedures and functions and to call them from java application the jdbc CallableStatement object will be utilized.

* since PL/SQL Procedures / functions resides and execute permenantly in the database software they are also called as stored procedures and functions.

* PL/SQL procedure doesn't return a value whereas PL/SQL function returns a value.

* The PL/SQL function, procedure related parameters contains mode along with type.

modes are

In (default)

out

Inout

* If logic is

```
y := x * x;
```

then

x → In parameter

y → out parameter

```
x := x * x;
```

x → Inout parameter.

* We can gather results from PL/SQL procedure from out parameters or Inout parameters.

* We can gather results from PL/SQL function either as return value or out parameters / Inout parameters.

* PL/SQL programming, SQL queries are database software dependent. so Jdbc persistance logic is database software dependent persistance logic. To develop database software independent persistance logic use ORM software like Hibernate. (Object Relational mapping - ORM)

steps to develop PL/SQL Procedure in oracle

step(1): Launch sql*plus tool

step(2): Execute any dummy query

step(3): Lanunch ed editor

```
SQL> ed
```

step(4): Type and save PL/SQL Procedure.

```
create or replace procedure first_pro (x in number, y out number)
as
begin
y := x * x;
end;
/
```

(x → parameter name; in → parameter mode; number → parameter type)

step(5): Execute the PL/SQL procedure

File menu → save → exit → SQL> /

Procedure to work with jdbc callable statement object to call PL/SQL procedure of Database software

step(1): Prepare query calling PL/SQL procedure.

String query = "{ call first_pro (?, ?)}";

(1) (2)

↓

Represents parameters

step(2): create CallableStatement object having the above query.

CallableStatement cs = Con.prepareCall(query);

↓

Represents the PL/sql procedure on the db s/w (first_pro)

step(3): set input values to in parameters of PL/SQL procedure call. by using setXxx(-) methods.

cs.setInt(1, 20);

↓ ↓

parameter index, parameter value

step(4): Register out parameters with jdbc data types.

cs.registerOutParameter(2, Types.INTEGER);

↓ ↓

parameter index, jdbc type

* Every out parameter must be registered with jdbc type. All jdbc data types are bridge datatypes between underlaying database software datatypes and java data types. They help jdbc driver to convert database software notation result into java notation result while working with PL/SQL procedure or functions.

Java data type	jdbc data type	oracle data type
int	Types.INTEGER	number
long	Types.LONG	number
String	Types.VARCHAR	varchar2
byte[]	Types.BINARY	BLOB
java.sql.Date	Types.DATE	date
⋮	⋮	⋮

* All jdbc datatypes are constants of java.sql.Types class.
* Public static final member variables of a java class/ java interface are called as constants.

NOTE: IN parameters of PL/SQL procedures and functions need not to be register with jdbc types because the jdbc driver can detect their jdbc datatypes automatically based on the values that are set and based on the setXXY(-) methods that are used to set the values to IN parameters.

Step(5): Make database software executing PL/SQL procedure.

```
cs.execute();
```

Step(6): Gather results from OUT parameters of PL/SQL procedure.

```
int result = cs.getInt(2);
```

↓ Decide this method based on the jdbc type that is used to register out parameter

↳ out parameter index.

Step(7): To call PL/SQL procedure for multiple times with same or different values then repeat step(3) to step(6) for multiple times (except step(4)).

Step(8): close jdbc callable statement object.

```
cs.close();
```

* If multiple applications of a module (or) multiple modules of a project (or) multiple projects of a company are looking to use same business logic (or) persistance logic then instead of writing those

logics in every application of a module (or) in every module of a project (or) in every project of a Company it is recommanded to write only once as centralized logics in DB s/w in the form of PL/SQL procedure or function and use them in multiple applications of a module (or) in multiple modules of a project (or) in multiple projects of a company having reusability.

Example application based on above steps

```
// CSTest.java
import java.sql.*;
import java.java.util.*;
public class CSTest
  {
    public static void main(String args[]) throws Exception
      {
        // read input values from key board
         Scanner sc= new Scanner (System.in);
         System.out.println ("Enter a value");
         int no= sc.nextInt();
        // create jdbc Connection object
         Class.forName ("oracle.jdbc.driver.OracleDriver");
         Connection Con= DriverManager.getConnection ("jdbc:oracle:thin:
                                                      @localhost:1521:orcl",
                                                      "scott", "tiger");
        // Prepare sql query calling PL/SQL Procedure
         String query = "{call first_pro(?,?)}";
        // create callable statement object
         CallableStatement cs = Con.prepareCall(query);
        // register OUT parameters with jdbc types.
         cs.registerOutParameter(2, Types.INTEGER);
        // set IN parameters
         cs.setInt(1, no);
```

```
            //make db s/w to execute PL/SQL procedure
              cs.execute();
            //gather results from OUT parameters
              int result = cs.getInt(2);
            System.out.println("The Result is"+result);
            //close jdbc objects
              cs.close();
              con.close();
          }//main
        }//class
```

//make sure that first_pro(-,-) PL/SQL procedure is available in oracle DB s/w.

//>javac CSTest.java

//>java CSTest

* Any mechanism based Jdbc driver supplied by any vendor can work with all the 3 types of jdbc Statement objects.

PL/SQL Procedure in oracle dealing with SQL query

```
create or replace procedure emp_pro(eno in number, name out varchar,
                                    bsal out number)
as
begin
select ename into name from emp where empno=eno;
select sal into bsal from emp where empno=eno;
       ↓          ↓
    column      outparameter
    name.
end;
```

```
//CSTest2.java
import java.sql.*;
import java.util.*;
Public class CSTest2
    public static void main(String args[])throws Exception
    {
```

```
        //read input values from keyboard
        Scanner sc=new Scanner(System.in);
        System.out.println("Enter emp no:");
        int no=sc.nextInt();
        //create jdbc con obj
        Class.forName("oracle.jdbc.driver.OracleDriver");
        Connection con=DriverManager.getConnection("jdbc:oracle:thin:@localhost
                                                    :1521:ord","scott","tiger");

        //Prepare query calling the PL/SQL procedure
        String query="{call emp_pro(?,?,?)}";
        //Create Callable statement object
        CallableStatement cs=con.prepareCall(query);
        //Register out parameters with jdbc types
        cs.registerOutParameter(2,Types.VARCHAR);
        cs.registerOutparameter(3,Types.VARCHAR//%INTEGER);
        //Set values to IN parameters
        cs.setInt(1,no);
        //execute the PL/SQL Procedure
        cs.execute();
        //Gather results from OUT parameters
        System.out.println("emp name is:"+cs.getString(2));
        System.out.println("emp sal is:"+cs.getInt(3));
        //close jdbc objects
        cs.close();
        con.close();
    }//main
}//class
```

* In PL/SQL programming of oracle we can use cursor datatype variables to store bunch of records given by SQL select query. For this we generally use sys_ref cursor data type of oracle PL/SQL programming.
* cursor of PL/SQL programming is like jdbc ResultSet object of java programming.

Example application on cursor

```
create or replace procedure find_emps( fchars in varchar2, myrecords
                                        out sys_refcursor) as
                                            ↓
                                        a cursor type datatype
begin
  open myrecords for
     select empno, ename, sal, job from emp where ename like fchars;
end;
```

* Sunmicrosystem is not supplying jdbc type in Types class to register cursor type out parameters so use oracle corporation supplied OracleTypes.CURSOR constant as jdbc data type for the above said situation.
* To receive results from cursor type out parameters use getObject(-) method to receive jdbc resultset object.

```
//CursorCsTest.java
import java.sql.*;
import java.util.*;
import oracle.jdbc.*; // for oracle Types class (this pkg is available in ojdbc14.jar).
class CursorCsTest
 {
   Public static void main (String args[]) throws Exception
    {
      //read cond character from keyboard
       Scanner sc = new Scanner (System.in);
       sop("Enter chars (first letters of emp name)");
       String cond = sc.next();
       cond = cond + "%"; // r becomes r%
      // write jdbc code
       class.forName("oracle.jdbc.driver.OracleDriver");
       Connection con= DriverManager.getConnection("jdbc:oracle:thin:@localhost
                                                   :1521:ord", "scott", "tiger");
      // create callableStatement object
       CallableStatement cs = con.prepareCall("{call find.emps(?,?)}");
```

```
        //register OUT parameters with jdbc types (oracle Corporation supplied
                                                    jdbc types)

         cs.registerOutParameter (2, oracleTypes.CURSOR);

        //set values to IN parameter
         cs.setString(1, cond);
        //execute PL/SQL Procedure
         cs.execute();
        //gather results from out parameter
         ResultSet rs = (ResultSet) cs.getObject(2);
        //display result
         while(rs.next())
          {
            sop(rs.getInt(1) + " " + rs.getString(2)+" " + rs.getString(3)+" "
                                                    + rs.getString(4));
          }
        // close jdbc objects
          rs.close();
          cs.close();
          con.close();
       }//main()
    }//class
```

* The oracle PL/SQL programming provides a ~~braild~~ built-in cursor variable called SQL%ROWCOUNT. This variable always holds no. of records that are effected when non select SQL queries are executed in PL/SQL Procedure or function.

Example application:

PL/SQL Procedure in oracle (Based on non-select query)

```
create or replace Procedure fire_emps (salrange in number, cnt out
                                                    number) as
begin
     delete from emp where sal > salrange;
     cnt := SQL%ROWCOUNT;
end;
```

↳ returns a numeric value indicating no. of records that are effected because of above delete query execution

```
//CsTest3.java
public class CsTest3 //Ja
{
   public static void main(String argsL]) throws Exception
    {
     //read input values from keyboard
      scanner sc = new scanner (system.in)
      system.out.println("Enter salary range");
      int salrange = sc.nextInt();
      // create jdbc connection object
       class.forName("oracle.jdbc.driver.OracleDriver");
       Connection con = DriverManager.getConnection("jdbc:odbc:oci8:@ord",
                                                    "scott", "tiger");
      // Create callableStatement object
       CallableStatement cs = con.prepareCall("{call fire_emps(?,?)*}");
      // register OUT parameters with jdbc types
      cs.registerOutParameter(2, Types.INTEGER);
      // set IN values
      cs.setInt(1, salrange);
      // execute PL/SQL Procedure
      cs.execute();
      // gather results from OUT parameters
      system.out.println("cnt of emps who are fired:"+cs.getInt(2));
      //close jdbc objects
       cs.close();
       con.close();
    } // main()
} // class
```

Working with PL/SQL functions:

* PL/SQL function returns a value.
* Results from PL/SQL function can be gathered from either from out parameters or from return value. (we can even use both).
* while calling PL/SQL function the query must have a parameter representing return value and that parameter should be treated as out parameter

and should also be registered with jdbc type.

Example PL/SQL function:

```
create or replace function first_fx(x in number, y out number) return
                                                              number
as
z number;
begin
  y:=x*x;
  z:=x*x*x;
  return z;
end;
```

Procedure to call PL/SQL function from java application by using jdbc CallableStatement object

1) Prepare query calling PL/SQL function

```
String query = "{ ? = call first_fx(?,?) }";
                  1          2  3
```

parameter to hold return value of PL/SQL function

2) create CallableStatement object representing PL/SQL function.

```
CallableStatement cs = con.prepareCall(query);
```

3) register return parameter, out parameter with jdbc types

```
cs.registerOutParameter(1, Types.INTEGER); // return parameter
cs.registerOutParameter(3, Types.INTEGER);
```

4) set values IN parameters

```
cs.setInt(2,100);
```

2 → parameter index; 100 → value

5) Make database software executing PL/SQL function

```
cs.execute();
```

6) Gather results from return parameter, out parameters

```
int res1 = cs.getInt(1); // gives cube value(z);
int res2 = cs.getInt(3); // gives square value(y);
s.o.p(res1 + " " + res2);
```

7) To call and execute PL/SQL function for multiple times with same or different input values repeat step(4) to step(6) for multiple times.

8) close CallableStatement object

```
cs.close();
```

```
// CsFxTest.java
import java.sql.*;
import java.util.*;
Public class CsFxTest
  {
    public static void main (String args[]) throws Exception
     {
       //read input values from key board
        Scanner sc = new Scanner (System.in);
        System.out.println("Enter value");
        int no = sc.nextInt();
       //create jdbc Connection object
       class.forName ("oracle.jdbc.driver.OracleDriver");
       Connection con = DriverManager.getConnection("jdbc:oracle:thin:@localhost
                                                    :1521:orcl", "scott", "tiger");
       //prepare query calling PL/SQL function
        String query = "{ ? = call first_fx (?,?)}";
       //create CallableStatement object
       CallableStatement cs = con.prepareCall (query);
       //register return, OUT parameters with jdbc types.
        cs.registerOutParameter (1, Types.INTEGER); // return parameter
        cs.registerOutParameter (3, Types.INTEGER);
       //set values to IN parameters
        cs.setInt (2, no);
       //execute PL/SQL function
        cs.execute();
       //gather results from return, OUT parameters
        System.out.println("cube value:" + cs.getInt(1));
        System.out.println("square value:" + cs.getInt(3));
```

```
            //close jdbc objects
             cs.close();
             con.close();
          }//main
       }//class
```

PL/SQL function in oracle (dealing with sql queries)

```
create or replace function emp_fx(stno number, endno number, name out
                                  varchar2, bsal out number) return varchar2
as
  desg varchar2(20);
begin
   select ename into name from emp where empno>=stno and empno<=endno;
   select sal into bsal from emp where empno>=stno and empno<=endno;
   select job into desg from emp where empno>=stno and empno<=endno;
   return desg;
end;
/
```

```
select ename, sal, job into name, bsal, desg from emp where
empno >= stno and empno<=endno
```

```
// CsFxTest2.java
Public class CsFxTest2
 {
    Public static void main(String args[]) throws Exception
     {
        //read input values from cmd line args
         int stno = Integer.parseInt(args[0]);
         int endno = new Integer(args[1]).intValue();
         // create jdbc con obj
         Class.forName("oracle.jdbc.driver.oracleDriver");
         Connection con = DriverManager.getConnection("jdbc:oracle:thin:@localhost
                                                      :1521:orcl", "scott", "tiger");
         //create callableStatement object
         callableStatement cs = con.prepareCall("{ ?= call emp_fx(?,?,?,?)}");
         cs.registerOutParameter(1, Types.VARCHAR);
         cs.registerOutParameter(4, Types.VARCHAR);
         cs.registerOutparameter(5, Types.INTEGER);
```

```
            //set values in parameters
             cs.setInt(2, stno);
             cs.setInt(3, endno);
            //execute PL/SQL function
             cs.execute();
            //gather results from OUT parameters, return parameters
             System.out.println("emp name is:" + cs.getString(4));
             System.out.println("emp salary is:" + cs.getInt(5));
             System.out.println("emp Desg is:" + cs.getString(1));
            //close jdbc objects
             cs.close();
             con.close();
          }//main
       }//class
//> javac CsFxTest2.java
//> java CsFxTest2 7900 8000
```

Example application

PL/SQL function having both select and non select SQL queries

```
create or replace function test_fx(no in number, cnt out number) return
                                                        sys_refcursor
                                                            ↑
                                                  cursor as a return type
as
  mycur sys_refcursor;
begin
  delete from emp where empno=no; → non select query
  cnt := SQL%ROWCOUNT; → holds a numeric value representing the no. of records
                                                    that are deleted.
  open mycur for
      select empno, ename, sal, job from emp; → Here cursor holds bunch of
                                                  selected records
  return mycur;
end;
/
```

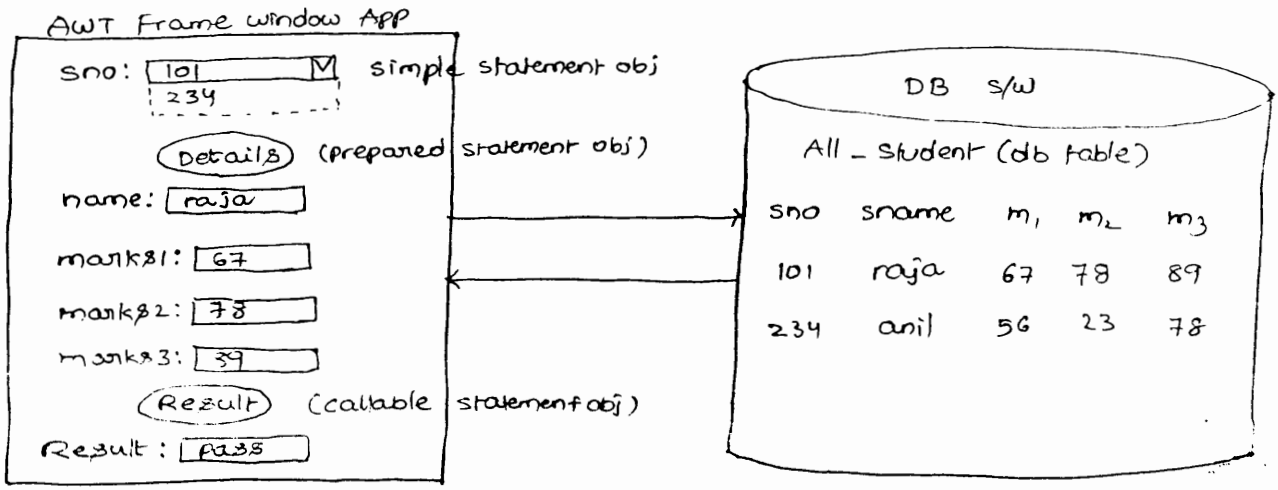
```
//CsTest3.java
import java.sql.*;
import java.util.*;
import oracle.jdbc.*;
public class CsTest3
  {
    public static void main(String args[]) throws Exception
    {
      // read input values from keyboard
      Scanner sc=new Scanner(System.in);
      System.out.println("Enter employee no.");
      int no=sc.nextInt();
      // create jdbc connection object
      Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");
      Connection con=DriverManager.getConnection("jdbc:odbc:oradsn",
      //use type4 driver instead of type1                    "scott","tiger");
      //create CallableStatement object
      CallableStatement cs=con.prepareCall("{?=call test_fx(?,?)}");
      //register out, return parameters with jdbc types
      cs.registerOutParameter(1,OracleTypes.CURSOR);
      cs.registerOutParameter(3, Types.INTEGER);
      //set values to IN parameter
      cs.setInt(2,no);
      //call PL/SQL function
      cs.execute();
      //gather results from OUT, return parameters
      ResultSet rs=(ResultSet)getcs.getObject(1); //from return parameter
      int count=cs.getInt(3); //from OUT parameter
      //Process the result
      if(count==0)
        System.out.println("Records not deleted");
      else
        system.out.println(count+" records deleted");
```

```
while (rs.next())
  {
    System.out.println(rs.getInt(1)+" "+rs.getString(2)+" "
                       +rs.getInt(3)+" "+rs.getString(4));
  }
//close jdbc objects
cs.close();
con.close();
}//main
}//class
```

* All jdbc drivers give support to work with all the three types of jdbc Statement objects, but type 1 driver can't use CallableStatement object to call cursor based PL/SQL functions and procedures.

Conclusion on three jdbc statement objects

* use simple statement object to execute SQL query only for one time in the entire application.
* use PreparedStatement object to execute same SQL query for multiple times with same or different values by making that query as pre-compiled query.
* To make persistance logic as centralized and reusable persistance logic take the support of PL/SQL functions and procedures of database software and use jdbc CallableStatement object to call them from java applications.
* Using AWT, swing concepts we can develop GUI front end based applications to interact with database software.



* By adding jdbc code in AWT, Swing applications we can make them interacting with database software.

* Example application to develop AWT based GUI front end application *

Resources required in the above AWT App

Container
- Frame window

Components
- Label ⟶ 6
- TextField ⟶ 5
- ChoiceBox ⟶ 1
- Button ⟶ 2

LayoutManager
- FlowLayout

Events
- ActionEvent

EventListener
- ActionListener

Event handling method
- actionPerformed(-)

For the source code of above App refer App 9 in booklet page no: 22-24

* In the above application all the three jdbc statement objects are used.

* student number to choice box should come only once in the entire execution of the application so its relevant query (select sno from student) should come be executed only once so simple statement object is recommanded.

* Based on the student number that is choosen from the choice box the remaining details of the student should come when details button is clicked. In this process we need to execute same sql select query (i.e. select * from student where sno = ?) for multiple times with different values so PreparedStatement object is used.

* To centralize re the student result generalization logic PL/SQL Procedure is taken in the db s/w so this PL/SQL procedure should be called when result button is clicked. For this CallableStatement object should be used.

* It is always recommanded to create jdbc objects in one time execution block like constructors and utilize them in repeatedly executing blocks like event handling methods for better performance.
* If java application wants to talk with database software for 10 times instead of creating 10 no. of jdbc connection objects create one jdbc connection and use it for ten no. of times.
* Method called from constructor executes as one time execution block.
* self class methods and super class public/protected methods in sub class can be called without object.
* The logic that you want to execute automatically during application startup is called as load on startup logic. we need to place these logics either in static block (or) constructor.
* In the above diagram based application values in the choice box should be populated as load on startup values.
* Jdk supplies buit-in tool called javaw to make the GUI java Apps as windows applications/desktop applications.
* Basic jdk setup is enough to work with this tool.

Example:

Right click on desktop → new → short cut → `javaw AllStmtsTest` →

next → name for short cut `collegeapp` → finish → Right click on collegeapp icon → properties → Short cut tab → start in `Application path`

Short cut key `Ctrl+Alt+Y`

→ change icon → apply → ok.

NOTE: The above javaw tool can not be used to make CUI applications as windows application.

How to execute CUI java applications as windows based desktop application?

- Keep .class file as batch file as shown below in any place of windows operating system.

myapp.bat (create on the desktop)

```
cd E:\NApps\Apps\advjava\jdbc
e:
javac selectTest.java
java selectTest
pause
```

what is the difference between AWT and swing?

AWT	swing
(1) Components look and feel is OS dependent	(1) OS independent
(2) Every Component maintains one peer Component at OS level, so these Components are heavy weight	(2) No peer Component at OS level so these Components are light weight
(3) gives minimum Components support (16+)	(3) gives more Components support (32+)
(4) Gives minimum LayoutManager support (4)	(4) Gives more LayoutManager support (6+)

Procedure to develop AllStmts application as swing application by using the NetBeans IDE 6.7

step(1): create java project in netbeans IDE.

File menu → New Project → java → java application → next →

ProjectName: myproject → finish

Step(2): Add Jframe to the project source packages

Right click on source packages folder → new → Jframe Form

class Name: AllStmtsTest → finish

Step(3): Design the frame window as shown in the previous diagram. by using drag and drop operations.

step(4): Register actionListener on button Components

Right click on Details button → Events → action → actionPerformed

Perform the same operation on Result button.

Step(5): Declare the following instance variables in the source code of AllStmt&Test.java:

```
Connection con=null;
Statement st=null;
PreparedStatement ps=null;
CallableStatement cs=null;
ResultSet rs1=null, rs2=null;
```

Step(6): write following user defined method in AllStmt&Test.java having jdbc code and call that method from constructor.

```
public void myinit()
{
  try
  {
    Class.forName("oracle.jdbc.driver.OracleDriver");
    con=DriverManager.getConnection("jdbc:oracle:thin:@localhost
                                    :1521:ord11","scott","tiger");
    //load-on-startup logic
    Statement st=con.createStatement();
    rs1=st.executeQuery("select sno from All-Student");
    while(rs1.next())
    {
      jComboBox1.addItem((String)rs1.getString("sno"));
    }
   //create other jdbc statement objects
    ps=con.prepareStatement("select * from All_Student where
                                    sno=?");
    cs=con.prepareCall("{call FIND_PASS_FAIL(?,?,?,?)}");
    cs.registerOutParameter(4,Types.VARCHAR);
  }
  catch(Exception e)
  {
    e.printStackTrace();
  }
}//myinit()
```

```
public AllStmt&Test()
{                                    → Constructor
    initComponents();
    myInit();
}
```

step(7): write following code jButton1ActionPerformed(-) method as details button related Eventhandling code.

```
private void jButton1ActionPerformed(java.awt.event.ActionEvent evt)
{
    //Code related details button
    try
    {
        int no = Integer.parseInt((String)jComboBox1.getSelectedItem());
                                    ↓ gives the selected item of ComboBox
        ps.setInt(1,no);
        //execute the query
        rs2 = ps.executeQuery();
        //Set the record of ResultSet object to text boxes
        if(rs2.next()){
        jTextField1.setText(rs2.getString(2));
        jTextField2.setText(rs2.getString(3));
        jTextField3.setText(rs2.getString(4));
        jTextField4.setText(rs2.getString(5));
        }
        rs2.close();
    }
    catch(Exception e)
    {
        e.printStackTrace();
    }
}
```

step(8): write the following code in jButton2actionPerformed(-) method for result button.

```
private void jButton2ActionPerformed(java.awt.event.ActionEvent evt)
{
    //for result button
    try
    {
        //Set IN parameters values
        cs.setInt(1, Integer.parseInt(jTextField2.getText());
```

```
cs.setInt(2, Integer.parseInt(jTextField3.getText());
cs.setInt(3, Integer.parseInt(jTextField4.getText());
//execute PL/SQL Procedure
 cs.execute();
//gather result from OUT parameter and set result to textbox
  jTextField5.setText(cs.getString(4));
}
catch(Exception e)
 {
   e.printStackTrace();
 }
```

Step(9): Add ojdbc14.jar file to the libraries of the project.

Right click on libraries folder → Add JAR → Browse and select ojdbc14.jar

Step(10): run the application

Right click on source code of AllStmtsTest.java → Run File

How to run a compiled java application from any folder of the project?

(A) Add the directory location of the application related .class file to classpath environment variable, to run that java application from any folder of the computer.

```
e:\apps\temp
        |--- TestApp.java
```

TestApp.java

```
public class TestApp
 {
    public static void main(String args[])
     {
       sop("TestApp: main(-)");
     }
 }
//>javac TestApp.java
```

Add e:\apps\temp folder to CLASSPATH environment variable

My Computer → properties → advanced tab → env variables →

variable name : CLASSPATH

value : e:\apps\temp; <other values>; .

→ ok → ok → ok

e:\> java TestAPP (success)

d:\> java TestAPP (success)

c:\xyz> java TestAPP (success)

* Large scale companies perprefer working with the commercial oracle db s/w whereas small scale companies prefer working with the open source db s/ws like mysql, postgre sql etc...

MySQL

type : multiuser db s/w

vendor: Devx or Sun microsystems (java soft)

open source or free db s/w

default portno : 3306

default username and password : root, root

(can be changed during installation/activation of s/w)

version : 4.x

Allows to create multiple logical databases

Default logical db names : mysql, test

To download software : www.devx.com

* After installing MySQL db s/w we must activate it seperately. For this go to mysql_home\bin and use winmysqladmin tool and look for traffic signal symbol in the system tray area.

Procedure to modify username and password of MySQL db s/w after installation

Right click on traffic signal symbol in system tray area → show me → my.ini setup tab → modify user and password → save modification

Procedure to create logical db, db table in mysql db s/w

Step(1): Launch the SQL prompt of MySQL database software

c:\mysql\bin ⟶ click on mysql.exe file

Step(2): create logical database

mysql> create database mydb1;

↓
Name of the logical database

Step(3): Connect to this logical database to mydb1

mysql>Connect mydb1;

Step(4): create db table in mydb1 logical db.

mysql> create table Employee (EID int(5) Primary key, FIRSTNAME varchar(20), LASTNAME varchar(20), EMAIL varchar(20));

Step(5): Insert records into this database table.

msql> insert into employee values(345, 'ramesh', 'rao', 'rao@gmail.com');

* If you are not aware of the sql queries of certain database software then you can use some third party GUI based db tools to perform various operations on the db s/w.

Ex: mysql front for mysql db s/w
sqlyog for mysql db s/w
TOAD for mysql db s/w
TOAD for oracle db s/w
Pl/sql developer for oracle db s/w

NOTE: All the above tools must be installed ~~seperately~~ seperately.

Procedure to perform above given operation (like logical db creation and etc) by using sqlyog tool.

(1) Launch sqlyog tool

Launch → Continue

MySQL Host address : localhost
User name : root
password : root1

port : 3306

→ connect

(2) create new logical database.

right click on root@localhost → create database → mydb2 → create

(3) create database table in mydb2 logical database.

right click on mydb2 → create table → Enter values for table →

create table → table name : Employee

(4) Add records to the table.

right click on employee table → insert/upgrade data → values

Procedure to develop java application to interact with mysql db s/w by using type-1 jdbc driver:

step(1): Install odbc driver for mysql seperately in your Computer(It will not come as default odbc driver).

step(2): create dsn for odbc driver for mysql.

Control panel → performance and maintainance → adminstrative tools → data sources → user dsn → add → MySQL ODBC 5.1 Driver → finish →

Data source Name: mysqldsn

server : localhost

port : 3306

user : root

password : root1

Database : mydb1

→ Test → ok

step(3): Develop java application as shown below by using type1 jdbc driver.

```
// SelectTest.java
import java.sql.*;
Public class SelectTest
    {
```

```
public static void main (String args[]) throws Exception
 {
   //load jdbc driver class
   Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");
   //establish connection with db s/w
   Connection con= DriverManager.getConnection (" jdbc:odbc:mysqldsn",
                                                "root", "root1");
   //create Prepared Statement object
   PreparedStatement ps = con.prepareStatement ("select * from
                                                 employee");
   //execute query
   ResultSet rs = ps.executeQuery();
   //process results
   while (rs.next())
     { sop(rs.getInt(1)+" "+rs.getString (2)+" "+rs.getString(3)+
                                 " "+rs.getString (4));
     }
   //close jdbc objects
     rs.close();
     ps.close();
     con.close();
 }
}
```

* The Devx Company supplied type-4 mechanism based jdbc driver for mySQL is technically called as Connector/J jdbc driver.

Connector/J jdbc driver

Target db s/w : mysql db s/w
version: 3.0.8
vendor : Devx
seperate installation is required
driver mechanism : type 4
open source jdbc driver
jdbc driver class : org.gjt.mm.mysql.Driver (or)
com.mysql.jdbc.Driver

Jdbc URL : jdbc:mysql:///<logical db name>

(or)

jdbc:mysql:/<hostname>:<portno>/<logical db name>

ZIP file that represents this jdbc driver : "mysql-Connector-java-3.0.8-stable.ZIP"
(download from deux.com).

Jar file that represents this jdbc driver : mysql-Connector-java-3.0.8-stable-bin.jar file.
(collect from the above ZIP file extraction).

Procedure to work with Connector/J jdbc driver in java application to interact with mysql database software

Step(1): Gather the above said jar file mysql-connector-java-3.0.8-stable-bin.jar file and add in the CLASSPATH.

Step(2): Gather Connector/J jdbc driver details as shown above.

Step(3): Develop java application by using Connector/J jdbc driver as shown below.

```
//SelectTest.java
import java.sql.*;
public class SelectTest
{
    public static void main(String args[]) throws Exception
    {
        //load jdbc driver class
        class.forName("org.gjt.mm.mysql.Driver");
        //or
        //class.forName("com.mysql.jdbc.Driver");
        //establish Connection with DB s/w
        Connection con=DriverManager.getConnection("jdbc:mysql:///mydb1",
                                                   "root","root1");
        //or
        Connection con=DriverManager.getConnection("jdbc:mysql://localhost:
                                                   3306/mydb1","root","root1");
    }
}
```

(protocol)

* To work with jdbc driver of any database software supplied by any vendor we must gather the following details.

(i) jdbc driver class name that implements java.sql.Driver interface

(ii) jdbc URL (or) db URL to locate database software.

(iii) Jar file that represents Jdbc driver.

* while developing small scale projects having the installation of MSoffice take the support of MSAccess as database software.

MS-Access

Type: single user db s/w

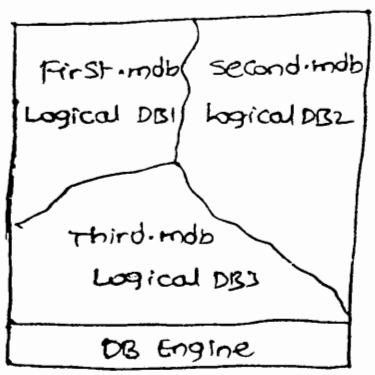
Version: 11.6 (MS-Office 2003)

Vendor: microsoft

Commercial software

Allows to create logical databases

Each .mdb file acts as one logical database.



(Physical MS-Access DB s/w

Procedure to create logical database and database table in MS-Access db s/w

Start → Programs → MS-office → MS-Access → create new file → blank database → Filename: mydb → Create → create table in design view

Field Name	Data type
pid	Number
Pname	text
Pqty	Number

→ Ctrl+w → yes → ok → NO → Launch Product table → Enter data

as records → click on save button.

* Industry generally prefers working with type-1 jdbc driver to interact with MS-Access database software because the type-2 and type-4 jdbc drivers of MS-Access db s/w are commercial jbc drivers.

Procedure to develop java application to interact with MS-Access db s/w

Step(1): create dsn for odbc driver for MS-Access.

Control panel → performance and maintainance → administrative tools → datasources → add → Microsoft Access Driver → Finish →

Data Source Name : accdsn

→ select → Browse and select the mydb.mdb file → ok

Step(2): Develop java application by using type-1 driver as shown below.

```
//AccessTest.java
import java.sql.*;
public class AccessTest
  {
    public static void main(String argS[]) throws Exception.
     {
        Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");
        Connection Con=DriverManager.getConnection("jdbc:odbc:accdsn");
                                                    dsn created above
        Statement st=Con.createStatement();
        ResultSet rs=st.executeQuery("select * from product");
        while(rs.next())
         {
           sop(rs.getInt(1)+" "+rs.getString(2)+" "+rs.getInt(3));
         }
        rs.close();
        st.close();
        Con.close();
     } //main
  } //class.
```

* The single user of MS-Access is nothing but the windows login user.

3 overloaded forms of DriverManager.getConnection() method

1) public static Connection getConnection(String url, String username, String password) throws Exception.

2) public static Connection getConnection(String url);

3) public static Connection getConnection(String url, Properties pro)

↓
Map datastructure of java.util package

* use (1) and (3) form to interact with multiuser db s/ws (like oracle...).
* use (2) form to interact with single db softwares (like MS-Access).

understanding java.util.Properties class

* It is a map data structure.
* It is subclass to java.util.Hashtable.
* It allows only String objects as keys and values of elements.
  (other map data structures allows any objects as keys and values in elements).
* It allows the programmer to gather the element values from outside the application as text properties file content.

abc.txt (text properties file)

```
name = ramesh
age = 30
address = hyd
# it is demo file
```

- The text file that maintain the entries in the form of key=value pairs is called as text properties file.

Example application to create java.util.Properties class object by utilizing the data of text properties file.

```
e:\apps
   |--> advjava
          |--> jdbc
                 |--> myfile.properties
                 |--> PropsTest.java
```

myfile.properties

```
# Demo file
name = raja
age = 30
address = hyd
```

P(Properties class object)

keys	values	
name	raja	(element-0)
age	30	(element-1)
address	hyd	(element-2)

PropsTest.java

```
//PropsTest.java
import java.io.*;
import java.util.*;
public class PropsTest
  {
    public static void main(String args[]) throws Exception
      {
        //locate properties file
        FileInputStream fis = new FileInputStream("myfile.properties");
        //create properties class object and load myfile.properties data
        //into that object
        Properties p=new Properties();
        p.load(fis);

        System.out.println("data of Properties class obj is:"+p.toString());
        System.out.println("data value of name key is:"+p.getProperty("name"));

      }//main
  }//class.
```

* In real time projects the dba changes the username and password of database software at regular intervals for security reasons. (like once in a month).
* To make jdbc code as flexible code it is recommanded to pass database username and password values to application from outside the application by taking the support of properties file.
* The standard principle in software industry is don't hard code any values in your java application that are changeble in the future. It is recommanded to collect them from outside the application by using properties file.

Example on (3) form of DriverManager.getConnection (-) method

```
e:\apps
    |--> advjava
            |--> jdbc
                    |--> DbDetails.properties
                    |--> SelectTest.java
```

DbDetails.properties

```
# DB details
user = scott
password = tiger
```

fixed keys as given in documentation

SelectTest.java

```
//SelectTest.java
import java.io.*;
import java.sql.*;
import java.util.*;
public class SelectTest
  {
    public static void main (String args[]) throws Exception
     {
        //locate properties file
        FileInputStream fis = new FileInputStream ("dbdetails.properties");
        //store dbdetails.properties file data into java.util.Properties class object
        Properties p = new Properties();
        p.load(fis);
        //register jdbc driver with DriverManager service
        Class.forName ("oracle.jdbc.driver.OracleDriver");
        Connection con= DriverManager.getConnection ("jdbc:oracle:thin:@local
                                                      host:1524:orcl", p);
        ------
        ------
        ------
     }
```

```
c:\javac SelectTest.java
c:\> java SelectTest
```

Postgresql

allows to create logical dbs the default logical db is postgres

Type: multi-user db s/w

Version: 8.3

Vendor: university of california & Mr. Scott.

open source db s/w

default portno: 5432

to download s/w: download as zip file from www.postgresql.org website (postgresql-8.3.13-1.zip)

* To install postgresql software extract postgresql-8.3.13-1.zip file to a folder and use the setup file that comes after extraction.

Procedure to create ~~logical database and~~ database table in postgresql in database software (in the default logical database postgres)

step(1): start postgresql database software.

start → programs → postgresql 8.3 → start service

step(2): open sql prompt of postgresql database software.

start → programs → postgresql → psql to postgres → password for user postgres: sathya

↳ choosen during installation

NOTE: Here we are connected to default logical database called postgres

step(3): create database table

```
create table product (pid int, pname varchar(20), price real);
```

step(4): Insert records into database table

```
insert into product values (101, 'table', 678.45);
insert into product values (102, 'pen', 348.45);
commit;
select * from prodcut;
```

pid	pname	price
101	table	678.45
102	pen	348.45

* The type4 mechanism based third party vendors like jdbc drivers for postgresql db s/w is technically called as postgresql thin driver.
* The type4 mechanism jdbc drivers are called as thin drivers whereas the type1 and type2 mechanism based drivers are called as thick drivers.

postgres thin driver (type4)

target db s/w : postgres db s/w
mechanism : type4
vendor : Open Community (University of California)
open source jdbc driver
jdbc driver class: org.postgresql.Driver
URL : jdbc:postgresql:<logical db name>
jar file that represents jdbc driver : postgresql-8.4-701.jdbc3.jar
(for jdbc3 spec based jdbc driver)
postgresql-8.4-701.jdbc4.jar
(for jdbc4 spec based jdbc driver)

Procedure to develop Java application interacting with postgresql db s/w by using the above thin driver

Step(1): Arrange the postgresql database software related thin driver based jar file in a folder.

E:\postgresql jarfiles
|→ postgresql-8.4-701.jdbc4.jar
(for jdk 1.6 environment)
|→ postgresql-8.4-701.jdbc3.jar
(for jdk 1.5 environment)

Step(2): Add one of the above given two jar files to classpath.

Step(3): Develop java application as shown below by using the postgresql thin driver.

PostgreSQLApp.java

```
import java.sql.*;
```

```
public class postgreSQLApp
 {
  public static void main(String args[]) throws Exception
   {
    //create jdbc con
    Class.forName("org.postgresql.Driver");
                   ↑package   ↳ jdbc driver class name
    //Connection Con= DriverManager.getConnection("jdbc:postgresql:
                                                    Protocol
                                                  postgres", "postgres", "satya");
                                                  logical db name  db user name  password
    // or
    Connection Con = DriverManager.getConnection("jdbc:postgresql://
                                                  localhost:5432/postgres",
                                                  "postgres", "satya");
    //create jdbc statement object
    Statement st = Con.createStatement();
    //send and execute SQL query
    ResultSet rs = st.executeQuery("select * from product");
    //process the result
    while(rs.next())
     {
       sop(rs.getInt(1)+" "+rs.getString(2)+" "+rs.getString(3));
     }
    rs.close();
    st.close();
    Con.close();
   }
 }
```

Step(4): compile and execute the java application.

* Latest specification of jdbc is : jdbc 4.0
* Each that is developed based on jdbc specification contains version

  type1 version : 2.x

  Oracle thin driver version : 9.2.x

  postgres thin driver version : 8.4.x

* J EE module is not a installable software, it is a big s/w specification having lots of sub specifications like servlet specification JSP specification, EJB specification and etc.

* Latest JEE specification is JEE5

In that

servlet 2.5 spec

JSP 2.1 spec

EJB 3.0 spec

JMS 1.2 spec are there as sub specifications.

Meta-Data

* Data about data is called as meta data that means gathering much information about given information is nothing but meta-data.
* If data is one website then search engine website looks like meta data.
* In jdbc meta data programming is useful to know limitations, capabilities and facilities of underlaying database software and its resources.
* Jdbc supports 3 meta data programming Concepts.

1) DatabaseMetaData
2) ResultSetMetaData
3) ParameterMetaData

Database MetaData :

* DataBase MetaData object means it is the object of a class that implements "java.sql.DataBaseMetaData" interface
* By invoking various methods on DataBaseMetaData object we can gather the following details.

```
DataBaseMetaData dbmd = Con.getMetaData();
```

Method declared in java.sql.Connection interface

→ Database name and version
→ support for PL/SQL procedures and functions
→ Max characters in table name and Column name etc...

ResultSetMetaData

* It gives various details about the database table that is represented by Jdbc ResultSet object like Column names, Column datatypes, Column Count and

etc...

```
ResultSet rs = st.executeQuery("select *from student");
ResultSetMetaData rsmd = rs.getMetaData(); // method declared in
                                           java.sql.ResultSet
```

* ResultSetMetaData object means it is the object of a class that implements "java.sql.ResultSetMetaData" interface

Parameter MetaData:

* Gives details about parameters available in sql queries of jdbc PreparedStatement object and callable statement object.

```
PreparedStatement PS = Con.prepareStatement("insert into student values
                                              (?,?,?)");
                                                ↓
                                          parameters of the query
ParameterMetaData pmd = PS.getParameterMetaData();
```

* ParameterMetaData object means it is the object of a class that implements "java.sql.parameterMetaData" interface.
* Using pmd object we can gather the details of parameter like parameter name, type and other details.
* MetaData programming is no way related with performing CURD operations on database tables. It is given to gather advanced details about database software and other resources.

For example application on DataBaseMetaData refer application 16 of the booklet (page no-29)

* If any method calls of MetaData programming returns 0 or null then we need to understand the underlaying jdbc driver is not capable of gathering that information.
* The metaData programming related output will change based on the database software and the jdbc driver we use.
* DatabaseMetaData is useful to develop certain database tables like SQL DB tools like sqlyog, mysqlfront, toad etc....

```
Class.forName("oracle.jdbc.driver.OracleDriver");
Connection con=DriverManager.getConnection(jdbc:oracle:thin:@localhost
                                        :1521:ord", "scott", "tiger");
Statement st = con.createStatement();
ResultSet rs = st.executeQuery("select * from student");
  while(rs.next())
     {
     }
```

* The above code gives db table (student records) but inorder to display them with column names and values take the support of "ResultSetMetaData" object.

Example code to print ResultSet object data along with column names and column datatypes

```
Class.forName("oracle.jdbc.driver.OracleDriver");
Connection con=DriverManager.getConnection("jdbc:oracle:thin:@localhost:
                                        1521:ord", "scott", "tiger");
Statement st=con.createStatement();
ResultSet rs = st.executeQuery("select * from student");
ResultSetMetaData rsmd = rs.getMetaData();
int colcnt = rsmd.getColumnCount();
//to print Column names
for(int i=1; i<colcnt; i++)
   {
     System.out.println(rsmd.getColumnLabel(i) + "\t\t\t");
   }
sop("\n");
// to print datatypes
for (int i=1; i<=colcnt; i++)
   {
     sop(rsmd.getColumnTypeName(i) + "\t\t\t");
   }
sop("\n");
```

```
            while (rs.next())
              {
               sop( rs.getInt(1) + "\t\t\t" + rs.getString(2) + "\t\t\t" +
                                                    rs.getString(3));
              }
             rs.close();
             st.close();
             con.close();
          }
       }
```

* ResultSetMetaData programming is quite useful towards reports generation on database table data.

How can you retrieve database table records by passing table name to application dynamically (without knowing columns)?

Use ResultSetMetaData object as shown below

```
Import java.sql.*;
Public class xyz
 {
   Public static void main (String args[]) throws Exception
    {
      class.forName("oracle.jdbc.driver.OracleDriver");
      Connection Con = DriverManager.getConnection ("jdbc:oracle:thin:@localhost:
                                                    1521:orcl", "scott", "tiger");
      Statement st = con.createStatement();
      ResultSet rs = st.executeQuery ("select * from" + args[0]);
      ResultSetMetaData rsmd = rs.getMetaData();
      int colcnt = rsmd.getColumnCount();
      //to print column names
      for (int i=1; i<=colcnt; i++)
        {
         sop( rsmd.getColumnName(i) + "\t\t");
        }
      sop();
```

```
//to print col datatypes
 for (int i=1 ; i<= colcnt ; i++)
   {
    sop(rsmd.getColumnTypeName(i) + "\t\t");
   }
 sop();
//to display records
 while(rs.next())
  {
    for (int i=1 ; i<= colcnt ; i++)
      {
        sop(rs.getString(i) + "\t\t");
      }
     sop();
  }
  rs.close();
  st.close();
  con.close();
 }
}
```

o/p

Eid	Ename	Add
Number	Varchar	Varchar
101	Raja	hyd
235	Ramesh	hyd

getColumnLable

getColumnTypeName

getString

NOTE: while executing the above code pass database table name as Command Line argument.

What is the difference between ResultSet object and ResultSet MetaData object ?

* ResultSet object points to table data whereas ResultSetMetaData object points to database table structure like column names, column data types and etc...
* Most of the jdbc drivers are not giving support for parameterMetaData programming.
* The maximum vendors of jdbc drivers are not providing implementation for "java.sql.parameterMetaData" interface.

DB s/w

1. Conventional DB s/w

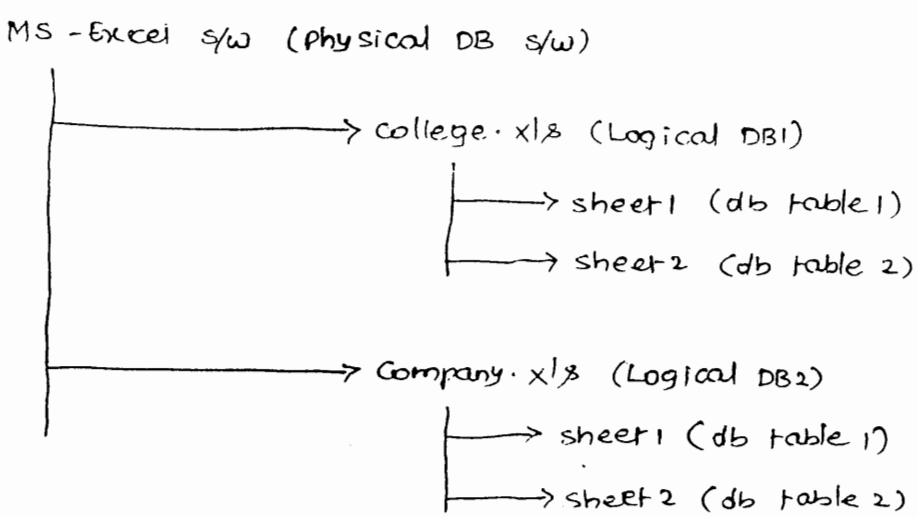
oracle
sybase
MS-Access
Mysql
postgresql

2. Non-conventional DB s/w

MS-Excel
CSV-files
tab-files

* Non-conventional DB s/w will be used as helping/supporting DB s/w when conventional DB s/ws are used as main DB s/ws.
* In ICICI banking project oracle is taken as Main DB s/w and MS-Excel is taken as helper DB s/w to get the printouts of accounts statements after every 3 months for this they move last 3 months data from oracle to MS-Excel.
* MS-Excel gives good performance towards print operations when compare to oracle.
* Since odbc drivers are already available to communicate with all non-conventional DB s/ws. So use jdbc driver type-1 for java application to non-conventional DB s/w communication.

MS-Excel :



sheet1 of college.xls

sno	sname	add
101	Ravi	hyd
345	Anil	hyd
123	Rajesh	Vizag

← (db table column names)

db table col values / db tables records

* Here College.xls and Company.xls are workbooks

Procedure to interact with MS-Excel from java application

step(1): Create work book in MS-Excel s/w (College.xls as shown above)

E:\Apps\temp\College.xls file

step(2): create DSN for the Microsoft Excel driver

Control panel → performance and maintanence → Adminstrative tools →

Data Sources → User DSN tab → Add → Microsoft Excel (.xls) →

Finish →

Data Source Name : xlsdsn (anyname)

→ select work book → browse and select → E:\Apps\temp\College.xls → ok

step(3): Develop java application by using type1 jdbc driver as shown below

```
// ExcelTest.java
import java.sql.*;
Public class ExcelTest
 {
   Public static Void main(String args[]) throws Exception
    {
       Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");
                     driver.OracleDriver
       Connection con = DriverManager.getConnection("jdbc:odbc:xlsdsn");
                                    (Excel is single user db so no
                                     need to username and password)
       Statement st = con.createStatement();
       ResultSet rs = st.executeQuery("select * from [sheet1$]");
                                    The MS-Excel sheet name as db table name
```

```
while(rs.next())
  {
   sop(rs.getInt(1)+" "+rs.getString(2)+" "+rs.getString(3));
  }
rs.close();
st.close();
con.close();
}//main
}//class
```

CSV - files (cama seperator values) :

E:\apps
- ⟶ temp (Logical DB)
  - ⟶ abc.csv (db table1)
  - ⟶ Myfile.tab (db table2)

abc.csv

sno, sname, add ⟶ db table Column names

101	Raja	hyd	} db table records
345	Rajesh	hyd	

Myfile.tab

sno	sname	add	⟶ Column names
101	Raja	hyd	} db table records
345	Ramesh	hyd	

* The total file system of OS (windows) acts as physical DB s/w, in that each directory acts as one logical DB, in that directory each file that contains formatted data acts as one DB table.

Procedure to Communicate with text file (csv file) from java application

step(1): Prepare the csv file ready as shown above

E:\NAPPS\Apps\temp\abc.csv

step (2): create DSN for Microsoft text Driver

Control panel → performance and maintainence → Administrative tools → Data Sources → User DSN tab → Add → Microsoft Text driver → Finish

Data source Name : txtdsn (any name)

→ select directory → E:\NAPPS\Apps\temp\ → ok

step(3): Develop java application by using type-1 driver as shown below.

```
// TextTest.java
import java.sql.*;
class TextTest
 {
   public static void main(String args[]) throws Exception
    {
     Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");
     Connection Con=DriverManager.getConnection("jdbc:odbc:txtdsn");
     Statement st = Con.createStatement();
     ResultSet rs =st.executeQuery("select * from abc.csv");
                                                  ↓
                                          filename as db table name
     while(rs.next())
      {
       sop(rs.getInt(1)+" "+rs.getString(2)+" "+rs.getString(3));
      }
     rs.close();
     st.close();
     Con.close();
    } //main
 } //class
```

* In one Jdbc application there can be multiple Connection objects, multiple Statement objects and multiple ResultSet objects.

* one JDBC application can interact with multiple DB softwares at a time.

Write a program to transfer the records of MS Excel to oracle DB S/w

Refer Application 13 of Page no 27, 28

Batch Process:

* The process of combining related queries into single unit, sending that unit to DB S/w, executing queries of that unit in DB S/w and gathering results of those queries as a single unit is called as "batch processing".

* Batch processing reduces network round trips between java application and database software especially when java application wants to send and execute multiple SQL queries in database software.
* Instead of sending multiple queries to database software individually one by one it is recommanded to take the support of batch process.
* The query is placed in batch will not be executed by enabling do everything or nothing principle (that means if one query execution fails the remaining queries executions should also be failed/rollback).

Procedure to perform JDBC batch processing

(1) create jdbc statement object

```
Statement st = con.createStatement();
```

(2) Add non select queries to batch.

select queries can't be added to the batch of batch processing

```
st.addBatch("insert into student values(111,'xxx','yyy')");
st.addBatch("update student set sadd = "hyd" where sno = 101");
st.addBatch("update student set add = "vizag" where sname = "Raju"");
st.addBatch("delete from student where sno = 111");
```

→ keep all these SQL queries in a single batch.

NOTE: Any no. of non-select queries pointing to same database table or different database tables can be added to the above batch for batch processing.

(3) Send and execute the above batch of queries in database software.

```
int res[] = st.executeBatch();
```

res[]

1	0
1	1
0	2
3	3

This int res[] contains the batch related queries execution results.

* select query execution gives Jdbc ResultSet object, executeBatch() returns int[] and ResultSet object can't be stored in this int[] so we can't add select SQL query to the batch of batch processing.

* For example application on batch processing refer page no 31 and application 21.

* The process of Combining related operations into a single unit and executing them by applying do everything or nothing principle is comes under transaction management.

Ex: Transfer money task is the combination of 2 operations.

- withdraw amount from source account
- Deposite amount into destination account.

* we need to execute these two operations by applying do everything or nothing principle performing transaction management.

* To perform transaction management in jdbc applications we use batch processing along with some additional logics.

Transaction Management in JDBC:

* The process of Combining set of related operations into single unit and executing those operations by applying "do everything or nothing" principle is called as transaction management.

Example application (performing transfer money operation)

SQL> select * from Account.tab;

Ac no	Holdername	Balance
101	Raja	10,000
102	Ravi	30,000

keep DB table ready as shown in the table.

develope the application to transfer the funds between two accounts.

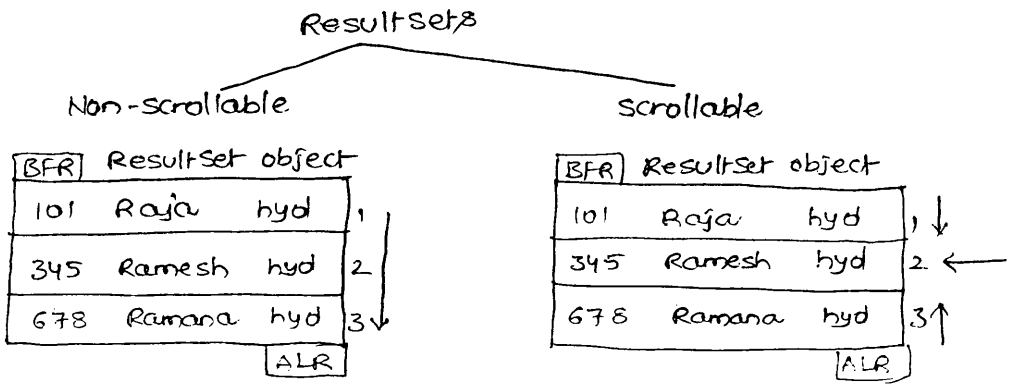
```
// TxMgmt.java
import java.sql.*;
import java.util.*;
public class TxMgmt
 {
    public static void main(String args[]) throws Exception
      {
```

```
//read input values from key board
Scanner sc = new Scanner (System.in);
sop( "Enter source account no:");
Int srcno = sc.nextInt();
sop("Enter destination account no:");
Int destno = sc.nextInt();
sop("Enter the amount to transfer");
Int amt = sc.nextInt();
//create jdbc Connection object
Class.forName("oracle.jdbc.driver.OracleDriver");
Connection con = DriverManager.getConnection("jdbc:oracle:thin:@localhost:
                                                1521:orcl", "scott", "tiger");
//Disable autoCommit mode(begining of transaction (Tx))
 con.setAutoCommit(false);
//perform transfer money operation(through batch processing)
 Statement st = con.createStatement();
//for withdraw operation on source account
st.addBatch("update Account_tab set balance=balance-"+amt+"where
                                                acno="+srcno);
//for deposite operation on destination account
st.addBatch("update Account_tab set balance=balance+"+amt+"where
                                                acno="+desno);
//execute the batch
Int res[] = st.executeBatch();
//write logic to Commit or rollback the transaction
 boolean flag=false;
 for(Int i=0; i<res.length ; i++)
  {
     if(res[i]==0)
        {
          flag = true;
          break;
        }
  }
  if(flag== true)
     {
       con.rollback() ;
       Sop(" Transaction rolledback");
     }
  else
     {
       con.Commit();
```

```
            sop("Transaction committed");
        }
        st.close();
        con.close();
    } //main
} //class
```

* Batch processing is not possible with preparedStatement object and callable statement object.

Working with different types of Resultsets :



* The Resultset object that allows to access the records sequentially, uni-directionally (top to bottom) is called as non-scrollable object.
* By default every jdbc resultset object is non-scrollable Resultset object.
* The Resultset object that allows the programmer to access records randomly or non-sequentially or directly or bidirectionally is called as Scrollable Resultset object.
* Using scrollable Resultset object we can access the records of Result-set object quickly.

To create non-scrollable Resultset object

```
Statement st = con.createStatement();
ResultSet rs = st.executeQuery("select * from student");
```

rs → Non-scrollable Resultset object

To create scrollable Resultset object

```
Statement st = con.createStatement(type, mode);
ResultSet rs = st.executeQuery("select * from student");
```

rs → Scrollable Resultset object.

Possible values for Type (int):

ResultSet.TYPE_SCROLL_SENSITIVE ; (1005)

ResultSet.TYPE_SCROLL_INSENSITIVE ; (1004)

Possible values for Mode (int):

ResultSet.CONCUR_UPDATABLE ; (1008)

ResultSet.CONCUR_READ_ONLY ; (1007)

NOTE: The possible values of type, mode parameters are int Constants internally in java.sql.ResultSet interface.

* Public static final member variables of java classes/ java interface are called as Constants.

* The jdbc statement object that is created without type and mode values can create only non-scrollable ResultSet object. The jdbc statement object that is created having type and mode values can create scrollable ResultSet object.

The methods invokable on non-scrollable ResultSet object

```
next();
getXXX();
getRow();
close();
```

The methods invokable on scrollable ResultSet object

```
next();
getXXX();
```

getRow();

close();

previous();

beforeFirst(); → keeps cursor in BFR position

afterLast(); → keeps cursor in ALR position

↓

Record pointer.

first(); → keeps cursor in first record

last(); → keeps cursor in last record

isFirst(); → checks whether cursor is there in first record or not.

isLast(); → checks whether cursor is there in last record or not.

absolute(-/+n);

relative(-/+n);

what is the difference between absolute(-) and relative(-) methods?

* Absolute(-) method moves the record pointer in resultset object in forward and reverse direction. positive no. indicates forward direction with respect to first record. Negative no. indicates with respect to last record.

Ex: absolute(3) → moves the cursor to (from first) 3rd record

absolute(-2) → moves the cursor to 2nd record from last.

* Relative(-) method moves the cursor (record pointer) with respect to current position of cursor. positive no. indicates forward direction, negative no. indicates reverse direction.

What is the difference between absolute path and relative path?

```
e:\apps
    |--> Demo
        |--> abc.txt
```

The absolute path of abc.txt file is

```
e:\apps\Demo\abc.txt
```

The relative path of abc.txt file from e:\apps folder

```
.\Demo\abc.txt
```

The relative path of abc.txt file from e:\apps\xyz folder

```
..\Demo\abc.txt
```

The relative path of abc.txt file from e:\apps\demo\abc folder

```
..\abc.txt
```

"." → indicates current directory

".." → indicates parent directory

* In relative path the file/directory location will be given with respect to the current working folder location.

* In absolute path the directory or file location will be given completely with respect to right from root directory.

---

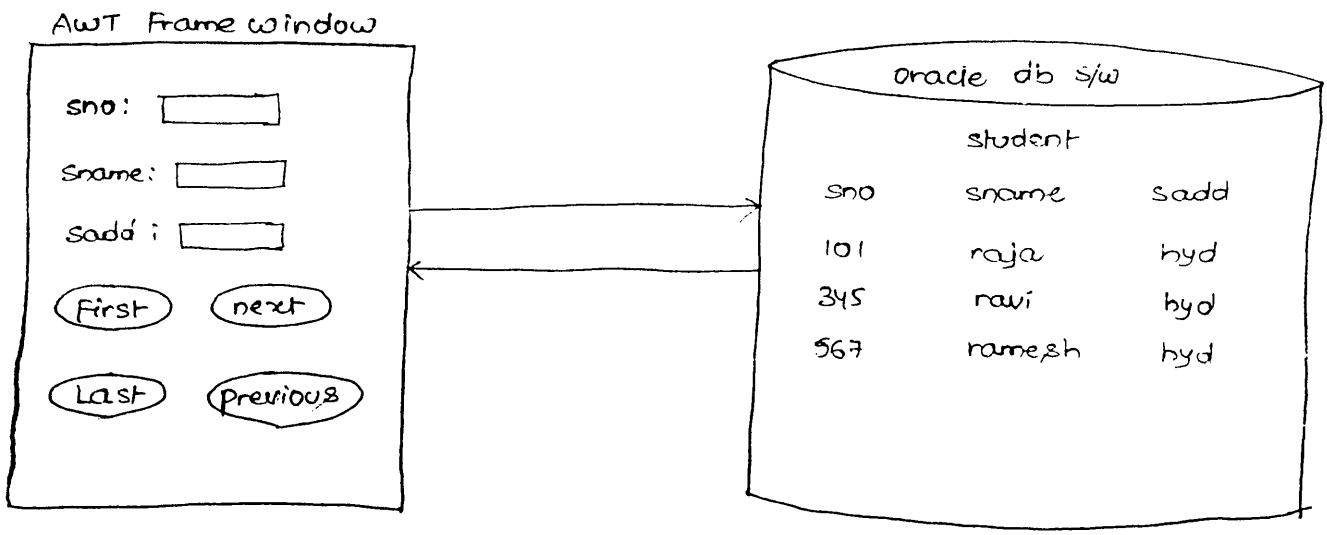
* To display the records of jdbc resultset object bidirectionally (top to bottom and bottom to top) we need to work with scrollable resultset object as shown below.

```
// ScrollTest.java
import java.sql.*;
public class ScrollTest
 {
    public static void main(String args[]) throws Exception
     {
        // create jdbc Connection object
        Class.forName("oracle.jdbc.driver.OracleDriver");
        Connection con = DriverManager.getConnection("jdbc:oracle:thin:@localhost
                                                    :1521:orcl","scott","tiger");
```

```
// create jdbc statement object with type and mode values
Statement st = Con.createStatement (ResultSet.TYPE_SCROLL_SENSITIVE,
                                    ResultSet.CONCUR_UPDATABLE);
                                    ↳ type (1005)
                                    ↳ mode (1008)
// get scrollable ResultSet object
ResultSet rs = st.executeQuery ("select * from student");
// print records in bottom to top order
rs.afterLast(); // keeps cursor in ALR position
while (rs.previous())
 {
  System.out.println(rs.getInt(1)+" "+rs.getString(2)+" "+rs.getString(3));
 }
// print records in top-bottom order
rs.beforeFirst(); // keeps cursor in BFR position
System.out.println("records... (top to bottom)");
while (rs.next())
 {
  System.out.println(rs.getInt(1)+" "+rs.getString(2)+" "+rs.getString(3));
 }
// 3rd record is
rs.absolute(3);
 System.out.println(rs.getRow()+" record is:"+ rs.getInt(1)+" "
                    +rs.getString(2)+" "+rs.getString(3));
// last but 2 record
rs.absolute(-2);
 System.out.println(rs.getRow()+" record is:"+rs.getInt(1)+" "
                    +rs.getString(2)+" "+rs.getString(3));
// 4th record from the bottom
rs.relative(-2);
 System.out.println(rs.getRow()+" record is:"+rs.getInt(1)+" "
                    +rs.getString(2)+" "+rs.getString(3));
// last but one record
rs.relative(3);
System.out.println(rs.getRow()+" record is:"+rs.getInt(1)+" "+
```

```
                rs.getString(2) + " " + rs.getString(3));
            //The first record is
             rs.first();
             System.out.println(rs.getRow()+"record is:"+rs.getInt(1)+" "
                                    +rs.getString(2)+""+rs.getString(3));
            //the last record is
             rs.last();
             System.out.println(rs.getRow()+"record is:"+rs.getInt(1)+" "
                                    +rs.getString(2)+" "+rs.getString(3));
            //close jdbc objects
             rs.close();
             st.close();
             con.close();
           }//main
        }//class
```

* Non scrollable Resultset objects are forward only resultset objects by default. That means they allow the programmer to move the cursor in resultset object only in forward direction. (top to bottom)
* All jdbc drivers support scrollable resultsets.



Container → Frame window

Components → label - 3, Button - 4, TextField - 3

LayoutManager → FlowLayout

Event → ActionEvent

EventListener → ActionListener

Event handling method → actionPerformed (-)

* For the above diagram based application refer application 10 of page nos 24-26.

Procedure to develop scrollFrame application (given in previous class) as an awt application by using netbeans IDE.

Step(1): create java project in netbeans IDE

File menu → new → java → java application → next → myproject1 → Finish.

Step(2): Add awt frame to the project

Right click on project → new → other → AWT GUI form → Frame form → next →

Classname: scrollFrame

→ Finish

Step(3): Design frame window as shown in the diagram with awt components.

NOTE: Before performing above step operation change the Layout of frame window from the default BorderLayout to NullLayout

[inspector window → right click on BorderLayout → Free design / NullLayout]

Step(4): Register actionListener on these button components.

Right click on button (any button) → Events → Action → actionPerformed (do this on all 4 buttons)

Step(5): Declare the following instance variables in the source code of ScrollFrame class.

```
Connection Con=null;
ResultSet rs=null;
Statement St=null;
```

Step(6): Develop user defined method in ScrollFrame class having JDBC code to create scrollable ResultSet object.

```
public void makeConnection()
 {
   try
    {
      Class.forName("oracle.jdbc.driver.oracleDriver");
      Connectio Con=DriverManager.getConnection("jdbc:oracle:thin:
                                        @localhost:1521:orcl","scott","tiger");
      St = Con.createStatement (ResultSet.TYPE_SCROLL_INSENSITIVE,
                                        ResultSet.CONCUR_UPDATABLE);
      rs = St.executeQuery ("Select * from student");
    }
   Catch(Exception e)
    {
      e.printStackTrace();
    }
 }// makeConnection
```

Step(7): call the above makeConnection method from the Constructor.

```
Public scrollFrame();
     initComponents();
     makeConnection();
```

Step(8): Add following Logic in button1ActionPerformed()method (for first button)

```
private void button1ActionPerformed(-)
  {
    try
     {
       rs.first();
       textField1.setText(rs.getString(1));
       textField2.setText(rs.getString(2));
       textField3.setText (rs.getString(3));
     }
```

```
        catch (Exception e)
          {
            e.printStackTrace();
          }
```

Step(9): write following code in button2ActionPerformed(-) method (for (next) button)

```
Private void button2ActionPerformed (-)
  {
    try
      {
        if (!rs.isLast())
          {
            rs.next();
            textField1.setText (rs.getString(1));
            text Field2.setText (rs.getString(2));
            text Field3.setText (rs.getString(3));
          }// if
      }// try
    catch(Exception e)
      {
        e.printStackTrace();
      }
  }
```

Step(10): write following code in button3ActionPerformed(-) method (for (previous) button)

```
Private void button3ActionPerformed (-)
  {
    try
      {
        if (!rs.isFast())
          {
            rs.previous();
            textField1.setText (rs.getString(1));
            text Field2.setText (rs.getString(2));
            text Field3.setText (rs.getString(3));
          }// if
      }// try
    catch(Exception e)
      {
        e.printstackTrace();
      }
  }
```

step(11): write the following code in buttonyActionPerformed(-) method (for (Last) button)

```
private void buttonyActionPerformed(-)
  {
    try
     {
        rs.last();
        textField1.setText(rs.getString(1));
        textField2.setText(rs.getString(2));
        textField3.setText(rs.getString(3));
     }//try
     catch(Exception e)
      {
       e.printStackTrace();
      }
  }
```

step(12): Add ojdbc14.jar file to the libraries of the project.

Right click on libraries folder → Add jar → browse and select ojdbc14.jar file

step(13): Run the application.

Right click in the source code of scrollFrame.java → Run file.

* A scrollable ResultSet object can also be developed having some additional behaviors like SensitiveResultSet, InSensitiveResultSet, UpdatableResultSet, ReadonlyResultset.

What is the difference between SensitiveResultSet object and InsensitiveResultSet object?

* when Resultset object is representing db table data if modifications done in the db table are immediately reflected to ResultSet object then it is called as SensitiveResultSet object. otherwise it is called as InsensitiveResultSet object.

To create SensitiveResultSet object

```
Statement st = con.createStatement(ResultSet.TYPE_SCROLL_SENSITIVE,
                                   ResultSet.CONCUR_UPDATABLE);
```

ResultSet rs = st.executeQuery("select * from student");

To create InSensitive ResultSet object

Statement st = con.createStatement(ResultSet.TYPE_SCROLL_INSENSITIVE,
ResultSet.CONCUR_UPDATABLE);

ResultSet rs = st.executeQuery("select * from student");

* Type1 JdbC driver supports sensitive ResultSets but not InSensitive ResultSets.
* Oracle thin driver, oci driver, Connector/j driver of mysql, postgresql thin driver supports INSENSITIVE ResultSets but not SensitiveResultSets.
* For example application on sensitive and InSensitive ResultSets refer application 15 of page nos 28 and 29
* Don't use sensitive and Insensitive ResultSet object features in your jdbc based projects because no driver is supporting these sensitive and Insensitive behaviours properly.

Creating Scrollable ResultSet objects by using PreparedStatement object

PreparedStatement ps = con.prepareStatement("select * from student",
ResultSet.TYPE_SCROLL_SENSITIVE,
ResultSet.CONCUR_UPDATABLE);

ResultSet rs = ps.executeQuery();

* We can use simple Statement object or PreparedStatement object to work with scrollable ResultSet object.

what is the difference between readonly resultset object and updatable resultset object?

* If the modifications done in resultset object are reflecting immediately to database table that result set object is called updatable resultset object. Through updatable result set object we can perform all the four CURD operations on the database table.

The ResultSet object that allows only read operations on database table is called as readonly resultset object.

what is the difference between sesitive resultset object and updatable resultset object?

* In Sensitive resultset object the modifications done in database table will reflect to resultset object whereas in updatable resultset object the modifications done in resultset object will reflect to database table.

TO prepare Scrollable Resultset object as Readonly Resultset object

```
Statement st = con.createStatement (ResultSet.TYPE_SCROLL_SENSITIVE,
                                    ResultSet.CONCUR_READ_ONLY);
ResultSet rs = st.executeQuery ("select * from student");
```

TO prepare Scrollable Resultset object as Updatable ResultSet object

```
Statement st = con.createStatement (ResultSet.TYPE_SCROLL_SENSITIVE,
                                    ResultSet.CONCUR_UPDATABLE);
ResultSet rs = st.executeQuery ("select * from student");
```

* All non-Scrollable resultset objects are Readonly ResultSet objects by default.

* we can perform both select and non-select operations on database table by using updatable resultset object as shown below

TO perform select operation

```
while (rs.next())
  {
    system.out.println(rs.getInt(1) + " " + rs.getString(2) + " " + rs.getString(3));
  }
```

TO insert new record

```
rs.moveToInsertRow();  → creates an empty record
rs.updateInt(1, 567);
rs.updateString(2, "rajesh");   } sets values to the columns of the empty
rs.updateString(3, "hyd");      } record.
rs.insertRow(); ⟶ inserts records in database table
```

TO update the existing record

```
rs.absolute(4); → moves to 4th record in ResultSet object
rs.updateString(2, "rakesh"); → modifying 2nd column value of 4th record.
```

rs.updateRow(); → updates the record in database table.

To delete the existing record

```
rs.absolute (3);
rs.deleteRow ();
```

* using updatable ResultSet object we can perform insert, update and delete operations on database table without using the respective sql queries.

* Limitations with updatable ResultSet objects *

(1) Very few jdbc drivers support updatable ResultSets.

(2) we can not perform criteria based non-select operations on database table.

(3) Bulk non-select operations are not possible.

NOTE: only Type-1 jdbc driver is giving support for updatable result set objects. oracle thin driver, oci driver, mysql connector/j driver, postgresql thin driver are not supporting updatable ResultSets.

* All jdbc drivers are giving support for Readonly ResultSet objects.

Summary table on jdbc ResultSet objects

Type-1 driver

Support for Non-scrollable ResultSet	: yes
Support for scrollable ResultSet	: yes
support for Readonly ResultSet	: yes
support for Sensitive ResultSet	: yes
Support for Insensitive ResultSet	: no
support for updatable ResultSet	: yes

oracle thin / oci / Connector-j driver for mysql / postgresql thin Driver

Support for Non-scrollable ResultSet	: yes
support for Scrollable ResultSet	: yes
Support for Readonly ResultSet	: yes
Support for Sensitive ResultSet	: no
Support for Insensitive ResultSet	: yes
Support for Updatable ResultSet	: no

* For example application on updatable ResultSet object refer application 18 of page no. 30.

Rowsets :

* Jdbc ResultSet objects are not serializable objects so we can not send those objects over the network.
* To solve this problem we can use Rowsets as alternate for ResultSets.
* Jdbc Rowsets are serializable objects by default.
* Rowset object means it is the object of a java class that implements javax.sql.RowSet interface. This interface is sub interface of java.sql.ResultSet interface.
* we can develop Rowsets in three modes.
  1) AS cached Rowset → It is a disConnected Rowset.
     → It is like insensitive ResultSet object.
  2) As Jdbc Rowset → It is a connected Rowset.
     → It is like sensitive ResultSet object.
  3) As web Rowset → It is a connected Rowset.
     → Allows to read data from xml files.
* Oracle corporation gives support for two types of Rowsets (cached Rowset, Jdbc Rowset). For this it supplies oracle.jdbc.rowset.OracleCachedRowset class and oracle.jdbc.rowset.OracleJdbcRowset class. These two classes are available in ojdbc14.jar file of oracle 10g. whereas in oracle 9i these two classes are available in ocrs12.jar file and for this jar file ojdbc is ojdbc14.jar file is dependent jar file.
* working with Rowsets is always an easy process because it allows getter and setter methods based programming.
* For example application on Rowsets refer application 19 and 20 of page nos 30 and 31.

Advantages with Rowsets

(1) Gives simplified programming (allows single object based setter and getter methods oriented programming).

(2) Rowsets are serializable objects so they can be sent over the network.

Disadvantages

(1) very few drivers are supporting Rowsets.

(2) Non-select operations on database table using Rowsets is not possible.

NOTE: All Collection frame work data structures are serializable objects by default. Since very few jdbc drivers are giving support for Rowsets the programmers copy ResultSet object records to collection frame work data structure (like arrayList) and sends that collection frame work data-structure over the network.

* If classes of certain jar file uses the classes and interfaces of certain other jar files then the other jar files are called as dependent jar files to the first main jar file.

Ex: The classes of ocrs12.jar file internally uses the classes of ojdbc14.jar file so we can say ojdbc14.jar file dependent jar file to ocrs12.jar.

Working with Date values

* While dealing DOB, Date of joining, bill purchase date and etc values we need to see jdbc persistance logic performing insert, update, delete and select operations on date values.

* Don't insert Date values in database table columns as String values because we can't apply the Date related sql functions on those Date values.

* It is always recommanded to insert the date values in "date" data-type columns of database table.

* Different database softwares support different patterns of Date values* like oracle supports → dd-mmm-yy pattern (ex: 20-Jan-99)

mysql supports → yyyy-MM-dd pattern (ex: 1999-10-20)

* once java.sql.Date class object is given to jdbc driver representing Date value then the Jdbc driver inserts that date value in the database table column in the pattern that is supported by underlaying database software.

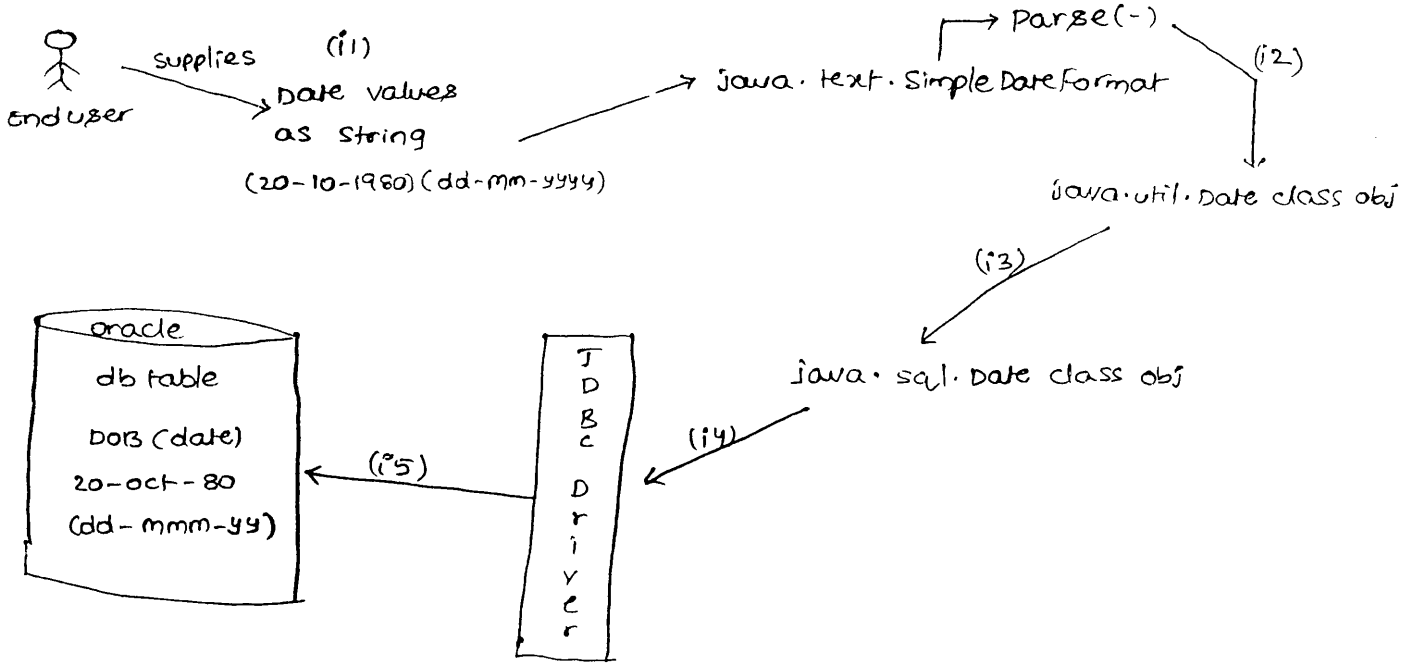
Wrong database table design

name	dob
(VC2)	(VC2)
raja	20-10-1982
ramesh	30-10-1987

Good database table design

name	dob
(VC2)	(date)
raja	20-Oct-1982
ramesh	30-Oct-1987

Database software and Jdbc driver independent logic to insert Date value in date data type column of database table in the pattern that is supported by underlaying database software.



With respect to diagram

i1 – End user supply string datavalue to application

i2 – The parse(-) method of simple Date format class converts string date value to java.util.Date class object.

i3 – Application converts java.util.Date class object to java.sql.Date class object.

i4 – Application gives this java.sql.Date class object to jdbc driver using PreparedStatement object

i5 – This jdbc driver inserts the date value to database table column in the pattern it is supported by database software.

What is the difference between java.util.Date class and java.sql.Date class ?

java.util.Date	java.sql.Date
* This class object represents ordinary date values. but it can't represent db table column date values.	* Represents db table Column date values.
* Jdbc driver can't recognize the date value of this class object.	* Jdbc driver can recognize the date value of this class object.

* java.sql.Date is the sub class of java.util.Date.

```
//Converting String date value to java.util.Date class object
String s1 = "20-10-99"; // dd-MM-yy
SimpleDateFormat sdf1 = new SimpleDateFormat ("dd-MM-yy");
java.util.Date ud1 = sdf1.parse(s1);
SOP("util date is :" +ud1.toString());

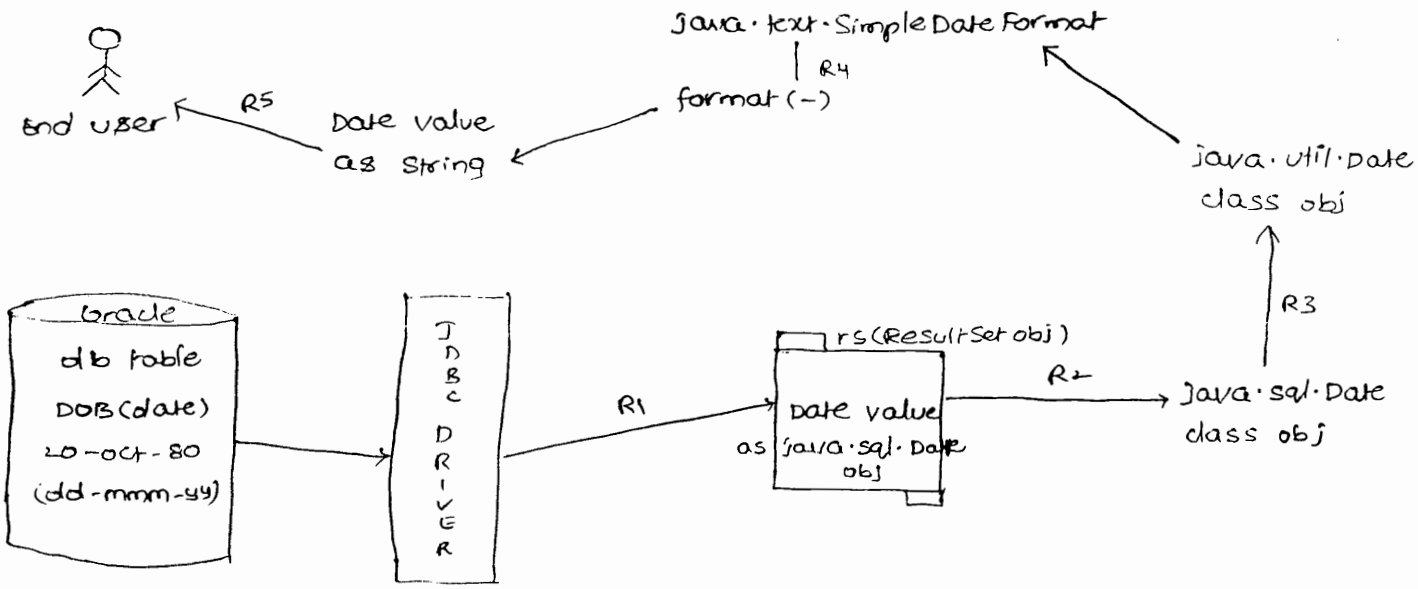
//Converting java.util.Date class obj to java.sql.Date class object
long ms = ud1.getTime(); ①
java.sql.Date sqd1 = new java.sql.Date(ms);
SOP("sql Date is :" + sqd1.toString());
```

The date pattern of input date value

① – gives no. of milliseconds that are elapsed between January 1st 1970 00:00 hours (Epoch standard) and the date and time represented by udl object (java.util.Date class object).

```
//Converting String date value directly to java.sql.Date class object
// (note: the String value must be there in yyyy-MM-dd pattern)
String s2 = "1982-10-20"; //yyyy-MM-dd
java.sql.Date sqd2 = java.sql.Date.valueOf(s2);
sop("the sql date is:" + sqd2.toString());
```

The db s/w independent and jdbc driver independent procedure to retreive dates from database table column



R1 – Application executes select query on database table and gets jdbc resultset object having date value

R2 – Application retreives date value from ResultSet object as java.sql.Date class object.

R3 – Application Converts java.sql.Date class object to java.util.Date class object.

R4 – The format(-) method of simpleDateFormat class converts java.util.Date class object to String Date value in the pattern that is expected by end user.

R5 – Application displays string date value for end user.

```
//Converting java.sql.Date class obj to java.util.Date class obj
        java.util.Date ud2 = (java.util.Date) sqd2;
        sop("the util date value is:" + ud2.toString());
//Converting java.util.Date class object to string date value
    SimpleDateFormat sdf2 = new SimpleDateFormat("MMM-yy-dd");
                                                  ↓ Expected date pattern of o/p string date value
        String s3 = sdf2.format(ud2);
        sop("String date value is:" + s3);
```

Example application to insert and retrieve data values

DB Table (oracle)

```
create table person_tab (no number, name varchar2(20), DOB date,
                                                    DOJ date);
```

DB Table (mysql)

```
create table person_tab (no int(5), name varchar(20), DOB date,
                                                    DOJ date);
```

```
//DateInsert.java
import java.util.*;
import java.sql.*;
import java.text.*;
public class DateInsert
 {
    Public static void main(String args[]) throws Exception
     {
        //read input values from key board
         Scanner sc = new Scanner(System.in);
         sop("Enter person number:");
         int no = sc.nextInt();
         sop("Enter person name:");
         String name = sc.next();
         sop("Enter person date of birth");
         String pdob = sc.next();
         sop("Enter person date of joining");
         String pdoj = sc.next();
```

```
//convert string date values to java.sql.Date class object.
// for Date of Birth
   SimpleDateFormat  sdf1= new SimpleDateFormat ("dd-mm-yy");
   java.util.Date  udob = sdf1.parse(pdob);
   java.sql.Date  sqdob = new java.sql.Date(udob.getTime());
// for Date of joining (yyyy-MM-dd)
   java.sql.Date  sqdoj = java.sql.Date.valueOf (pdoj);
// create Jdbc Connection object
   class.forName ("oracle:jdbc:driver:oracleDriver");
   Connection con= DriverManager.getConnection ("jdbc:oracle:thin:@local
                                                host:1521:ord","scott", "tiger");
//create Prepard statement object pointing to insert query
   PreparedStatement ps = con.prepareStatement ("insert into person_tab
                                                values (?,?,?,?)");
// set values to the parameters of query
   ps.setInt(1,no);
   ps.setString (2,name);
   ps.setDate (3, sqdob);
   ps.setDate (4, sqdoj);
// execute the query
   int res = ps.executeQueryUpdate();
// processing the result
   if (res==0)
      sop ("record not inserted");
   else
      sop("record is inserted");
// close jdbc objects
      ps.close();
      con.close();
  }//main
}//class
//javac DateInsert.java
//java DateInsert
```

* using simple statement object we cannot insert date, large object values (files in database table). But we can do this work by using PreparedStatement object.

Application

```
//DateRetrive.java
import java.sql.*;
import java.util.*;
import java.text.*;
public class DateRetrive
 {
   public static void main (String args[]) throws Exception
    {
      //create jdbc connection object
      class.forName("oracle:jdbc:driver:oracleDriver");
      Connection con = DriverManager.getConnection("jdbc:oracle:thin:
                                          @localhost:1521:orcl","scott","tiger");
     //create jdbc statement object
      Statement st = con.createStatement();
     // execute the query
      ResultSet rs = st.executeQuery("select * from person_tab");
     // process the resultset
      while (rs.next())
        {
          int no=rs.getInt(1);
          String name=rs.getString(2);
          java.sql.Date sqdob = rs.getDate(3);
          java.sql.Date sqdoj = rs.getDate(4);
         // Convert java.sql.Date class object to java.util.Date class
         //object
         java.util.Date udob = (java.util.Date) sqldob;
         java.util.Date udoj = (java.util.Date) sqdoj;
        // Convert java.util.Date class objects to String class values
         SimpleDateformat sdf1 = new SimpleDateFormat("MMM-yy-dd");
         String pdob = sdf1.format(udob);
         SimpleDateformat sdf2 = new SimpleDateFormat("yyyy-dd-MMM");
         String pdoj = sdf2.format(udoj);
```

```
            sop(no + " " + name + " " + pdob + " " + pdoj);
        }//while
        //close jdbc objects
          rs.close();
          con.close();
          st.close();
      }//main
    }//class.
```

* Don't hard code any values in your java application that are changeble in the future, pass them to application from outside the application by taking the support of properties file.
* To make jdbc code of java application as flexible code to modify collect the following details from outside the application by taking the support of .properties file the following
  * Jdbc classname
  * URL
  * Database username
  * database password
  * sql query

```
E:\
  └→ Napps
       └→ Apps
            └→ adv.java
                 └→ jdbc
                      ├→ properties.java
                      └→ myfile.properties
```

properties file

```
# jdbc details
  my.driver = oracle.jdbc.driver.OracleDriver
  my.dburl = jdbc:oracle:thin:@localhost:1521:orcl
  my.dbuser = scott
  my.dbpwd = tiger
  my.sql = select * from student
```

→ save myfile.properties

```
//PropTest.java
import java.sql.*;
import java.util.*;
import
public class proptest
{
   public static void main(String arg S[]) throws Exception
    {
      //Locate text properties file
      FileInputStream fis = new FileInputStream("myfile.properties");
      //create an empty object of java.util.Properties class
      Properties p = new properties();
      //load the content of myfile.properties into java.util.Properties
      //class
       p.load(fis);
      //read values from java.util.Properties class of object (p)
       String dname = p.getProperty("my.driver");
       String url = p.getProperty("my.dburl");
       String user = p.getProperty("my.dbuser");
       String pwd = p.getProperty("my.dbpwd");
       String qury = p.getProperty("my.sql");
      //create jdbc connection object
       Class.forName(dname);
       Connection con = DriverManager.getConnection(url, user, pwd);
      //create statement object
        Statement st = con.createStatement();
        ResultSet rs = st.executeQuery(qury);
       while(rs.next())
        {
          sop(rs.getInt(1)+" "+rs.getString(2)+" "+rs.getString(3)+" "
                                                  +rs.getString(4));
        }
       //close jdbc objects
        rs.close();
        con.close();
        st.close();
    }
```

* In database table files are called as large objects.
* There are two types of large objects
  (1) BLOB → Ex: image files, avi files
  (2) CLOB → Ex: text files, rich text files, ms word documents
* matrimony, job portal applications we must deal with these large objects.
* To insert LOB values to database table use Jdbc PreparedStatement object and take column data types has BLOB (or) CLOB.

standard procedure to insert LOB values to database table column

1. create inputstream object pointing to file

```
FileInputStream fis = new FileInputStream ("c:\abc\ash.gif");
```

2. create PreparedStatement object pointing insert query based on jdbc con object

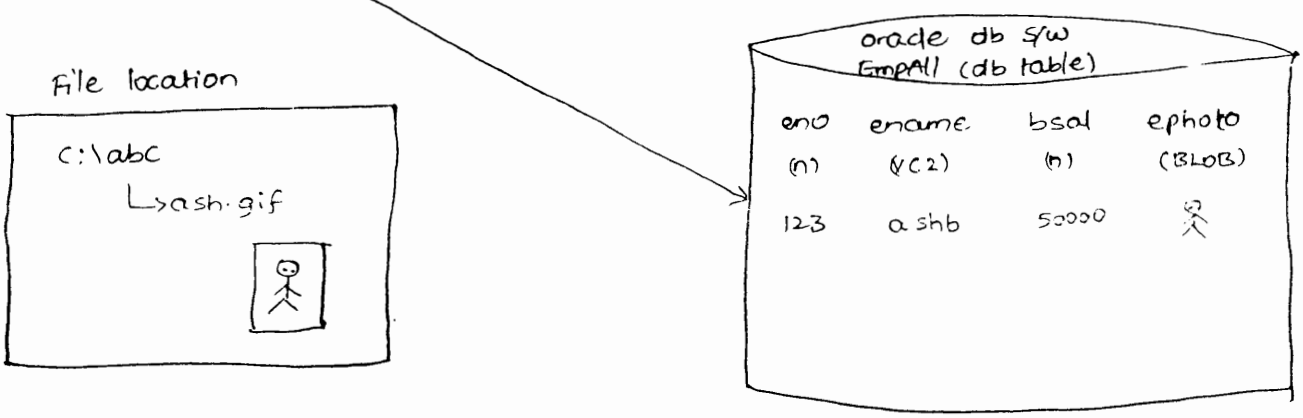
```
PreparedStatement ps = con.PrepareStatement("insert into EmpAll values(?,?,?,?);
```

3. set values to parameters of query

```
ps.setInt(1,123); ps.setString(2,"ashb"); ps.setInt(3,50000);
ps.setBinaryStream(4, fis.length);
```

4. Execute the query

```
ps.executeUpdate();
```



* All files and directories are belonging to various drives together it's called as File System.
* To binary large object values to query we can use setBinaryStream(3P) (or) setBlob(3P) method.
* Similarly to set character large object values to sql query we can use

setCharacterStream( ) or setClob() methods.

* For example application on working with large object refer application 23 and 24 of handBook.

Standard procedure to follow to retrive large object values from database table column

Step(1): Execute select query on database table and get Jdbc ResultSet object.

```
Statement st = con.createStatement();
ResultSet rs = st.executeQuery("select * from EmpAll");
```

Step(2): Get access ephoto value from db table

```
rs.next();
FileInputStream fis = rs.getBinaryStream("ephoto");
```

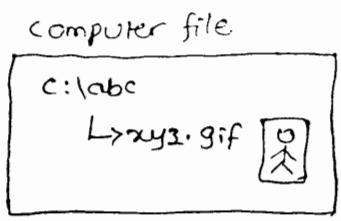
↑ Input stream object pointing photo db table column through ResultSet object

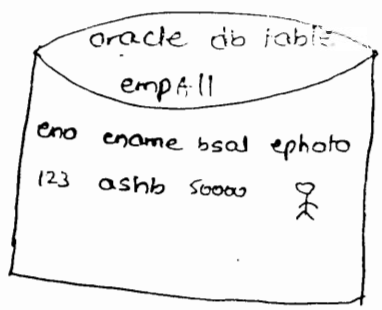
Step(3): Create FileOutputStream object pointing to destination file (xyz.gif)

```
FileOutputStream fos = new FileOutputStream("c:\\abc\\xyz.gif");
```

Step(4): write db table column photo to the destination file (xyz.gif) using both input, output stream objects.

```
int bytesRead=0;
byte[] buffer = new byte[4096];
while (bytesRead = fis.read(buffer)! = -1)
  {
    fos.write(buffer, 0, byteRead);
  }
```

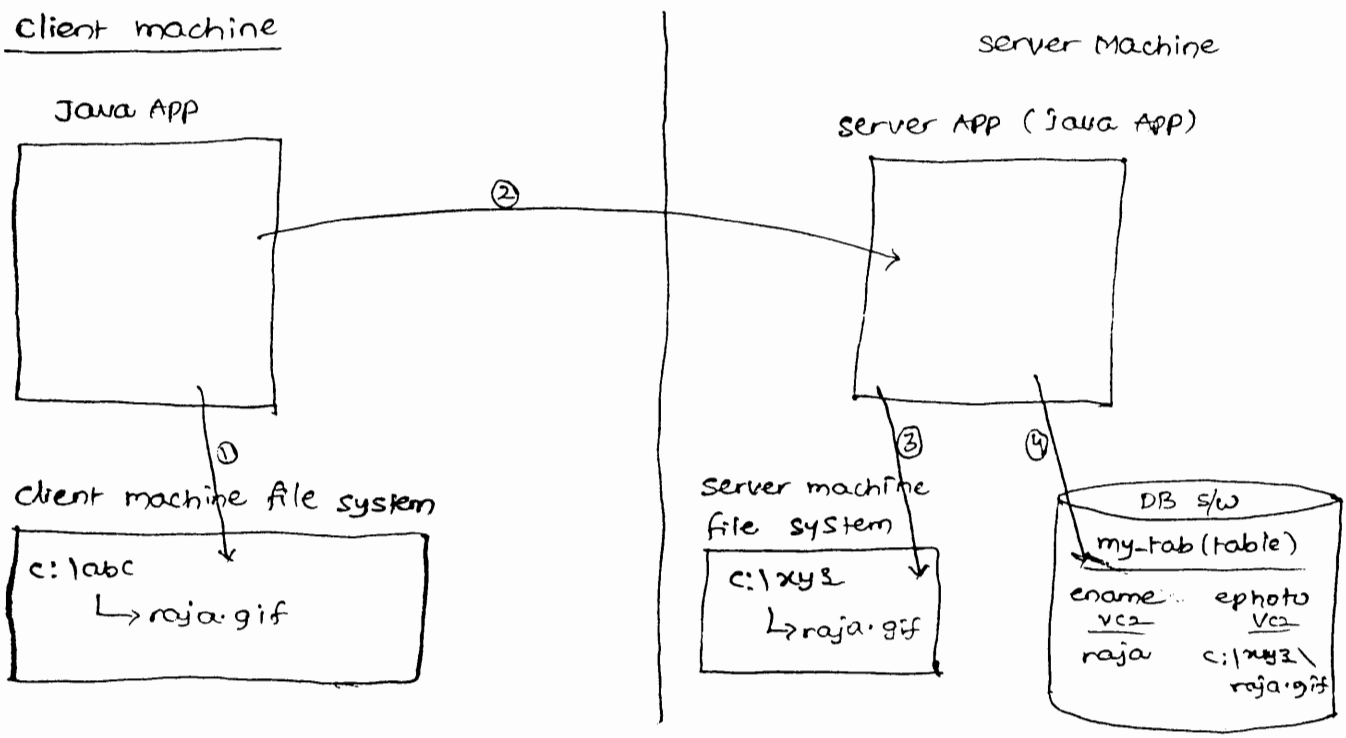




* Buffer is temporary memory to store temporary data.
* The process of storing data in buffer before sending to final destination is called as buffering.
* while transfering source file data to destination file if buffer support

is taken, we can reduce no. of read and write operations on source and destination files respectively.

* In the above code the byteRead variable keeps track of the no. of bytes that are stored into buffer each time by reading the content from source file (image of ResultSet object) so that we can use that data value while writing buffer content to destination file.
* To read BLOB value from ResultSet object use rs.getBinaryStream() or rs.getBlob() method.
* To read clob value from ResultSet object use either rs.getCharacterStream() or rs.getClob() method.
* In real time projects no one prefers to store large object value like image files text files etc...
* Directly in database table column because they may degrade performance of database software, by allocating memory upto 4GiB.
* In industry level projects instead of storing LOB files directly as DB table column values they store these files in Computer file system/ directories of hard disk drives and they will store the paths of these files as String values in String data type table columns.



* The process of sending file from client machine to server machine is called as file uploading and reverse is called as file downloading.

Sample Senario to know how realtime projects manages large objects (towards insertion)

(1) client application locates and selects the file from client machine file system.

(2) client application sends the file to server application through file uploading process.

(3) server application writes/saves the received file to the file system of server machine.

(4) server application writes the path of the file on server machine as db table column value.

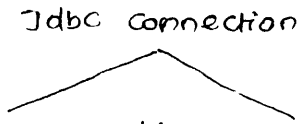
* Jdbc connection pool is a factory that contains set of readly available jdbc connection objects before actually being used.

* The advantage with jdbc connection pool is we can use minimum no. of connection objects to give service to maximum clients.

* There are two types of jdbc connection pools.

(1) JDBC Driver managed/stand alone Jdbc Connection

(2) Web server/application server managed jdbc connection.



1. Jdbc driver managed/ Stand alone jdbc Con pool (Resides outside the server)

2. web server/application server managed Jdbc connection pool (Resides inside the server)

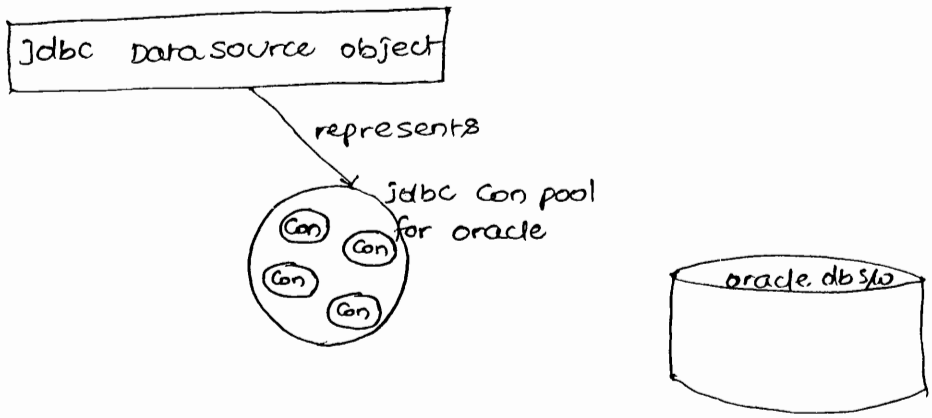
* while developing small scale applications like stand alone, desktop application we can use driver managed jdbc connection pool.

* while developing large scale applications like web sites and enterprise applications we can use server managed jdbc connection pool.

* All jdbc connection objects in a jdbc connection pool represents connectivity to same database software.

* Jdbc Connection pool for oracle means all jdbc connection objects in that connection pool represents connectivity with oracle database software.

* oracle thin driver/ oci driver gives one built-in jdbc Connection pool for oracle.
* Jdbc data source object represents jdbc Connection pool so our application should always get a Jdbc Connection object from Connection pool through data source object



* Jdbc data source object means it is the object of a java class that implements javax.sql.DataSource (I)
* The jdbc Connection object that is created by the programmer manually is called as direct Jdbc Connection object.
* The jdbc Connection object that is collected from jdbc Connection pool is called as pooled Jdbc Connection object.

Example application on oracle thin driver managed jdbc Connection pool

```
//ConnectionPoolTest.java
  import java.sql.*;
  import javax.sql.*; // for pooledConnection (I)
  import oracle.jdbc.pool.*; // (for oracle Connection Pool DataSource (C))
  public class ConnPoolTest
   {
      public static void main(String args[]) throws Exception
       {
         //create jdbc data source object representing empty driver managed
         //connection pool of oracle
          OracleConnectionPoolDataSource ds = new oracleConnectionpoolData
                                                    Source();

         //gives details to create Jdbc Connection objects in the connection pool
           ds.setDriverType("thin");
```

```
        ds.setServerName("localhost");
        ds.setPortNumber(1521);
        ds.setServiceName("orcl");
        ds.setUser("scott");
        ds.setPassword("tiger");
       //get access to jdbc connection pool for oracle (this connection
                    pool will be created based on the above details)
        PooledConnection con = pcon.getConnection();
        //write jdbc based persistance logic
          Statement st = con.createStatement();
          ResultSet rs = st.executeQuery("select * from student");
          while (rs.next())
           {
            sop(rs.getInt(1)+" "+rs.getString(2)+" "+rs.getString(3));
           }
         //close jdbc objects
          rs.close();
          st.close();
          con.close(); // releases connection object back to jdbc con pool
       }//main
    }// class
   //add ojdbc14.jar file to classpath and then execute.
```

* Scrollable ResultSet, sensitive ResultSet, updatable ResultSet features are given from jdbc 2.0 onwards.
* Drivermanager connection pool save point connections are given from jdbc 3.0 onwards.
* Auto loading of driver class annotations based jdbc programming features are given from jdbc 4.x onwards.
* To work with Driver managed jdbc connection pool there is no necessity of dealing with type3 protocol. while working with servermanager connection pool we must deal with type3 protocol.
* Savepoint is a point within the current transaction that can be referenced from the Connection.rollback method.

* When a transaction is rolled back to a save point point all the changes made after that savepoint point will be under.
* After executing 10 queries if you want to commit first 5 query results and if you want to rollback last 5 query results then create savepoint after first 5 queries and rollback upto that savepoint position.
* There are two types of savepoints that can be created in jdbc environment.

(1) named savepoint

(2) unnamed savepoint

NOTE: For unnamed savepoint the db s/w automatically generates an identity value.

* In jdbc queries based transaction save point based rollbacks are possible but savepoint based commits are not possible.

```
//SavepointTest.java
import java.sql.*;
public class SavepointTest
  {
    public static void main(String args[]) throws Exception
     {
        //create jdbc connection object
        Class.forName("oracle.jdbc.driver.OracleDriver");
        Connection con=DriverManager.getConnection("jdbc:oracle:thin:@localhost
                                                   :1521:orcl","scott","tiger");
        Statement st=con.createStatement();
        //Begin Tx
        con.setAutoCommit(false);
        //query1
        st.executeUpdate("insert into student values(671,'xyz','hyd1')");
        //set a named savepoint to Connection object
        Savepoint svpt=con.setSavepoint("mysp"); //mysp is savepoint
                                                 name
        //query2 in savepoint area
        st.executeUpdate("update student set name='mahesh2' where
                                                 sno=345");
```

```
//rollback upto savepoint   con.rollback(svpt);
//Con.commit(svpt); // error
//Commit Tx
Con.Commit();
//record insertion will be Committed (query1) but record
//updation (query2) will be rolledbacks because query2 is
//there in save point area
}//main
}//class
//> javac SavepointTest.java
//> java SavepointTest
```

* To work with jdbc 3.0 features we need jdk 1.5 or higher version similarly to work with jdbc 4.0 features we need jdk 1.6 or higher version setup.
* The auto driver class loading feature of jdbc 4.0 can load the make the application loading Driver class automatically in one of the following two situations.

(1) If Driver class is specified in META-INF/service/java.sql.Driver file of application directory related work directory.

(2) If jar file that represents jdbc driver contains java.sql.Driver file having jdbc driver class name.

* ojdbc14.jar does not contains java.sql.Driver file so you must arrange that file explicitly as shown below to make application loading oracle thin Driver class automatically as shown below.

step(1): create java.sql.Driver file

```
E:\....\App1
    |--> META-INF
        |--> services
            |--> java.sql.Driver (file)
```

java.sql.Driver (file)

```
oracle.jdbc.driver.OracleDriver
```

step2): Develop java application as shown below

```
//SelectTest.java
import java.sql.*;
public class selectTest
 {
    public static void main (String args[]) throws Exception
      {
         //do notice that there is no class.forName(-)
         Connection con = DriverManager.getConnection ("oracle.jdbc:oracle:
                                                  thin:@localhost:1521:orcl","scott","tiger");

         if (con! =null)
              sop ("Connection is established");
         if (con == null)
              sop ("connection is not established");

      }//main
  }//class
//>javac selectTest.java
//> java selectTest
```

Example(2):

* The postgresql related thin Driver jar file is having java.sql.Driver file having the Driver class name. By just adding this jar file to class path we can make our java applications loading jdbc driver class automatically.

```
E:\.... \App2
        |--> selectTest.java
```

```
//SelectTest.java
import java.sql.*;
public class selectTest
  {
     public static void main (String args[]) throws Exception
      {
         //do notice that there is no class.forName(-)
         Connection con = DriverManager.getConnection("jdbc:postgresql:
                                                  postgres", "postgres", "postgres");
```

```
        if (con != null)
            sop("Connection established");
        else
            sop(Connection not established");
    }// main
}// class
// > javac SelectTest.java
// > java SelectTest.java
```

* Before executing above example add postgresql-8.4-701.jdbc4.jar file to class path.

——— *** ———

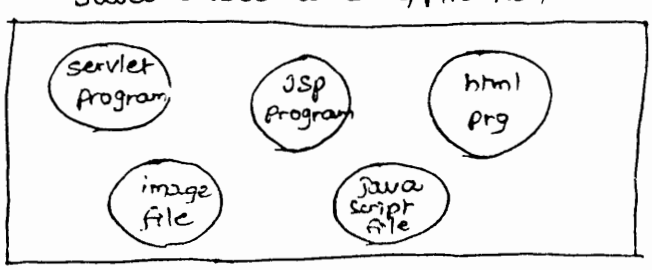
Servlets

* The logics and data of stand alone, desktop applications are specific to that computer where these applications are running.
* The logics and data of client-server applications, two-tier applications like jdbc applications are specific to one network where they are running. In the above said two tier, client-server applications the server allows only known clients (recognized clients).
* To provide global visibility and accessability to the logics and data of the application use the internet environment based websites or web applications. These applications allow both known and unknown clients having 24x7 access to the resources and logic data.
* When website is under development / testing in a software company then it is called as web application. Once web application is hosted on to the internet network by purchasing domain name (www.citibank.com) or host name and space in the internet network is called as website.
* A web application is a collection of web resource programs which can generate web pages. There are two types of web resource programs based on the web page they generate.

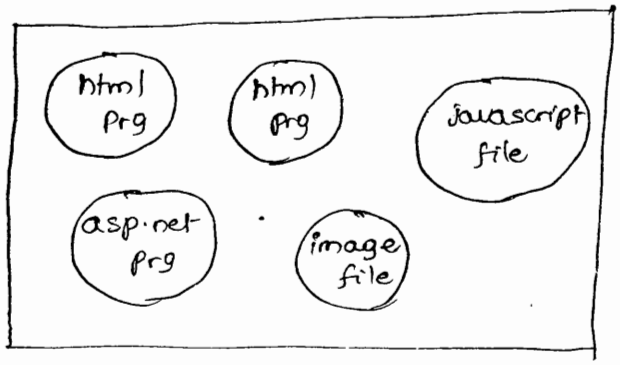
  1) static web resource programs → generates static webpages (Ex: HTML)
  2) dynamic web resource programs → generates dynamic webpages
     Ex: (servlet program, JSP programs, ASP programs, PHP programs)
     ↓
     Personal Home page for Hypertext processing

* Static web page contains fixed content forever. The content of dynamic web page changes based on the input values of the request or time of the request generation.

Java based web application



.net Based web Application



* The web resource programs like image file, javascript file can not generate web pages directly but they support other web resource programs to generate web pages.
* Stand alone applications, desktop applications and client-server applications will be executed by programmer manually. But the web resource programs of web application must be executed dynamically when they are requested by clients (Browser windows) so we can't think about manual execution for web resource programs. we can use a special software called web server software to execute the web resource programs dynamically when they are requested by clients.
* A web server software can also manage, execute multiple web applications simultaneously or parallelly.

Examples of web server / Http server Softwares

Tomcat
Resin
JWS (Java web server)
PWS (personal web server)
IIS (Internet Information server)
and etc....

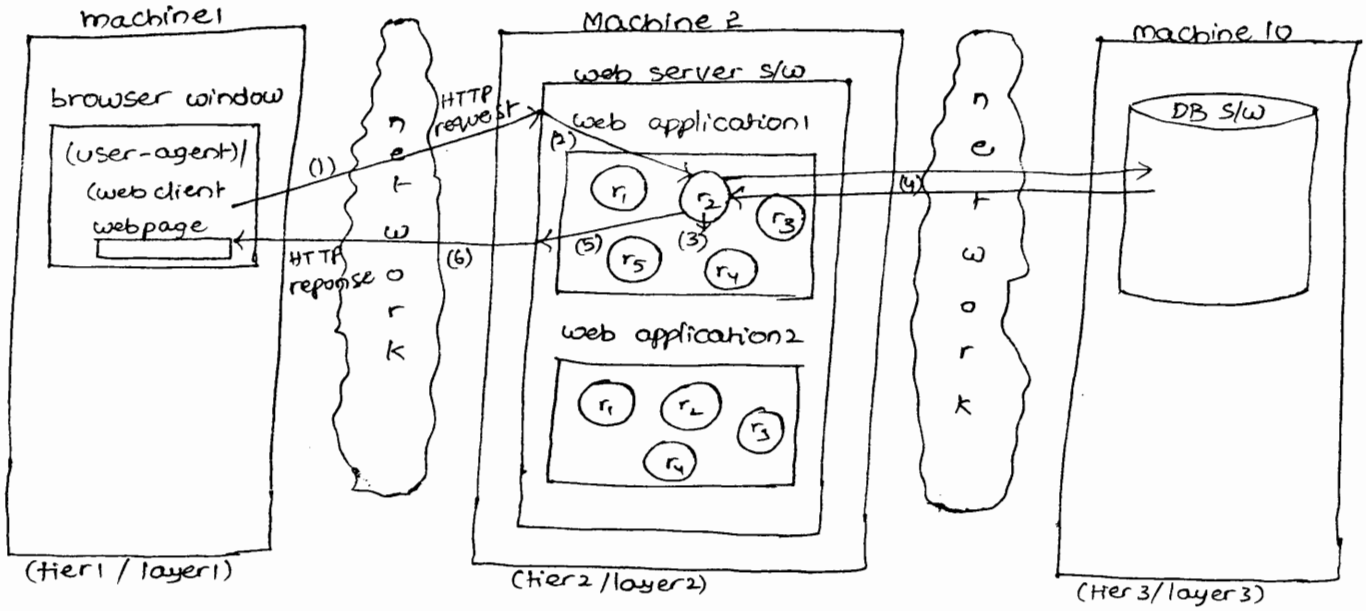
* To develop and execute jdbc application we need the following setup.
jdk s/w, Jdbc driver, DB s/w.
* Similarly to develop and execute web application we need the following setup.
browser s/w, web server s/w, db s/w, web technologies → To develop web resource programs

* Web server s/w used to manage and execute web application.

* browser s/w used to request to webapplication.

*java based web servers internally uses jdk s/w

setup required to develope and execute web application



* Each browser window acts as one client to web application

* A web application can be developed as two tier application without database software or three tier application with database software.

* With respect to diagram

(1) Browser window generates http request to web application.

(2) The web server traps and takes the request and passes the request to appropriate web resource program.

(3) The web resource program process the request.

(4) Web resource program interacts with data base software if necessary.

(5) The web resource program generated result goes to web server.

(6) Web server sends these results to browser window as HTTP response in the form of web page.

* In the above diagram $r_1, r_2 \cdots$ are web resource programs.

* Based on the place of executing where web resource programs execute there are two types of web resource programs.

(1) server side web resource programs (they execute in the web server).

Ex: servlet prg, JSP prg and etc.

(2) client side web resource program → They come to browser window from the web application placed in web server for execution.

Ex: HTML program, JavaScript program, Ajax program and etc...

* Decide whether web resource program is client side or server side based on the place where it executes not based on the place where it resides.
* A web application looks like thin client-fat server application.
* The process of keeping web application in web server is technically called as deployment and reverse process is called as undeployement.

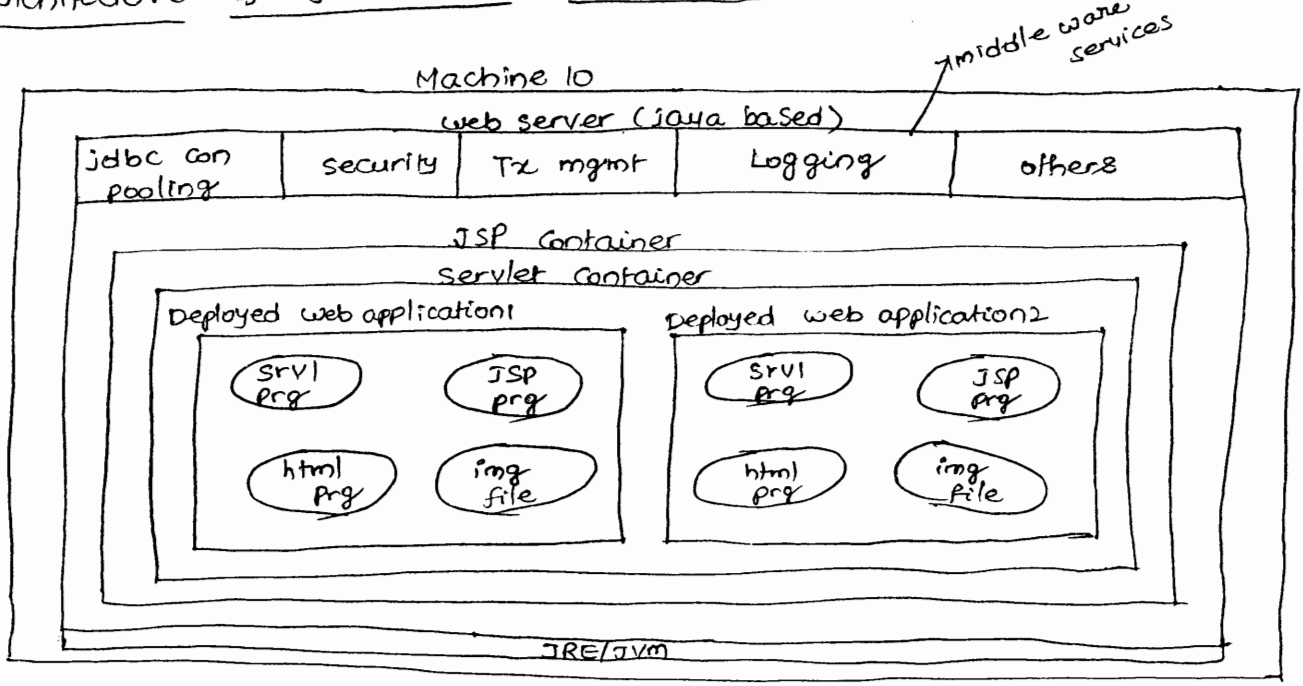
Responsibilities of web server

1) Listens to client request continuously (HTTP request)
2) Traps and takes client generated HTTP request.
3) Passes the HTTP request to an appropriate web resource program of web application (deployed web application)
4) Provides container software to execute server side programs (web resource programs)
5) Gathers output generated by web resource programs.
6) Passes output of web resource programs to browser window as http response in the form of web pages.
7) Provides environment to deploy manage and to ~~undeploy~~ undeploy the web application.
8) Gives middle ware services and etc...

* A container is a software or software application that can manage the whole life cycle (object birth to death) of given resource. (like java class).
* A file or program is called as resource of the application.
* Servlet Container takes care of servlet program lifecycle.
* JSP Container takes care of JSP program lifecycle.
* Applet Container (Appletviewer) takes care of applet program lifecycle.
* Servlet Container, JSP containers are part of web servers (java based).
* Container is like an aquarium who can take care of the whole life cycle of given resources called fishes.

* programmer is not responsible to develop containers and webservers but he is responsible to use them to execute web applications.

Architecture of java based web server



* once web server is started one daemon process will be started to listen to client's continuously and to trap and take the client generated HTTP request.
* The process that runs continuously is called as daemon process.
* Every java application contains two default threads.
  (1) main thread
  (2) Garbage collector thread (daemon thread).
* JSP container is the enhancement of servlet container and both containers use JRE/JVM supplied by web server.
* middle ware services are configurable additional services on the application to make applications executing perfectly in all the situations.
* Security middle ware service protects the application from unauthorised and unauthenticated users.
* Transaction management service executes the logics by applying do everything or nothing principle.
* Jdbc connection pool supplies set of readly available jdbc connection objects.

* Logging service keeps track of the application execution process through confirm conformation statements / messages / log messages.

* Middle ware services are not minimum logics of application development. They are additional or optional services to apply on applications.

* A web application can be there with or without middle ware services.

* Web server automatically activates servlet Container and JSP Container to execute servlet, JSP programs when they are requested by clients.

* The client side web resource programs of web application goes to browser window for execution whereas the server side programs will be executed by using the containers of web server.

* In web application executing environment browser software is called as client software and serweb server software is called as server software.

Examples of Java based web servers

Tomcat ⟶ from Apache Foundation
Resin ⟶ from Resin soft
JWS ⟶ from Sun microsystem

* HTTP is a application level protocol define in set of rules to get communication between browser window and web server. The text that allows sequential reading is called as normal text.
Ex: notepad text.

* The text that allows non-sequential reading through hyperlinks is called as hyper text.
Ex: any web page content.

* Protocol HTTP is useful to transfer hyper text between browser window and web server, web server and browser window.

Different browser softwares

IE — MicroSoft
Netscape Navigator — from Netscape
Fire Fox — from mozilla
Hot Java — from RedHat

chrome —— from Google

opera —— from Opera soft

Different client side Technologies (for developing client side web resource programs)

html ⟶ from W3C

java script ⟶ from netscape

VB script ⟶ from MicroSoft

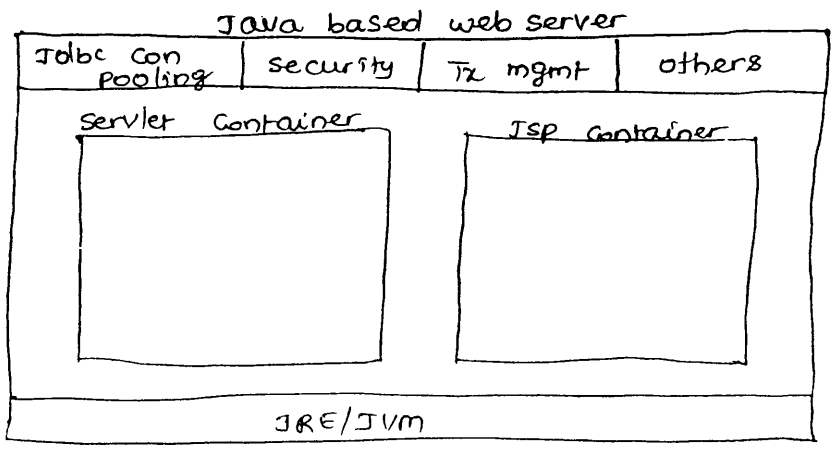
AJAX ⟶ from Adaptive path

NOTE: Java script is no way related with the programming language java.

* The code that can not be executed independently and which must be embedded in other technology based program for execution is called as script code.

* Java script code can not be executed independently and it must be executed by embedding with html code so java script is called as scripting language.

Another diagram that shows Java based web server architecture



Jsp container is developed based servlet container

* when java web application is deployed in java web server then the servlet programs of that web application will be executed by servlet container and Jsp progrms will be executed by Jsp Container.

* Directly or indirectly every java application (or) java resource (or) java Component uses JVM during execution.

Ex: appletviewer executes our applet program but this appletviewer internally uses (or) take support from JVM.

servlet container executes servlet program but this servlet container internally uses JVM support

Different server side Technologies (to develop server side web resource programs)

Servlet → from Sun microsystems → java based (2)

JSP → from Sun microsystems → java based (3)

asp → from misera microsoft → (non-java based)

asp.net → from microsoft → (non-java based) (4)

PHP → from Apache → (non-java based) (5)

COLDFusion → from Adobe → (non-java based)

SSJS → from Netscape → (non-java based)
(server side java script)

Different web server softwares

Tomcat → from Apache → java based (1)
Resin → from Resin soft → java based (2)
JWS → from Sun microsystems → java based
Jetty → from Adobe → java based
} for servlet, JSP programs

IIS → from microsoft → Non-java based (3)
PWS → from microsoft → (Non-java based)
} for asp, asp.net programs

AWS → from Apache → Non-java based (4) (for php programs)
(Apache web server)

Application server = web server + EJB Container + more middleware services

NOTE: EJB container is required to execute EJB Components.

→ reusable java class object is called as Component.

* Application server software is enhancement of web server software.

* Every application server software can act as web server.

Different application server softwares

Weblogic → from BEA systems (Oracle corporation) (1)

Websphere → from IBM (3)

Jboss ⟶ from Apache / Red Hat (4)

GlassFish ⟶ from Sun microsystems (2)

OlogAS ⟶ from Oracle Corporation

(oracle 10g Application server)

Jrun ⟶ from macromedia (Adobe)

*All application server softwares are java based softwares.

Different database softwares

oracle ⟶ oracle corporation (1)

mysql ⟶ from DevX (3)

SQLServer ⟶ from microsoft (2)

DB 2 ⟶ from IBM

Postgresql ⟶ from university of California (4)

* we can use client side technologies like html, java script along with any vendor supplied server side technologies in web application development.

* To develop & execute java based web application

a) choose any browser software

b) choose one or more client side technologies like HTML, Java script

c) choose servlet, JSPs as server side technologies.

d) choose any java based web server software or application server software.

e) choose any data base software.

* To develop and execute .net based web application.

a), b), e) are same as above

c) choose asp.net as server side technology.

d) choose microsoft supplied IIS, PWS server.

* To develop PHP based web applications

a), b), e) are same as above

c) choose PHP as server side technology.

d) use AWS as web server.

* To develop medium scale and large scale web applications use java environment.

* To develop small and medium scale web applications use .net environment.
* To develop small scale web applications use PHP environment.

Tomcat

Type : java based web server

version : Tomcat 6.0 (Compatable with Jdk 1.6)

Tomcat (7.0) (Compatable with Jdk 1.6/1.7)

Tomcat 5.0 (Compatable with Jdk 1.5)

Vendor : Apache foundation

open source software (the source code will be exposed)

default port no: 8080 (changable)

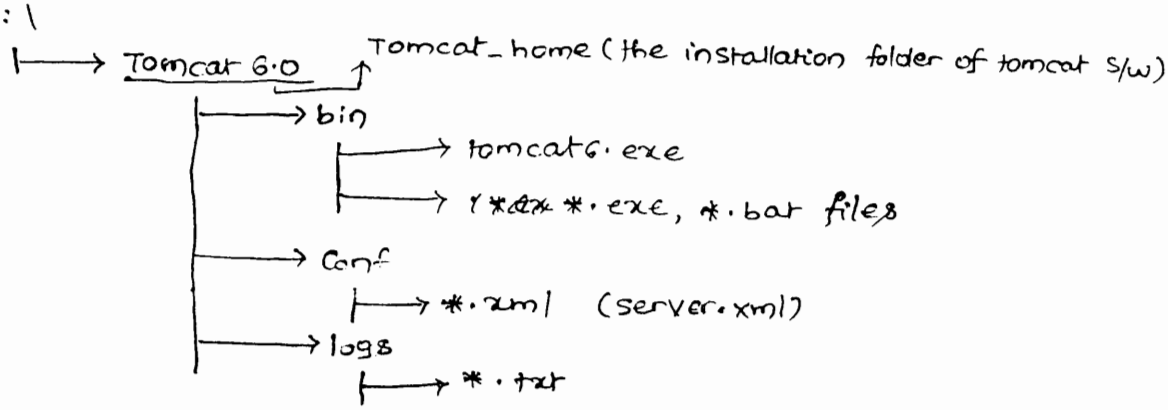
To download software: www.apache.org

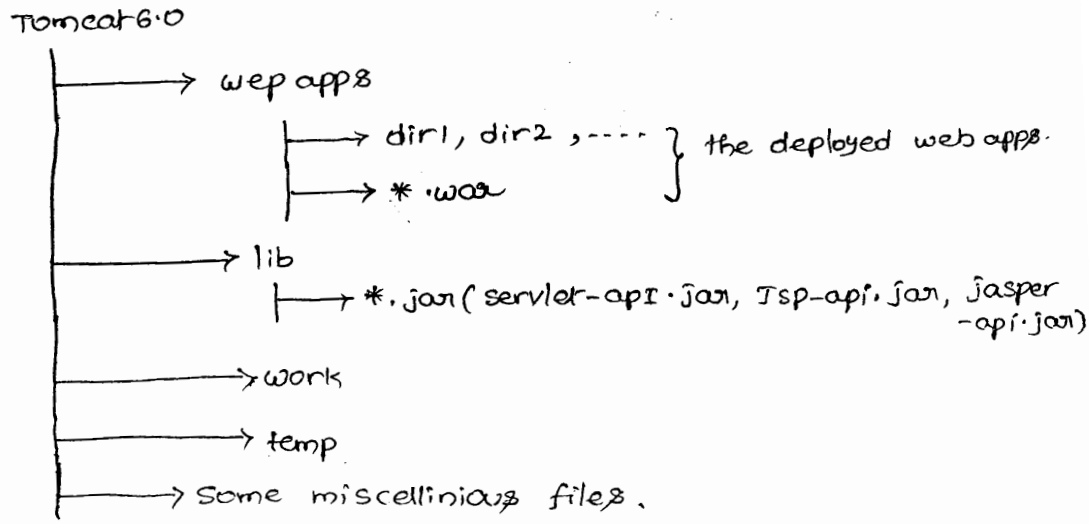
for docs : www.apache.org

for faqs : www.forum.apache.org

* To install Tomcat 6.0 software use Apache → Tomcat – 6.0.26 set up file and you can change the default port no, default username and password details during installation. If not changed default port no is "8080", default username is "admin", default password is "admin".
* Every software installed in a computer will reside in a logical position called software port and every software port will be identified with its port no. (Communication end point)

After installing tomcat you will get the following folders and files in the installing folder





* The installation folder of tomcat is called as <tomcat_home>.
* To start tomcat server use <tomcat_home>/bin/tomcat6.exe file
* To launch home page of tomcat the procedure is

open browser window → type http://localhost:8080/ in address bar

↑ port no. of tomcat server

Procedure to change port no. of tomcat server after installation:

Goto <tomcat_home>/conf/server.xml file and modify port attribute value of first <connector> tag → restart the server

* The browser window keeps every web page in the buffer before displaying it for end user.
* A buffer is a temperory memory which can hold the data for temperory period.
* while giving new port no. to any software service use 1025 to 65535 range number because 1 to 1024 range numbers are busy for with OS services.
* Programmer can supply Java web application to tomcat server either in the form of directory or war file.
* Programmer uses <tomcat_home>/webapps folder to deploy web applications.

* A specification is a document or reference that contains set of rules and guidelines to develop the softwares.

* There are two types of specifications.

(1) open specification (Any one can develop softwares based on this specification)

Ex: JDBC specification, servlet specification, JSP specification and etc.

(2) propriatory specification (only specific company which has given specification is allowed to develop the software)

Ex: .net specification.

* servlet, jsp, ejb, jms, java mail and etc are the sub specifications of JEE specification.

* servlet specification contains rules and guidelines to develop servlet container.

* JSP specification contains rules and guidelines to develop Jsp Container.

* EJB specification contains rules and guidelines to develop EJB Container by using various sub specifications of JEE the web server and application server softwares will be developed and these server softwares contains various containers like servlet container, JSP Container and etc...

* JEE and its concepts like servlets, JSP, EJB and etc are not installable softwares because they are just given as specification.

* working with web server and its containers or application server and its containers is nothing but indirectly working with JEE and its concepts. (like servlet, JSP and etc..)

* Every software specification contains an API representing rules and guidelines. These rules are nothing but methods declared in the interfaces and abstract methods declared in the abstract classes. These guidelines are nothing but concrete methods defined in abstract classes and concrete classes.

* while developing software based on software specification rules will be implemented (must) and guidelines will be utilized (optional).

* Servlet specification supplied api is the classes and interfaces of javax.servlet package and javax.servlet.http package.

* The methods of interfaces and the abstract methods of abstract classes available in above two packages represents rules of servlet specification whereas the concrete methods defined in classes and abstract classes of above two packages represent guidelines of servlet specification

What is Servlets

* Servlets is an open/open source api specification that contains set of rules and guidelines to develop servlet container software.

* Based on this servlet specification given by Sun micro systems different vendors develops different servlet container softwares and supplies them to programmers along with web server and application server software installation.

* In earlier days servlet container used to call as servlet engine.

* The tomcat serve the main name of servlet container is catallina

* Since JEE is not a installable software the tomcat server supplies sun micro system's servlet api in the form of <tomcat_home>/lib/servlet-api.jar and it gives its servlet container software in the form of <tomcat_home>\lib\catalina.jar.

* Every spe software specification supplied api will be used in two angles.
  (1) Vernods companies use specification api to develop the softwares.
  (2) Programmers use specification api to develop programs or applications.

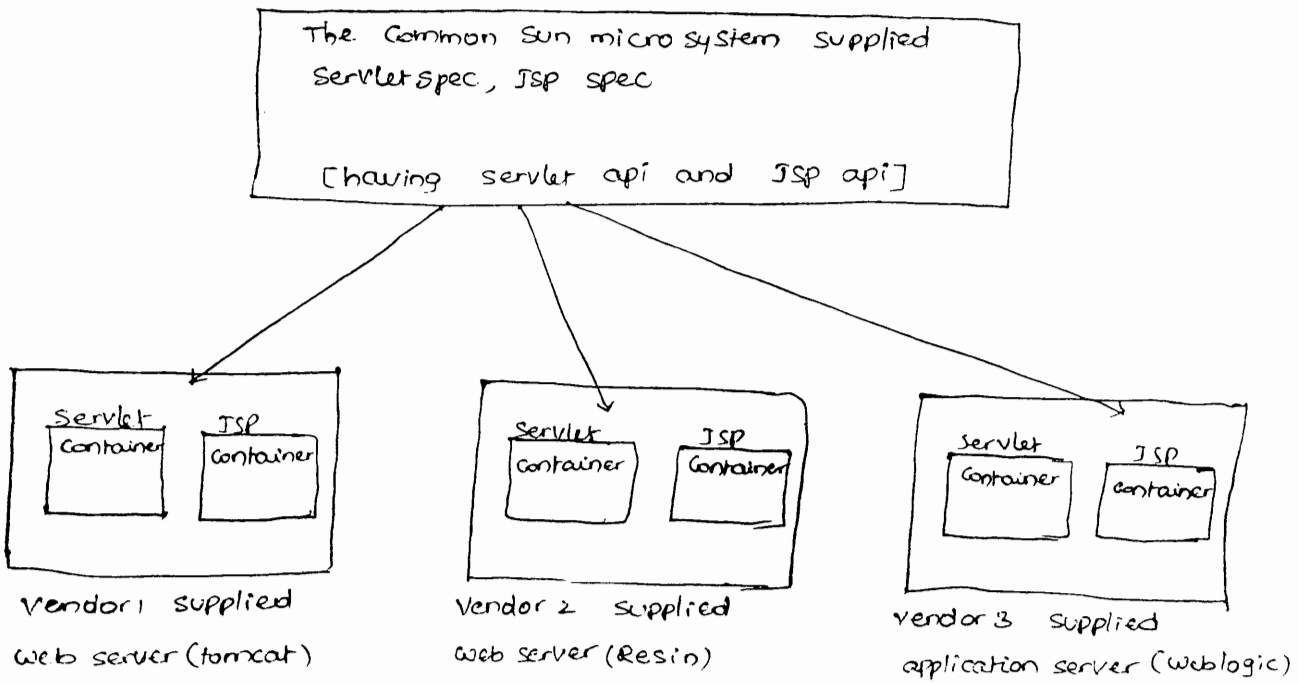
* The servlet api of servlet specification will be used by vendor companies to develop servlet container software. whereas programmers use the same servlet api to develop servlet programs as web resource programs of web application. Due to this servlet container is always capable of executing servlet programs.

What is JSP?

* It is sun micro system supplied an open specification having set of rules and guidelines to develop Jsp container.

* JSP specification is enhancement of servlet specification so JSP container is also the enhancement of servlet container.
* In tomcat server JSP container is called as Jasper.
* The tomcat server supplies sun microsystems JSP api in the form of <tomcat_home>\lib\jsp-api.jar and supplies its own JSP container in the form of jasper.jar.
* The sun microsystems JSP api contains following packages

javax.servlet.jsp
javax.servlet.jsp.el
javax.servlet.jsp.tagext



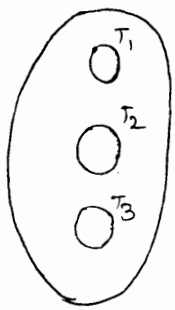
* Since all webserver softwares and their container softwares given by different vendors are developed based on common Sun micro system supplied servlet specification, JSP specification so the way we work with all these web server softwares and their container softwares in java based web applications development is going to be much similar.

Server side Technologies

(required to develope server side ~~web~~ web resource prgs of web apps)

- Process Based
  - CGI - perl
- Thread Based
  - servlets
  - JSP
  - asp
  - asp.net
  - PHP

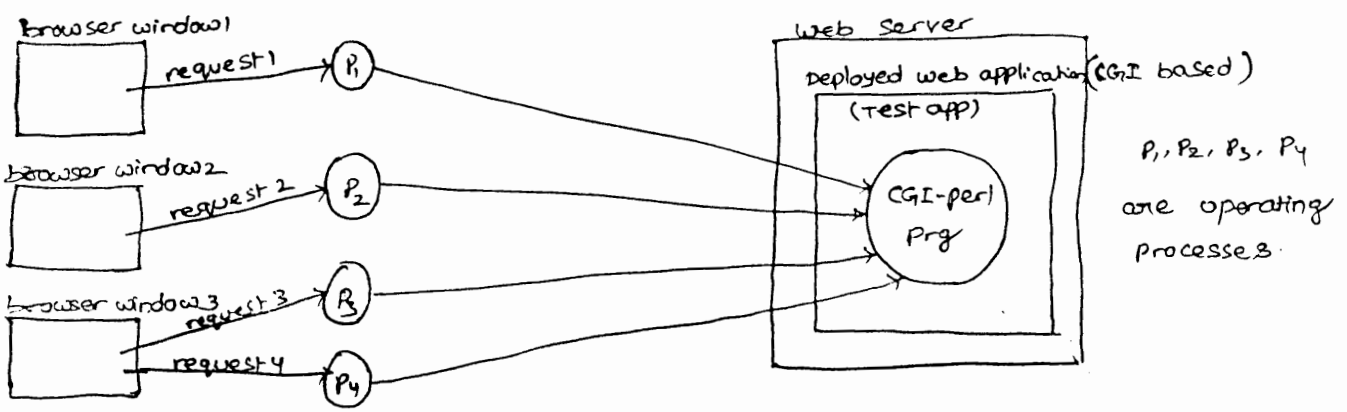
OS managed process



T_1, T_2, T_3 are threads

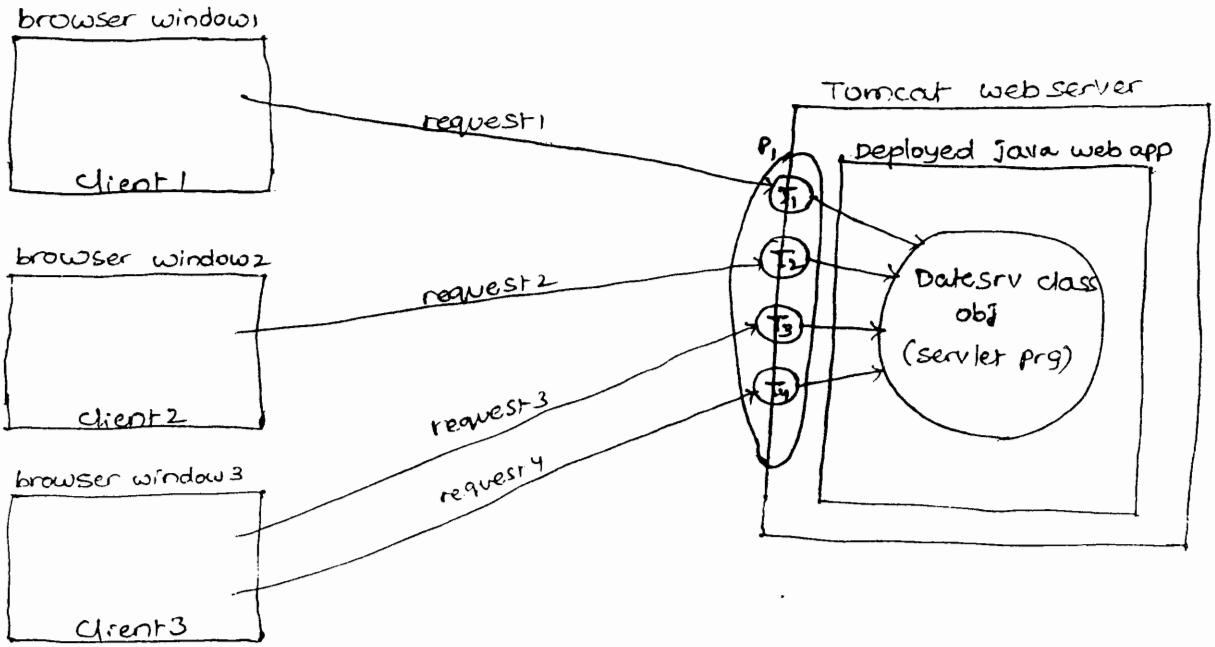
* A thread is a light weight process in OS managed process.
* Every java application contains two default threads.
  (1) main thread
  (2) Garbage collector thread
* CGI is a specification to develop server side web resource program so perl programming language will be used to develop the webresource programs based on CGI specification.
* perl → practical extraction reporting language

Understanding Process based serverside technologies. (CGI)



* The procedure of transfering control from one process to another process or from one thread to another thread is called as the scheduling based control jumping or context switching.
* since OS processes are heavy weight processes so they take lot of time for control jumping or context switching.
* Due to this when more requests are given CGI based web application more OS processes will be created on one per request basis and performance of website will be degraded. (That means web application becomes non scalable web application).
* If application is giving same performance irrespective of increase or decrease in number of clients / no. of requests is called as scalable application.
* Every servlet program is a java class using servlet api.

Understanding thread based server side technologies like servlet



P1 → OS managed process representing the webserver startup

T1, T2, T3, T4 are threads started on servlet program related object representing request given by clients.

* The control jumping between threads of a process takes very much less time when compare to control jumping between two seperate OS processes. So we can say thread based server side technologies give better

performance when compared to process based server side technologies.

* The web application that is developed based on thread based server side technologies is scalable application.
* servlet program is a single instance multiple threads based java component of java web application as shown above. That means if 100 requests are given to single servlet program then the servlet container creates only one object for that servlet program class but starts 100 threads on that object representing 100 requests as shown above.

Definitions of servlet

(1) Servlet is server side technology (basically specification) that allows the programmers to develop dynamic web resource programs or server side web resource programs in java based web application.

(2) Servlet is a software specification given by Sun microsystems that provides set of rules and guidelines for vendor companies to develop the softwares called servlet containers.

(3) servlet is a single instance multiple threads ~~component~~ based java component in java web application to generate dynamic web pages.

(4) servlet is a server side technology (basically specification) that can enhance functionalities of web server software or application server software.

* servlet program is server independent because one servlet program can be executed in any java based web server or application server software without modifications.
* securing web application is nothing but applying authentication and authorization on the web application.
* checking the identity of a user is called as authentication.
* checking the access permissions of the user on particular resources of the project is called as authorization.
* For basic information on servlets, servlet container refer the page no.s 36 to 43.

jee 5 specification

Servlets 2.5 spec
JSP 2.0 spec
EJB 3.0 spec
:

jee 6 specification

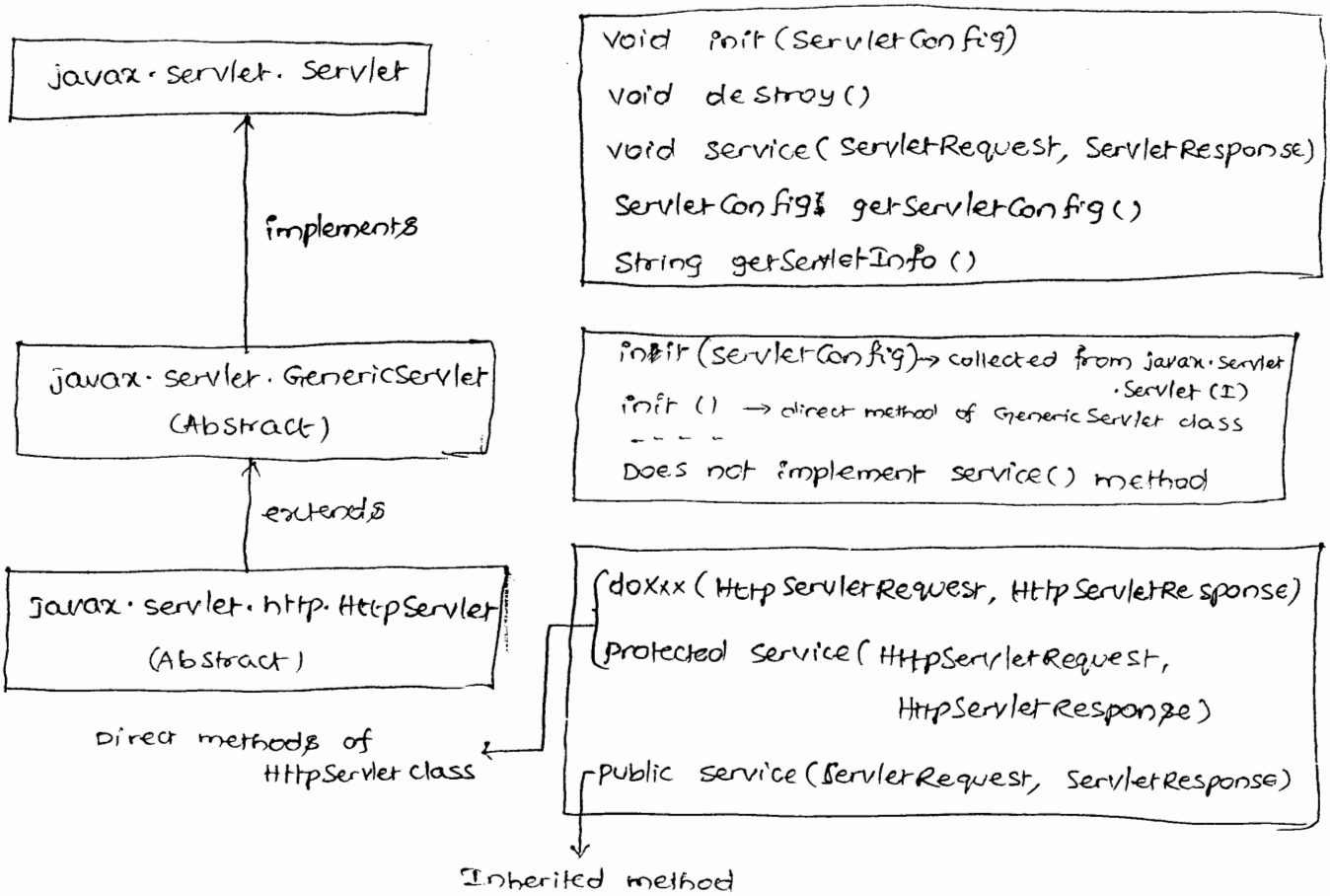
Servlets 3.0 spec
JSP 2.1 spec
EJB 3.1 spec
:

* In Jee module of java all are specifications.
* In Jse module of java awt, swing are technologies, JDBC, JNDI are specifications.
* The latest version of the servlet api specification is 3.0 but industry is using servlet 2.5 api specification.
* Tomcat 6.0 supplies servlet container software based on servlet 2.5 specification whereas tomcat 7 use servlet container software based on servlets 3.0 specification.

The 3 important resources of servlet api

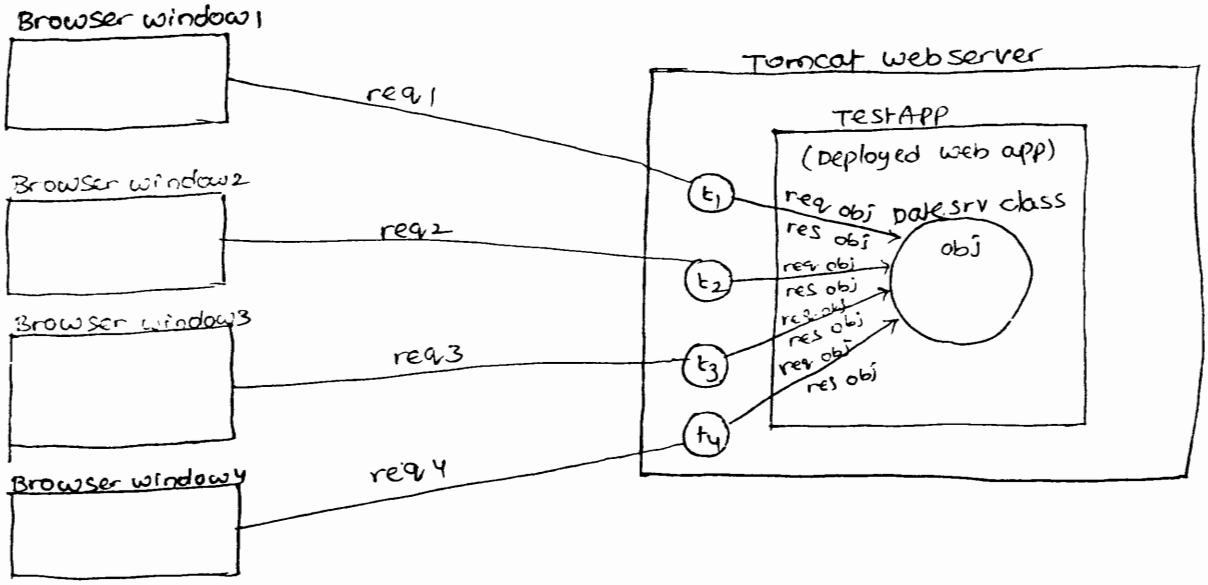
javax.servlet.Servlet (I)
javax.servlet.GenericServlet (Abstract class)
javax.servlet.http.HttpServlet (Abstract class)

* In java an abstract class can contain only abstract methods or only concrete methods or mix of both.
* The above give predefined HttpServlet class is an abstract class containing no abstract methods.
* If you want to make methods/logics of one java class accessable only through subclasses then make that class as abstract class even though that class contains only concrete methods.



* Every servlet program is a java class that is developed based on servlet api. There are three ways to develop servlet program.
  (1) Take a java class implementing javax.servlet.Servlet Interface and provide implementation for all the 5 methods of that interface.
  (2) ~~Provide~~ Take java class extending from javax.servlet.GenericServlet class and provide implementation for service() method.
  (3) Take java class extending from javax.servlet.http.HttpServlet class and override one of the 7 doXxx() methods or one of the two service(-) methods.
* In the above said three approaches the overridden/implemented service(-,-) / doXxx(-) programmer places request processing logic that generates web page by processing the request. But programmer never calls these methods manually but servlet container calls these methods automatically for every request given by client to servlet program

* Programmer just supplies his servlet program related java class to servlet container then onwards servlet container is responsible to manage the whole life cycle of servlet program. (from object birth to death every operation will be taken care by container).
* For all the request given to servlet program the servlet container creates one object but for every request given to servlet program, the servlet container creates one seperate request, response objects and calls service(-,-)method by keeping these requests, response objects as argument values.



* servlet program uses request object to read details from the request like form data.
* servlet program uses response object to send response content to browser window through web server.

When 10 requests are given to a servlet program from single or different browser windows (clients)

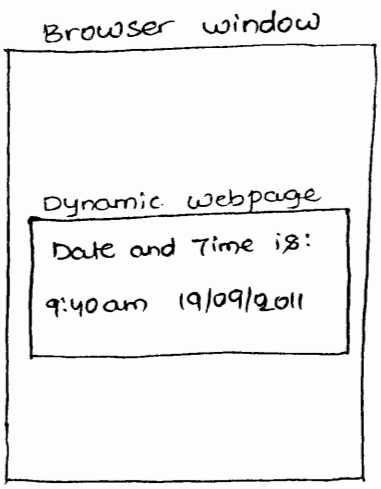
--> ServletContainer creates only one object for servlet program related java class.

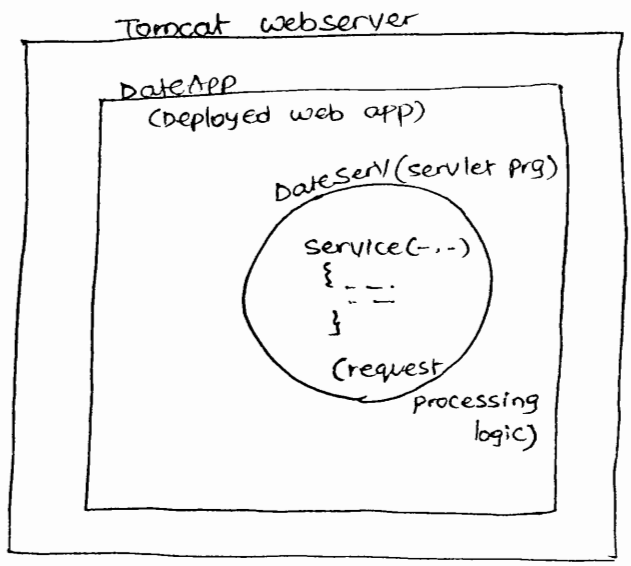
→ servletContainer creates 10 threads on servlet program object representing 10 requests.

→ ServletContainer creates 10 sets of request, response objects and calls service(-,-) or methods for 10 times having request, response objects as argument values.

* To make our servlet program java class visible to servlet container the java class must be taken as public class.

Procedure to develop first java web application by taken servlet program as webresource program

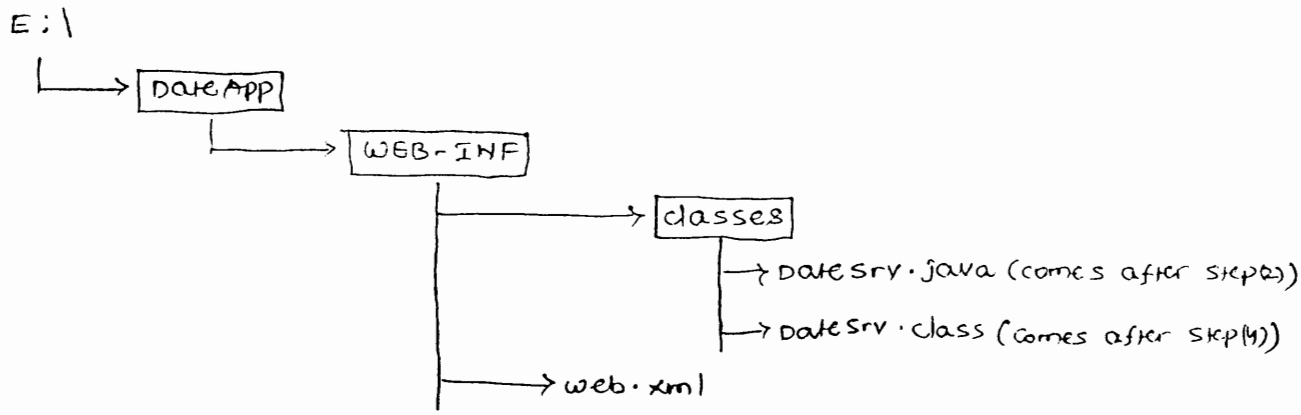




Setup required JDK 1.6
Tomcat 6, IE any version

Step(1): create the following deployment directory structure or packing directory structure to combine multiple webresource programs into single unit.

NOTE: A web application is a directory or war file containing multiple web resource programs. (one or more).



* Here DateApp folder is a webroot folder.
* After deployment this DateApp becomes name of the web application.
* The web application packing or deployment directory structure is common for all the servers and can be created in any location of computer.

Step(2): Develop servlet program (DateSrv.java) save that one in WEB-INF \classes folder of web application.

```
//DateSrv.java
import javax.servlet.*;
import java.io.*;
public class DateSrv extends GenericServlet
{
    //implement service(-,-)
    public void service(ServletRequest req, ServletResponse res) throws
                                        ServletException, IOException
    {
    //Set response content type
     res.setContentType("text/html");
    //get stream obj
     PrintWriter pw = res.getWriter();
    //write req processing logic
     java.util.Date d1 = new java.util.Date();
    //write response content to browser window through webserver
     pw.println("Date and time is: "+d1.toString());
    //close stream object
     pw.close();
    }//service(-,-)
}//class
```

req(1): Represents the servlet container supplied request object

req(2): Represents the servlet container supplied response object.

text/html: MIME (multipurpose Internet mail Extention/Content type).

res.setContentType("text/html");

* Represents that passes instructions to browser window through web-server that this servlet program generate html code base response text.

other MIME types

* text/html
* application/ms word
* application/ and etc

printwriter pw = res.getWriter();

* Here pw is a stream object pointing to response object getWriter() called on res object returns the printwriter stream object.

NOTE: A stream object can represent either file or object as destination

pw.println("---");

* This method makes the stream object pw writing data to the destination object response (res). This response object passes that data (argument value of println("...") method) to webserver and this webserver writes data to browser window as http response in the form of a web page.

pw.close();

* closes the stream connection with response object.

* Add the tomcat server supplied servlet-api.jar (contains servlet api) to classpath.

my computer → properties → advanced variables → environment variables →

variable name: CLASSPATH

value : D:\Tomcat 6.0\lib\servlet-api.jar ; <other existing jar files>;

→ ok → ok → ok

NOTE: Since servlet-api is not part of jdk software and since our servlet program is using servlet api to make java compiler (javac) recognising api the above jar file is added to the classpath.

Step(3): Compile our servlet program source code.

E:\DateApp\WEB-INF\classes > javac DateSrv.java

Step (4): Develop web.xml file having the servlet programs Configuration.

NOTE :

(1) A program or file of application is called as resource of the application

(2) servlet program is called as the web resource program of the web application. All servlet programs must be configured in web.xml file.

(3) specifying the details of certain resource program and making underlaying software like Container, Server and etc recognising the resource programs is called as resource configuration. By configuring servlet programs in web.xml file we make servlet Container recognising servlet programs.

web.xml

```
<web-app>
    <servlet>
        <servlet-name> abc </servlet-name>
        <servlet-class> DateSrv </servlet-class>
    </servlet>
    <servlet-mapping>
        <servlet-name> abc </servlet-name>
        <servlet-url-pattern> /test </url-pattern>
    </servlet-mapping>
</web-app>
```

→ logical name/ object name for our servlet class when servlet container creates the object.

→ java class that is acting as servlet program

→ must match abc with above logical name

→ url pattern of servlet program

* servlet program will be identified in the web.xml file through its logical name (abc). more ever servlet container uses this logical name as the object name when it creats object for our servlet program java class.

* The web server servlet Container and the web resource programs of web application and clients identifies the servlet program through its url pattern (/test).

* web.xml file is called as deployment descriptor (dd) file because servlet Container reads and verifies the entries of web.xml file the moment web application is deployed.

* web.xml file is called as Configuration file because it contains servlet program Configurations.

* physical presence of servlet program in WEB-INF/classes folder is not sufficient to make servlet container to recognize servlet program. It must be configured in web.xml file.

NOTE: step(1) to step(4) represents web application development.

Step(5): Start the tomcat server.

Tomcat_home\bin\tomcat6

Step(6): Deploy the above DateApp web application

copy e:\DateApp folder to Tomcat_home\webapps folder.

NOTE: Step(5) and Step(6) performs the deployement of web application.

Step(7): Test the web application

open browser window → type http://localhost:2020/DateApp/test

- http: Protocol
- localhost:2020 → host name & port no. of tomcat server
- DateApp → web application name / Context path / context route
- test → URL pattern of DateSrv servlet program

NOTE:

* URL pattern given to servlet is useful, to hide the servlet related program class name from end users.

* Java web applications are WODA (write once Deploy Anywhere) applications because

(1) The deployment directory structure is common for all webservers.

(2) The web resource programs like servlet, JSPs are server independent programs.

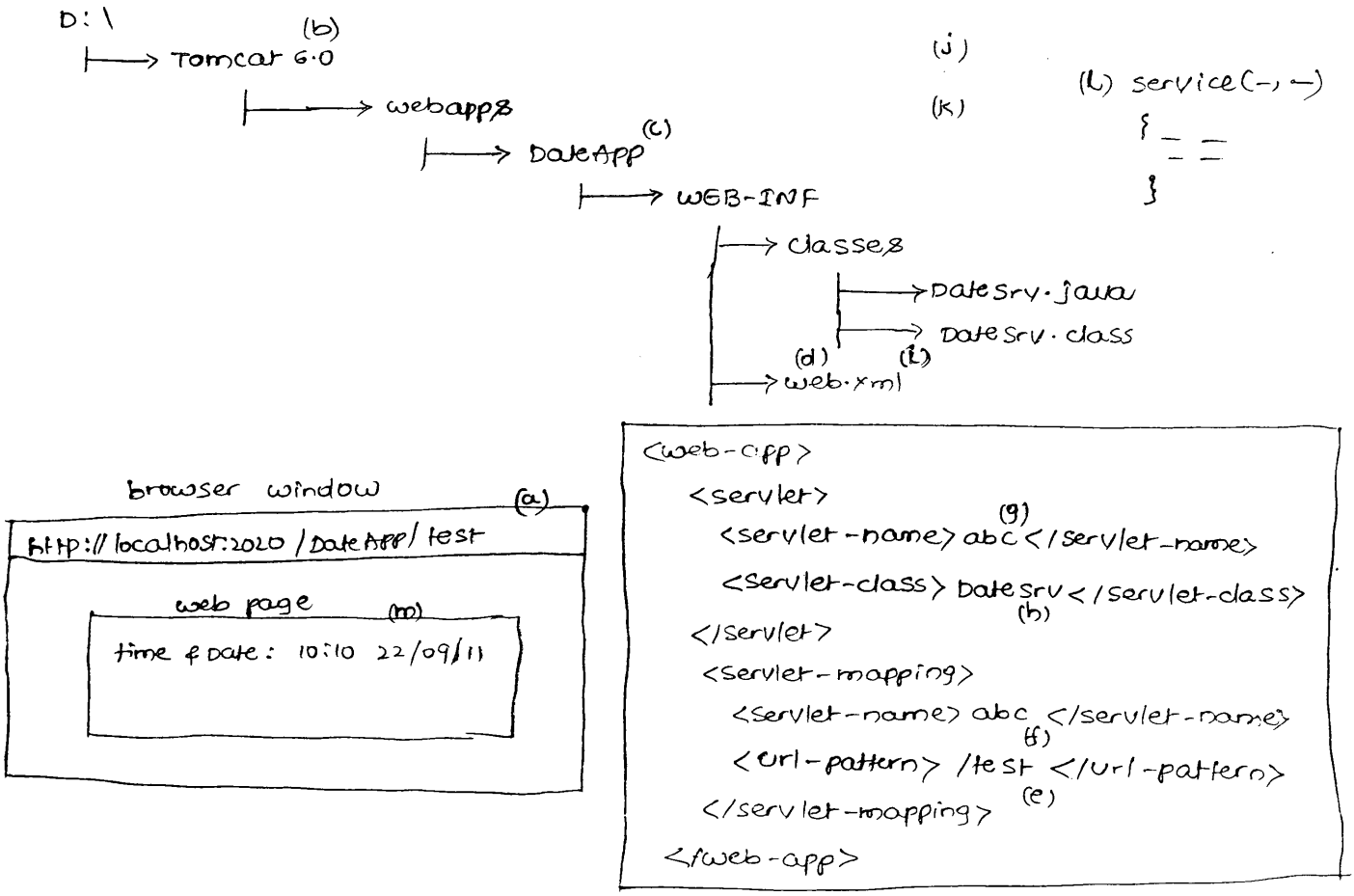
Can you explain flow of execution from request to response generation with the aid of an example application?

With respect to the given code

(a) End user types request URL in the address bar of browser window to generate request to servlet program.

(b) Based on localhost:2020 content of request url the request goes to the tomcat webserver of local machine.

(c) Based on DateApp word of request url the request goes to the



deployed DateApp web application of web server.

(d) The servlet Container uses web.xml file file entries to locate the servlet program that is requested.

(e), (f), (g), (h) Based on the url pattern /test the logical name abc the servlet Container gathers the class name of servlet program (DateSrv).

(i), (j) servlet Container loads DateSrv class from WEB-INF\classes folder of web application.

(k) servlet Container creates and locates DateSrv class object. If created the servlet Container finishes initialization process on our servlet class object.

(k) servlet Container creates one set of request, response object for the current request and calls service (-,-) method by keeping them as argument values.

(l) The service (-,-) method processes the request and generates the output.

(ro) This output goes to browser window through webserver as http response in the form of dynamic web page.

PrintWriter pw = res.getWriter();

* In the above statement we are getting access to PrintWriter stream object that is available as built-in stream object of res object. This stream object points to res object as outputStream object so we can use this stream object to write some output content to response object from servlet program.
* Java I/o stream objects can perform read and write operations on the files / objects.
* The System.out.print statements generated output from servlets will appear on server console whereas the pw.println statements generated output will come on webpage of browser window.
* modifications done in web.xml file of deployed web application will be recognized by server automatically whereas the modifications done in servlet program source code will be reflected only after recompilation of servlet program and reloading of the web application.
* To reload web application in Tomcat web server the procedure is

open Tomcat home page (http://localhost:2020/ → Tomcat manager →

user name : admin
password : admin } choosen during installation

→ DateApp application → Reload.

* Servlet Container uses 0-arg Constructor to create object of our servlet class so make sure that our servlet class is having 0-arg Constructor directly or indirectly. Directly means programmer should place 0-arg constructor explicitly. Indirectly means servlet program deals with the javac generated default 0-arg constructor.
* The service method placed in servlet program will be called by Servlet-Container for every request that is given to servlet program from

any browser window (client).

javax.servlet.ServletRequest (I)

↑ extends

javax.servlet.http.HttpServletRequest (I)

javax.servlet.ServletResponse (I)

↑ extends

javax.servlet.http.HttpServletResponse (I)

* The req object of service(-,-) method is not the object of javax.servlet.ServletRequest interface, it is the object of servlet Container supplied Java class implementing javax.servlet.ServletRequest interface directly or indirectly. But programmer never specifies this class name in servlet program because its ~~classname~~ name changes based on the webserver/servlet container we use. In tomcat server req object class name is "org.apache.catalina.connector.RequestFacade".

* The res object of service(-,-) method is not the object of javax.servlet.ServletRespose interface, it is the object of servlet Container supplied Java class implementing javax.servlet.ServletResponse interface directly or indirectly. In Tomcat server the res object class name is "org.apache.catalina.connector.ResponseFacade".

* To know req, res objects class names in any server call getClass() methods on those objects from service(-,-) method of servlet program as shown below.

```
pw.println("<br> req obj class name is:" + req.getClass());
pw.println("<br> res obj class name is:" + res.getClass());
```

* All the request coming to our servlet program will use only one object of our servlet class but every request gets seperate request, response objects and also starts one seperate thread on servlet class object (refer

* TO prove this practically write following code in the service(-,-) method of our servlet class.

```
pw.println("<br> req obj class name is:" + req.getclass());
pw.println("<br> req obj class name is:" + res.getClass());
```

gives seperate values for each request

```
pw.println("<br><br> hash code of req obj : " + req.hashCode());
pw.println("<br><br> hash code of res obj : " + res.hashCode());
```

gives seperate thread name for each request

```
pw.println("<br><br> current req thread name is:" + Thread.currentThread
                                                    ().getName());
```

gives same value for all the requests

```
pw.println("<br><br> hashCode of our servlet class obj:" + this.hashCode());
try
{
Thread.sleep(40000);
}
catch (Exception e)
{
  e.printStackTrace();
}
```

NOTE: once server is down all objects (including our servlet class objects) will be destroyed.

How many types of servlets are there?

* There is a possibility of developing n- types of servlets like http servlet, FTP servlet, SMTP servlet and etc...
* Since the entire internet and all web server softwares are given based on the protocol http the programmers prefer developing their servlet programs as http servlet programs.
* Javax.servlet.GenericServlet is not a seperate type of servlet. programs It is given as Common super class for all protocol specific servlet classes like http servlet and etc...
* As of now servlet api is giving only one sub class for GenericServlet class that is HttpServlet class because all servers are there supporting only http protocol.

How a servlet program developed based on GenericServlet class can work with HttpRequest and HttpResponse?

* Since GenericServlet class is Common super class for protocol specific servlet classes. It can take HttpRequest but it can not gather all the

details from that HttpRequest. whereas the servlet program that is developed based on HttpServlet class can gather all the details of HttpRequest.

* Even though there are lot of details in servlet program but the servlet program will be identified with its URL pattern.
* we can provide multiple URL patterns for a single servlet program in web.xml file as shown below.

```
<web-app>
   <servlet>
      <servlet-name> abc </servlet-name>
      <servlet-class> DateSrv </servlet-class>
   </servlet>
   <servlet-mapping>
      <servlet-name> abc </servlet-name>
      <url-pattern>/test </url-pattern>
   </servlet-mapping>
   <servlet-mapping>
      <servlet-name> abc </servlet-name>
      <url-pattern>/test1 </url-pattern>
</web-app>
```

* If web application is having multiple servlet programs then all the servlet programs must be configured in web.xml file having URL patterns.
* By specifying different MIME Types as response content types we can make Servlet programs sending web pages to browser window in different formats.
- Every software installed in your Computer will provide MIME type or Content type for its files, to know them we can use the windows registery editor (regedit) tool
- Browser window can display web pages in different formats based on the response content type of the output that is generated by web resource programs. To know MIME type or content type of any windows operating system managed file, the process is

start --> Run --> regedit --> HKEY_CLASSES_ROOT --> choose file extension --> get Content type from right side panel.

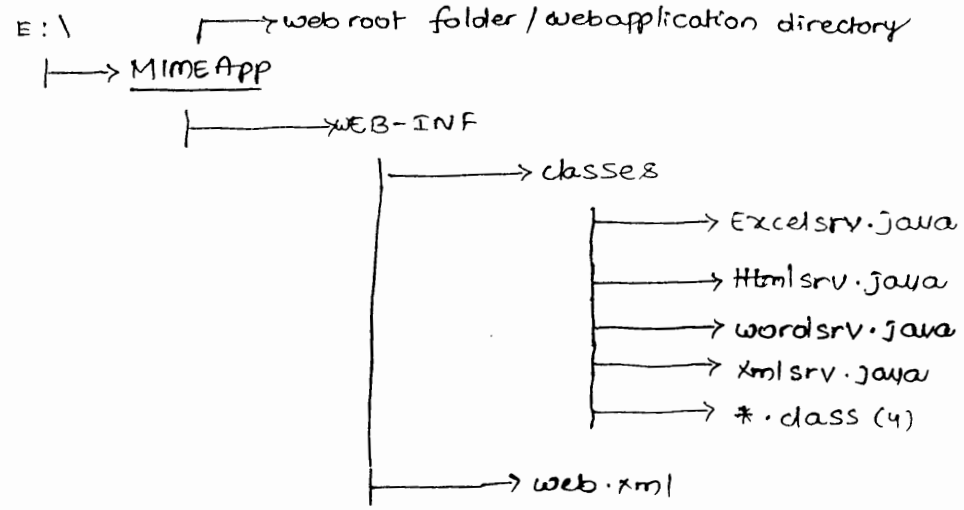
- .doc (ms word) ------ application/msword
- .xls (msexcel) ——————→ application/vnd.ms-excel
- .html ——————→ text/html
- .xml ——————→ text/xml
- .bmp ——————→ image/bmp
- .avi ——————→ video/avi

and etc.

Develop a java web application having multiple servlet programs and each servlet should generate a different format based web page.

(Servlet programs or HtmlSrv, ExcelSrv, WordSrv, XmlSrv)

Step(1): Prepare the deployment directory structure of web application.



Step(2): Develop the source code of above servlet programs.

* For the source code of HtmlSrv.java, WordSrv.java, XmlSrv.java, ExcelSrv.java refer page nos 58 and 59 of the booklet

NOTE: The approach no. (1), (2) based servlet programs work with ServletRequest, ServletResponse objects where as the approach no. (3) based servlet program works with HttpServletRequest object and HttpServletResponse object.

NOTE: The simple ServletRequest, ServletResponse objects can't work with all the facilities of the protocol Http where as HttpServletRequest, HttpServletResponse objects can work with all the facilities of protocol HTTP.

Step(3): Add servlet-api.jar file to classpath.

Step(4): Compile the source files of all the servlets programs.

E:\MIMEApp\WEB-INF\classes> javac *.java

Step(5): Configure all the four servlet programs in web.xml file having four different URL patterns.

Refer the web.xml file of page no.s 59 and 60

Step(6): Start the server (Tomcat)

Step(7): Deploy the web application.

copy E:\MIMEApp folder to Tomcat_home\webapps folder

Step(8): Test the web application.

open browser window → type these URLs

http://localhost:2020/MIMEApp/hturl

http://localhost:2020/MIMEApp/wdurl

http://localhost:2020/MIMEApp/xlsurl

http://localhost:2020/MIMEApp/xmlurl

* A servlet program can generate both static and dynamic web pages. In the above application all the servlet programs are designed to generate static web pages.

* The WEB-INF folder of java webapplication deployment directory structure is called as private directory because it is not visible outside the web-server and it is visible only to that web server where our java web-application is deployed. more ever the web resource programs that are placed in WEB-INF folder or in its sub folders (like classes folder) will be recognised by servlet container only when they are configured in web.xml file.

* Since we place our servlet programs in WEB-INF\classes folder so they must be configured in web.xml file.

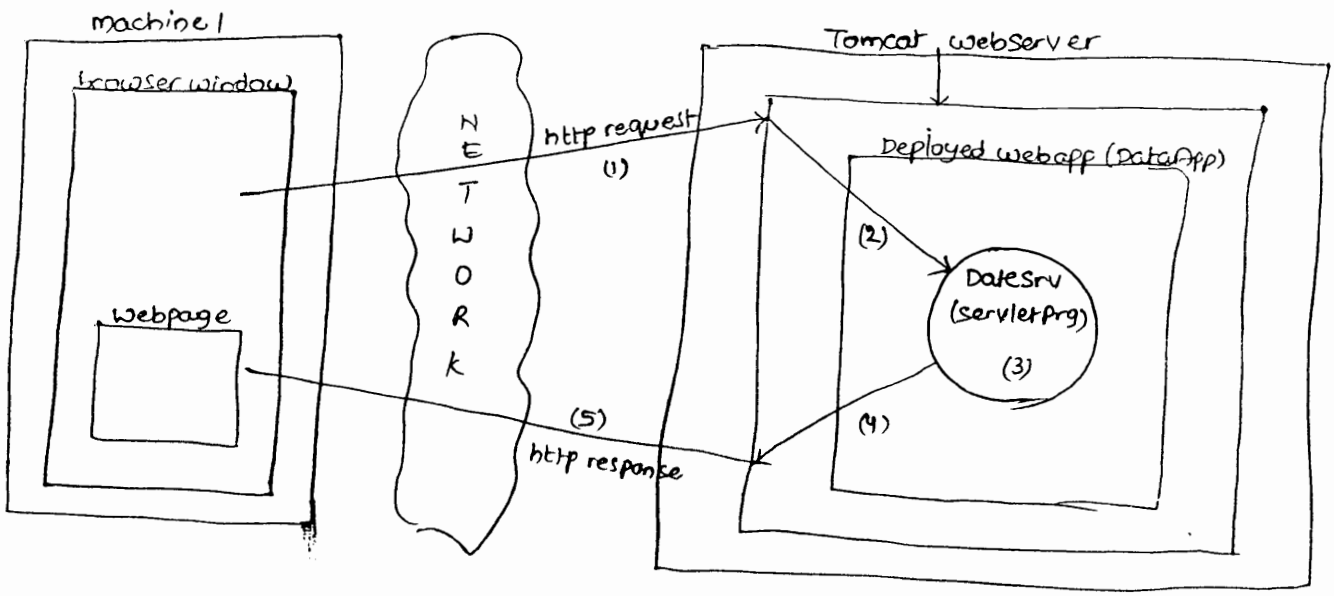
* If multiple servlet programs of a web application are configured with same pattern the last servlet in that list (web.xml order) will be executed for that url pattern based request generation.

understanding HTTP

* Protocol defines set of rules that are required to get communication between two parties.
* HTTP is a application level protocol that runs over network level protocol called TCP/IP having set of rules to get communication between browser window and web server.
* Application level protocols defines set of rules that are required to get communication between two softwares or software applications.

  Ex: HTTP, JDBC:odbc, Jdbc:oracle, SMTP and etc...
* Network level protocol defines set of rules to get communication between two physical computers of network.

  Ex: TCP/IP



Request url:

http:// machine 10 : 2020 / DateApp / test ? sno=101 & sname=raja

(query string having req params)

* The query string appended to the request URL sends data to destination web resource program (like servlet) along with the request.
* When request URL is typed in the address bar of browser window the browser window generates one http request having lots of details and we can remember these details as H^r p^r details.

H ----> http request method (GET/POST/DELETE/.....)

H ----> http request headers (like user-agent, accept, accept-language....)

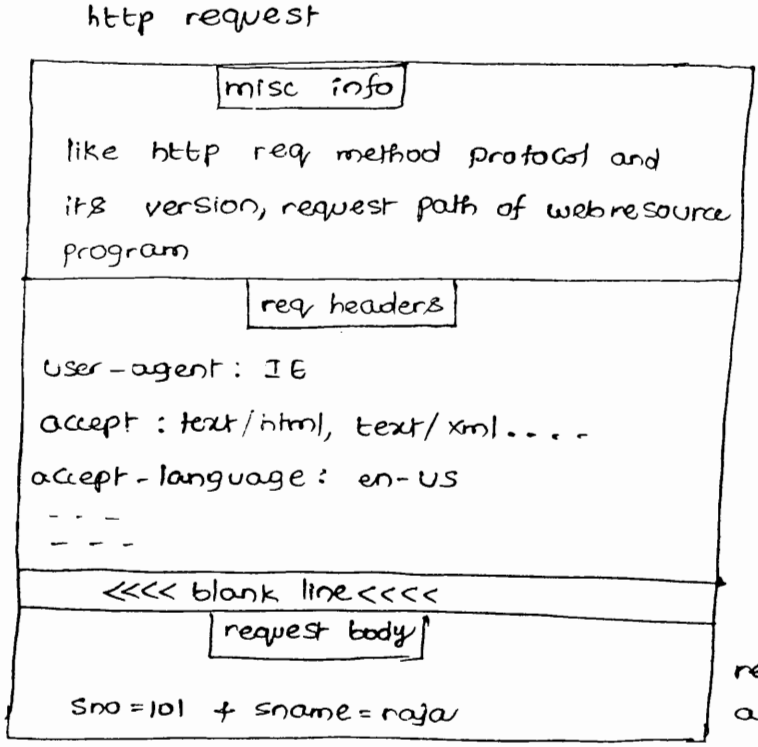
P ----> request path (http://machine10:2020/DateApp/test)

P ----> request parameters/query string (sno=101 & sname=raja)

request parameter name — request parameter value

* Browser window is responsible to generate http request having multiple details based on the request URL that is generated.

The structure of http request



request body is also called as request payload

An example http request with content

Http request method — Path to the resource on web server — Protocol & its Version

POST / DateApp / test ? Http/1.1 } - misc info

Host : www.nataraj soft.com

User-agent : mozilla/IE 5.0

Accept : text/html, text/xml, image/jpeg, application/msword, */*

Accept-language : en-us

Accept-Encoding : gzip, deflate

Accept-charset : ISO-6431
Keep-Alive : 300
Connection : Keep-Alive
} request headers

<<<< blank line <<<<<<

sno = 101 & sname = raja } request body / payload having request parameters.

* "GET" method based request can send limited amount of data along with request (max of 256 kilo bytes).
* "POST" method based request can send unlimited amount of data to server along with the request.

What is the difference between request parameters and request headers of http request ?

* Both are there to send input values to target web resource program like servlet program along with the request. The differences are

Request parameters	Request headers
* Represents end user supplied input values to web resource program.	* Represents the browser window supplied input values to web resource program.
* End user supplies these values either through form page or by appending query string in the URL.	* Browser window internally adds request headers and their values to http request automatically.
* Request parameter names and values are not fixed.	* Request header names are fixed, predefined and their values will be generated by browser window through browser settings.
* multiple request parameters can have same names.	* Request header names are unique names.
Ex : sno = 101, sname = raja and etc.	Ex: user-agent : IE accept-language : en-us

* For related information about all http request headers refer page no. 48.

* To know various details about http request page nos pages 46 to 49.
* servlet program can use all the details of http request as input values while processing the request.
* A simple servletRequest object can gather only misc info, req param values from http request given by client (browser window).
* A HttpServletRequest object of servlet program can gather all the details from http request (including headers) given by client.

Request URL

http://localhost:2020/DateApp/test?sno=101&sname=raja&sadd=hyd&sadd=vizag

req params

Different approaches of gathering req parameters values/ request body values from Http request being from servlet program

Approach 1: (by using req.getParameters(-)

Code in service(-,-) of Servlet program

String s1 = req.getParameters("sno"); ⟶ gives "101"

String s2 = req.getParameters("sname"); ⟶ gives "raja"

String s3 = req.getParameters("sadd"); ⟶ gives "hyd"

NOTE: In this approach we must know req parameter name to get its value, if request parameter is having multiple values than it gives only first value.

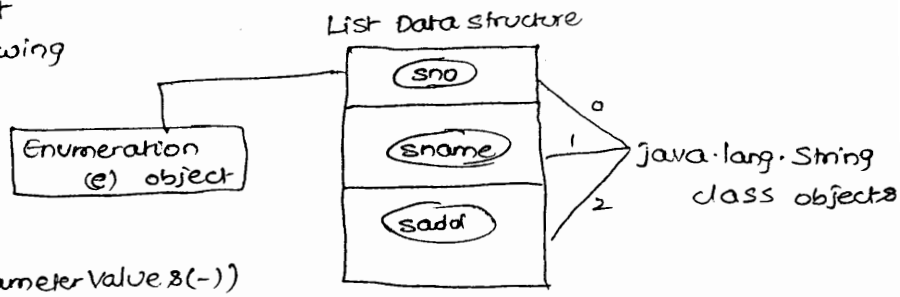
Approach(2) (by using req.getParameterNames())

Code in service(-,-) of servlet prg

```
Enumeration e = req.getParameterNames(); // given enumeration obj pointing to
                                            list data structure having req
                                            param names
while(e.hasMoreElements())
{
   String pname = (String)e.nextElement(); // gives each req param name
   String pvalue = req.getParameter(pname); // gives each req param value
   pw.println(pname + " ⟶ " + pvalue);
}//while
```

Output: sno => 101
sname => raja
sadd => hyd

* In this approach we can get req param values without knowing their names.
* If one req param is having multiple values this gives only first value.



Approach(3) (By using req.getParameterValues(-))

```
String s[] = req.getParameter("sadd");
for(int i=0; i<s.length; i++){ pw.println(s[i]+"....");} //gives hyd...vizag values
String s1 = req.getParameterValues("sadd")[0]; //gives hyd
String s2 = req.getParameterValues("sadd")[1]; //gives vizag
```

* This approach is useful to gather multiple values of single req parameter.
* To read a req param values from http request we can use either ServletRequest obj or HttpServletRequest obj.
* while opening tomcat manager through browser window if username and password related problems are raised, then add following two lines of code in Tomcat_home/user.xml

```
<role rolename="manager"/>
<user username="admin" password="admin" roles="managers"/>
```

* res.setContentType("text/html");

* According to the above statement setContentType is the method that is declared in javax.servlet.ServletResponse Interface and that method is implemented in container supplied java class implements this ServletResponse interface.
* In case of Tomcat the implementation class name is responseFacade.
* String s1=req.getParameter("sno"); → gives 101
* In the above statement getParameter is the method that is declared in predefined ServletRequest interface and implemented in servlet container supplied java class that implements ServletRequest interface (In case of tomcat this class name is RequestFacade).

Ex:

```
interface xyz
{
  public void x();
}
class Test implements xyz
{
   public void x()
    {
    }
   public void y()
    {
    }
}
xyz obj1=new Test(); // Here obj1 is reference variable of interface but not object.
   obj1.x();         // obj1 is object of implementation class Test
```

* When interface reference variable points to its implementation class object then it becomes object of implementation class and it is not the object of interface.

* So when you call method on this reference variable then interface method will not be executed but the method defined in implementation class will be executed.

* Different approaches of gathering request header values from Http request being from servlet program

Approach(1): (by using req.getHeader(-)

Code in service(-,-) of servlet Program

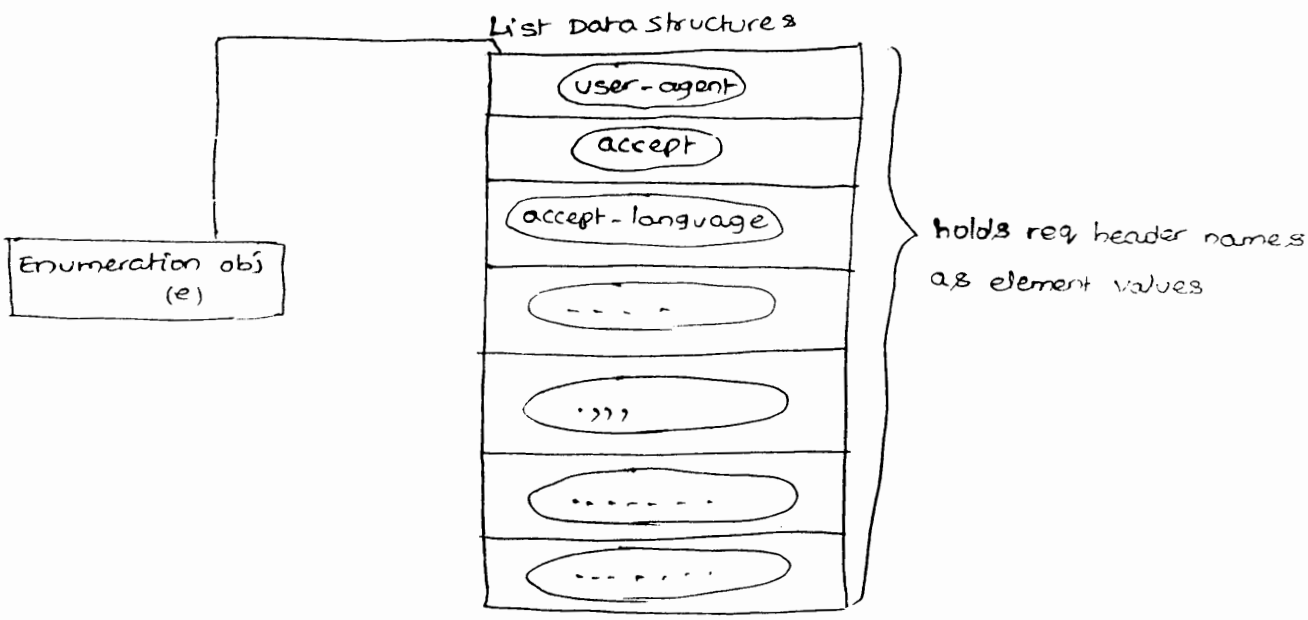
```
String s1 = req.getHeader("user-agent"); //gives browser s/w name
String s2 = req.getHeader("accept"); //gives the mime types supported by browser window like text/html, text/xml etc...
```

* In this approach we must know request header name to get its header value.

Approach(2): (by using req.getHeaderNames())

Code in service(-,-) of servlet program

```
Enumeration e = req.HeaderNames(); ①
while (e.hasMoreElements())
{
  String hname = (String) e.nextElement(); ②
  String hvalue = req.getHeader(hname); ③
  pw.println(hname + " ==> " + hvalue);
}
```



① Gives enumeration object pointing to list data structure having request header names

② Gives each request header name

③ Gives each request value.

* This code gives all request header names and values.
* The above code must be executed with the support of HttpServletRequest object.
* Through browser language setting we can set or modify accept-language request header value.

  InternetExplorer → tools menu → Internet options → languages → add → choose language.

* In Nestcape Navigator Edit → preferences → Navigator → languages → add → choose language.

How to gather current browser window software name being from servlet program:

(A) use user-agent request header that holds browser software name as shown below.

In service (-) method

```
pw.println("<br><br> the current brower window" + req.getHeader("user-agent"));
                                                              ↓
                                                        request header name
```

---

* The URL pattern of servlet program is technically called as servlet path.
* we can gather the misc info from HttpRequest by calling various getXx methods on request object as shown below.

```
//example request url : http://localhost:2020/DateApp/test?sno=101
pw.println("<br> misc info of http request");

//methods available in javax.servlet.ServletRequest (I)

pw.println("<br> req Content length:" + req.getContentLength());
// gives request data length in bytes like 345 bytes (if not known gives -1)
pw.println("<br> req Content type:" + req.getContentType());
//gives req Content type like text/html (if not known gives null)
pw.println("<br>req protocol:" + req.getProtocol());
//gives http/1.1
pw.println("<br> req scheme:" + req.getScheme());
// gives HTTP
pw.println("<br> req browser window machine IP address:"
                                        + req.getRemoteAddr());

//gives 127.0.0.1
```

```
pw.println("<br>browser window machine host name:"+req.getRemoteHost());
// gives current computer name other wise IP address
pw.println("<br>browser window port no:" + req.getRemotePort());
// gives 2922 for Nestcape, 2926 for IE
pw.println("<br>server name"+req.getServerName());
//gives the domain name typed in the URL like local host
pw.println("<br>server port"+req.getServerPort());
// gives 2020
//methods available in javax.servlet.HttpServletRequest (I)
pw.println("<br>Context path:"+req.getContextPath());
//gives DateApp
pw.println("<br>req method is:"+req.getmethod());
// GET
pw.println("<br>req path info is:"+req.getPathInfo());
//gives additional information kept in req url (otherwise null)
pw.println("<br>query String is:"+req.getQueryString());
//gives sno=101
pw.println("<br>request uri:"+req.getRequestURI());
// gives /DateApp/test
pw.println("<br>request url:"+req.getRequestURL());
//givest http://localhost:2020/DateApp/test
pw.println("<br>servlet path:"+req.getServletPath());
// givest /test
```

* servlet program can avoid browser dependent tags while generating html tags based response if by knowing browser software name through the request header user-agent.

+ when web resource program (like servlet program) sends response to browser window the generated Http Response contain multiple details including response Content. Those details can be remembered as SCH details.

S ⟶ Response status code (100 - 599)

100 - 199 ⟶ Info
200 - 299 ⟶ Success
300 - 399 ⟶ Redirection
400 - 499 ⟶ Incomplete web resource
500 - 599 ⟶ Server Error

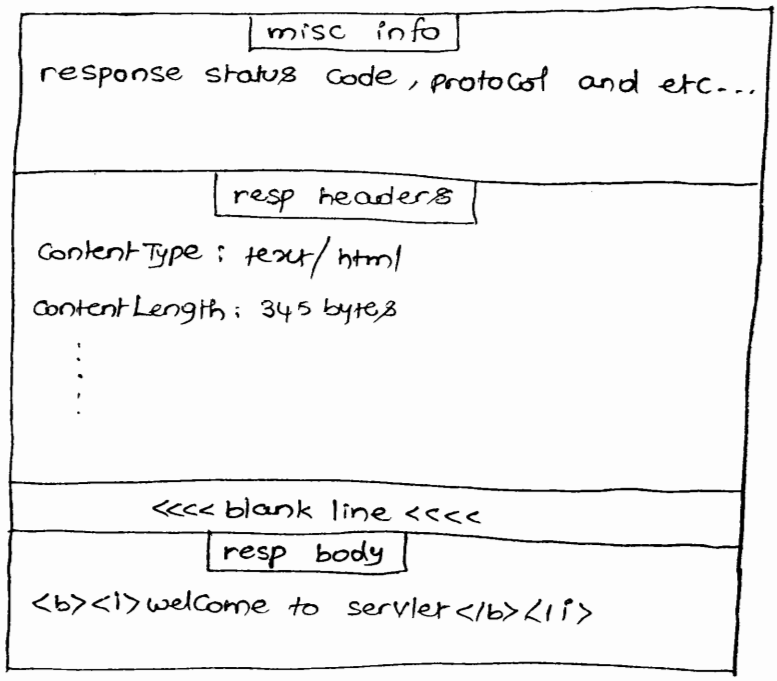
C ——→ Content type (webpage Content)

H ——→ response headers

like ContentType, ContentLength, refresh and etc.

* Response status code indicates the status of generated response to display on the browser window.
* Every generated Http response contains one http status code, default status code is 200.
* If web resource program generates warnings based web page then the status code is 100-199.
* If web resource program generates successful web page then the status code is 200-299.
* If request given to one web site is forwarded to another website then the status code will be 300-399.
* If our web resource program is incomplete or invalid to process the request then the status code will be 400-499.
* If server fails to execute our web application the status code will be 500-599.
* 400-599 are error status codes, using them the programmer can debug the problems related to web application execution and server.
* 100-399 indicates success response status codes that means they display web pages on browser window having output content so these status codes will not appear on the web pages.
* For related information on http response status codes refer page no 49 and 50.
* Response headers provide instructions to browser window through web server towards displaying web pages on the browser windows.
* The Content that we have placed in pw.println statements becomes response body http response.

Http Response Structure



resp body also called as resp content/resp payload

Http response with example content

Http/1.1 200 ok → Http status code

→ misc info

Set-Cookie : JSESSIONID = 148937A5EA179EA
Content-Type : text/html
Content-length: 1354
Date : Fri Feb 2008 8:40 34 GMT
Server : Apache-Tomcat/5.0
Connection: close

} response Headers

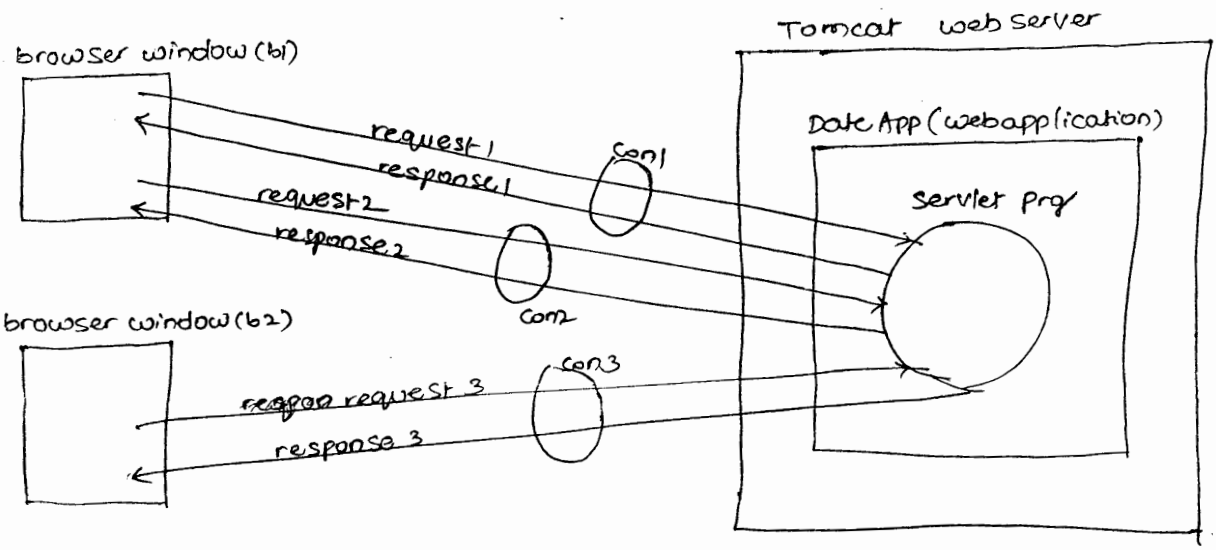
<<<< blank line <<<<

<html> <body> welcome servlet </body> </html> → response body

↓

response content/response body

* For every request generated by browser window one new connection will be created between browser window and web server and this connection will be closed automatically once request related response comes back to browser window.



* For related information on response headers refer page no's 50 and 51.

The list of all http request headers

accept, accept-encoding, authorization, connection, cookie, host, if-modified-since, referrer, user-agent, keep-alive, accept-charset and etc..

List of http response headers

Location, refresh, set-cookie, cache-control / Pragma, content-encoding, content length, ContentType, last-modified, date, server, connection and etc...

How to make browser window refreshing its web page automatically at regular intervals ?

* Give instructions to browser window from servlet through server by using response header called refresh as shown below.

```
res.setHeader("refresh", "10");
                          ↓
                    time in seconds
```

By default every web resource program generated output / response content will be stored in the buffer before it is getting displayed on browser window as web page content. Due to this browser window may show old output collected from the buffer even though the web resource program of web application is capable of generating new and updated output.

* To solve the above problem instruct browser window for not storing the output in the buffer from web resource program through web server. by using response headers as shown below.

res.setHeader("cache-control", "no-cache"); for Http 1.1 based servers

res.setHeader("pragma", "no-cache"); for Http 1.0 based servers

* Buffer is a temporory memory which stores the data for temporory period. Buffer is also called as cache.

* for related information on Http request details, Http response details refer page nos 46 - 51

* Every software and non-software objects life cycle (Considering all the operations that are taken place from object birth to object death).

* our servlet class object life cycle will be managed by Servlet Container.

* Event is an actionPerformed / raised on the object.

* servlet Container raises the following life cycle events in the life cycle of our servlet program. They are

1) Instantiation Event (raises when servlet Container creates our servlet class object)

2) RequestArrival Event (raises when ~~browser window~~ servlet container takes the browser window generated request)

3) Destruction Event (raises when servlet-Container is about to destroy our servlet class object)

* when Container raises these events on our servlet class object it looks to call certain methods automatically so these methods are called as life cycle methods or Container callback methods. They are

init(ServletConfig cg)

service(ServletRequest req, ServletResponse res)

destroy()

Prototype of servlet life cycle methods

public void service(ServletRequest req, ServletResponse res) throws ServletException, IOException

public void init(ServletConfig cg) throws ServletException

public void destroy()

* For one life cycle event only one life cycle method will be called by ServletContainer so init(), protected service(HttpServletRequest, HttpServletResponse), doXxx() methods are not life cycle methods.
* ServletContainer calls init(-) life cycle method for instantiation event.
* ServletContainer calls service(-,-) life cycle method for RequestArrival Event.
* ServletContainer calls destroy() lifecycle method for Destruction Event.
* The method that is called by underlaying container automatically based on the event that is raised on the object is called Container callback methods.
* Programmer never calls life cycle methods manually, they will be called by underlaying container automatically.
* Servlet API as supplied life cycle methods
  (i) to allow programmer to place his choice logics in the execution of servlet Programs.
  (ii) to supply Servlet container created objects to servlet program (also for Programmer as parameters of cy lifecycle methods. (like request object, response object, ServletConfig object and etc...)
* while overriding super class method in sub class, the method can have same modifier or excess modifier.
* while overriding protected service() method of predefined HttpServlet class in its sub class we can take either protected or public modifiers for that method.

```
Public class TestSrc extends HttpServlet
{
    Public void service(HttpServletRequest req, HttpServletResponse res) throws
                                                    ServletException, IOException
    {
        ==   here protected method is overridden.
    }
}
```

service() method-1

public void service(ServletRequest req, ServletResponse res) throws ServletException, IOException

service() method-2

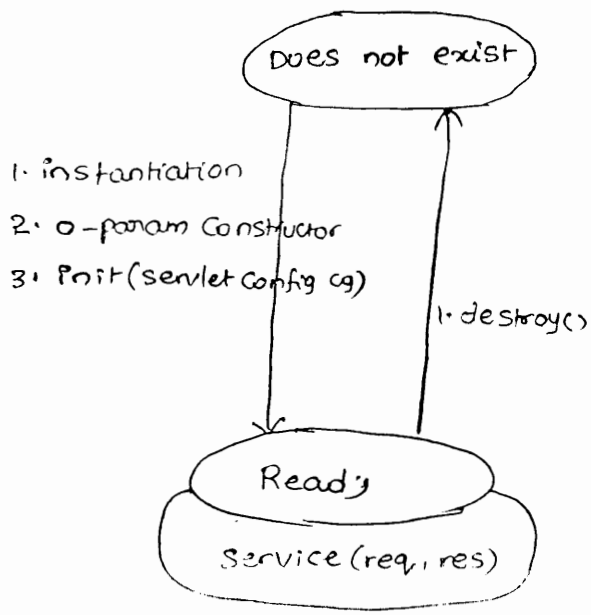
Protected void service(HttpServletRequest req, HttpServletResponse res) throws ServletException, IOException

* servlet container is responsible to raise various life cycle events on our servlet class objects so logics placed in life cycle methods are not responsible to raise these events but they are responsible to process these events by executing programmers supplied logic.

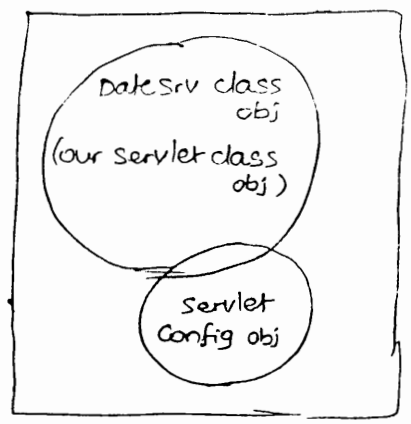
Ex: Code which is written in init() life cycle method never creates our servlet class object but programmer places this code has initialization logic to execute for instantiation event raised by servlet container by creating our servlet class object.

* Life cycle method and its logic are not capable of raising life cycle events on the object but when underlaying container raises life cycle events on the objects it automatically calls a relavent life cycle methods.

Servlet life cycle Diagram :



(1)



(2)

(1) → ServletConfig object is right hand object to our servlet class object it is one per servlet class object.

(2) → Programmer uses this object to pass additional information to our servlet program and to gather information from our servlet program.

What happens when our servlet program gets 1st request from browser window?

(1) Servlet Container loads our servlet class from WEB-INF\classes folder of deployed web application.

(2) ServletContainer instantiates (object creation) our servlet class object as Class.forName("DateSrv").newInstance();.

* class.forName("DateSrv") → loads our servlet class

* newInstance() → creates object for the loaded class DateSrv.

(3) During instantiation process the 0-param constructor of our servlet class executes.

(5) Servlet Container calls init(-) life cycle method having ServletConfig object as argument value on our servlet class object.

(4) Servlet Container creates one servlet Config object for our servlet class object.

* (1) to (5) steps completes instantiation and initialization on our servlet class object.

(6) Servlet Container calls next life cycle method service(-,-) on our servlet class object. This will process the request and generated response goes to browser window as web page through web server.

What happens when our servlet program gets other than first request?

* Servlet Container checks the availability of our servlet class object.

(1) if available, servlet container calls service(-,-) (Public) on existing object of our servlet class to process the request and this response goes to browser window.

(2) if not available, servlet Container performs all operations of 1st request.

Example servlet program to understand the life cycle of servlet

Adding this servlet program to MIMEAPP web application.

Step(1): Add LCTestSrv.java in WEB-INF/classes folder of deployed MIMEApp web application.

```
//LCTestSrv.java
package P;
import javax.servlet.*;
import javax.servlet.http.*;
import java.io.*;
public class LCTestSrv extends HttpServlet
{
   public LCTestSrv()
    {
      System.out.println("LCTestSrv: 0-param Constructor");
    }
   public void init(ServletConfig cg)
    {
      System.out.println("LCTestSrv: init(-)");
    }
   public void service(ServletRequest req, ServletResponse res) throws
                                          ServletException, IOException
    {
      System.out.println("LCTestSrv: service(-,-)");
      PrintWriter pw = req.getWriter();
      req.setContentType("text/html");
      //write business logic
       pw.println("Date and time is "+new java.util.Date().toString());
      //close stream objects
       pw.close();
    }//service(-,-)
   public void destroy()
    {
      sop("LCTestSrv: destroy()");
    }
}//class
```

* Compile the servlet program source code.

D:\Tomcat 6.0\webapps\MIMEApp\WEB-INF\classes> javac -d . LCTestSrv.java

Step(2): Configure the above servlet program in web.xml having url pattern.

In web.xml (already some code is there it is added to that one)

```
<servlet>
   <servlet-name> abc4 </servlet-name>
   <servlet-class> p.LCTestSrv </servlet-class>
</servlet>
   <servlet-mapping>
     <servlet-name> abc4 </servlet-name>
     <url-pattern> /lck/url-pattern>
   </servlet-mapping>
```

Request URL to test App is

* The servlet Program is there in package that must be specified in web.xml while configuring that servlet program.

Step(3): Start the server

* once webServer is down all objects created and managed by that Server will be destroyed automatically (including our servlet class objects).

* init(-) is a one time execution block of servlet Program life cycle.

* This method executes when servlet container raises instantiation event on our servlet class so the programmer generally keeps initialization logic like creating JDBC connection in this method.

* service(-,-) method is repeatedly executing block of servlet program life cycle. Servlet Container calls this method for every request given to servlet Program. So programmer generally keeps request processing and response generation logics in this method.

* destroy( ) is the one time execution block of our servlet program life cycle. Servlet Container calls this method when it is about to destroy our servlet class object. So programmer generally keeps uninitialization logic like closing JDBC Connection in this method.

When does servlet container creates our servlet class object?

(A) When <load-on-startup> is not enabled on servlet program

(a) When servlet program gets first request from client.

(b) When servlet program gets first request after restarting web application.

(c) When servlet program gets first request after reloading the web application.

(d) When servlet program gets first request after restarting the server.

(e) When servlet program gets first request after redeploying the web application.

When <load-on-startup> is enabled

(a) The servlet container creates our servlet class object either during server startup or during the deployment of the web application.

Example code in web.xml

```
<servlet>
    <servlet-name> abc4 </servlet-name>
    <servlet-class> P.LCTestSrv </servlet-class>
    <load-on-startup> 5 </load-on-startup>
</servlet>                    └→ load-on-startup > priority value
<servlet-mapping>
    <servlet-name> abc4 </servlet-name>
    <url-pattern> /lc </url-pattern>
</servlet-mapping>
```

When servlet container destroys our servlet class object through garbage collector?

(a) When server is stopped or restarted.

(b) When web webapplication is stopped or reloaded.

(c) When webapplication is undeployed.

(d) When our servlet object sits idle continuously for long time.

NOTE: No special priority towards destroying our servlet class objects even though load-on-startup is enabled on the servlets.

Why should we enable load-on-startup on servlet program?

The first request given to servlet goes to service() method only after the completion of instantiation and initialization process whereas other

than first request coming to servlet program go to service(-,-) method directly. Due to this the request processing and response generation time of first request will be little bit high when compare to other than first request.

To minimize first request request processing, response generation time and equalise that time with other than first request make servlet container to complete instantiation and initialization operations on servlet program before first request either during server start up or during deployment of web application by enabling <load-on-startup> on servlet program.

NOTE: Don't enable <load-on-startup> on every servlet program. Enable it only on those servlet programs of web application which will be gaurenteely requested by clients immediately after deploying the web applications like the servlet programs which generate home pages or main menus and etc..

---

* when multiple servlet programs of web application are enabled with <load-on-startup> in which order the servlet container creates those servlet program objects will be decided based on their <load-on-startup> priority values.
* High value indicates low priority. Low value indicates high priority

NOTE: <load-on-startup> with negative value is equal to not enabling <load-on-startup> on servlet.

Servlet prgs of WA1 web application

	<l-o-s>
Srv 1	1 (I)
Srv 2	3 (III)
Srv 3	2 (II)
Srv 4	10 (IV)

Servlet prgs of WA2 webapplication

	<l-o-s>
Srv1	11 (III)
Srv2	0 (I)
Srv3	2 (II)
Srv4	-1 (Ignores <l-o-s>)

Servlet prgs of WA3 web application

	<l-o-s>	
Srv 1	4	Server decides the order
Srv 2	4	
Srv 3	4	

* In Tomcat 5, 5.5 versions "0" is having least priority towards <load-on-startup> priority values. where as in tomcat 6.0 "0" is having first priority or higher priority as shown above.

* If you keep <load-on-startup> tag with out any priority value then that servlet program gets first / higher priority compare to other servlet programs whose <load-on-startup> tags are there with values.

Ex: <load-on-startup></load-on-startup>

↓

This tag <load-on-startup> priority value is zero.

* when no value is passed explicitly as priority value then the <load-on-startup> tag takes "0" as the priority value.

HTML to Servlet Communication:

* So far we have sent request to servlet program from browser window by typing request URL in the browser window. In this process to send data along with request we need to add query string to the request URL explicitly. But this work can be done only by technical people. The non technical end users like civil engineers, chemical engineers, kids can't do this work. So they need a graphical user interface to generate request with or with out data. For that purpose we can use either HTML form page or hyperlink to generate request with or with out data.

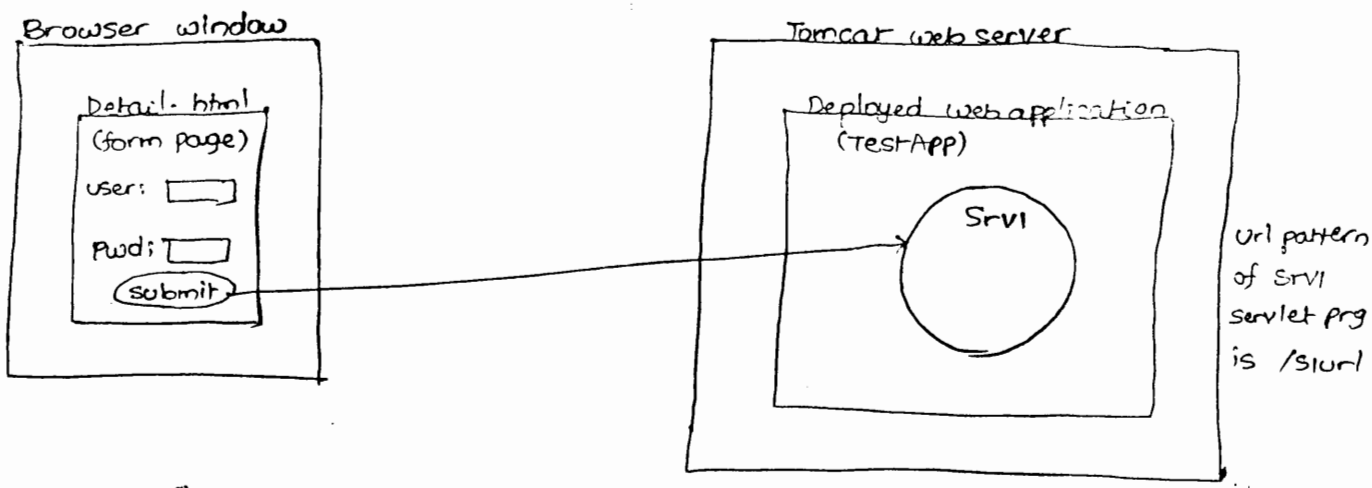
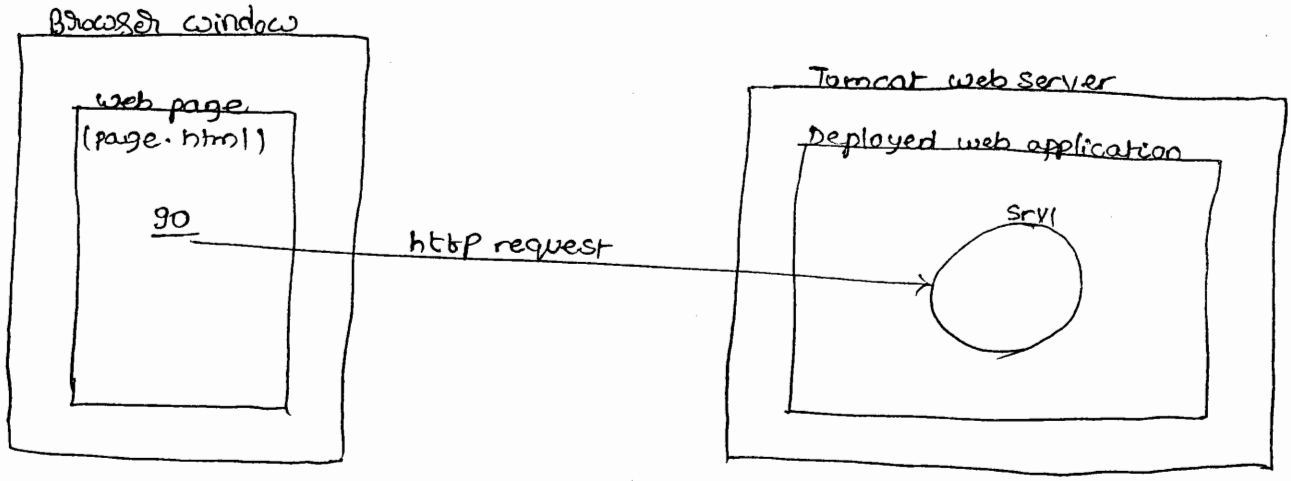


fig: Form page to servlet Communication

* The form page generated request carries form data as request parameters along with request.

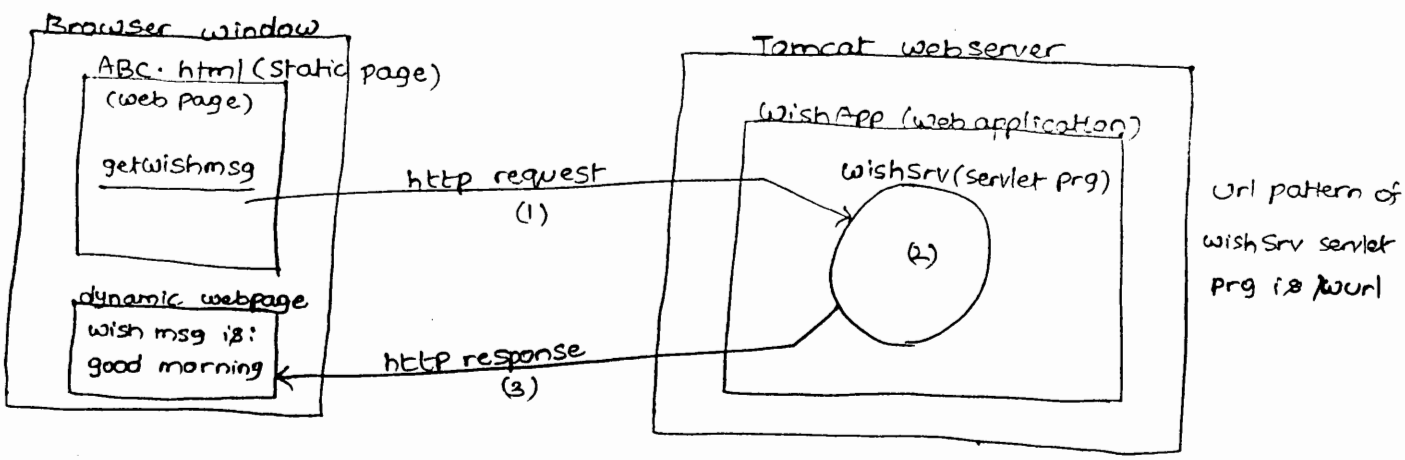
Hyperlink based webpage to servlet communication



url pattern of srv1 servlet prg is \s1url

* Generally the hyperlink generated request is blank request that means it can not carry any data along with the request.
* .html files of web application must be placed parallel to WEB-INF folder in deployment directory structure of web application. There is no need of configuring them in web.xml file.

Example application (hyperlink based web page servlet program Communication)

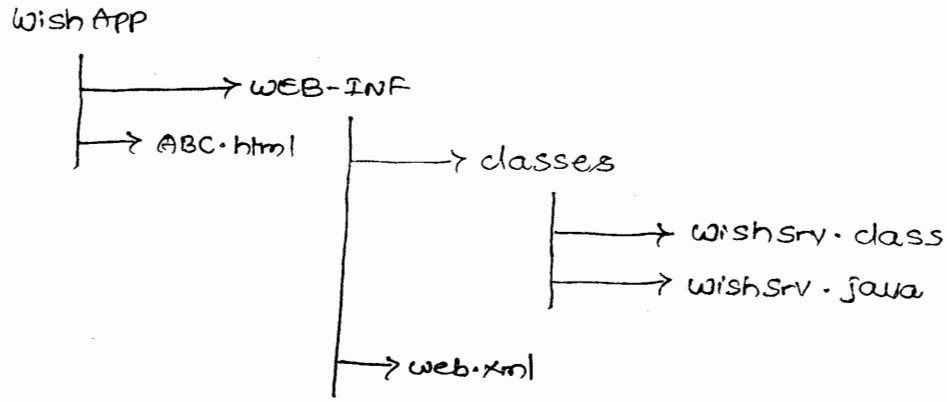


url pattern of wishSrv servlet prg is /wurl

* WishSrv servlet program generates the wish message based on the server machine's current time.

* .html based web pages are static pages always, whereas servlet, JSP programs based web pages can be static pages or dynamic pages.

Deployment Directory structure



* place servlet program request URL with URL pattern as the value of h-ref attribute.

↓

in <a> tag to make hyper link generate request communicating with servlet program.

ABC.html

```
<!-- web page having hyper links -->
<a href = "http://localhost:2020/wishApp/wurl"> get wishmsg </a>
```

↓ url pattern of wishSrv servlet program

WishSrv.java

```
//wishSrv.java
import javax.servlet.*;
import javax.servlet.http.*;
import java.io.*; import java.util.*;

Public class wishSrv extends HttpServlet
 {
    Public void service(HttpServletRequest req, HttpServletResponse res)
                                        throws ServletException, IOException.
     {
        // set response content type
         res.setContentType("text/html");
```

```
                //get printWriter obj
                PrintWriter pw = res.getWriter();
                //write request processing logic to generate wish message
                Calender cl = Calender.getInstance(); //gives current date
                                                        and time
                //get current hour of the day
                int h = cl.get(Calender.HOUR_OF_DAY); //in 24hrs format
                //generate wish message
                if (h<12)
                   pw.println("Good morning");
                else if (h<16)
                   pw.println("Good afternoon");
                else if (h<20)
                    pw.println("Good evening");
                else
                    pw.println("Good night");
                //close stream obj
                pw.close();
            } //service(-,-)
        } //class
```

web.xml

```
<web-app>
    <servlet>
        <servlet-name>abc</servlet-name>
        <servlet-class>wishSrv</servlet-class>
    </servlet>
    <servlet-mapping>
        <servlet-name>abc</servlet-name>
        <surl-pattern>/wurl</url-pattern>
    </servlet-mapping>
</web-app>
```

NOTE: Do observe that html program is not configured in web.xml

Request URL to test the application

http://localhost:2020/wishApp/ABC.html

* In absolute request URL all details will be specified including URL pattern or identification name of the target web resource program.

Ex: In ABC.html

http://localhost:2020/wishApp/ABC.html

* In relative request URL either URL pattern or identification name of target web resource program will be specified.

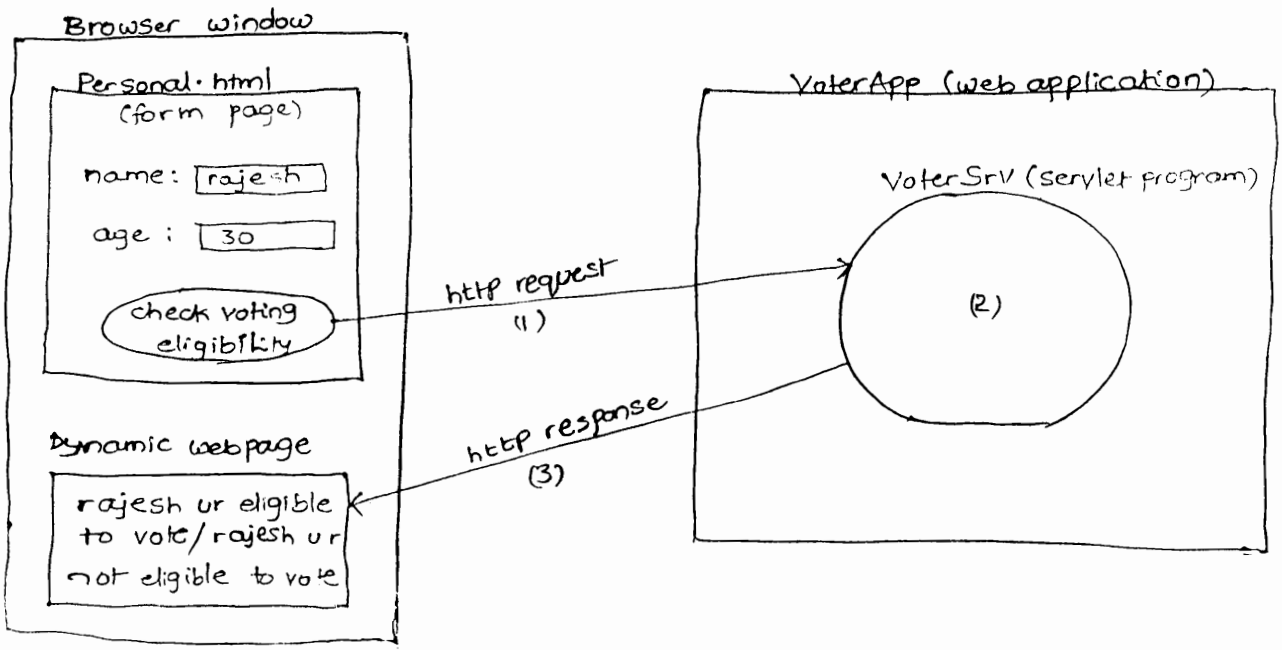
Ex: In ABC.html

getwishmsg

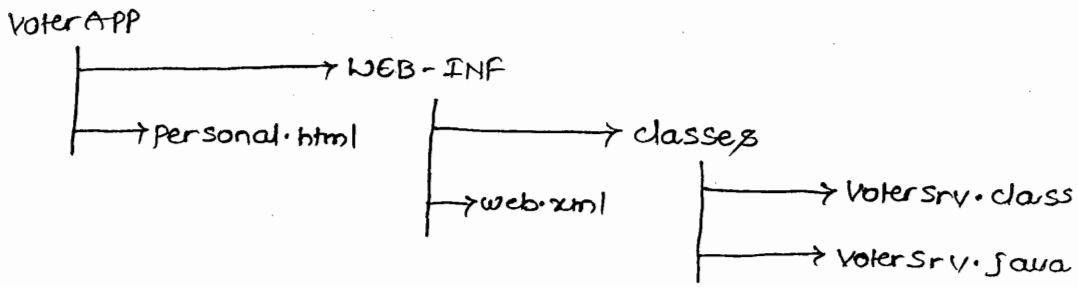
* If source and destination web resource programs are there in the same web application then it is recommanded to relative URL. otherwise it is recommanded to use absolute URL.

* To make web applications as "WODA" applications it is recommanded to use relative URLs.

Example application on form page to servlet communication



deployment directory structure



personal.html

```
<!-- form page -->
<form action="vtUrl" method="get">
```

vtUrl ↓ URL pattern of VoterSrv servlet program (relative URL)

```
Name : <input type="text" name="pname"><br>
Age : <input type="text" name="page"><br>
<input type="submit" value="check Voting Eligibility">
</form>
```

(pname) ← name of the component

(pname, page)

* When form page is submitted the form component names will go to server as request parameter names and the form component values will go to server as request parameter values.
* The "get" method based request generated by form page is recommended to process by using doGet(-,-) method in servlet program. Similarly use doPost(-,-) method to process "post" method based request generated by form page.
* Since doXxx() methods of servlet api are designed based on HTTP standards so it is recommended to override doXxx() methods in our servlet programs to process the request.

VoterSrv.java

```
// VoterSrv.java
import javax.servlet.*;
import javax.servlet.http.*;
import java.io.*;
```

```
public class VoterSrv extends HttpServlet
{
  public void doGet(HttpServletRequest req, HttpServletResponse res)
                                  throws ServletException, IOException
  {
    //get printWriter obj
    PrintWriter pw = res.getWriter();
    //set response content type
    res.setContentType("text/html");
    //read form data from form page as request parameter values
    String s1 = req.getParameter("pname");
    String s2 = req.getParameter("page");      } Form components acting as request parameter
    //convert age value to numeric value
    int age = Integer.parseInt(s2.trim());
    //write logic to generate dynamic web page
    if(age >=18)
      pw.println("<font color='green' size='4'>"+s1+"U r eligible to
                                                vote </font>");
    else
      pw.println("<font color='green' size='4'>"+s1+" ur not eligible
                                                to vote </font>");
    //add hyperlink to dynamic web page
    pw.println("<br><br><a href='personal.html'> home </a>");
  }//doGet(-,-)
}//class
```

relative URL

web.xml

configure VoterSrv servlet program with "/vturl" URL pattern

Request URL to test the application

http://localhost:2020/voterApp/personal.html

* To make our servlet program as flexible servlet program working for both "get", "post" http request methods we can use one of the following three approaches.

Approach (1) : By keeping service(-,-) method in our servlet program

```
Public class VoterSrv extends HttpServlet
 {
    public void service (HttpServletRequest req, HttpServletResponse res)
                                    throws ServletException, IOException
     {
        ------
        ------  //request processing logic
        ------
     }
 }
```

NOTE: The service(-,-) method of our servlet program can process both "get", "post" methods based request. But keeping request processing logic in service(-,-) method is not industry standard so try to keep request processing logic in doXxx methods.

Approach (2): By overriding both doGet(-,-), doPost(-,-) methods but keep request processing logic in one method and call that method from another method

```
public class VoterSrv extends HttpServlet
 {
    public void doGet (HttpServletRequest req, HttpServletResponse res)
                                    throws ServletException, IOException
     {
        ------
        ------  //request processing logic
        ------
     }
    public void doPost(HttpServletRequest req, HttpServletResponse res)
                                    throws ServletException, IOException
     {
        doGet(req, res);
     }
 }
```

NOTE: Here servlet program can process both Get, Post method based request. Here request processing logic is not duplicated.

Approach (3): keep request processing logic in a user-defined method and call that method from both doGet(-,-), doPost(-,-) methods

```
Public class VoterSrv extends HttpServlet
{
    public void xyz(HttpServletRequest req, HttpServletResponse res)
                                    throws ServletException, IOException
    {
        ======  // request processing logic
    }
    public void doGet(HttpServletRequest req, HttpServletResponse)
                                    throws ServletException, IOException
    {
        xyz(req, res);
    }
    public void doPost(HttpServletRequest req, HttpServletResponse res)
                                    throws ServletException, IOException
    {
        xyz(req, res);
    }
}
```

* Here the key benifits are same as approach (2).
* Programmers generally prefer using approach (2) or approach (3) to make their servlet programs as flexible servlet programs against the http request methods "get", "post".

What happens if programmer calls destroy() method explicitly from the service(-,-) method of servlet program?

a) servlet container will not destroy our servlet class object but logic of destroy() method executes along with service(-,-) method.
* When life cycle event is raised the servlet container calls life cycle method. Since the programmer has called life cycle method automatically

the servlet container never raises the life cycle event.

What happens if init(-) method is called explicitly from the service(-,-) method of servlet program?

(A) servlet container never creates new object of our servlet class for the above call but logic of init(-) method executes along with service(-,-) method.

What is the difference between HttpRequest method GET and POST?

GET	POST
* Design to gather data from server by generating request.	* Design to send data to the server along with the request.
* GET send limited amount of data along with the request. (max of 256 kb)	* It can send unlimited amount of data along with the request.
* The form page generated query string will appear in browser's address bar so no data secracy.	* The form page generated query string will not appear in browser's address bar. so data secracy is available
* Not suitable for file uploading operations.	* suitable.
* Can not send data in encrypted format.	* can send.
* use doGet() method or service(-,-) method to process the request	* use doPost() method or service(-,-) method to process the request.
* Get is not idempotent	* Post is not idempotent
* Default request method of HttpRequest.	* is not default and should be applied explicitly.

* It is recommanded to design form pages by having Post method.
* Hyperlink generated requests are get method based requests where as the form page can generate either get or post method based request.
* Before completing the request processing of given request if client is

allowed to generate next request and web application is processing both the request then it is called as double posting problem or idempotent problem.

* when form page submit button is clicked for multiple times this problem may raise. To prevent this problem take request method as Post and process that request in doPost(---) method with additional logics. This indicates Post is not idempotent because it can prevent double posting by canceling all the requests.
* The first web page of web application that comes automatically when request is given to web application is called as welcome page or home page of web application.
* In java based web applications the html or jsp programs can be configured as welcome pages.

Ex: In web.xml of VoterApp web application

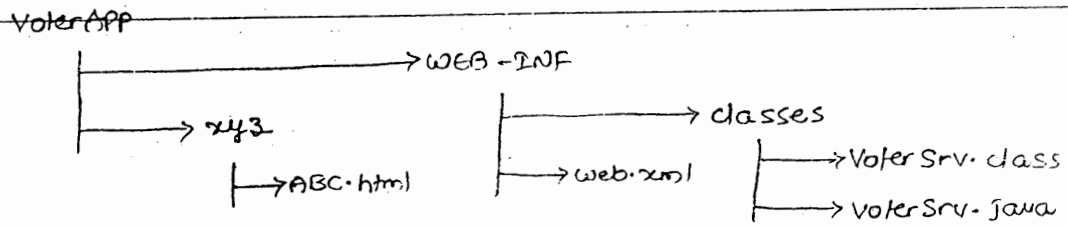
```
<welcome-file-list>  → must be taken as subtag of <web-app> tag
    <welcome-file> Personal.html </welcome-file>
    <welcome-file> Personal.htm </welcome-file>
    <welcome-file> ABC.htm </welcome-file>
    <welcome-file> ABC.jsp </welcome-file>
</welcome-file-list>
```

} only one file will become as welcome file here.

* when no welcome file is explicitly configured the java web application looks to take either index.jsp or index.html as default welcome file.
* If both are available the index.html will be taken as default welcome file.
* If multiple welcome files are configured then the web application picks up one welcome file based on the availability and configuration order.
* If all explicitly configured welcome files are not available and if index.html or index.jsp files are available then web application runs without welcome file.

NOTE: we can not configure a servlet program as welcome file (in Tomcat 5.0, 6.0)

* The html files of web application can be placed in the sub directories of web root folder. while configuring these html files as welcome files we must specify that folder name.



In web.xml

```
<welcome-file-list>
    <welcome-file> xyz\ABC.html </welcome-file>
</welcome-file-list>
```

* Tomcat 6.0,5.5 server allows to configure servlet program as welcome file by specifying its URL pattern.

In web.xml

```
<welcome-file-list>
    <welcome-file> test </welcome-file>
</welcome-file-list>
```

test → URL pattern of servlet program

* To make our servlet program related java class visible and accessable to servlet container, the java class must be taken as public class.

What happens if I place main(-) method in our servlet program?

(A) servlet program execution will be taken care by servlet container through life cycle methods. Since main(-) is not the life cycle method of servlet program so it will not called by servlet container. only in stand alone applications we need main(-) method to begin the application execution. The container software managed programs or components will be executed through life cycle methods so main(-) method is not required in that components or programs.

How Servlet program is executing without main(-) method?

(A) The stand alone application that will be executed by JVM directly needs to have main(-) method to begin the execution. Since servlet program is not stand alone application and it is a web resource program which will be executed by servlet container through life cycle methods so there is no need of main(-) method in servlet program.

* If already running java application wants to create certain other java class object and wants to call methods of that class then that other class need not have main(-) method.
* Servlet Container is a continuously running java application/software which creates our servlet class object and calls life cycle methods on that object for executing servlet program. So our servlet program need not to have main(-) method.

—•—

NOTE: Since we never give our servlet program to JVM directly for execution so there is no need of placing main(-) method in our servlet program.

* Verifying the pattern and format of the form data before it is getting used in business logic as input values is called as form validation, and logic written for this is called as form validation logic.

Ex: checking whether required components are filled up with values or not, checking whether age value is given as numeric value or not, checking whether email id is having @, . symbols or not.

What is the difference between form validation logic and business logic?

(A) Business logic uses the form data as input values and generates the results where as form validation logic verifies the pattern and format of the form data.

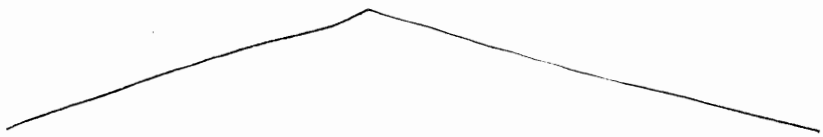
Ex(1): checking whether a, b values are typed or not and checking whether they are typed as numeric values or not comes under form validation

* performing c = a+b operation comes under business logic.

Ex(2): * checking whether username and password values are typed or not comes under form validation.

* checking given username and password values against db s/w, username and password details comes under business logic.

Form validations in web applications



Client side

Use VBScript code or Java script code in form page

Server side

write java code in servlet/ JSP programs before b.logic.

* Since script code that is embed with form page executes by coming to browser window we can say client side form validations are good compare to server side form validations because the client side form validation reduces network round trips between browser window and web server.
* Since there is a chance of disabling script code execution through browser settings it is recommanded to write both client side and server side form validations so that there is a gaurentee of performing server side form validations even though client side form validations are not done.
* To disable script code execution in IE

  Tools → internet options → security tab → custom level → scripting → active scripting → disable
* In netscap navigator

  Edit → Preferences → Advanced → Scripts & plugins → Enable java script for Navigator (deselet)
* return statement without value in any java method definition removes the control from current method definition and further statements in that method definition will not be executed.

* For example application that performs both client and server side - form validations refer supplementary hand out of given on 13-10-2011

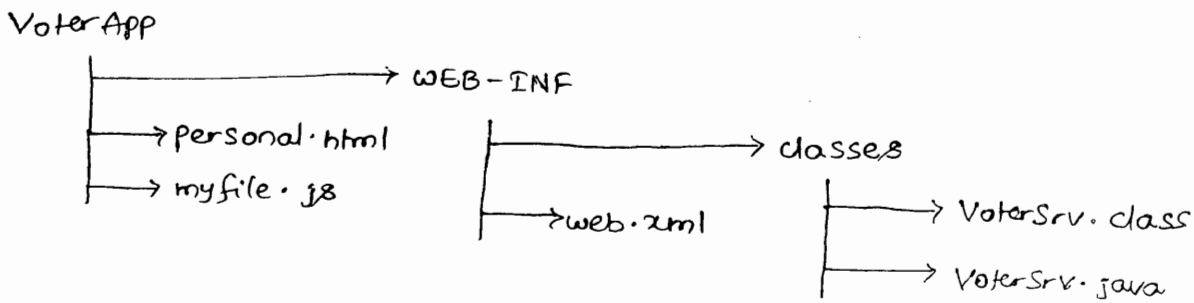
<form action = "vturl" method = "post" onsubmit = "return validate(this)">

* In the above statement the return key word takes the return value of validate function (true or false). If it is true the browser and sends to browser window.

* If that value is true then browser window sends the request to the requested web resource program of web application. If the value is false then the browser window blocks / stops request going to server.
  - true → No validation errors
  - false → validation errors are there.

NOTE: on safe side the software industry writes both client side and server side form validations as shown in the hand out example.

* To hide java script code from end users through view source option. Then and to make javascript code as reusable code for multiple html programs of web application then keep javascript code in .js file and link with html programs as shown below.



Personal.html

```
<!-- form page -->
<html>
    <head>
        <script language = "JavaScript" src = "myfile.js">
        </script>
    </head>
    <form action = "vturl" method = "post" onSubmit = "return validate(this)">
    </form>
```

myfile.js

```
function validate (frm)
{
    ---
    ---        //refer hand out code
    ---
}//validate (-)
```

* The code that can not be executed independently and it must be embedded along with other technology based program is called as script code.
* Javascript code can not be executed independently and it must be embedded with HTML program for execution. so the Javascript code is called as script code.

What is the difference between Java and Javascript?

NOTE: Javascript is no way related with java

Java	Javascript
* It is a programming language to develop all kinds of applications.	* It is a scripting language given for form validations and for other operations.
* Java application code can be executed independently.	* JavaScript code must be embedded with html code for execution.
* Needs JRE/JVm for execution.	* Needs Java script engine for execution.
* It is object oriented language.	* It is object based language.

```
Public class Test
{
    Public void x() (b)
    {
        y(); (c)
    }
    public void y()
    {
    }
}
```

```
public class Demo extends Test
  {
      public void y()
        { (d)
          ---
          ---
        }
  }

    Demo d= new Demo();
    d.x(); (a)
```

Understanding init() vs init(ServletConfig cg) methods

* init(ServletConfig cg) is lifecycle method of servlet program and init() is not lifecycle method and it is convienence method given to programmer to place initialization logic of servlet program.

Scenario (1) :

GenericServlet.java (Pre-defined class)

```
public abstract class GenericServlet implements Servlet
  {
      ServletConfig cg;
      public void init(ServletConfig cg)
       { (4)
         this.cg=cg; //initialization logic of ServletConfig obj
         init();
       }
      public void init() //empty method
       {
       }
        public ServletConfig getServletConfig() { return cg; }
        ---------- //other methods
        ----------
  }
```

NOTE: In predefined HttpServlet class there are no init() methods.

(1) (2) (3)

TestSrv.java (our servlet program)

```
public class TestSrv extends GenericServlet/HttpServlet
  {
      public void init()
        { (5)
          ---
          ---   Programmer choice initialization logic like creating jdbc
                Connection obj
        }
```

```
public void service(SReq req, SRes res) throws SE, IOE
    {  (6)
       ____
       ____   request processing logic
       ____
    }
}
```

* When servlet Container raises instantiation event by creating our servlet class object then Container calls init(-) method but not init() method. So init(-) method is called as life cycle method and init() method is not life cycle method.

* With respect to the above Code

(1) End user gives first request to our servlet program.

(2) ServletContainer creates our servlet class object (TestSrv) and also creates one ServletConfig object.

(3) ServletContainer calls init(-) method as life cycle method by keeping ServletConfig object as argument value, on our servlet class object.

(4) Since init(-) is not available in our servlet program the super class (predefined GenericServlet class) init(-) method executes.

(5) The super class init(-) method initializes the received ServletConfig object in GenericServlet class and calls init() method. Since init() method is available in our servlet program that will be executed. Here programmer places his servlet program related initialization logic.

(6) Servlet Container calls next lifecycle method called public void service(-,-) on our servlet class object.

NOTE: In scenario (1) programmer need not to initialise the ServletConfig object explicitly in his servlet program. More ever we can call getServletConfig() method to get access to servletConfig object from any life cycle method of his servlet program.

Scenario (2)

GenericServlet.java

```
==  // Same as above code in scenario (1)
==
=
```

```
public abstract class GenericServlet implements Servlet
 {
        ServletConfig cg;
        public void init(ServletConfig cg)
          {
            this.cg = cg;
            init();
          }
        public void init()
          {
          }
        Public ServletConfig getServletConfig()
          { return cg;
          }
        ---- //other methods.
 }
```

(1) (2) (3)

TestSrv.java (our servlet prg)

```
Public class TestSrv extends GenericServlet/HttpServlet
 {
        ServletConfig cg;
        public void init(ServletConfig cg)
          { (4)
            this.cg = cg;
            ---- Programmer choice initialization logic like creating
            ---- jdbc connection obj
          }
        Public void service(SReq req, SRes res) throws SE, IOE
          { (5)
            ---- request processing logic
          }
 }
```

* Here (1), (2), (3) are same as scenario (1)

(4) → init(-) method of servlet program executes having initialization logic

(5) → same as step(6) of scenario (1).

NOTE (1): In scenario (2) the programmer must explicitly initialize ServletConfig object from init(-) method to give access to ServletConfig

object from other lifecycle methods (refer above). If programmer forgets to do this ServletConfig object initialization then the service() destroy() lifecycle methods can not get access to ServletConfig object.

NOTE(2): In scenario (2) control did not pass on to init(-) method of super class (predefined GenericServlet class) so our servlet program can not call getServletConfig() method to get ServletConfig object.

Scenario (3)

GenericServlet.java (predefined servlet)

```
Public class GenericServlet implements Servlet
 {
    ServletConfig cg;
    Public void init(ServletConfig cg)
     {
         this.cg=cg; (6)
         init();
     }
     Public void init()
     { (7)
     }
     Public ServletConfig getServletConfig()
     {
        return cg;
     }
 }        ====  //other methods
```

TestSrv.java (our servlet program)

```
(1) (2) (3)
Public class TestSrv extends HttpServlet/GenericServlet
 {
    public void init(ServletConfig cg)
     { (4)
       ========  // Programmer choice initialization logic like create jdbc
       ========                                              Connection

       super.init(cg); (5)
     }
    Public void service(ServletRequest req, ServletResponse res) throws SE, IOE
     } (8)
       ========  //request processing logic
       ========
     }
 }
```

(1), (2), (3) steps of Scenario (3) are same as (1), (2), (3) steps of scenario(2)

(4) init(-) method of our servlet program executes.

(5) Because of super.init(cg) method call the super class (GenericServlet) init(-) method executes.

(6) init(-) method of GenericServlet class initializes the ServletConfig object Contains servletConfig initialization logic

(7) Since init() method is not available in our servlet init() method of GenericServlet class will execute.

(8) same as step (6) of Scenario (1)

NOTE: In scenario (3)

(i) Programmer should call super.init(cg) method in the definition of init(-) method of our servlet program and he can use getServletConfig() method call to get servletConfig object.

(ii) If Programmer forgets to call super.init(cg) method the service(-,-), destroy() lifecycle methods can not get access to ServletConfig object even after calling getServletConfig() method.

* The best scenario to keep init() method in our servlet program is scenario (1) (overriding init() method).

* Always give chance to execute init(-) method of pre defined GenericServlet class during our servlet class instantiation and initialization process. Because this method contains servletConfig initialization logic and makes programmer not performing that work explicitly.

Why servlet API has given init() method when it is not lifecycle method ?

=> Refer the 4 paragraphs of init() Vs init(ServletConfig cg) discussion of page no. 55 of booklet.

---

* A good servlet programmer always overrides init() method in his servlet program instead of init(-) method (follow scenario (1)).

* self class methods and super class public/protected methods in sub class can be called without object.

In any life cycle method of servlet program

```
ServletConfig cg = getServletConfig(); //gives access to ServletConfig obj
```

↓

It is public method of GenericServlet class so it can be called without object in our servlet programs.

understanding two service(-,-) methods and seven doXxx(-,-) methods of predefined HttpServlet class

* we can use one of the two service(-,-) methods or one of the seven doXxx(-,-) methods to place request processing logic in our servlet program but only public service(-,-) method is life cycle method of servlet program.

HttpServlet.java (Predefined class)

```
Public abstract class HttpServlet extends GenericServlet
 {
   public void service(ServletRequest req, ServletResponse) throws SE, IOE
     { (4)
       // type casting
        HttpServletRequest hreq = (HttpServletRequest)req;
        HttpServletResponse hres = (HttpServletResponse)res;
       //calls protected service(-,-) method
         service(hreq, hres);
     }
    //protected service(-,-) / service(-,-) method2
     protected void service(HttpServletRequest hreq, HttpServletResponse hres)
                                                    throws SE, IOE
     { (5)
     //reads the request method of given request
       String method = req.getMethod();
     //calls one of 7 doXxx(-,-) methods based req method
       if(method.equals("GET")
            doGet(hreq, hres);
       else if(method.equals("Post"))
            doPost(hreq, hres);
       else if(method.equals("head"))
            doHead(hreq, hres);
```

```
    }
    protected void doGet(HttpServletRequest hreq, HttpServletRequest hres)
                                                    throws SE, IOE
    {
        ------
        ------   logic send 405 error response to browser window
        ------
    }
    protected void doPost(HttpServletRequest hreq, HttpServletResponse hres)
                                                    throws SE, IOE
    {
        ------
        ------   logic send 405 error response to browser window
        ------
    }
    ------
    ------   // implementation of other doXxx(-,-) methods sending 405
    ------      error response to browser window.

}// class
```

TestSrv.java (our Servlet program)

```
(1) (2) (3)
public class TestSrv extends HttpServlet
{
    public void doGet(HttpServletRequest req, HttpServletResponse res)
    {  (6)
        ------
        ------   our request processing logic
        ------
    }
```

with respect to above code

(1) client gives request to our servlet program having "get" method.

(2) servlet container creates or) locates our servlet class object. If created servlet container also completes the initialization process.

(3) The servlet container creates servletRequest, servletResponse objects for current request and calls the life cycle method public service(-,-) on our servlet

class object. keeping ServletRequest, ServletResponse objects as argument values.

(4) Since public service(-,-) method is not available in our servlet program, the super class public service(-,-) method executes (Predefined HttpServlet class). This method converts ServletRequest object to HttpServletRequest object, ServletResponse object to HttpServletResponse object and calls protected service(-,-) method having those objects as arguments.

(5) Since protected service(-,-) method is not available in our servlet program, the protected service(-,-) method of super class (Predefined HttpServlet class) executes. This protected service(-,-) method calls doGet() method based on "get" method of given request.

(6) Since doGet(-,-) method is available in our servlet program that will be executed and generated response goes to browser window.

NOTE: Don't let doXxx(-,-) methods of predefined HttpServlet getting executed directly (or) indirectly because they send "405" error response to browser window indicating that our servlet is incomplete servlet program towards processing request.

* When request arrival event is raised (means request comes to container) servlet container calls public service(-,-) method on our servlet class object and it never calls protected service(-,-) method, doXxx(-,-) methods for that event. So only public service(-,-) method is called as servlet life cycle method, protected service(-,-) and doXxx(-,-) methods are called convenience methods given to programmer to keep request processing logic in our servlet programs.

* For understanding flow of execution in our servlet programs towards the request processing refer the six scenarios given in page no. 56 of booklet.

* Even though public service(-,-) method is lifecycle method but the good programmer always prefers overriding doXxx(-,-) methods in his servlet

program to place request processing logics. Because these methods are given based on the http protocol's standard. Moreover the super class (pre-defined HttpServlet class) version of this methods can generate appropriate error messages like 405 when those methods are really not overridden in our servlet program.

* most of the programmers in real world overrides both doGet(-,-) and doPost(-,-) methods in servlet program, but they place request processing logic only in one method and calls that method from another method.
* In java Abstract class can have only abstract methods (or) only concrete methods (or) mix of both.

In predefined HttpServlet class when all methods are given as Concrete methods. why that class itself is given as abstract class?

(A) Refer the last 3 paragraphs of page no. 56 of booklet.

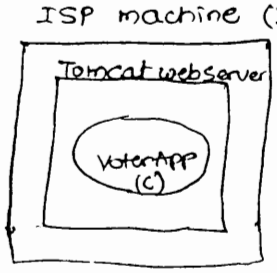
* Every website that is hosted in the internet will be having one domain name like www.yahoo.com, www.gmail.com and etc...
* These domain names along with home page request URLs will be registered in DNS ~~registe~~ registering.

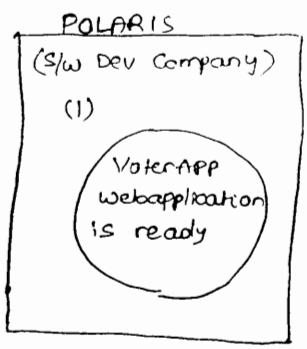
Procedure to host web application on to the internet having domain name

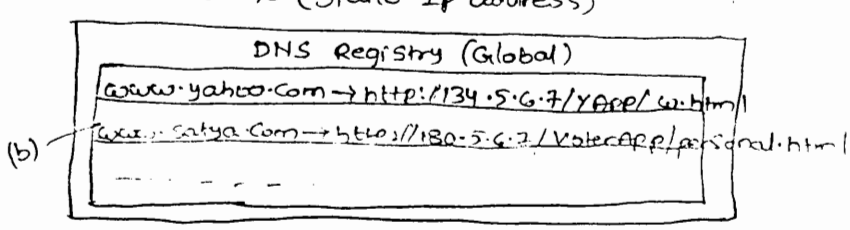
(2) Purchase domain name from ISP
like www.satya.com

(3) Purchase space in webserver/application server maintained by ISP in static Ip address based machine.

(4)



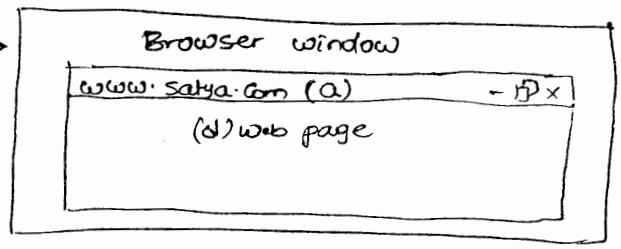




(4) move VoterAPP app the server of ISP machine

(5) register VoterAPP app in DNS registry.

(6) Access the web site →



* Registering web application in DNS registry is nothing but keeping domain name of web application along with home page URL.
* The ISP machine where ever web application hosted will be having static / fixed IP address.
* once we purchase space from the server of ISP machine they supply FTP application along with user name and password to us and we can use that FTP application to interact with our space of ISP machine from any place so using this we host or move web applications to the server of IP machine.

Flow of accessing

(a) End user types domain name as URL in browser windows address bar.

(b) Through internet network the request goes to DNS registry, gathers home page request URL of given domain name (like satya.com) from DNS registry.

(c) Based on home page request URL the web resource program of hosted web application in tomcat server of ISP machine will be executed.

(d) The web resource program generates one web page to browser window.

* we can add or remove web resource programs in the hosted website of ISP server from our local machine through FTP application. (This is nothing but updating web pages content after hosting).

http request methods

GET
POST
HEAD
DELETE
PUT
TRACE
OPTIONS

* The regularly used two request methods of real world programming are get, post.

GET:

* Given to gather / to get more data from the server.
* The response of this method based request contains both headers and body. ~~Head~~

Head:

* Same as get but this method based request generated response contains only response headers.
* This method is useful to check the availability of web resource programs.

NOTE: Get() method is useful to gather data from server by sending limited amount of data (upto 256 kb) from the request.

post:

* Design to send unlimited amount of data along with the request

put:

* useful to add new web resource program in web application from client.

delete

* useful to remove web resource program of web application from client.

NOTE: put(), delete are useful in the development of FTP applications.

options

* server determines using which request methods the current web resource program can be requested.

NOTE: If our servlet Program Contains doGet() method overriding then the options request method based request given to that servlet Program

returns get, Head, Trace, options.

Trace

* A Trace request method based request sends the flow of execution details of certain web resource program like servlet. So these details can be used for debugging operations.

List of form Components in form page

(1) TextBox
(2) Password Box
(3) Select box/ComboBox
(4) List box
(5) TextArea
(6) RadioButtons
(7) checkBox
(8) Button (Standard button, submit button, reset button)
(9) File uploading Component

TextBox

In form page

```
Name: <input type="text" name="pname">
```

In servlet program (to read comp value)

```
String s1 = req.getParameter("pname");
```

Password box

In form page

```
Age: <input type="password" name="page">
```

In servlet program

```
String s1 = req.getParameter("page");
```

TextArea

In form page

```
Address: <textarea name="taddress" rows="4" cols="5">
            enter ur address
         </textarea>
```

In servlet program (to read component value)

```
String s1= req.getParameter("taddress");
```

Select Box / Combo Box (allow us to create select one item a time)

Qualification :
```
<select name="qlfy">
    <option value="engg">B.E/B.Tech</option>
    <option value="medico">MBBS</option>
    <option value="arts">B.A</option>
</select>
```

in servlet program

```
String s1 = req.getParameter("qlfy");
```

* while working with select Box the selected item will not come to server as request parameter value, the value available in the value attribute of option tag for selected item will come to server as request parameter value.

Ex: From the above select Box if BE/B.Tech is choosen then the value engg will go to server as request parameter value.

List box (allows us to select multiple items at a time)

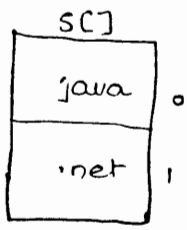
In form page

```
Courses: <select name="crs" multiple>
    <option value="java">JAVA PKG</option>
    <option value=".net">.NET PKG</option>
    <option value="Oracle">Oracle PKG.</option>
</select>
```

In servlet program

```
String s[] = req.getParameterValues("crs");
```

* If JAVA PKG, .NET PKG items are choosen then s[] holdes java, .net as elements values.



Radio buttons

* By giving same name for multiple radio buttons we can group them into single unit. So only one radio button can be selected at a time.

In form page

```
Gender : <input type="radio" name="g" value="m">Male   
         <input type="radio" name="g" value="f"> Female
```

In Servlet Program

```
String s1 = req.getParameter("g") ; // gives m when male radio is selected
                                    // gives f when female radio is selected.
```

checkBoxes

* When multiple checkBoxes are grouped into single unit by giving same name then we can select multiple checkBoxes at a time.

In form page

```
Hobbies : <input type="checkbox" name="ch1" value="read">Reading
          <input type="checkbox" name="ch1" value="sleep">sleeping
          <input type="checkbox" name="ch1" value="roaming">roaming
```

In Servlet program

```
String s[] = req.getParameterValues("ch1");
```

* If reading, roaming check boxes are selected then the s[] holdes read, roam values.

NetBeans

Type: IDE for java environment

Version: 6.7.1 (comparable with jdk1.6)

Vendor: Sun micro system

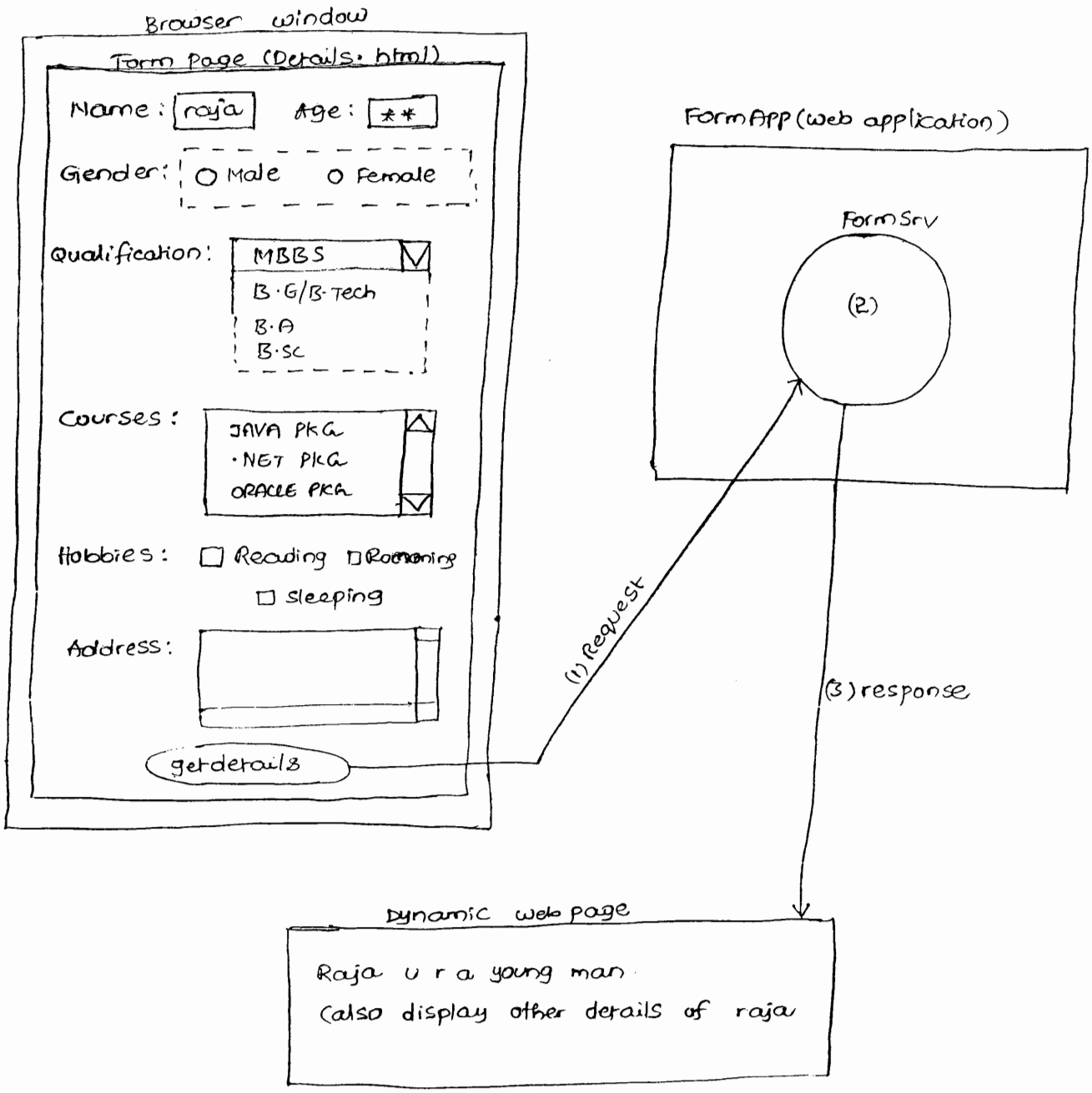
gives GlassFish 2.x as Built-in Server

open source s/w

to download s/w: www.netbeans.org

for docs: www.netbeans.org

* It is always recommanded to design the form components of form page as HTML table rows and column values. This allows us to align the form page easily.



Procedure to develop and execute above application by using NetBeans IDE

Step(1): Create java web project having name FormApp.

File menu → new project → java web → web application → next →

Project Name : FormApp

→ next → server : GlassFish 2.1 → next → Finish

Step(2): Add form page to the project (Details.html)

Right click on web pages folder → new → HTML → HTML File Name : Details

→ Finish

```
<form action="furl" method="get">
                    ↓
              url pattern of FormSrv servlet program
    <table border="1">
        <tr>
            <td> Name: </td>
            <td> <input type="text" name="pname" ></td>
        </tr>
        <tr>
            <td> Age: </td>
            <td> <input type="text" (password) name="page"> </td>
        </tr>
        <tr>
            <td> Gender: </td>
            <td>
                <input type="radio" name="g" value="m" check
                <input type="radio" name="g" value="F"
            </td>
        </tr>
        <tr>
            <td> Qualification </td>
            <td>
                <select name="qlfy">
                    <option value="engg"> JAVA PKG (B.G/B.Tech) </option>
                    <option value="medico"> MBBS </option>
                    <option value="arts"> B.A </option>
                    <option value="Science"> B.Sc </option>
                </select>
            </td>
        </tr>
        <tr>
            <td> Courses </td>
            <td>
                <select name="crs" multiple>
                    <option value="java"> JAVA PKG </option>
                    <option value="Net">  .NET PKG </option>
                    <option value="oracle"> oracle PKG </option>
            </td>
        </tr>
```

```
<tr>
    <td> Hobbies </td>
    <td>
        <input type = " checkbox"  name = "ch1"  value = "read" checked />
                                                    reading    
        <input type = "checkbox"  name = "ch1"  value = "roam" /> Roaming
                                                       
        <input type = "checkbox"  name = "ch1"  value = "sleep" /> sleeping
    </td>
</tr>
<tr>
    <td> Address </td>
    <td>
        <textarea  name = "address"  rows = "4"  cols = "20>
        enter address
        </textarea>
    </td>
</tr>
<tr>
    <td colspan = "2" > < input type = "submit" value = "getdetails" /> </td>
</tr>
```

merges two cells into single cell

```
</table>
</form>
```

Step (3): Add FormSrv servlet to source packages folder of project

Right click on Source packages folder → new → servlet →

class name : [FormSrv] → next →

url pattern : [/url] → Finish

NOTE: The above IDE generated servlet program gives processRequest(-,-) method to programmer for keeping request processing logic and this method will be called internally from doGet(-,-), doPost(-,-) methods like approach (3) of developing our servlet program as flexible servlet program. (Refer approach (3))

Step (4): keep the following request processing logic in processRequest(--) method of above servlet program.

```
{
//general settings
PrintWriter pw = res.getWriter();
res.setContentType("text/html");
//read from form page
String name = req.getParameter("pname");
int age = Integer.parseInt(req.getParameter("page"));
String gen = req.getParameter("g");
String q/fy = req.getParameter("q/fy");
String crs[] = req.getParameterValues("crs");
String hb[] = req.getParameterValues("ch1");
String address= req.getParameter("taddress");
if (gen.equals("F"))
  {
     if( age<=5)
          pw.println( name+"ur baby girl");
     else if(age<=12)
          pw.println(name+"ur child girl");
     else if(age<=19)
          pw.println(name+"ur teenage girl");
     else if(age<=30)
          pw.println(name+"ur a young woman");
     else if(age<=45)
          pw.println(name+"ur middle age woman");
     else
          pw.println(name+"ur old lady");
  }
else if(gen.equals("M"))
  {
     if(age<=5)
          pw.println(name+"ur baby boy");
     ==========
  }
```

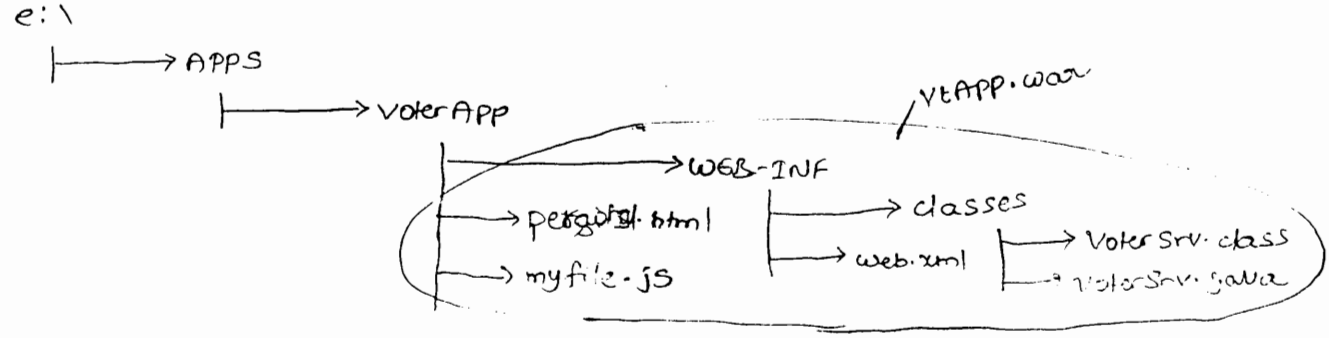
```
        //close stream obj
        Rw.close();
    } // print form data
    pw.println("<br>name = "+name);
    pw.println("<br> age = "+age);
    pw.println("<br> Address = "+address);
    pw.println("<br> Gender = "+gen);
    pw.println("<br> qualification = "+qlfy);
    pw.println("<br> courses = ");
      for(int i=0; i<crs.length; ++i)
          {
            pw.println(crs[i]+"....");
          }
    Pw.println("<br> hobbies = ");
      for(int i=0; i<hb.length; ++i)
          {
            pw.println(hb[i]+"....");
          }
      //close stream obj
        pw.close();
  }
```

Step(4) : Run the project

Right click on FormApp project → Run

* we can prepare WAR file on deployment directory structure of web application. Each war file represents one web application.

Procedure to prepare war file



To prepare war file

e:\Apps\VoterApp> jar cf VtApp.war .

* Gives war file by combining everything of VoterApp folder.

* There are 3 ways to deploy web applications in servers.

(1) Hard deployment (Copy .war or directory of web application to a fixed (or) installation folder of server software like Tomcat_home\webapps folder).

(2) Console Deployment (use admin console window for deployment)

(3) Tool based deployment (Deployment using tools like Ant, maven, IDE)

* (3) is recommanded process.

Procedure to deploy webapplication in Tomcat server through console deployment process

Step(1) : Prepare war file (VtApp.war) as shown above

Step(2): Launch Tomcat web application manager window

start tomact server → launch home page → Tomcat manager → Submit user name password

Step(3) : Deploy the war file

Tomcat web application manager window → Select WAR file to upload → browse → select the above VtApp.war file → Deploy

* Test the application by using the following request URL

http://localhost:2020/VtApp/personal.html

↓

war file name as context path

NOTE: In most of the servers when war file is deployed, its file name automatically becomes context path of the web application.

* To perform hard deployment of web application in tomcat server.

copy VoterApp folder (or) VtApp.war file to tomcat_home\webapps folder.

Weblogic

type : ApplicationServer S/w

Vendor: BEA Systems (oracleCorp)

Version: 10.3 (Comparable with JDK 6)

Commercial S/w

default port no : 7001

jar file that represents whole Jee parts apis : weblogic.jar

to download software : www.oracle.com or www.commerce.bea.com

for docs : www.edocs.bea.com or www.oracle.com

allows to create domains. each domain act as one application server.

* If multiple projects of a company are using

then weblogic S/w will be installed only once in a comman machine

but for multiple projects multiple domains will be created in that S/w.

Procedure to create user defined domain server in weblogic 10.3

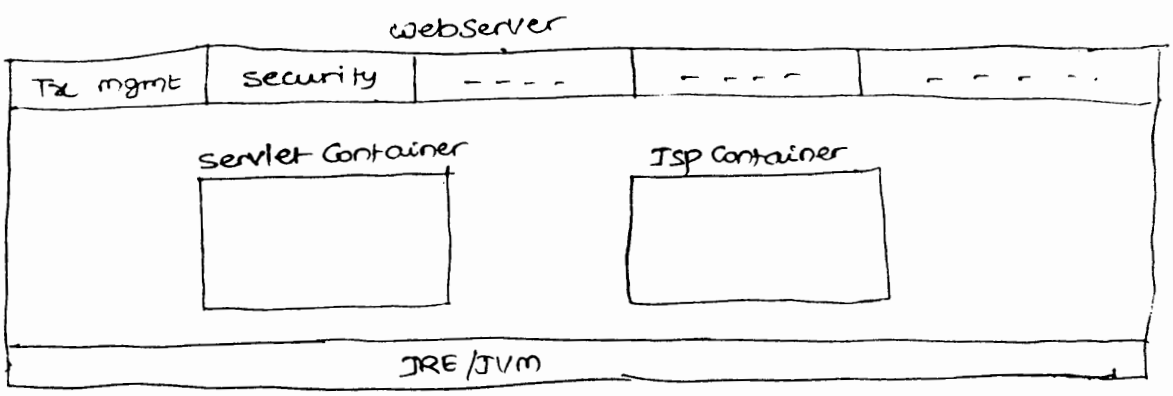
start → programs → oracle weblogic → Quick start → Get Started with weblogic server → create new weblogic domain → next → Gereate a domain .... → next →

Domain name : Adv.JavaBatchDomain

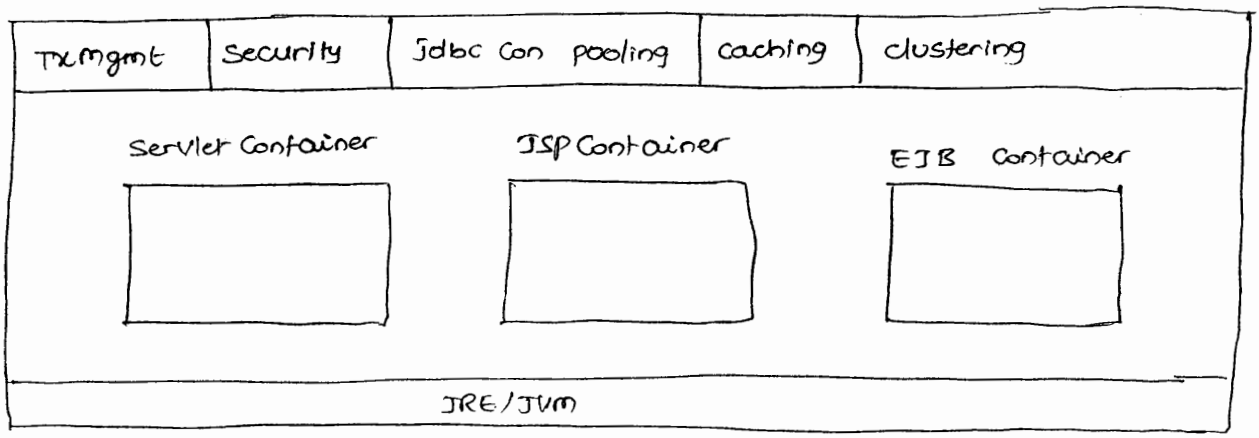
→ next → User name : javaboss password : javaboss1 Conform

User password : javaboss1 → next → select Adminstration Server → next → Listen port : 7070 → next → create.

Application Server = web server + EJB Container + middle ware services



Application Server



* EJB Container is required to manage and execute EJB component.

* war file → web application archive (represents web application)

* jar file → Java archive (represents java api / Zip file)

* ear file → entriprise application archive (jar file + war file + jar file + war file + ...)

* rar file → resource adaptor archive (represents a Jee application interacting with Siebel / SAP s/w)

What is the difference between webserver and application server ?

webserver	application server
(1) Allows to deploy and execute web applications.	(1) Allows to deploy and execute web applications, EJB Components, Enterprise applications and resource adapter apps.
(2) Developed based on servlet, JSP api specifications.	(2) developed based on all JEE api specifications. (Servlets, JSP, EJB, JMS etc)
(3) Gives servlet Container, JSP Container.	(3) Gives both servlet Container, JSP Container and EJB Container.
(4) Doesn't allow to create domain	(4) Allows to create domains.
(5) Allows only Http protocol request	(5) Allows both Http and non Http (IIOP, T3 and etc) protocol based request.
(6) Gives minimum no. of middleware services.	(6) Gives more no. of middleware services.

(7) suitable for small scale and medium scale web applications.	(7) suitable for large scale web applications and for JEE applications.
(8) Recognizes .war file as application	(8) Recognizes .war, .ear, .jar, .rar files as applications.
(9) Ex: JWS, Tomcat, Resin and etc.	(9) Ex: weblogic, websphere, JBoss, Glass_Fish and etc.

Procedure to perform the console deployment of web application Adv.bat Adv.javaBatchDomain

Step(1): Prepare war file representing your web application like VtApp.war (refer previous class)

Step(2): Start Adv.javaBatchDomain server of web logic.

Start → Programs → Oracle weblogic → User projects → Adv.javaBatchDomain → Start admin server for web logic.

Step(3): open administration console of the above domain server.

open browser window → type below URL.

http://localhost:7070/Console

username: javaboss

password: javaboss1

Step(4): Deploy the web application through admin console.

Admin Console → environment → ~~Ser~~ Deployments → Install → Upload your files → Deployment archive → Browse and select VtApp.war file → next → next → next → next → finish → save

Step(5): Test the above deployed web application

Admin Console → deployments → launch VtApp.war file → select personal.html file related URL

(10) undeploy the web application in web logic

Admin Console → deployments → ☑ VtApp → delete

* To perform hard deployment in Adv.javaBatchDomain of weblogic server copy webapplications directory (VoterApp) or its war file (VtApp) to oracle weblogic_home\user_projects\domains\autodeploy folder.

* When war file is used for hard deployment (VtApp.war) use the following request URL to test the application.

http://localhost:7070/VtApp/Personal.html

* The web application that is deployed in weblogic server through hard deployment process can not be undeployed from admin console but can be undeployment through hard deployment process itself.

* When wep application is hard deployed in web logic server through directory based hard deployment (copying VoterApp folder to autodeploy folder) then use following request URL to test the application.

http://localhost:7070/VoterApp/Personal.html

↓

Directory name as Context path

*In web logic server when you modify the source code of Servlet programs placed in deployed web application, just recompilation is enough to get the effect of modifications and there is no need of performing the reloading of web application.

* The web application and its web resource programs that are having relative URLs is called as WODA applications.

+ Each server supplied seperate implementation classes implementing various interfaces of servlet, JSP apis but we never specify these implementation class names in our servlet programs to make our servlet programs as WODA programs.

GlassFish

+ type: Application Server S/W

Vendor: Sun microsystems

Version: 2.x (Compatable with jdk 1.6/1.5)

open source s/w

Allows to create domains. Default domain is domain1

Jar file that represents all jee apis : javaee.jar

default port no: 8001 (for accessing web applications)

4848 (for accessing admin console)

* The Glassfish 2.x software that comes with netBeans 6.7.1 IDE can be used with or without IDE.

Procedure to change the default http protocol service related portno in domain1 server of Glassfish

Go to Glassfish_home\AppServer\domains\domain1\config\domain.xml file and change port attribute value of first <http-listener> tag

Procedure to perform console deployment of web application in domain1 server of Glassfish

Step(1): Preparing war file representing web application like VtApp.war

Step(2): start domain1 server of Glassfish.

Start → Programs → Sun microsystems → Application Server → Start default server

Step(3): open admin console of domain1 server

open browser window → type http://localhost:4848

username : admin

password : admin admin

Step(4): Deploy the web application

Admin Console → applications → web applications → deploy →

Type : Web Application (.war)

location : Browse and select VtApp.war → ok

Step(5): Test the web application

open the browser window → type below URL

http://localhost:5151/VtApp/Personal.html

war file name as context path

* The Glassfish server supports only war file based hard deployment and it does not support directory based hard deployment.

Procedure to perform hard deployment of web application in GlassFish 2.x server.

Step(1): create war file representing the web application (like VtApp.war)

Step(2): Start domain1 server of GlassFish.

Start → programs → Sun microsystems → Application server → Start default server.

Step(3): Deploy the web application.

copy the above VtApp.war file to <GlassFish_home>\appserver\Domains\domain1\autodeploy folder.

Step(4): Test the application.

open browser window → type below URL.

http://localhost:5151/VtApp/personal.html

↓ war file name as context path.

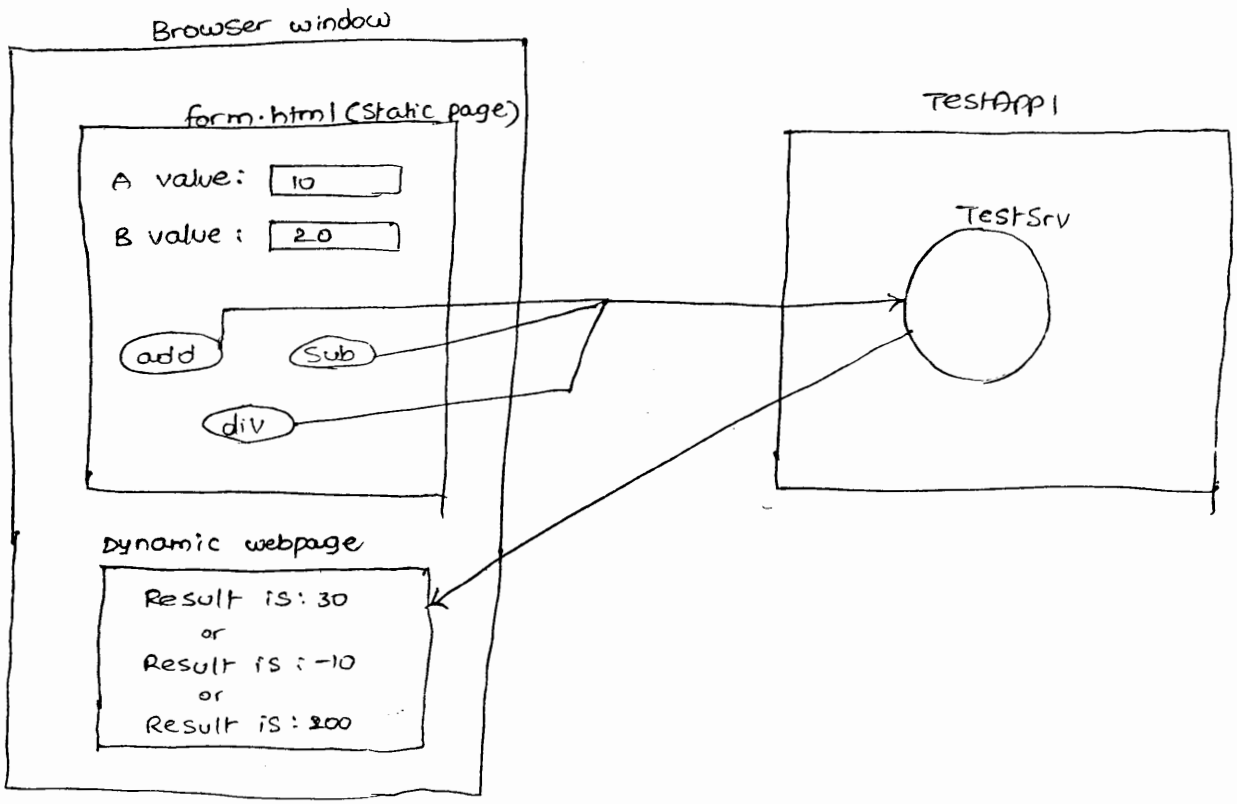
* When form page submit button is taken with logical name then the caption of the submit button will go to server as request parameter value. otherwise submit button caption never goes to server as request parameter value.

```
<form action="Vturl" method="get">
    ---  ---  ---  ---
    ---  ---  ---  ---
  <input type="submit" name="s1" value="check">
</form>
```

Here s1=check will go to server as request parameter name and value.

```
<form action="Vturl" method="get">
    ---  ---  ---
    ---  ---  ---
  <input type="submit" value="check">
</form>
```

NOTE: Here related to submit button no value will go to server as request parameter value.



To handle the above form page related request processing give same name to all the three submit buttons with different captions and use that caption as criteria value in servlet program to differentiate request processing logic for each submit button.

form.html

```
<form action="url" method="get">
    A value: <input type="text" name="t1"><br>
    B value: <input type="text" name="t2"><br>
    <input type="submit" name="s1" value="add">
    <input type="submit" name="s2" value="sub">
    <input type="submit" name="s1" value="div">
</form>
```

TestSrv.java

```
public class TestSrv extends HttpServlet
{
    Protected void service(HttpServletRequest request, HttpServletResponse response)
                                        throws ServletException, IOException
    {
```

```
response.setContentType("text/html");
printWriter pw = response.getWriter();
//read form data
 int a = Integer.parseInt(request.getParameter("t1"));
 int b = Integer.parseInt(request.getParameter("t2"));
//read caption of submit button
 String cap = request.getparameter("s1");
 if(cap.equals("add"))
    {
       pw.println("sum is on result is:" + (a+b));
       pw.println("add btn is clicked");
    }
  else if(cap.equals("sub"))
     {
        pw.println("result is : +(a-b));
        pw.println("sub btn is clicked");
     }
  else
     {
       pw.println("result is : +(a/b));
       pw.println("div btn is clicked");
     }
   //close stream object
    pw.close();
}
```

web.xml

configure TestSrv servlet program with "/turl" pattern

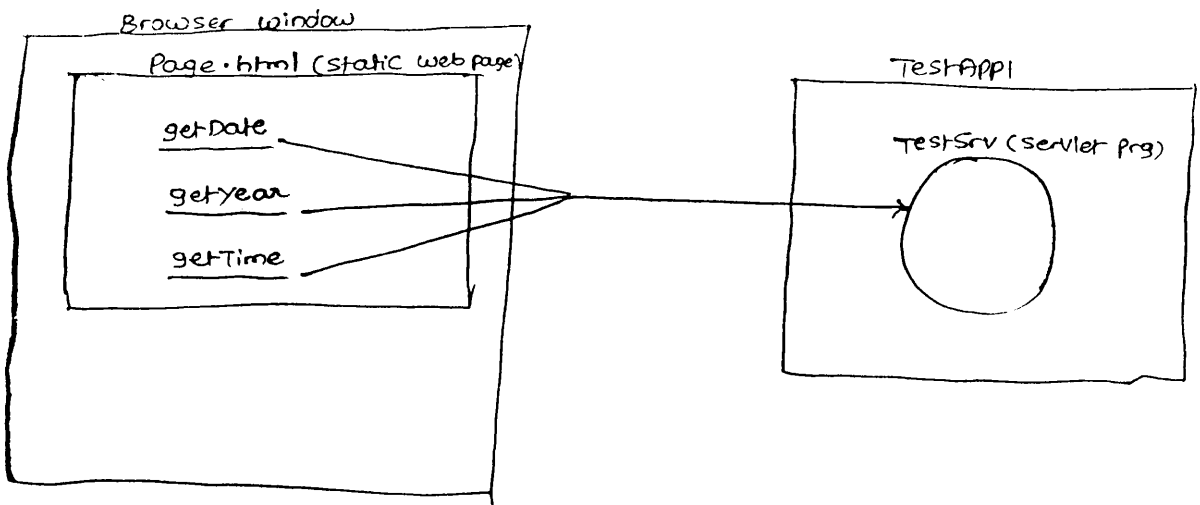


fig: Handling with multiple Hyperlinks

Page.html

```
<form action="">
<a href = "url? P1=link1" >getDate </a>
 <br><br>
<a href = "url ? P1 = link2" > getyear </a>
 <br><br>
 <a href = "url ? P1= link3" > getTime </a>
```

query string

NOTE: The query string that is appended to the URL of href attribute sends request parameter to server along with the request.

TestSrv.java

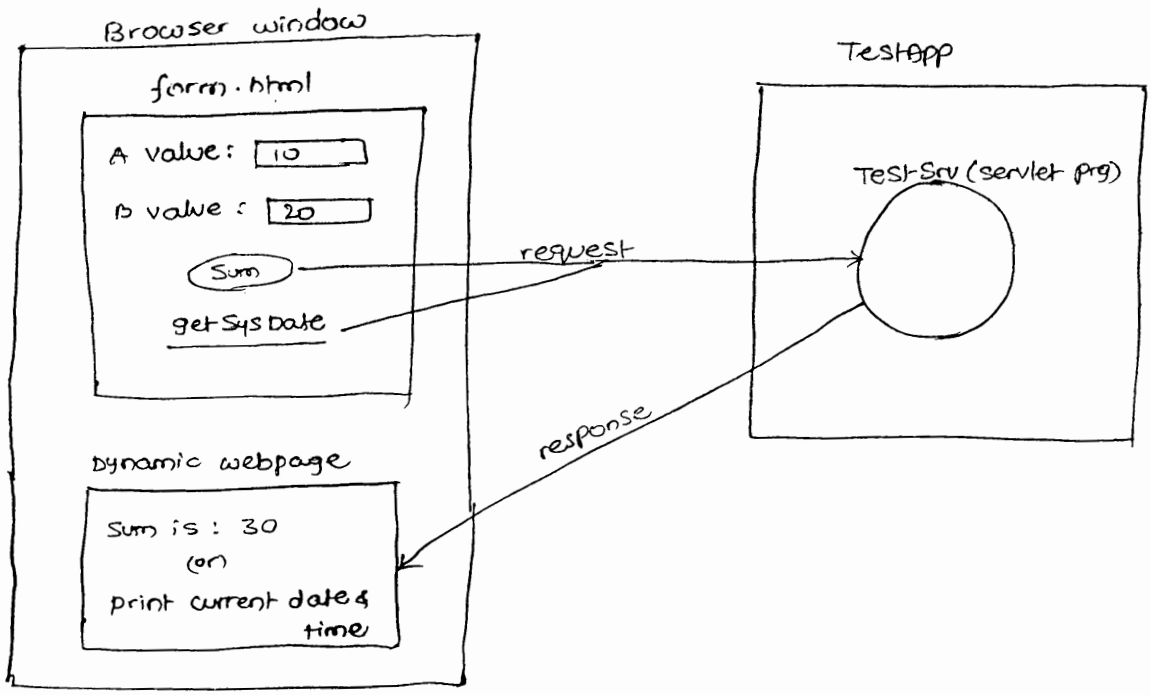
```
public class TestSrv extends HttpServlet
 {
  protected void service( HttpServletRequest request, HttpServletResponse response)
                                  throws ServletException , IOException
   {
      //general settings
      response.setContentType ("text/html");
      PrintWriter pw = response.getwriter();
      //read P1 request parameter value
      String pval = request.getParameter("P1");
      //write req processing logic based on hyperlink that is clicked
      Calendar cl = new Calendar .getInstance(); //give sys date
      if (pval.equals ("link1"))
        {
          pw.println ("link1 is clicked");
          pw.println("current Day is :" + cl.get( Calendar.DATE));
        }
      else if (pval.equals ("link2"))
        {
          pw.println ("link2 is clicked");
          pw.println ("current year is:" +cl.get(Calendar.YEAR));
        }
      else
        {
          pw.println("link3 is clicked");
          pw.println ("current time is:" +cl.get (Calendar.HOUR) +":"
                                  +cl.get(Calendar.MINUTE));
        }
```

```
        //close stream object
        pw.close();
    }
}
```

web.xml

configure TestSrv servlet program with "/tur" url pattern.

Scenario (3) (Handling both Hyperlink, Submit buttons based form page generated request)



* Give same name for Submit button and hyperlink related additional request parameter having two different values. use that value as criteria value in servlet program to differentiate the logics for submit button, hyperlinks.

* Calendar cl = Calendar.getInstance();

This getInstance() is static method of Calendar class returning one sub class object of Calendar class as return value. Calendar class is an abstract class.

* In the above statement cl is not the object of Calendar class it is one subclass object of Calendar class.

form.html

```
<form action="url" method="get">
        ↳ URL pattern of TestSrv servlet program
    A value <input type="text" name="t1"><br>
    B value <input type="text" name="t2"><br>
    <input type="submit" name="s1" value="add">
</form>
<br><br>
<a href="url?s1=link1">getSysDate</a>
```

TestSrv.java

```
public class TestSrv extends HttpServlet
{
  protected void service(HttpServletRequest request, HttpServletResponse response)
                                  throws ServletException, IOException
  {
    //general settings
    response.setContentType("text/html");
    PrintWriter pw = response.getWriter();
    //read s1 req parameter value
    String pval = request.getParameter("s1");
    //write req. processing logic based on hyperlink or submit
    //button on that is clicked
    if (pval.equals("add")) //if add btn is clicked
    {
      //read form data
      int a = Integer.parseInt(request.getParameter("t1"));
      int b = Integer.parseInt(request.getParameter("t2"));
      pw.println("sum is:"+(a+b));
    }
    else if(pval.equals("link1")) //if hyperlink is clicked
    {
      Date d=new Date();
      pw.println("Date is:"+d);
    }
    //close stream object
    pw.close();
  }
}
```

web.xml

* same as previous application.
* A servlet program is identified through its URL pattern according to servlet specification given by Sun micro systems there is a possibility of giving three types of URL patterns.

(1) Exact match
(2) Directory match
(3) Extension Math.

Exact match

* URL pattern begins with "/" symbol and should ~~not~~ contain * symbol.
* multiple words can be there in the URL pattern seperated with "/" symbol.

Ex: In web.xml

```
<url-pattern> /test1 </url-pattern>
```

request URLs from browser window to request the above servlet program

http://localhost:2020/DateApp/test1 (valid)

http://localhost:2020/DateApp/xy3/test1 (invalid)

/DateApp/test1/abc (invalid)

/DateApp/abc2 (invalid)

other examples

ex(1) <url-pattern> /test1/abc </url-pattern>

ex(2) <url-pattern> /abc/test1/xy3 </url-pattern>

ex(3) <url-pattern> /test1.cpp </url-pattern>

* The URL pattern provided for servlet programs can hide the servlet technology and the class names of servlet programs from the end users of the websites. This gives little bit prevention for website from hackers and jackers.

Directory Match

* The URL pattern must begin with "/" symbol. and must end with * symbol. and can also have multiple words seperated with "/" symbol.

Ex(1) : /test1/xyz1/*

request URLs from browser window to request the above servlet program

http://localhost:2020/DateApp/test1/xyz1/abc (valid)

/DateApp/xyz1/test1/abc (invalid)

/DateApp/test1/xyz1/abc.c (valid)

/DateApp/x/y/test.do (invalid)

/DateApp/test1/xyz1 (valid)

/DateApp/test1 (invalid)

other example directory match URL pattern

Ex(1): <url-pattern> /x/y/* </url-pattern>

Ex(2): <url-pattern> /abc/xyz/* </url-pattern

Ex(3): <url-pattern> /abc/* </url-pattern

Extension Match URL pattern

* URL pattern must begin with * symbol and must end with extension word/letter

Ex. <url-pattern> *.do </url-pattern>

↓

extension word.

request urls from browser window to request the above servlet program

http://localhost:2020/DateApp/abc.do (valid)

/DateApp/abc/xyz/abc1.do (valid)

/DateApp/.do (valid)

/DateApp/abc.do/xyz (in valid)

/DateApp/abc.do/xyz.do (valid)

/DateApp/abc.xyz (invalid)

other example extension match URL patterns

Ex(1): <URL-pattern> *.abc </url-pattern>

Ex(2): <url-pattern> *.a </url-pattern>

* we can not prepare URL pattern of a servlet program by mixing up multiple styles because all servers are designed to just recognize only the above 3 styles of URL patterns.

<url-pattern> /xyz/abc/*.abc </url-pattern>

↓

Invalid URL pattern formation

* If two servlets are configured with two different styles of URL patterns like exact match and extension match if matching same request URL given by browser window then server gives important priority to exact match url pattern based servlet.

Example scenario

In web.xml

DateSrv ⟶ with /test.do url pattern (exact match)

DateSrv1 ⟶ with /*.do url pattern (extension match)

request from browser window

http://localhost:2020/DateApp/test.do

↓

This test.do matches both DateSrv, DateSrv1 servlet program but priority will be given to DateSrv program with URL pattern exact match.

MyEclipse = Eclipse + Built-plugins to work with advanced technologies

* A plugin is patch software or software application that can ~~ehh~~ enhance the functionalities of existing s/w and s/w applications.

* In java environment plugins come as jar files. In IDE softwares plug ins provides wizards to develop s/w technologies based applications

What is the difference between Eclipse IDE and MyEclipse IDE?

Eclipse	MyEclipse
(1) open source	(1) Commercial
(2) Provides environment to develop JSE module applications.	(2) Provides environment to develop JSE, JEE and frame software based applications.
(3) Does not provide built-in plug in to work with advanced technologies we must ~~them~~ supply them manually.	(3) It provides built-in plug in to work with advanced technologies and also allows to add more plugins explicitly.
(4) suitable for small scale companies.	(4) suitable for medium scale and large scale companies.

Basic information of myEclipse:

Type : IDE Software for java environment.

Version : 8.x (Compatable with Jdk 1.6)

Vendor : Eclipse organization.

Commercial software

Gives tomcat as built-in server. But also allows the programmer to configure other external servers.

To download software : www.myeclipseide.com

for documents : www.myeclipseide.com

MyEclipse 8.x cheat code:

Subscriber : gocto.cn

Subscription code : ELR8ZC-855444-666653573987614Z

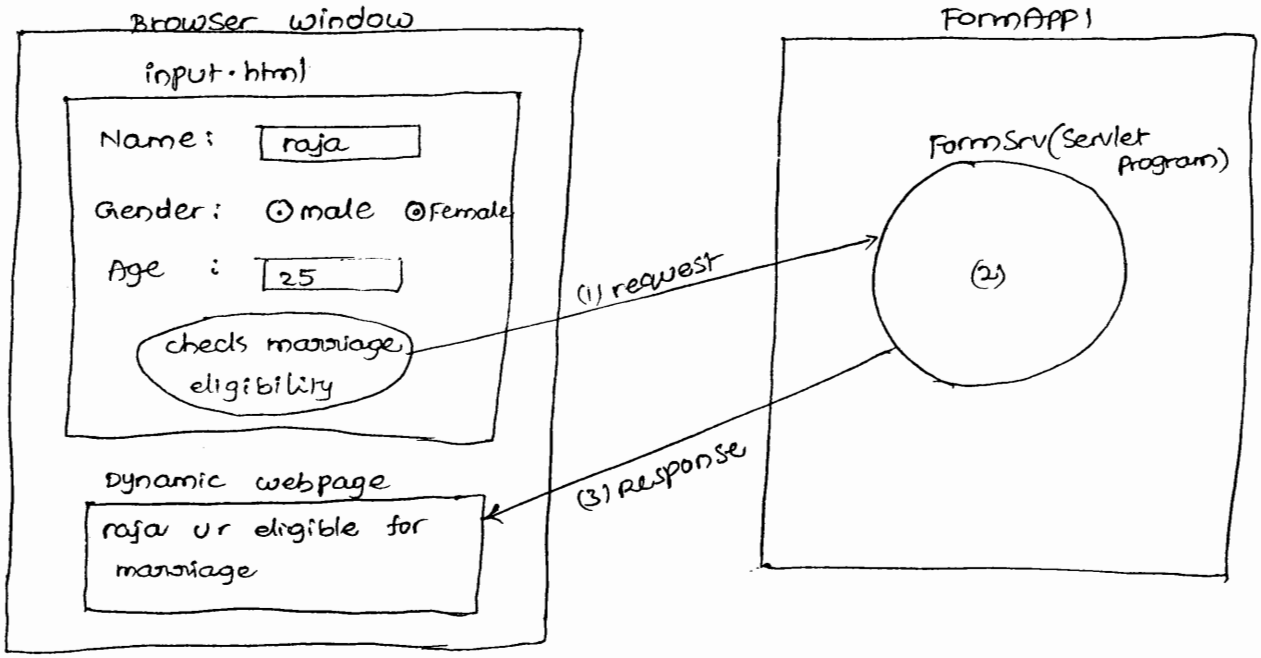
* Eclipse Geleelio, GG Eclipse are alternate IDEs for myEclipse which are given based on Eclipse IDE.

Procedure to develop below application by using myEclipse IDE

Step(1): Lanch myEclipse IDE by choosing one work space folder to save the projects.

work space : E:\myEclipse 8.x

Example application



Step (2): Submit the cheat code

MyEclipse menu → Subscription information →

Subscriber : [ ]

Subscription code : [ ] → Finish

Step (3): create web project in myEclipse IDE.

File menu → New → web project →

project name : [FormApp1]

Context root URL : [/FormApp1] → finish

Step (4): Add form.html to web root folder of the project

Expand project → Right click on web root folder → new → html → Filename

form.html

```
<form method = "get" action = "url" name = "f1">
Name : <input type = "text" name = "pname" /> <br/>
Gender: <input type = "radio" name = "g1" value = "M" checked/> Male
        <input type = "radio" name = "g1" Value = "F"/> female <br>
```

```
Age:<input type="text" name="page"/> <br/>
    <input type="submit" value="checked marriage eligibility"/>
</form>
```

* Add servlet program to the scr folder of the project.

Right click on scr folder → new → servlet

Name: FormSrv → select doGet, doPost methods → next →

servlet/JSP mapping URL: / → Finish

step(5); In the generated FormSrv servlet keep the following logic and call that method.

In FormSrv.java

```
public void doGet(--,--) throws SE, IOE
{
  //general settings
    response.setContentType("text/html");
    PrintWriter pw=response.getWriter();
   //read form data
    String name=request.getParameter("pname");
    int age = Integer.parseInt(request.getParameter("page"));
    String gen=request.getParameter("gl");
    if (gen.equals("m"))
     {
        if (age >=21)
          {
           pw.println("mr"+name+"ur eligible to marriage");
          }
        else
          {
           pw.println("mr"+name+"ur not eligible to marriage");
          }
     }//if
    else if(gen.equals("f"))
     {
       if(age >=18)
          {
            pw.println("miss"+ name + "u r eligible to marriage");
          }
```

```
                else
                  {
                    pw.println("miss"+name+"u r not eligible to marriage");
                  }
              } // else if.
          } // doGet(-,-)
        public void doPost(-,-) throws SE,IOE
          {
            doGet(-,-);
          }
```

Step(6): Configure Tomcat 6.x server with myEclipse IDE

window menu → preferences → myEclipse → servers → Tomcat →

Tomcat 6.x → Tomcat → Tomcat_home directory : D:\Tomcat 6.0

[⊙Enable / ○Disable]

→ apply → ok.

Step(7): Start the tomcat server from myEclipse IDE

Go to servers icon in the tool bar → Tomcat 6.x → Start.

Step(8): Deploy the above project in Tomcat server.

Go to deploy icon of the tool bar → project FormAppl →

add server [Tomcat 6.x] [V] → [Finish] → [Ok]

Step(9): Test the application.

open browser window → type this URL

http://localhost:2020/FormAppl/form-html ↵

Procedure to Configure Adv.JavaBatchDomain server of weblogic 10.3 with MyEclipse IDE:

window menu → preferences → myEclipse → servers → weblogic →

weblogic 10.x → ⊙ enable →

BEA Home directory :: c:\oracle\middleware

Adminstration username: javaboss

Adminstration password: javaboss1

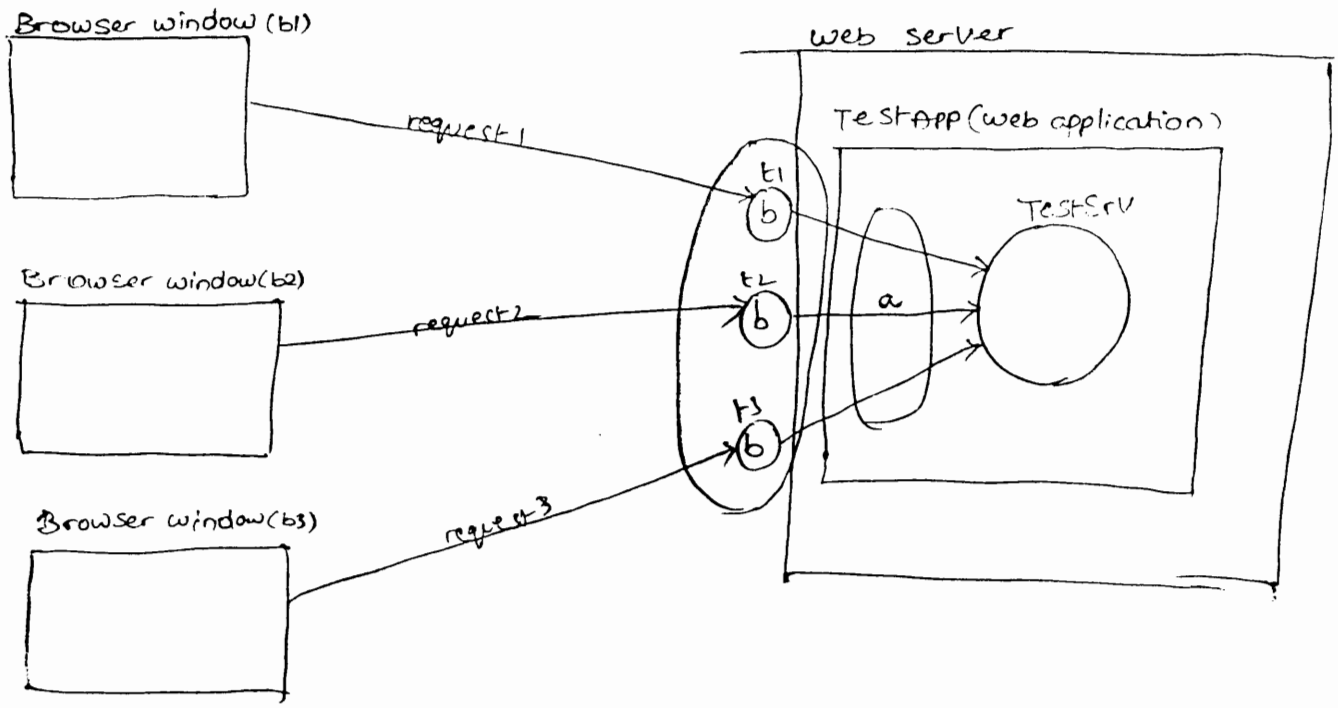
Execution domain root : e:\middleware\user_Projects\domains\
Adv-javaBatch Domain.

→ Apply → ok.

Procedure to configure the domain/ server of "GlassFish 2.x" with myEclipse IDE:

Window menu → preferences → myEclipse → servers → GlassFish → GlassFish 2.x → ⊙ Enable → Home_directory : D:\Sun\APPServer → Apply → ok.

* If multiple threads are acting on single variable (or) object simultaneously (or) concurrently then we say object/ variable is not thread safe. Because, there is a chance of data corruption in that situation.
* To make the above said variable/object as thread safe use Synchronization Concepts.
* Instance variables of our servlet program are not thread safe by default whereas the local variables declared in the service(-,-) method of servlet program are thread safe by default.



```
public class TestSrv extends HS
 {
    int a;
    public void service(-,-) throws SE,IOE
     { int b;
       = =
     }
 }
```

t1, t2, t3 are threads representing requests

* In the above diagram the multiple threads representing multiple requests of servlet program will act on single copy of instance variable "a" and every thread gets its own copy of local variable "b" so, we can say variable "a" is not thread safe and variable "b" is thread safe.
* By default our servlet class object and its instance variables are not thread safe. To make them as thread safe, use synchronisation concept.
* To store results, inputs of servlets program in a perminant place like database software (or) to gather input values of servlet program from perminant storage unit we need to make servlet program interacting with database software where by placing JDBC code in servlets program.
* Every JDBC code contains three important operations.
  (1) create JDBC Connection object
  (2) use JDBC Connection object to create other JDBC objects and to develop persistance logic
  (3) close JDBC Connection object.
* There are three approaches to place JDBC code in our servlet program.

Approach (1)

(a) create jdbc Connection object in init()
(b) use jdbc connection object and other object in service(-,-) /doXxx(-,-) methods.

(c) close JDBC Connection object and other objects in destroy() method.

* In approach(1) the JDBC Connection object must be declared as instance variable so it is not thread safe (it is disadvantage). Programmer must use synchronisation concepts to make the Connection object as thread safe.
* In approach (1) all the requests given to servlet program will use single JDBC Connection to interact with database software. This is improve the performance. (It is advantage).

Approach(2):

(a) Create JDBC Connection object in service(-,-) / doXxx(-,-) methods
(b) Use JDBC connection object and other objects in service(-,-) / doXxx(-,-) methods
(c) close JDBC Connection object and other objects in service(-,-)/ destroy() method.

* Here JDBC Connection object is local variable to service(-,-) method so it is thread safe object (it is advantage).
* Each request given to servlet program will establish one new Connection with database software. This kills the performance (It is disadvantage).

Approach(3):

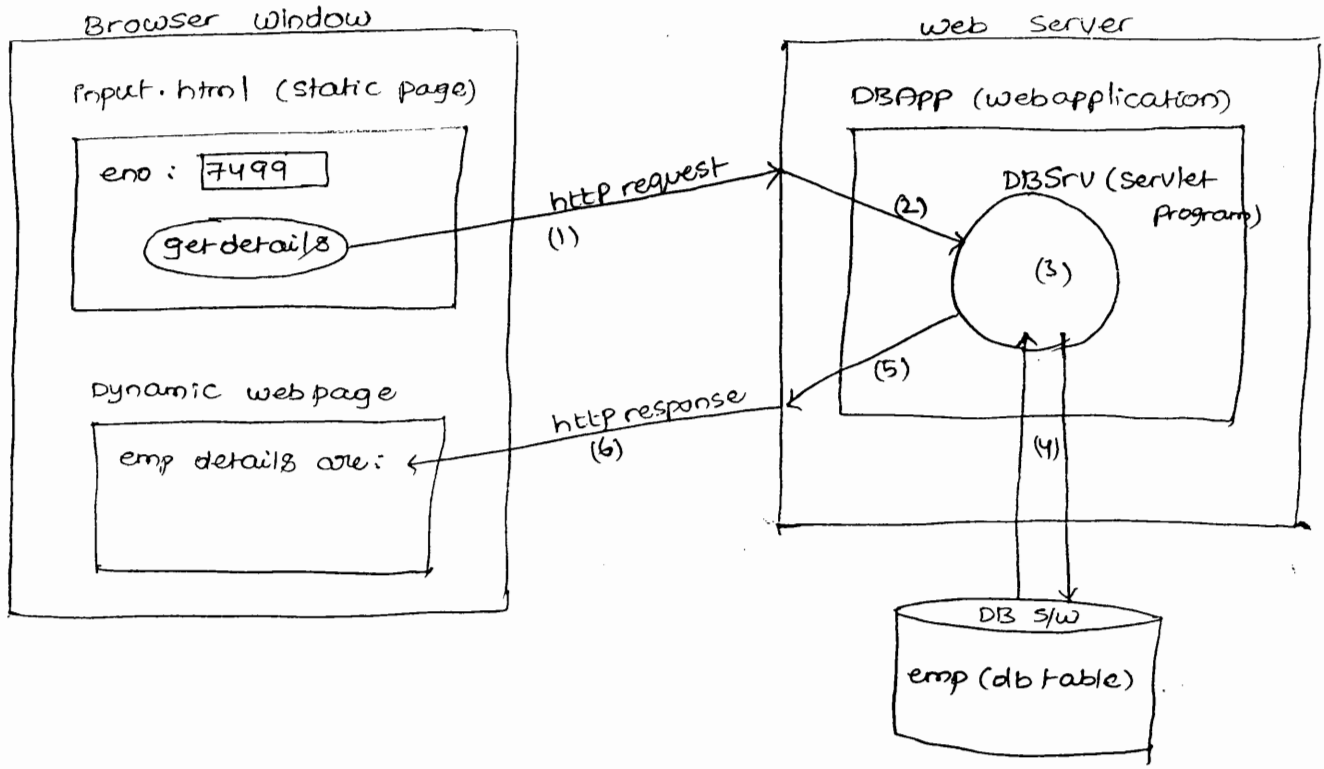
(a) Get JDBC Connection object from server managed JDBC Connection pool being from service(-,-) method / doXxx(-,-) method.
(b) Use JDBC Connection object and other JDBC objects in service(-,-)/ doXxx(-,-) methods.
(c) Return JDBC Connection object back to JDBC Connection pool from service(-,-) / doXxx(-,-) methods.

* Here Connection object is local variable of service(-,-) method. so it acts as thread safe object. (It is advantage).
* By using minimum no. of JDBC Connection objects of Connection pool we can make more servlet programs and all requests coming to

servlet programs interacting with database software. This gives better performance. (It is also advantage).

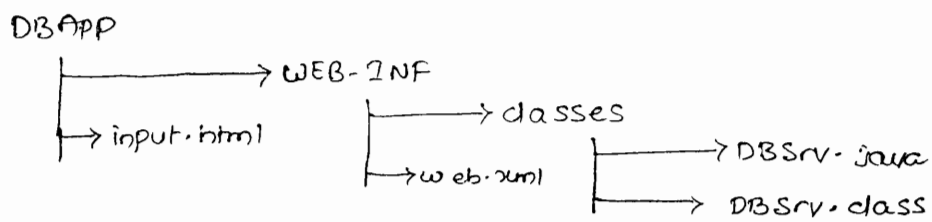
* Approach (3) is the best for servlet to database software communication.

Example application on servlet to database software communication

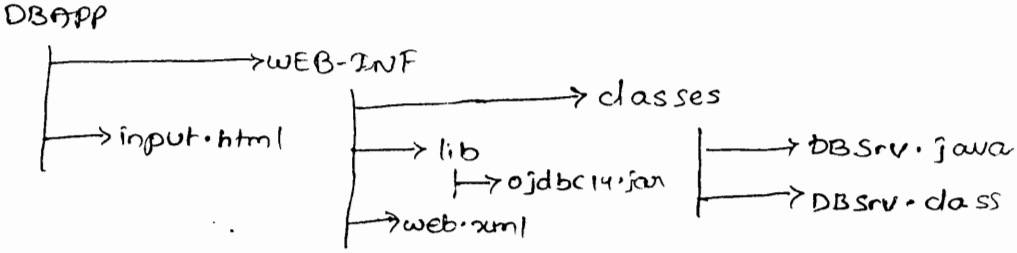


Deployment structure

when DBSrv program uses type 1 JDBC driver:



when DBSrv program uses type 4 (oracle thin) jdbc driver



* when stand-alone java application uses 3rd party API (other than JDK APIs) then the third party API related jar files must be placed in class path to make java compiler and JRE to recognize and use third party API.
* If java web application uses third party API in its web resource programs (like servlet, JSP programs) then third party API related jar file should be added to class path and should also be added to WEB-INF\lib folder of web application. Here jar files added to class path will be used by javac to recognize third party API during the compilation of servlet program. Similarly jar files added to WEB-INF\lib folder will be used by servlet container to recognize and use third party API during the execution of servlet program.

Ex: If servlet program uses oracle thin driver then keep ojdbc14.jar file in class path and WEB-INF\lib folder of web application as shown above.

* The jar files added to class-path are not visible in IDE softwares and in container softwares of servers.
* Stand alone java application will be compiled and executed from command prompt. whereas servlet program compilation takes place in servlet container of ~~servlet~~ server/Application server.
* If above web application is created in IDE software then the third party API related jar file must be placed in the libraries folder of the project.

Source code of above diagram based application

input.html

```
<form action="dburl" method="get">
    Employee no: <input type="text" name="eno"> <br>
                 <input type="submit" value="getEmpDetails">
</form>
```

DBSrv.java (to interact with DB s/w using type 4 driver and approach (1))

```
import javax.servlet.*;
import javax.servlet.http.*;
import java.io.*;
import java.sql.*;
public class DBSrv extends HttpServlet
 {
    Connection con=null;
    PreparedStatement ps=null;
     public void init()
     {
       try
        {
          // create jdbc connection object
           Class.forName("oracle.jdbc.driver.OracleDriver");
           con = DriverManager.getConnection("jdbc:oracle:thin:@localhost:1521:
                                                satya", "scott", "tiger");
            ps = con.prepareStatement("select ename, job, sal from Emp
                                                where empno = ?");
          }// try
          catch(Exception e)
           {
             e.printStackTrace();
           }
      }//init()
     public void doGet(HttpServletRequest req, HttpServletResponse res) throws
                                                ServletException, IOException
      {
       try
        {
          // read form data
           int no = Integer.parseInt(req.getParameter("eno"));
          //write business logic
          //set parameter values to SQL query
           ps.setInt(1, no);
          // execute the query
           ResultSet rs = ps.executeQuery();
          //process the resultSet object
           String name = null, desg = null, bsal=null;
```

```
        if (rs.next())
         {
           name = rs.getString(1);
           desg= rs.getString(2);
           bsal = rs.getString(3);
         }
        //display emp details as web page content
         PrintWriter pw= res.getWriter();
          res.setContentType("text/html");
          pw.println("Emp details: <br>");
          pw.println("<br> Emp Name: "+name);
          pw.println("<br> Emp salary: "+bsal);
          pw.println("<br> Emp desg : "+desg);
         //close stream objects
          pw.close();
          rs.close();
       }// try
       catch(Exception e)
        {
          e.printStackTrace();
        }
    }// doGet(-,-)
public void doPost(HttpServletRequest req,HttpServletResponse res) throws
                                        ServletException, IOException.
   {
     doGet(req,res);
   } //doPost(-,-)
 public void destroy()
  {
    try{
         if(ps!=null)
            ps.close
       }
      catch(Exception e)
        {
          e.printStackTrace();
        }
      try{ if(con!=null)
             con.close();
         }
       catch(Exception e)
         {e.printStackTrace();
         }
   }
}// class
```

web.xml

Configure DBSrv servlet program with "/dburl" as url-pattern.

Request URL to test application

http://localhost:2020/DBApp/input.html

*If multiple web applications of a server are utilizing same third party API (like oracle thin driver) then instead of keeping the 3rd party API related jar file (like ojdbc14.jar) in WEB-INF\lib folder of every application it is recommended to place only once in common library folder of server software instantiation.

Common library folder in Tomcat 6.0

<Tomcat_home>\lib\ folder

Common library folder in Tomcat 5.0/5.5

<Tomcat_home>\common\lib folder

<Tomcat_home>\server\lib folder

Common library folder in weblogic (Any version)

<BEA_home>\wlserver_<version>\common\lib folder

Common library folder in GlassFish 2.x (any version)

<GlassFish_home>\AppServer\lib folder

*when web resource program of web application uses the third party API class or interface then the servlet container looks for third party API class or interface in the following places and in the following order.

(i) In WEB-INF\classes folder itself (if not available then (ii))

(ii) In the jar files added to WEB-INF\lib folder (if not available then (iii))

(iii) In the jar files added to common library folder of underlaying server. (like Tomcat_home\lib folder) (if here also not comming then class not found exception is occured).

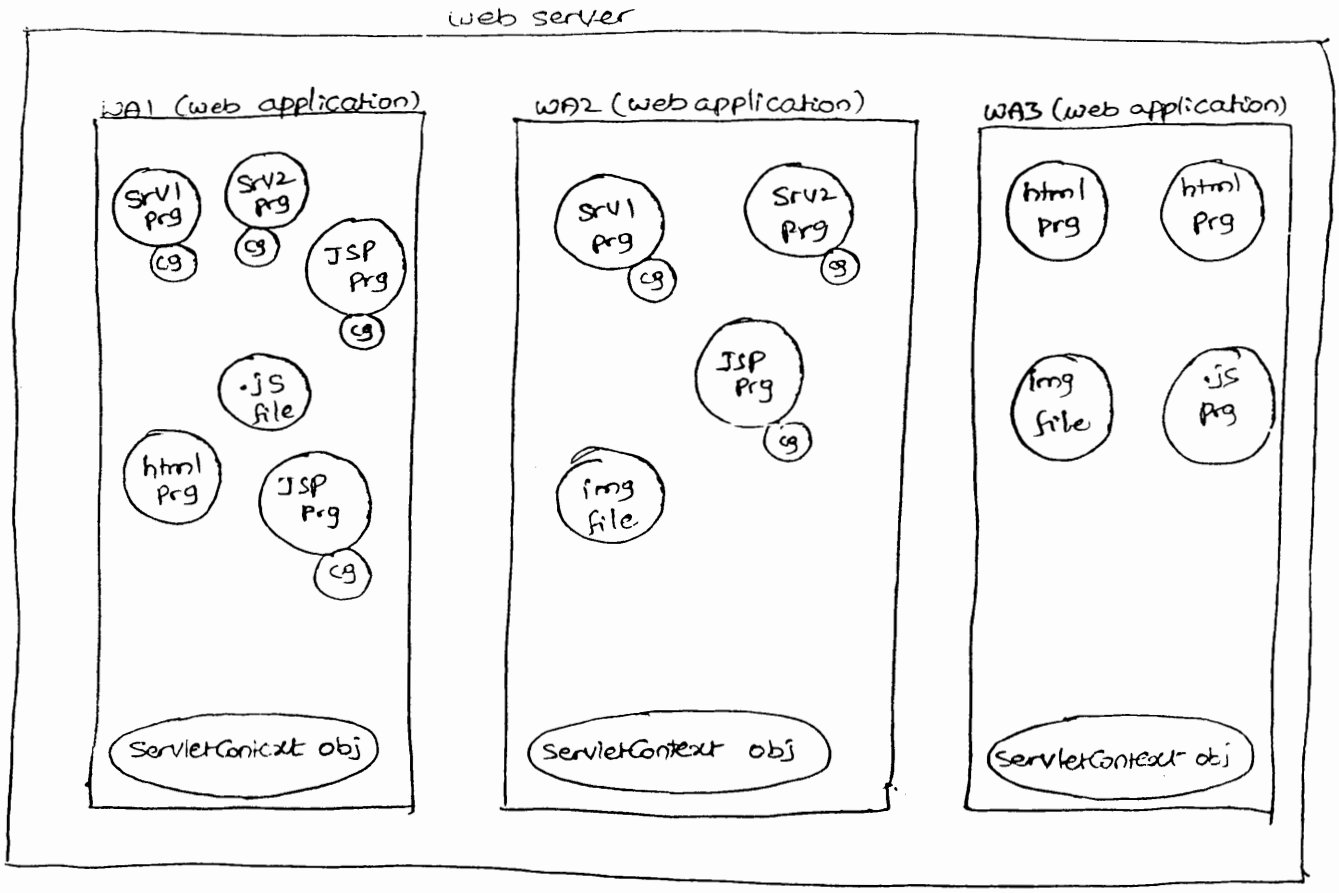
*Every server internally uses one Jdk software so then servlet program uses type-1 jdbc driver to interact with database software then there is no need of adding any jar files in WEB-INF\lib folder and classpath.

What is the difference between ServletContext object and ServletConfig object?

(A) ServletConfig object (cg):

*It is one for our servlet program/JSP program.

* ServletConfig object means it is the object of servlet Container supplied java class (implementing) implementing javax.servlet.ServletConfig interface.
* This object is useful to pass additional data to servlet and to read additional details from Servlet.
* This object is useful to read servlet init parameter values from web.xml file of the web application.
* Servlet Container creates the servletConfig object in the instantiation and initialization process of our servlet class object.
* Servlet Container destroys our servlet class object in the destruction process of our servlet object.



cg - ServletConfig object img file — image file

.js file - javaScript code Srv prg — servlet program

ServletContext object

* It is one per web application. So it is called as the global memory of web application.
* servlet context object means it is the object of a java class (Container supplier) implementing javax.servlet.ServletContext interface.
* Servlet Container creates this object either during deployment of the web application or during server start up.
* Servlet Container destroys this object automatically when web application is undeployed or reloaded or stopped or when server is stopped/re-started.
* Using this object we can read global init parameters or context parameters from the web.xml file of the web application.
* Using this object we can know the details of underlaying server like server name, version and the servlet-api version supported by the server.
* Using this object we can get context path of the current web application and absolute path of the any web resource program in web application.
* The data kept in servletContext object is visible and accessible in all servlet, JSP programs of web application.

In one web server 10 web applications are deployed. In that six web applications are there in running mode and four web applications are there in stopped mode. Can you tell me how many servletContext objects are currently available in that web applications server?

(A) Six

There is a web application with 10 servlet programs In that three servlet programs are already requested by clients and other 3 servlet programs are enabled with load-on-startup. can you tell me how many servletConfig objects are currently available in that web application?

(A) 3+3 = 6

NOTE: ServletContainer creates ServletConfig object for Servlet program only when the class of servlet program is instanciated (object creation).

* our servlet class object, request object, response object, ServletConfig object, ServletContext object can not be created by programmer manually. The servlet container creates all these objects but programmer can get access to these objects in servlet program.

+ To get access to our servlet class object use "this" keyword.

* To get access to request, response objects use parameters of service(-,-) doXxx(-,-)

* To get access to ServletConfig obj

approach(1):

```
public class TestSrv extends HS/GS
{
   ServletConfig cg;
   public void service(ServletConfig cg)   (init)
    {
       this.cg=cg;
       //use cg here
    }
   public void service(-,-)/doXxx(-,-)
    {
       use cg here
       ---  -
       --   -
    }
}
```

approach(2):

```
public class TestSrv extends HS/GS
{
    public void init()
    {
       ServletConfig cg= getServletConfig();
       use cg here          ↳ public method of predefined generic Servlet class
    }
    public void service(-,-)/doXxx(-,-) nexeth
    {
       ServletConfig cg = getServletConfig();
       ==  =
    }
}
```

NOTE: approach(2) is good.

NOTE: self class methods and super class public, protected methods can be called in the sub class with out object.

* To get access to ServletContext object

approach(1):

```
public class TestSrv extends HS/GS
{
    Public void init()
    {
       ServletConfig cg = getServletConfig();
       ServletContext sc = cg.getServletContext();
       // use sc here
    }
    public void service(-,-) / doXxx(-,-)
    {
       ServletConfig cg = getServletConfig();
       ServletContext sc = cg.getServletContext();
       //use sc here
    }
}
```

getServletContext() → method belongs to ServletConfig (I)

approach(2):

```
Public class TestSrv extends HS/GS
{
    Public void init()
    {
       ServletContext sc = getServletContext();
       // use sc here
    }
    public void service(-,-) /doXxx(-,-)
    {
       ServletContext sc = getServletContext();
       // use sc here
    }
}
```

getServletContext() → public method of predefined Generic Servlet class

* Directly or indirectly we need ServletConfig object to get access to ServletContext object.

NOTE: Approach(2) is good.

* getServletContext() method definition of predefined GenericServlet class internally uses servletConfig object to give servletContext objects.

* To make JDBC code of servlet program as flexible code to modify it is recommanded to gather the following four details of JDBC code from outside the servlet program. They are

(i) JDBC driver class name
(ii) JDBC URL
(iii) DB username
(iv) DB password.

* There are two ways to pass input values to servlet program from outside the servlet program.

(1) request parameters / form data.
→ end user / visitor of web application sends these values through browser window (from client side)
→ since end user is non-technical person this data will be non-technical data like name, age, address of end user.
→ servlet program uses request object to read request parameter values.

(2) Servlet init parameters
→ programmer passes this data to servlet program from web.xml (server side).
→ since programmer is technical person, this data can be technical data like jdbc driver, URL and etc details.
→ servlet program uses servletConfig object to read init parameter values of web.xml.

* To make the JDBC code of servlet program as flexible code to modify it is recommanded to gather the above said JDBC details (driver class, URL, username, password) from web.xml file as Servlet init parameter values.

* init parameters are specific to each servlet program and these programmer choice name and value.

* For revised DBApp application that uses servlet init parameter to make jdbc code of DBSrv servlet as flexible code to modify refer application (4) of the page nos 63-65.

* Send technical input values to servlet program from web.xml file as init parameter values. Send non-technical input values to servlet program as request parameter values / form data.

Different ways of reading servlet init parameter values

```
ServletConfig cg = getServletConfig();
```

approach(1)

```
String s1 = cg.getInitParameter("driver");
```

here we must know init parameter name to get the value

approach(2)

```
Enumeration e = cg.getInitParameterNames();
while (e.hasMoreElements())
  {
    String name = (String)e.nextElement();
    String val = cg.getInitParameter(name);
  }
```

//gives all init parameter names and values.

* When you configure multiple init parameters in web.xml file having same logical name then the last value will be effected as that init parameter value.

* Using ServletConfig object we can gather the container created object name of our servlet class.

```
S.O.P("current servlet logical name/ instance name" + cg.getServletName());
```

↓ gives <servlet-name> tag value from web.xml

* If multiple servlet / JSP programs of web application are looking to use same init parameters then instead of placing them in every servlet configuration it is recommanded to write them only once in web.xml as Context parameters (or) global init parameters and use ServletContext

object in all servlet/JSP programs to read these context parameter values.

* If multiple servlet/JSP programs of a web application are talking with DB software by using same JDBC driver to make their JDBC code as flexible code instead of specifying JDBC driver details as init parameters in every servlet program configuration it is recommended to place JDBC driver details only once in web.xml as context parameters as shown below.

In web.xml of DBApp web application

```
<web-app>
    <context-param>
        <param-name> driver </param-name>
        <param-value> oracle.jdbc.driver.OracleDriver </param-value>
    </context-param>
    <C-P>
        <P-N> dburl </P-N>
        <P-V> jdbc:oracle:thin:@localhost:1521:ord </P-V>
    </C-P>
    <C-P>
        <P-N> dbuser </P-N>
        <P-V> scott </P-V>
    </C-P>
    <C-P>
        <P-N> dbPwd </P-N>
        <P-V> tiger </P-V>
    </C-P>
```

Context parameters of web application

```
    <servlet>
        <servlet-name> db </servlet-name>
        <servlet-class> DBSrv </servlet-class>
    </servlet>
    <servlet-mapping>
        <servlet-name> db </servlet-name>
        <url-pattern> /dburl </url-pattern>
    </servlet-mapping>
```

```
        <servlet>
            <servlet-name> t </servlet-name>
            <servlet-class> TestSrv </servlet-class>
        </servlet>
        <s-m>
            <s-n> t </s-n>
            <u-p> /testurl </u-p>
        </s-m>
    </web-app>
```

In init() method of DBSrv servlet program belonging to DBApp webapplication

```
public void init()
{
    try
    {
        //read init param values from web.xml
        ServletContext sc = getServletContext();
        String s1 = sc.getInitParameter("driver");
                                  └──→ Context Parameter name
        String s2 = sc.getInitParameter("dburl");
        String s3 = sc.getInitParameter("dbuser");
        String s4 = sc.getInitParameter("dbpwd");
        ---
        }          //same as previous application code
        =
}
```

NOTE: init parameters are specific to that servlet/JSP program for which they are configured. Context parameters are visible in all web resource programs (java based) of web application.

Different ways of reading context parameter values

```
ServletContext sc = getServletContext();
```

approach(1) :

```
String s1 = sc.getServletInitParameter("driver");
```

here we must know context parameter name to get the value.

approach(2)

```
Enumeration e = sc.getInitParameterNames();
while (e.hasMoreElements())
   {
      String name = (String)e.nextElement();
      String val = sc.getInitParameter(name);
   }
```

//gives all context parameter names and values.

* we can use same name for both init parameter of Servlet program and for context parameter. we can access both these parameter values in our servlet program using ServletConfig, ServletContext objects respectively.
* TO gather misc info about underlaying server use servlet context object as shown below.

```
// gathering misc info using ServletContext obj
pw.println("server info is:" +sc.getServerInfo()); // gives Apache Tomcat 6.0
pw.println("servlet api spec version impl by underlaying server:"+
                    sc.getMajorVersion()+"."+sc.getMinorVersion());
                    gives 2.5
pw.println("context path of current web application:"+sc.getContextPath());
                                                    gives /DBAPP
pw.println("absolute path of input.html on server:"+sc.getRealPath(
                    "/input.html"));
                    ↓
                    gives Tomcat installation path and input.html
```

* Tomcat 5.x → given based on servlet api 2.4 spec
* Tomcat 6.x → given based on servlet api 2.5 spec
* Tomcat 7.x → given based on servlet api 3.0 spec

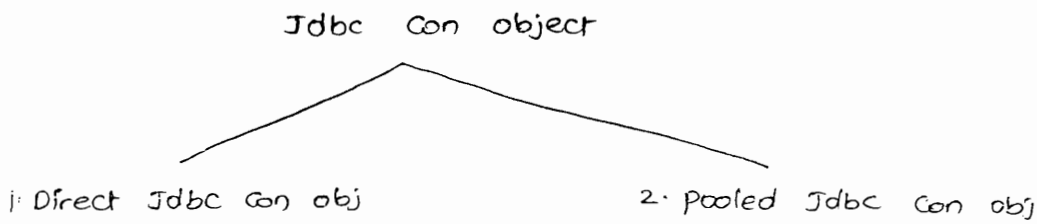
what is the difference between code placed in the constructor and code placed in the init() method of our servlet program?

Ans: Servlet Container creates ServletConfig object after constructor execution and before init() method execution in the instantiation

and initialization process of servlet program so ServletConfig object is not visible in constructor but visible and accessable in the init() method.

Since servletContext object can not be accessed without ServletConfig object so the servletContext object is not accessable in the constructor but servletContext object is accessable in the init() method.

The code placed in the constructor of servlet program can not work with init parameters and context parameters where as the code placed in init() method can utilize those parameters so it is always recommanded to place the initialization logic only in init() method.



To create direct Jdbc Con obj

```
class.forName("oracle.jdbc.driver.OracleDriver");
Connection con=DriverManager.getConnection(url, uname, Pwd);
```

* The Jdbc connection object that is created by programmer manually is called as direct Jdbc Connection object.
* The Jdbc connection object that is collected from Jdbc Connection pool is called as pooled Jdbc connection object.

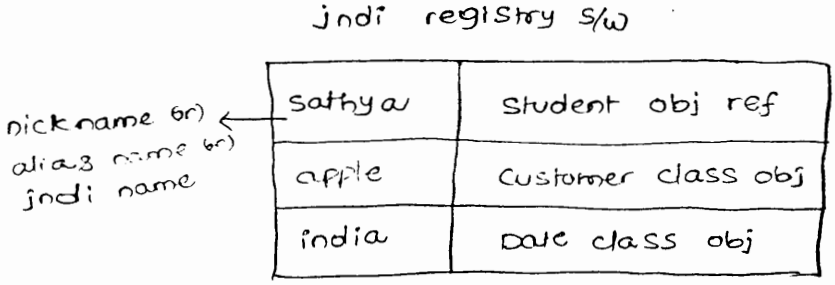
3 types of Jdbc Con pools.

(1) Driver managed jdbc con pool
(2) Third party s/w managed Jdbc Connection pool (like Apache DBCP, C3P0, proxool and etc).
(3) Server managed Jdbc Connection pool (In web server, application server environment).

* Oracle thin driver gives one built-in Jdbc Connection pool in its ojdbc14.jar file. That Connection pool is called as DriverManagerol Connection

pool use. (1), (2) type Jdbc Connection pools in stand alone applications. use (3) type connection pool in those applications that are deployable in the server like web applications, EJB Components and etc..

* In Drivermanaged, third party s/w managed jdbc Connection pool environment we need to use type1/ type2 / type 3 jdbc drivers.
* In server managed Jdbc connection pool environment we need to use type 3 with type1/ type2/ type4 jdbc drivers.
* Each Jdbc Data source object represents one Jdbc Connection pool to get each connection object from Connection pool we need to depend upon this data source object.
* Jdbc DataSource object means it is the object of a java class that implements javax.sql.DataSource interface.
* All Jdbc connection objects of connection pool represents connectivity with same database software. Jdbc Connection pool for oracle means all jdbc Connection objects in that Connection pool represents Connectivity with oracle database software.
* The advantage of Jdbc Connection pool is by using minimum no. of Jdbc Connection objects we can make more clients talking with DB software.



* The registry s/w can maintain objects / object references having nick names or alias name to provide global visibility to them.
* To provide global visibility to jdbc DataSource object we place that object reference in registry software.
* Java applications use jndi api to interact with registry s/w's.

* The process of getting object / object reference from registry based on its nickname or alias name is called as look-up operation

* jndi api means working with javax.naming and its sub packages.

* jndi registry s/w are

  Rmi registry

  COS registry

  weblogic registry

  Jnp registry and etc...

* Every application server software comes with one built-in registry s/w

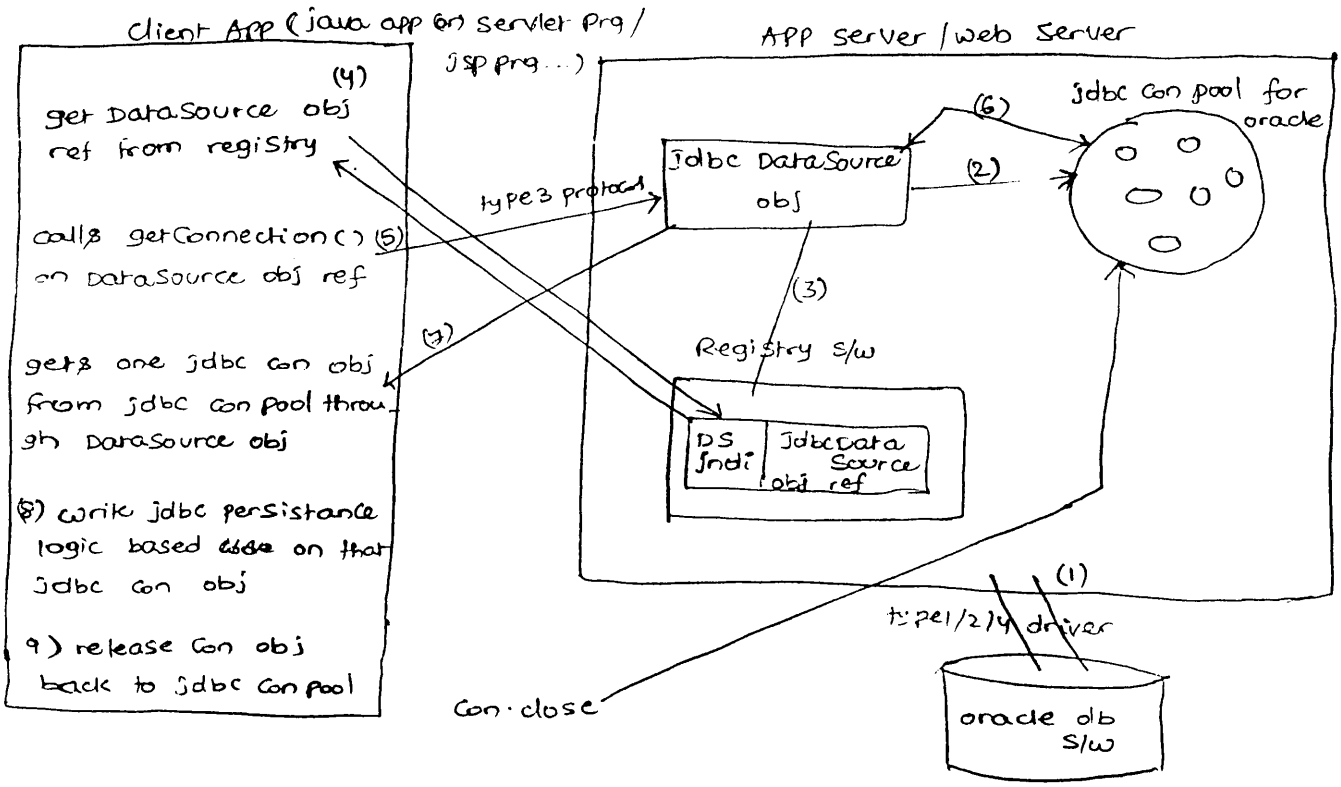
  Jboss ——> Jnp registry

  weblogic ——> weblogic registry

  websphere ——> cos registry

* The Connection object of Jdbc application represents Connectivity between java application and database software. Similarly the initial context object of jndi programming represents Connectivity between java application and registry software.

Understanding the server managed jdbc connection pool environment



* In server managed Jdbc Connection pool environment type3 with type4 jdbc drivers will be utilized. Here type4 will be utilized as a driver to interact with database software and to create Connection objects in the Connection pool where as type 3 will be used as a protocol by client to get one jdbc Connection object from jdbc Connection pool.

With respect to diagram

(1) T.L or P.L makes application server to interact with DB S/W using type1 or type2 or type4 drivers and creates jdbc connection pool having jdbc Connection objects.

(2), (3) → T.L (or) P.L creates datasource object representing the above Jdbc Connection pool and also keeps that datasource object reference in registry software having nickname for global visibility.

(4) → client application gets Datasource object reference from registry software through look-up operation.

(5), (6), (7) → client application calls getConnection() on DataSource object reference this call uses type3 protocol to get one jdbc Connection object from Connection pool.

(8) → refer diagram

(9) → refer diagram.

NOTE: The released Connection object in jdbc connection pool becomes ready to give service to next request or next client.

Procedure to create jdbc Connection pool for mysql Oracle and Jdbc DataSource in Glassfish 2.x server

Step(1): place ojdbc14.jar file in (SUN_HOME)\ AppServer\ domains\domain1\ lib\ ext\ folder.

Step(1): place mysql related Connector j jdbc driver jar file (mysql-connector-java-3.0.8-Stable-bin.jar file) in GlassFish_home\ appserver\ domains\domain1\lib\ext folder.

Step(2): Start Glassfish server and open its admin console (domain1 server)

Step(3): create jdbc connection pool for mysql.

Admin console → resources → jdbc → Connection pools → new →

Name : mypool1

Resource type : javax.sql.DataSource

Database vendor : mysql

→ next → initial and minimum pool size : 2 (initial capacity)

max pool size : 20

Pool resize quantity : 2

idle time out : 300

☑ Database name : db1

☑ Password : root

☑ Port : 3306

☑ Port number : 3306

☑ Servername : localhost

☑ URL : jdbc:mysql://localhost:3306/db1

☑ url : jdbc:mysql://localhost:3306/db1

☑ user : root

→ Finish → Launch mypool1 (click on mypool1) → Ping

Step(4): create jdbc Datasource representing the above jdbc connection pool

Admin console → resources → jdbc → jdbc resources → new →

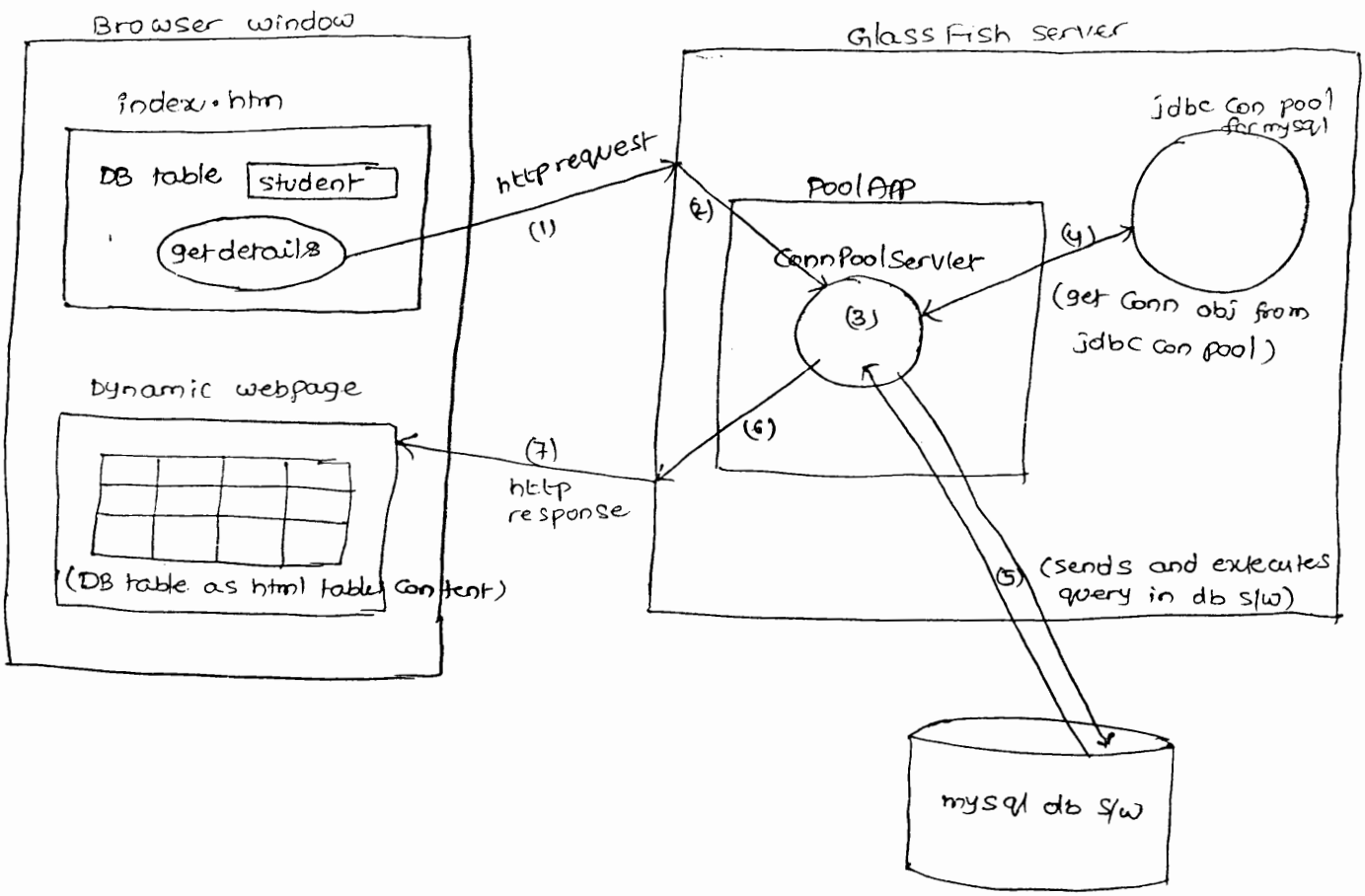
JNDI name : Ds Jndi (any name) (it becomes nickname or alias name once Datasource object is registered with registry s/w)

Pool name : mypool1

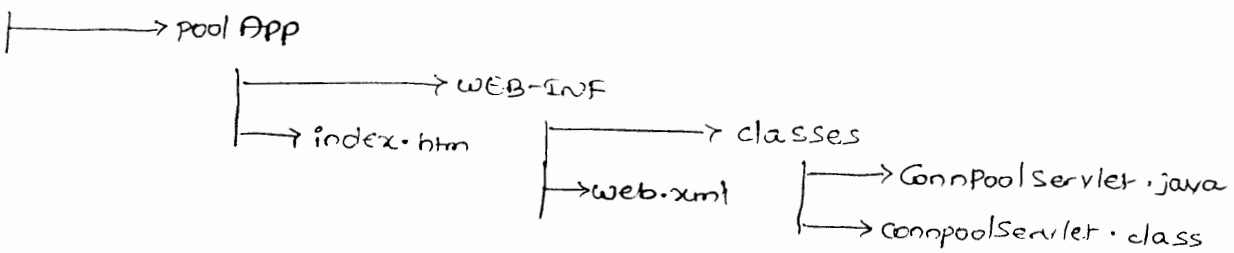
→ ok

NOTE: once ok button is clicked the above created DataSource object will be registered with registry software of GlassFish server automatically with JNDI name "DsJndi"

Ex Example web application where servlet program uses the Connection object of jdbc Connection pool to interact with DB s/w



E:\Apps



* Preparing war file

E:\Apps\poolApp>jar cf PoolApp.war .

* Deploy the above war file in GlassFish 2.x server

Copy PoolApp.war file to <GlassFish_home>\appserver\domains\domain1\autodeploy folder>

Request URL to test the application

(http://localhost:5151/PoolApp/index.html)

* For above diagram based example application refer application 5 of the page nos 65-67.
* Always develop web applications by keeping non-technical end-users in mind for that when exception is raised in web resource programs instead of displaying the exception related technical messages on browser window it is recommanded to display those messages as non-technical guiding messages for end user. For this we need to write some presentation logic in the catch blocks of servlet programs.
* Jndi api is part of java JSE module so we need not to add any jar files to classpath while working with Jndi api.

Procedure to create JDBC Connection pool for oracle Data source in oracle-weblogic 10.3

Step(1): Start Adv.javaBatchDomain Server of weblogic and open its admin console

start → programs → oracle weblogic → user_Projects → Adv.javaBatch Domain → start Admin server for weblogic server domain

step(2): open browser window the following URL

http://localhost:7001/console ↵

user name: javaboss

password: javaboss1

Step(3): create JDBC Data source pointing to the JDBC Connection pool for oracle

Admin console → services → JDBC → Data Sources → New →

Name: mydsl (logical name)

JNDI Name: DSJndi (Required for client while performing lookup operation)

Database type: Oracle

Database driver: "Oracle's Driver (thin) for service Connections"

→ next → next

Database name: satya

Host name: localhost

port no : 1521

Database username : scott

Database password : tiger

Conform password : tiger

→ Test Configuration → next → select admin server → finish

NOTE: when finish button is clicked the data source object that represents JDBC connection pool for oracle will be registered with registry software with jndi name "DSJndi".

Step(4): specify the JDBC connection pool additional properties.

Admin Console → Services → JDBC → Data sources → mydsl → Connection pool tab

→ initial capacity [1]

maximum capacity [30]

capacity increment [2] // specifies the no. of new JDBC connection objects that should be created at a time if there a need of creating new JDBC connection objects in the Connection pool.

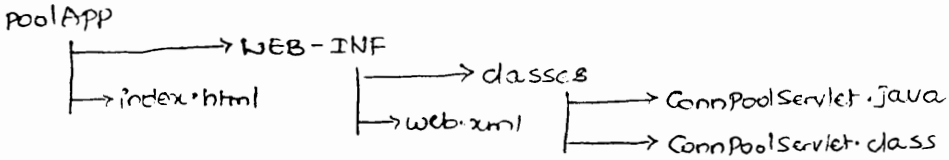
→ save → Advanced → shrink frequency : [1000] seconds → [save]

* for utilizing the above server managed connection pool we can run the application ⑤ of the booklet. For that

Step(i) ; make sure that the Adv. javaBatchDomain server is in running mode

Step(ii): make sure that DSJndi is specified as the argument value of ic.lookup(-) method in line no. 612.

Step(iii): Prepare the deployment directory structure or war file representing the web application.



Step(iv) : Deploy the above PoolApp application in Adv. javaBatchDomain Server of weblogic.

Copy the PoolApp folder to weblogic_home\middleware\User_Projects\Domains\Adv.javaBatchDomain\Auto Deploy folder.

Step(v): Test the application

http://localhost:7001/poolApp/index.html

Jboss server

Type : Application server software

version : 5.x (Compatable with jdk 1.6)

vendor : Apache / RedHat

Open source software

Default port no: 8080

Allows to create domains and also gives 5 built-in domains. They are

(1) default
(2) web
(3) standard
(4) all
(5) minimal

To download software: download s/w as zip file from www.apache.org website.

Installation process

* Extract Jboss-5.1.0.GA-JDK6.zip file to a folder.

* Jar file that represents all JEE APIs : jboss-javaee.jar (<jboss_home>\client)

Procedure to change the port no. of "default" domain server of Jboss 5.x

Go to Jboss_Home\server\default\deploy\jbossweb.jar\server.xml file and modify the post attribute of first <Connector> tag.

TO start jboss server ("default" domain)

use jboss_home\bin\run.batch file.

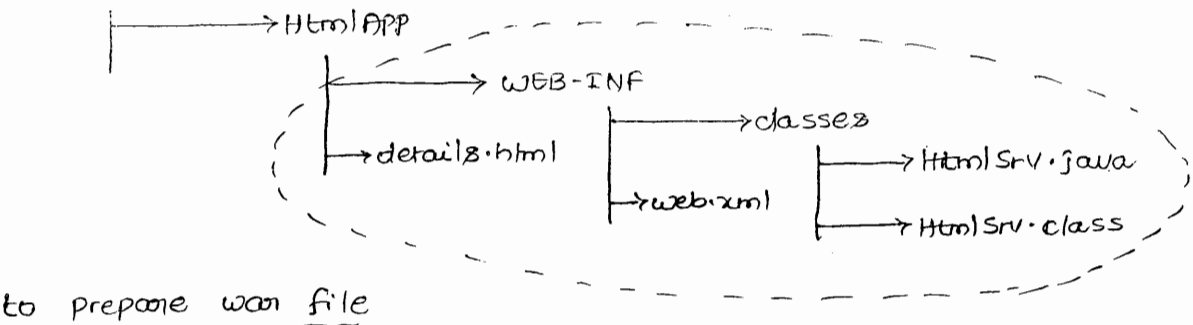
* In Jboss server there is no console deployment. But hard deployment is available (only war file based hard deployment is supported).

NOTE:

Jboss, GlassFish server directory based hard deployment. They only support .war file based hard deployment.

Procedure to deploy Java web application in "default" domain of Jboss 5.x server

Step(1): create .war file on the deployment directory structure of web application.



to prepare war file

E:\Apps\HtmlApp> jar cf HtmlApp.war .

Step(2): start "default" domain server of Jboss

Step(3): Deploy the web application in Jboss server

copy the above "HtmlApp.war" file to Jboss-home\server\default\deploy folder.

Step(4): Test the web application

open the browser window and type this url

http://localhost:5151/HtmlApp/details.html

Common library folder: The common library folder of 'default' domain of Jboss is: Jboss-home\server\default\Lib folder

Procedure to Configure Jboss with myEdipse IDE:

window menu ⟶ preferences ⟶ myEdipse ⟶ servers ⟶

Jboss 5.x ⟶ Enable Jboss_home directory: G:\Jboss 5.x soft\Jboss-5.1.0.GA

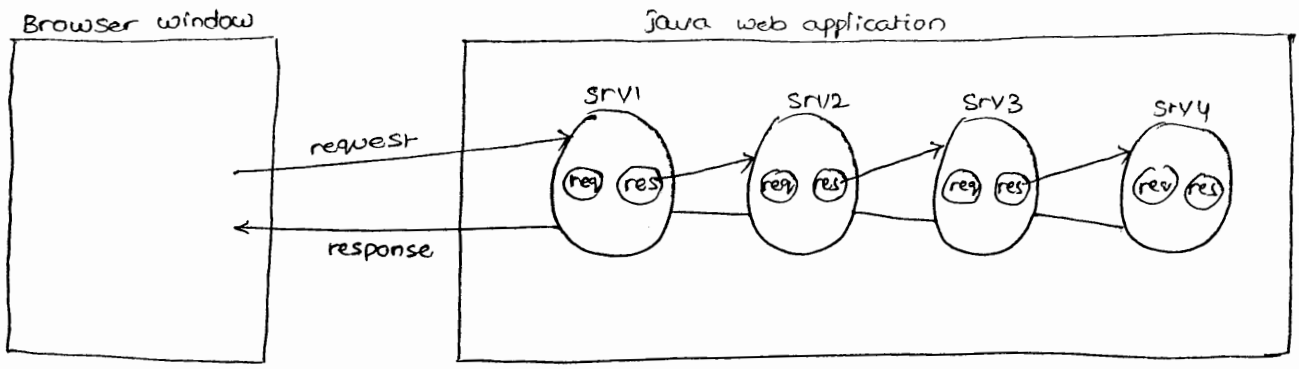
Server name: default (domain name)

⟶ apply ⟶ ok

31/10/11:

Servlet chaining

* Taking request from a browser window and processing that request by using multiple servlets as a chain is called as servlet chaining. In servlet chaining we perform communication between servlet programs to process the request given by a client.

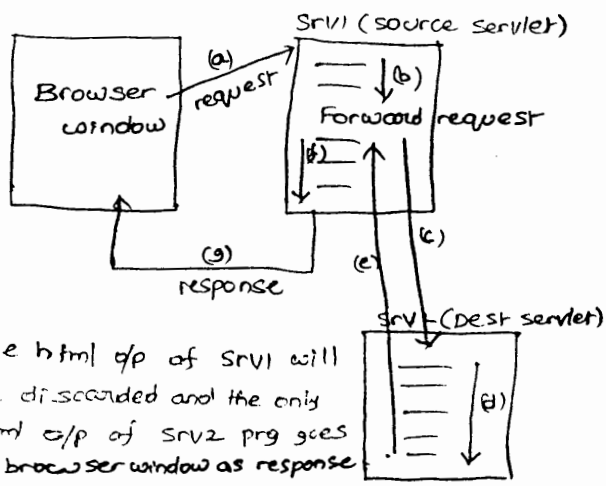


Servlet chaining based webapplication

* All servlet programs that participate in a servlet chaining will use same request and response objects because they process the same request that is given by client
* To perform servlet chaining we need RequestDispatcher object. Request_Dispatcher object means it is the object of a container supplied java class implementing javax.servlet.RequestDispatcher interface.

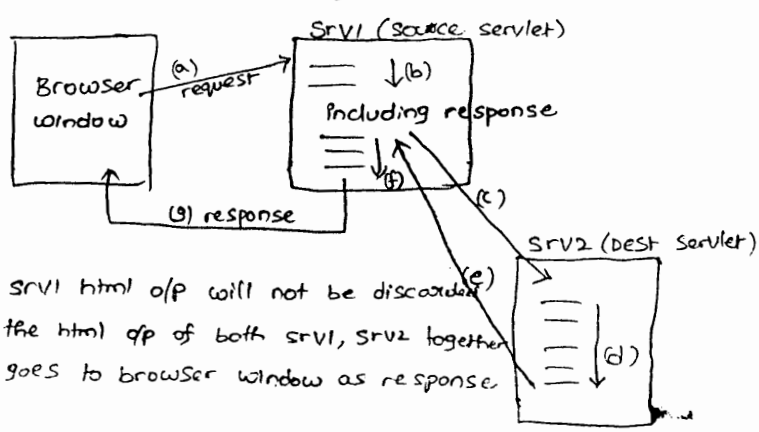
Servlet chaining

1. Forwarding Request mode of servlet chaining



The html o/p of Srv1 will be discarded and the only html o/p of Srv2 prg goes to browser window as response

2. including Response mode of servlet chaining.



Srv1 html o/p will not be discarded the html o/p of both Srv1, Srv2 together goes to browser window as response.

* In any mode of servlet chaining all the servlet programs / web resource programs will use same request and response objects. If Srv1, Srv2, Srv3, Srv4 servlet programs are there in forwarding request mode of servlet chaining then the html o/p of Srv1, Srv2, Srv3 programs will be discarded and only the o/p of srv4 servlet program goes to browser window.

* If Srv1, Srv2, Srv3, Srv4 servlet programs are there in including response mode of servlet chaining then the html o/p of all servlet programs together goes to browser window as response.

* The source servlet program uses RequestDispatcher object to perform servlet chaining with destination web resource program (like servlet program, JSP program html program and etc...)

There are 3 approaches to create RequestDispatcher object in source servlet program pointing to destination webresource program.

Approach(1) (by using req obj)

Ex: In Srv1 source code

```
RequestDispatcher rd = req.getRequestDispatcher("/s2url");
                                                  ↑ optional
                                                  ↳ URL pattern of destination servlet Program (Srv2)
rd.forward(req, res);
      (or)
rd.include(req, res);
```

Ex(2): In source Srv1 Program

```
RequestDispatcher rd = req.getRequestDispatcher("/abc.html");
                    (or)
RequestDispatcher rd = req.getRequestDispatcher("/abc.jsp");
rd.forward(req, res);
      (or)
rd.include(req, res);
```

Approach (2) (by using servletContext obj)

Ex(1): In source Srv1 program

```
ServletContext sc = getServletContext();
RequestDispatcher rd = sc.getRequestDispatcher("/s2url");
                                                  ↑ manditory
                                                  ↳ URL pattern of dest Srv Program (Srv2)
rd.forward(req, res)
   (or)
```

```
rd.include(req,res);
```

Ex(2): In source SrvI program

```
ServletContext sc = getServletContext();
RequestDispatcher rd = sc.getRequestDispatcher("/abc.html");   → mandatory
                    (or)
RequestDispatcher rd = sc.getRequestDispatcher("/abc.jsp");
rd.forward(req,res);
        (or)
rd.include(req,res);
```

Approach (3) (by using servletContext obj)

Ex(1): In src srvI program

```
ServletContext sc = getServletContext();
RequestDispatcher rd = sc.getNamedDispatcher("S2");
rd.forward(req,res);
        (or)
rd.include(req,res);
```

↳ logical name of destination servlet program (like srv2) given in web.xml file

Ex(2): In src SrvI program

```
RequestDispatcher rd = sc.getNamedDispatcher("j2");
rd.forward(req,res);
        (or)
rd.include(req,res);
```

↳ logical name of destination JSP program like abc.jsp

NOTE (1): If source servlet program calls rd.forward(req,res) method then it performs forwarding request mode of servlet chaining with destination web resource program.

NOTE (2): In source servlet program calls rd.include(req,res) method then it performs including response mode of servlet chaining with destination web resource program.

* servlet configuration in web.xml is mandatory so it contains logical name and URL pattern.

* JSP program configuration in web.xml file is optional so it may or may not contain logical name and URL pattern. we can not configure html

Program in web.xml file :-

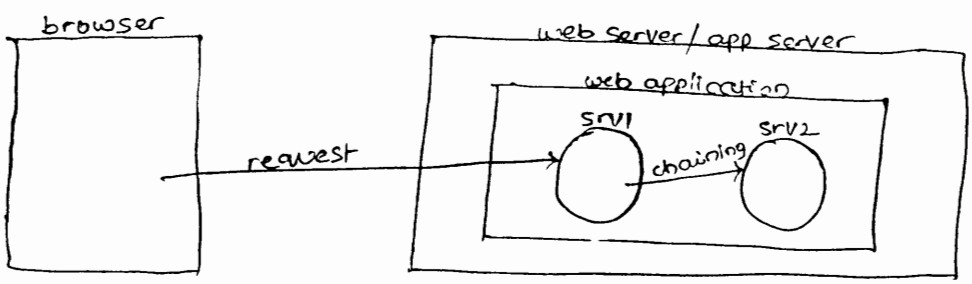
What is the difference between getRequestDispatcher( ) and getNamedDispatcher(-) methods ?

getRequestDispatcher( )	getNamedDispatcher( )
(1) Invokable on both request, servletContext objects.	(1) Invokable only on ServletContext object.
(2) Expects URL pattern of destination Servlet program or file names of destination JSP or html programs as argument value	(2) Expects logical name of the destination servlet/JSP program as argument value.
(3) This method generated RequestDispatcher object can point only the destination servlet, JSP program and html program.	(3) This method generated RequestDispatcher object can point only the destination servlet, JSP programs.

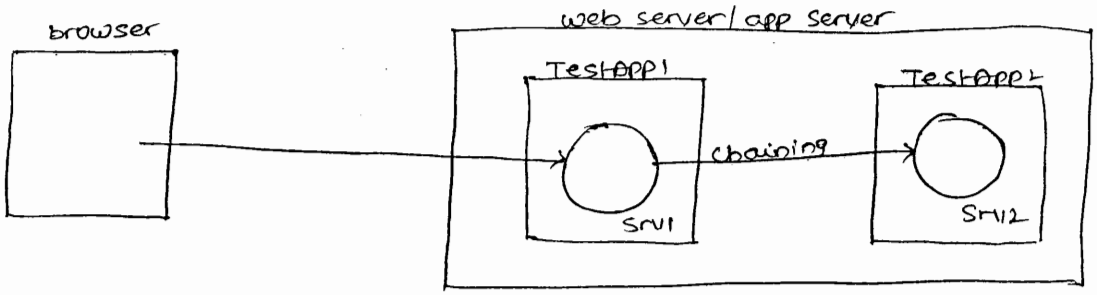
What is the difference between the RequestDispatcher object that is created based on RequestObject and ServletContext object ?

(1) The Request object based RequestDispatcher object expects that source servlet program and destination web resource program in a same web application.

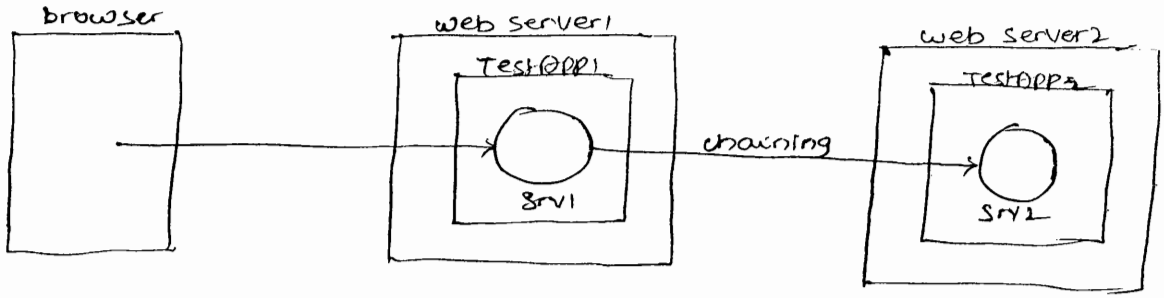
* The ServletContext object based RequestDispatcher object allows to keep the source servlet program and destination web resource program either in the same web application or in two different web applications of same server but they can not be there in two different web applications of two different servers.



Here Srv1 can use req obj or ServletContext obj based RequestDispatcher obj.



Here Srv1 should use ServletContext obj based RequestDispatcher obj



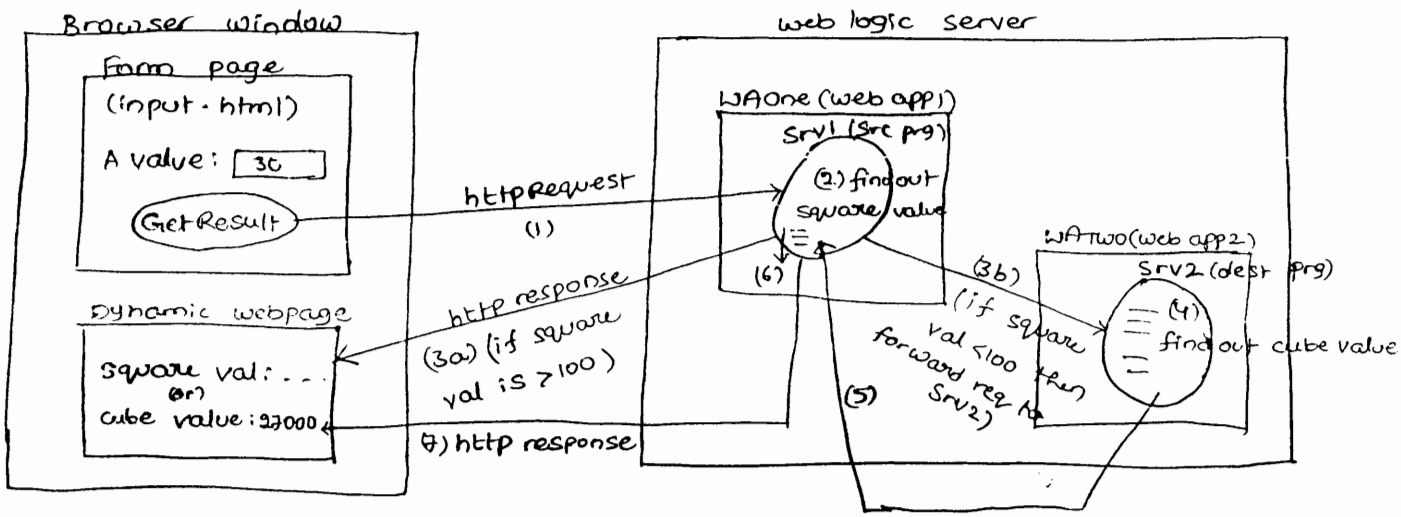
This kind of servlet chaining is not possible with RequestDispatcher obj

- use send redirection Concept

3/11/11

* Servlet chaining is all about performing servlet-servlet Communication.

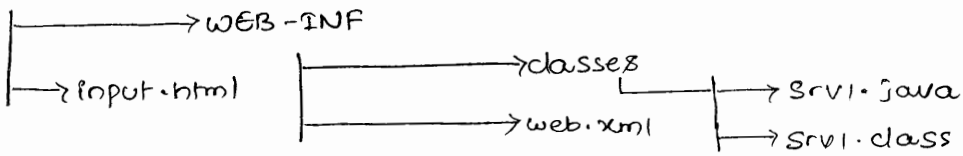
performing servlet chaining between two servlet programs of two different web applications which reside in the same server



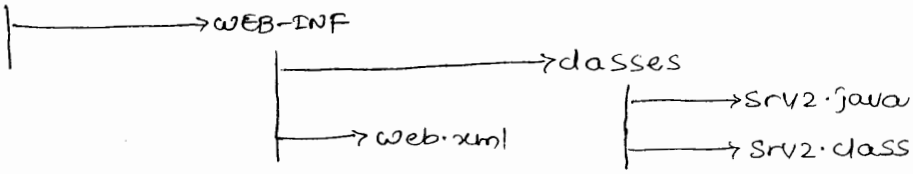
NOTE: In the above diagram Srv1 program forwards the request to Srv2 only when the generated square value is less than 100 other wise the Srv1 directly sends response to browser window displaying that square value.

deployment directory structure

WAOne



WATWO



Deploy both these webapplications in web logic server (Adv. javaBatchDomain).

Copy WAOne, WATWO web applications to <oracle weblogic_home>\user_projects\domains\Adv. javaBatchDomain\autodeploy folder.

* In Srv1 servlet program of the above WAOne webapplication we must create RequestDispatcher object based on ServletContext object.

Source Code (WAOne)

input.html

```
<form action="s1url" method="get">
                  └→url pattern of Srv1 servlet program
      A value : <input type="text" name="t1"><br>
      <input type="submit" value="getResult">
</form>
```

Srv1.java

```
//Srv1.java
import javax.servlet.*;
import javax.servlet.http.*;
import java.io.*;
public class Srv1 extends HttpServlet
{
   Public void service(HttpServletRequest, HttpServletResponse res) throws
                                                  ServletException, IOException
     {
        //general settings
        PrintWriter pw=res.getWriter();
        res.setContentType("text/html");
```

```
            //read form data
            int no = Integer.parseInt (req.getParameter ("ti"));
            //find out square value
            int resl= no * no;
            if (resl>=100)    //display square value on browser window
             {
               pw.println ("srv1: square val is :"+resl);
             }
            else
             {
               //forward the request to srv2 program of WATwo web application
               //get Access to servlet Context obj of wAone web application
                ServletContext sc1 = getServletContext ();
               //get Access to servlet Context obj of WATwo webapplication.
                ServletContext sc2 = sc1.getContext ("WATwo");
               //Create RequestDispatcher obj pointing to srv2 program of
                                                    WATwo web application
               RequestDispatcher rd= sc2.getRequestDispatcher ("/s2url");
               //forward the req to srv2 program from srv1 program
               rd.forward (req, res);
             }//else
          }// service(-,-)
       }//class
```

web.xml

Configure srv1 program with /s1url as URL pattern.

Source code (WATwo)

srv2.java

```
import javax.servlet.*;
import javax.servlet.http.*;
import java.io.*;
public class srv2 extends HttpServlet
 {
    Public void service (HttpServletRequest req, HttpServletResponse res) throws
                                                    servletException, IOException.
      {
         //general settings
         PrintWriter pw = res.getWriter();
```

```
        res.setContentType("text/html");
        //read form data
        int no = Integer.parseInt(req.getParameter("t1"));
        //find out cube value
        int res2 = no * no * no;
        //display cube value
        pw.println("srv2: cube value is:"+res2);
    }//service(-,-)
}//class
```

web.xml

Configure srv2 program with the URL pattern /s2url

URL to test the application

http://localhost:7070/wfone/input.html

* In most of the servers the getContext() method of javax.servlet.ServletContext interface is not implemented so when that method is called in those servers we will get java.lang.NullPointerException. Due to this servlet chaining between two different servlet programs of two different web applications is not possible in most of the servers (like weblogic 10.3, Tomcat all versions, JBoss versions, Glassfish all versions) but the above said chaining can be achieved in weblogic 8.x and 9.x servers.

* Since getContext() method of servlet context interface is not implemented properly in all servers so we can say the RequestDispatcher object based servlet chaining is good only when both source servlet program and destination web resource program resides in same web application. In remaining situations it is recommended to use send redirection concept.

What is the difference between rd.include(), rd.forward() methods?

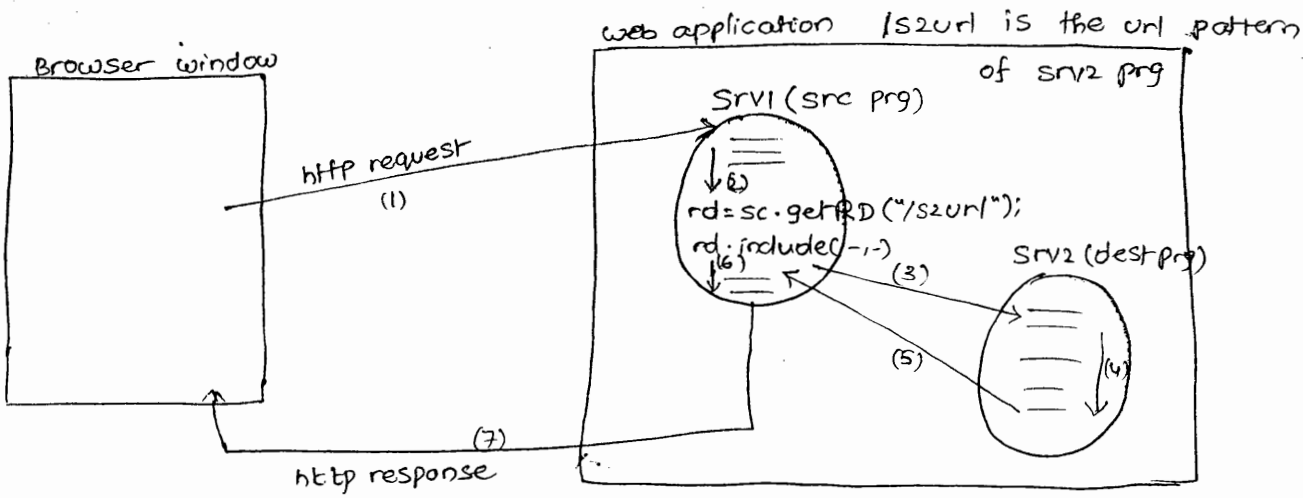
=) Refer respective diagrams and key points of previous concepts.

* To know the current java version >java -version

* when multiple jdk softwares are installed in your computer having different versions then that jdk version will be activated whose <jdkversion>\bin directory is placed as first value of path environment variables.

2/11/11

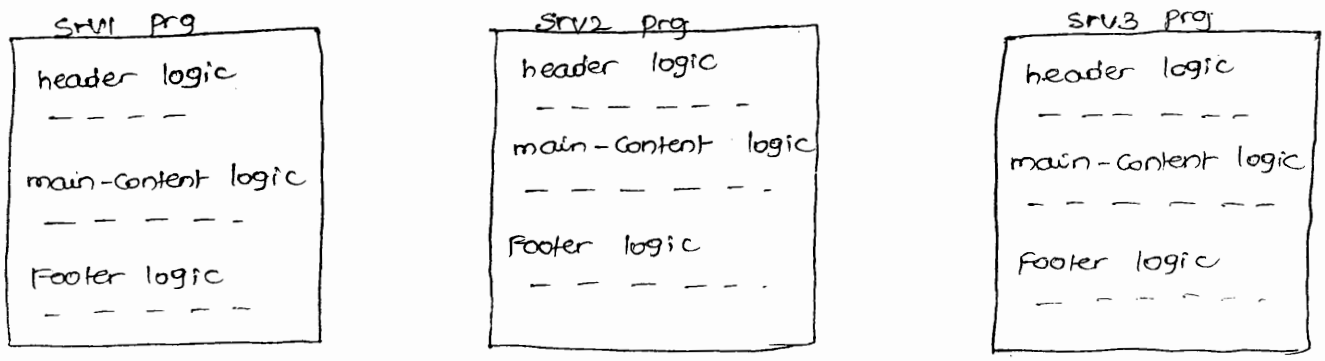
understanding rd.include(-,-)



Here srv1 program (src prg), html o/p will not be discarded. moreover srv1, srv2 servlet programs html outputs together goes to browser window as response.

* rd.include(-,-) method performs including response mode of servlet chaining.
* Both srv1 and srv2 programs will use same request and response object so the request data coming to srv1 is visible and accessable in srv2. To pass additional data between srv1 and srv2 use request attributes.
* Browser window gets response from srv1 program by having the html o/p of both srv1 and srv2 programs
* srv2 html o/p can be included any where in the html o/p of srv1 by calling rd.include(-,-) method in the required place.
* srv1 and srv2 can be there in the same web application (or) can be there in two different web applications of same server.
* Srv2 can be a servlet program, JSP program (or) html program.
* In real time rd.include(-,-) is useful to include the o/p of reusable logics like header and footer logics in the output of (html output) main web resource programs.

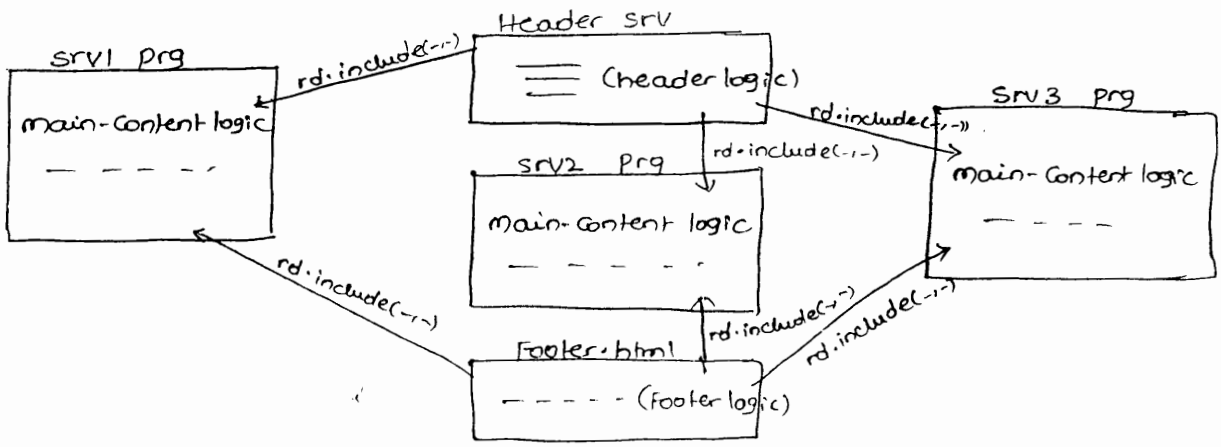
Problem :



(Generates web page1) (Generates web page2) (Generates web page 3)

* All web pages of website generally contains same header and footer contents but the main content will be changed in every web page.
* In the above diagram header and footer logics are not reusable because they are placed in all the three servlet programs eventhough they are same for all the three servlet programs.

Solution :



+ In solution diagram header and footer logics are seperated from main servlet programs of web application and they are placed in two seperate web resource programs but their html output is included in main servlet programs html output. This indicates header and footer logics are made as reusable logics
(included using rd.include(-,-))

sample code based on solution diagram

Headersrv.java (having header logic)

```
public class HeaderSrv extends HS
 {
    public void service(-,-) throws servletException, IOException
     {
        ------  PrintWriter pw = res.getWriter();
        ------
        ------
                res.SetContentType ("text/html");
                pw.println("<font color=redb SIZE=6> S A T H Y A </font>");
                pw.println("<br><br><br><hr>"); //Don't place pw.close() in
     }                                                    HeaderSrv.
 } /headurl is the Headersrv servlet program
```

Footer.html (having footer logic)

```
<hr>
<br><br><br>
<b><i> &copy; All rights reserved 2101-11 </b></i>
```

MainServlet prgs

```
Public class ~~Test~~Srv 1/2/3... In extends HS
 {
    Public void service(-,-) throws SE, IOE
     {
       RequestDispatcher rd1=null, rd2=null;
       try
        {
          //include header content from HeaderSrv
           rd1 = req.getRequestDispatcher ("/headerurl");
           rd1.include (req,res);
           ------
           ------  //logic to generate main content of web page
           ------
          //include footer content from Footer.html
           rd2=req.getRequestDispatcher ("/footerurl");
           rd2.include (req.res);
        }
       catch (Exception e)
        {
          e.printStackTrace();
        }
     }
 }
```

* once the rd.forward(-,-) method executed in source servlet program then the effect of rd.include(-,-) method call will not be there in that servlet program that means rd.forward(-,-) method call discards both directed and included html outputs of source servlet program.

* For example application on both rd.forward(-,-), rd.include(-,-) method calls refer app (6) of the page nos 67-70.

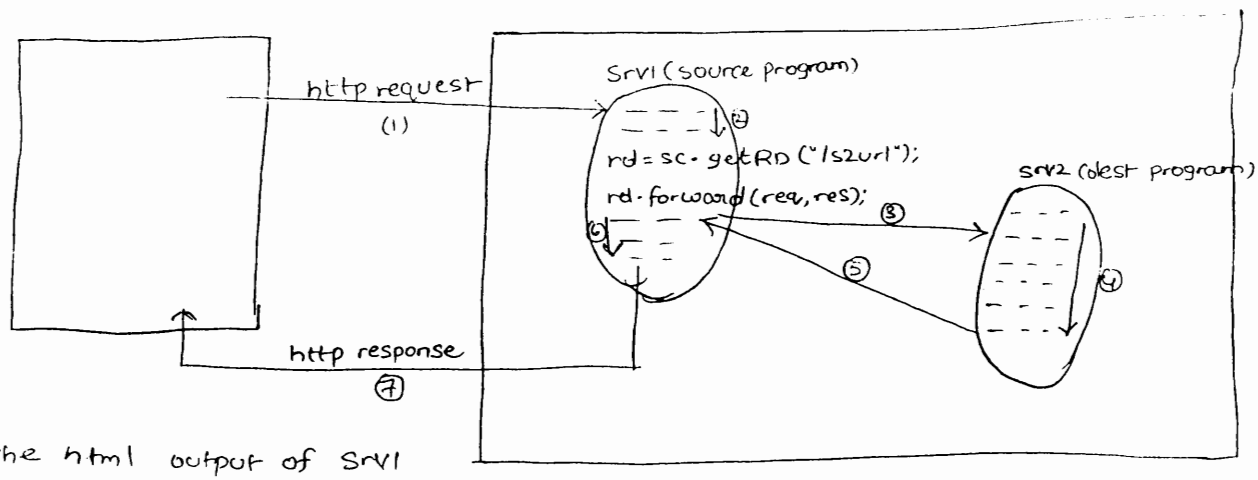
* For example application on rd.forward() and rd.include() also refer app (8) of material.

* while working with rd.include(-,-) method don't commit the response in destination web source program by calling pw.close() method because it won't allow us to add further output generated by source servlet program and other destination programs. But we can do this work while working with rd.forward(-,-) method.

* Don't commit the response in source servlet program by calling pw.close() method before rd.forward(-,-) method call because this process throws java.lang.IllegalStateException (Can't forward after response has been committed)

NOTE: pw.close() commit the response of web resource program that means it does not allow to add further content to response.

Understanding rd.forward(-,-) method



The html output of Srv1 will be discarded and only the html output of Srv2 will go to [illegible] response.

Key points :

* rd.forward(-,-) method is there to perform forwarding request mode of servlet chaining.
* Both SRV1 and SRV2 servlet programs will use same request and response objects so the request data coming to SRV1 (lik request parameters, headers are visible and accessable in SRV2 program.
* To pass additional data between SRV1 and SRV2 programs use "request" attributes.
* All the statements placed in SRV1 program before and after rd.forward method will be executed but the entire html output of SRV1 program will be discarded.
* Both SRV1 and SRV2 can be there either in the same webapplication (or) two different webapplications of same server.
* SRV2 can be a servlet program (or) JSP program (or) html program.
* rd.forward(-,-) is very useful to configure Error Servlet for other main servlet programs of the web application.

What is Error servlet?

(A) The servlet program that executes only when exceptions are raised in other servlet program is called as Error servlet. This servlet is useful to display exception related messages as non-technical guiding messages on browser window when exceptions are raised in other servlet programs of web application.

Sample code to configure Error Servlet

ErrSrv.java

```
public class ErrSrv extends HS/GS
 {
    public void service(-,-) throws SE, IOE
     {
        res.setContentType("text/html");
        PrintWriter pw = res.getWriter();
        pw.println("<font color=red>Internal Problem</font>");
```

```
        pw.println("<br> Go to <a href='input.html'> Home </a>");

    }// service(-,-)

}// class.
```

// "/eurl" is the url pattern of ErrSrv program (in web.xml).

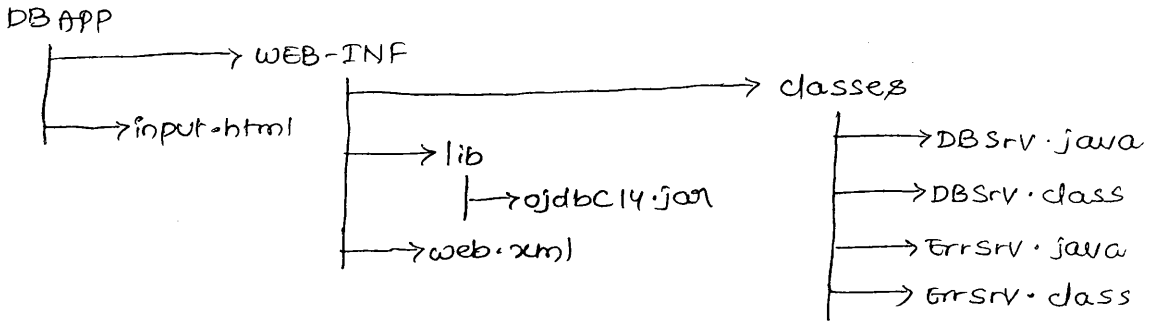
DBSrv.java (main servlet program of web application)

```
public class DBSrv extends HS
{
    public void init()
    {
        ---
        ---
        ----        Same as previous application
        ----        (JDBC code)
        ----
        ----
    }
    public void doGet(-,-) throws SE, IOE
    {
        try
        {
            ---
            ----        same as previous application
            ---         (JDBC code)
            ----
        }
        catch(Exception e)
        {
            try
            {
                RequestDispatcher rd = req.getRequestDispatcher("/eurl");
                rd.forward(req,res);
            }
            catch(Exception e1)
            {
                e1.printStackTrace();
            }
        }
    }
```

```
    public void doPost(-,-) throws SE, IOE
      {
        doGet(req,res);
      }
     public void destroy()
      {
        - - -
        - - -
      }
  }
```

* In the above sample code when exception is raised in the doGet(-,-) method of DBSrv servlet program the Error servlet (ErrSrv) program will be executed automatically because of the rd.forward(-,-) method call.
* If servlet program contains rd.forward(-,-) method call then that servlet program html output will be discarded but their s.op("-") statements output will not be discarded.
* Always keep rd.forward(-,-) method call in source servlet program with condition statement show that the html output of that source servlet program will be discarded only when that condition is satisfied otherwise the html output will not be discarded.
* In real time projects rd.forward(-,-) useful to we Configured Error servlet and to make web applications as more end user friendly web applications.

Deployment directory structure of the above sample code based web application

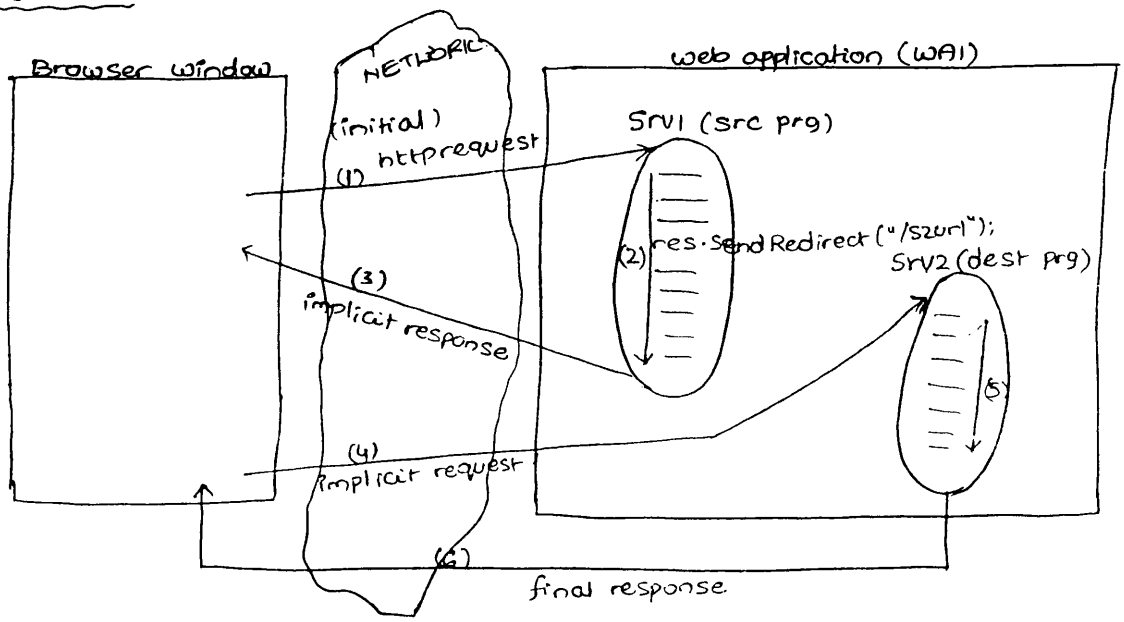


* Develop the above web application resources based on above sample code and application ④ of the booklet.

* The limitation with RequestDispatcher object based servlet chaining is it can not be used when source servlet program and destination web resource program are placed in two different web applications of same server (very few servers are supporting this) (or) in two different web applications of two different servers. To overcome this problem use send redirection concept of response.sendRedirect(-) method.

Understanding sendRedirection

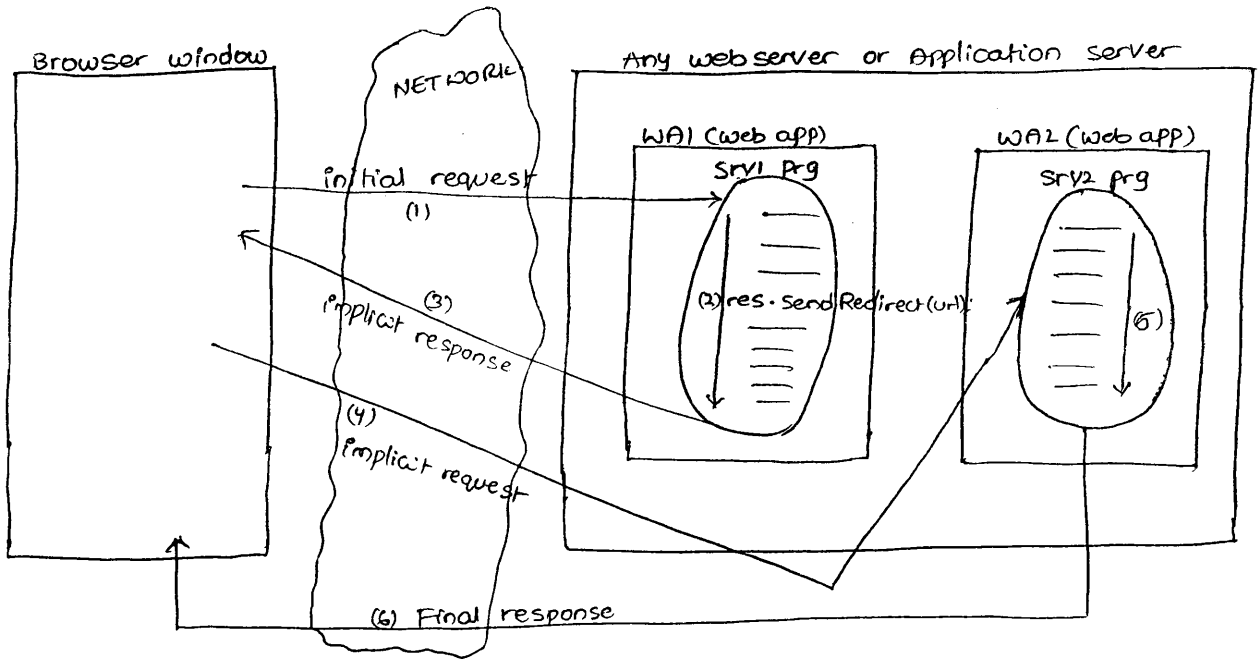
Diagram (1)



The html output of Srv1 (src) servlet program will discarded and only the html output of Srv2 (dest) servlet program will be displayed on browser window as response.

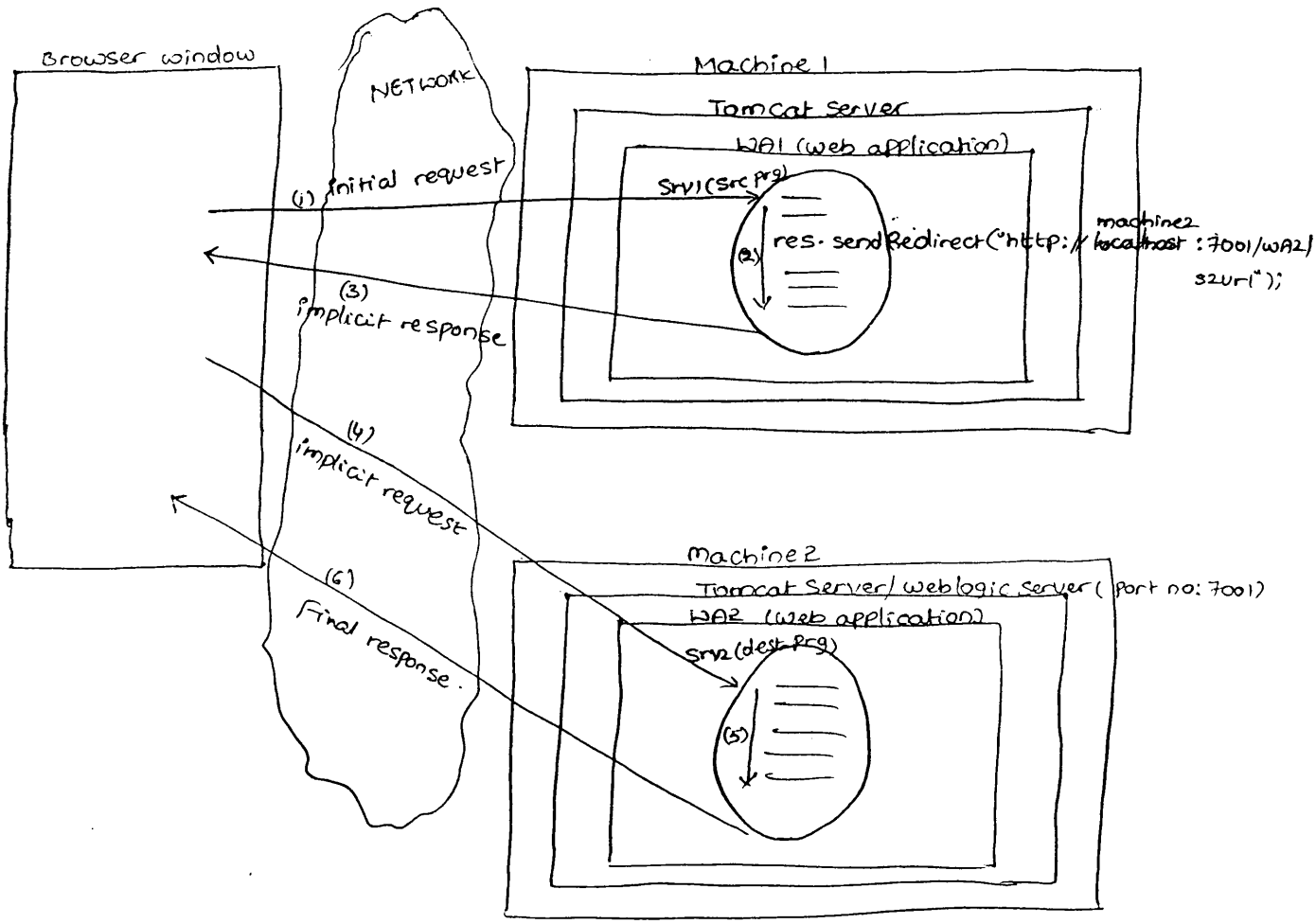
/s2url is the URL pattern of Srv2 servlet program.

Diagram(2)



url : http://localhost:2020/WA2/s2url (Request url of Srv2 program)

Diagram(3)



* Using send redirection concept we can not perform response inclusion but we can forward request from one web servlet program to destination web resource program.

* With respect to the diagrams

(1) Browser window gives initial request to SrV1 servlet program.

(2) All the statements of SrV1 servlet program executes including res.sendRedirect(-) method.

(3) SrV1 program generates implicit response to browser window having the URL placed in sendRedirect(-) method as argument value. The response status code of this implicit response is 300-399 (indicates redirection).

(4) Browser window uses the URL coming from implicit response (because of 300-399 status code) and generates implicit request to SrV2 program.

(5) All the statements of SrV2 program executes.

(6) The output of SrV1 program will be discarded and only the html output of SrV2 program goes to browser window as final response.

Key points

* SrV1 and SrV2 will not use same request and response objects. They use different sets of req, res objects. So the request data coming to SrV1 program is not visible and accessible in SrV2 program.

* To send additional data from SrV1 program SrV2 program append query string to the URL of res.sendRedirect(-) method in SrV1 program and read those values in SrV2 program as request parameter values.

In SrV1 prg

```
res.sendRedirect("/s2url?p1=val1&p2=val2");
        (or)
res.sendRedirect("http://localhost:2020/WA2/s2url?p1=val1&p2=val2");
        (or)
res.sendRedirect("http://machine2:7001/WA2/s2url?p1=val1&p2=val2");
```

In SrV2 prg

```
String s1 = req.getParameter("p1");
String s2 = req.getParameter("p2");
```

* Srv1 and Srv2 can be there in the same web application or can be there in two different web applications of same server or two different servers (these two servers can be there in the same machine or in two different machines).

* If Srv1 and Srv2 resides in the same web application we can pass relative path in sendRedirect(-) method otherwise we must pass absolute URL.

* Srv2 can be a servlet program or JSP program or html program or ASP program or ASP.net program or php program etc...

* The movement res.sendRedirect(-) method executes in Srv1 program the entire html output of Srv1 program will be discarded.

* In real time to pass the request of one website to another website without worrying about their technology environment we can use send Redirection.

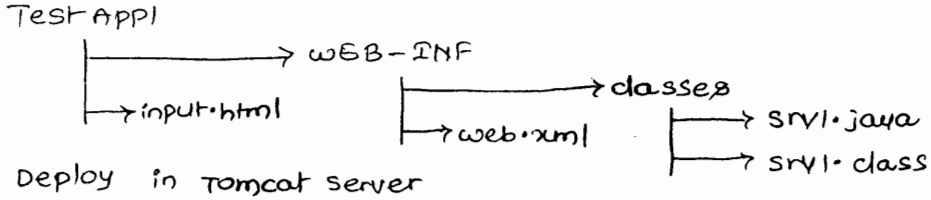
Ex: IBM has acquired rational.com so the request given to rational.com will be redirected to certain web resource program of ibm.com.

Oracle Corporation has acquired sun microsystems so the request given to sun.com will be redirected to a web resource program of oracle.com.

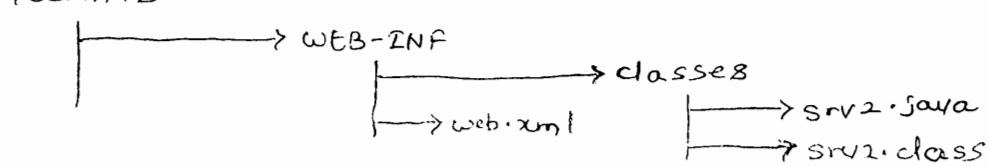
Example application on send Redirection

NOTE: In the below application Srv1 program is sending three values to Srv2 program by appending query string to the url of response.send Redirection() method. Those values are form data, sum result.

Directory structure



Deploy in Tomcat server

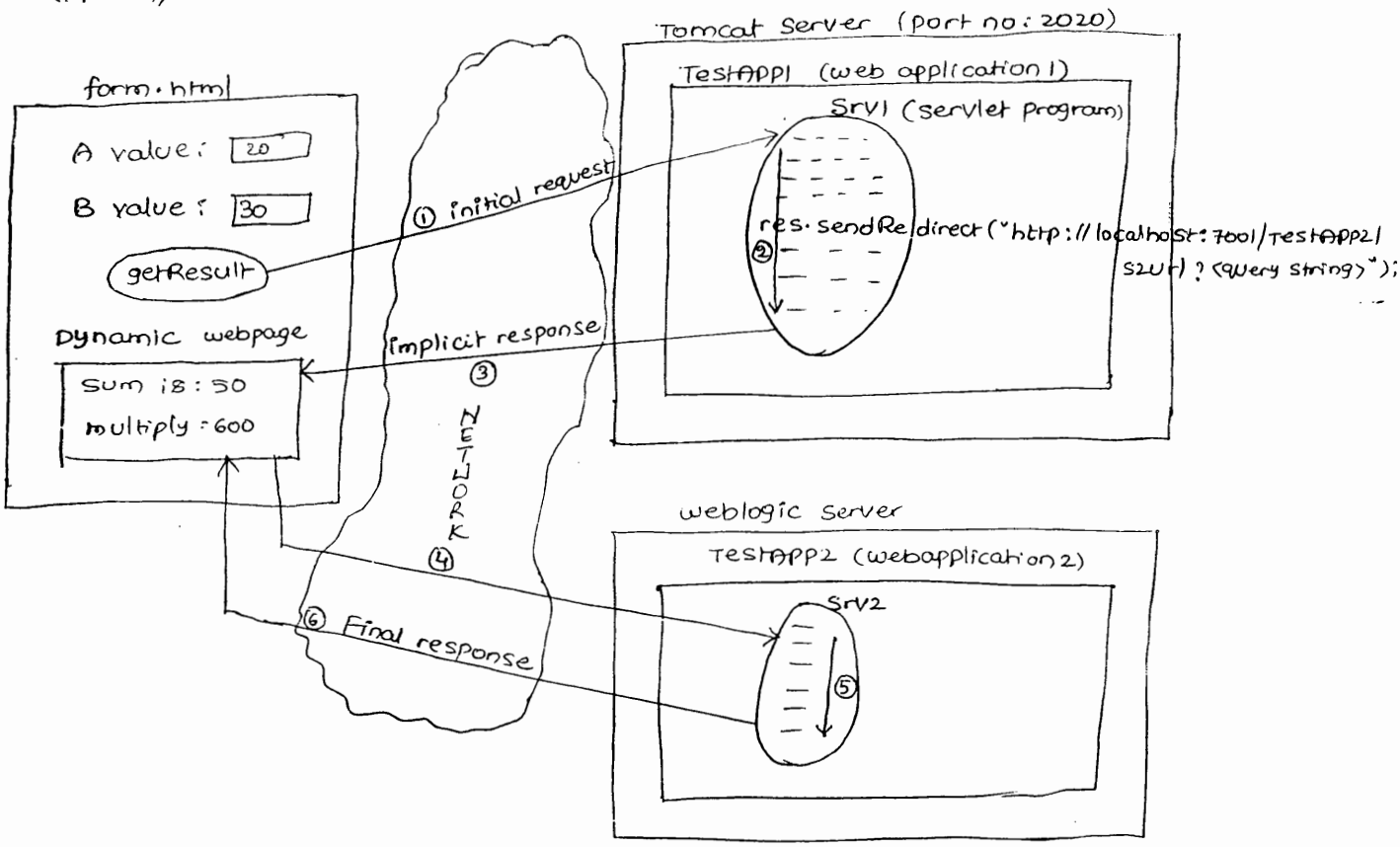


Deploy in Adv.javaBatchDomain of weblogic server.

TestApp1 (web application)

input.html

```
<form action = "s1url">
    A value : <input type = "text" name = "t1"> <br>
    B value : <input type = "text" name = "t2"> <br>
              <input type = "submit" value = "getResult">
</form>
```



Srv1.java

```
import javax.servlet.*;
import javax.servlet.http.*;
import java.io.*;
public class Srv1 extends HttpServlet
{
   Public void service (-,-) throws SE, IOE
    {
       // general settings
       PrintWriter pw = res.getWriter();
       res.setContentType ("text/html");
       //read form data
       int val1 = Integer.parseInt (req.getParameter("t1"));
```

```
            int val2 = Integer.parseInt(req.getParameter("t2"));
            //find sum value
                int sum = val1+val2;
            pw.println("<b> srv1 : sum is </b>" +sum);
           //redirect the request to srv2 program of testapp2
            s.o.p("srv1 : before res.sendRedirect(-)");
            res.sendRedirect("http://localhost:7001/testapp2/s2url?p1="+val1+"&p2=
                                                    "+val2+"&p3="+sum);
            s.o.p("srv2 : after res.sendRedirect(-)");
           //close stream obj
             pw.close();
         }//service(-,-)
    }//class
```

query string having data to send data to srv2 program.

---web.xml

Configure Srv1 servlet program with "/s1url" url-pattern.

TestApp2 source code (web application2 of different server)

Srv2.java

```
import javax.servlet.*;
import javax.servlet.http.*;
import java.io.*;
public class srv2 extends HttpServlet
 {
    public void service(-,-) throws SE, IOE
      {
        //general services
         PrintWriter pw = res.getWriter();
         res.setContentType("text/html");
        //read request param values send srv1 program
         int val1 = Integer.parseInt(req.getParameter("p1"));
         int val2 = Integer.parseInt(req.getParameter("p2"));
         int sum = Integer.parseInt(req.getParameter("p3"));
        //find out multiply value
          int mul = val1 * val2;
        //display results
          pw.println("srv2 : sum is :" +sum);
          pw.println("srv2 : multiply is :" +mul);
          s.o.p("form srv2 program");
          pw.close();
      }
 }
```

web.xml

Configure Srv2 program with "/s2Url" url-pattern.

Request URL to test the application

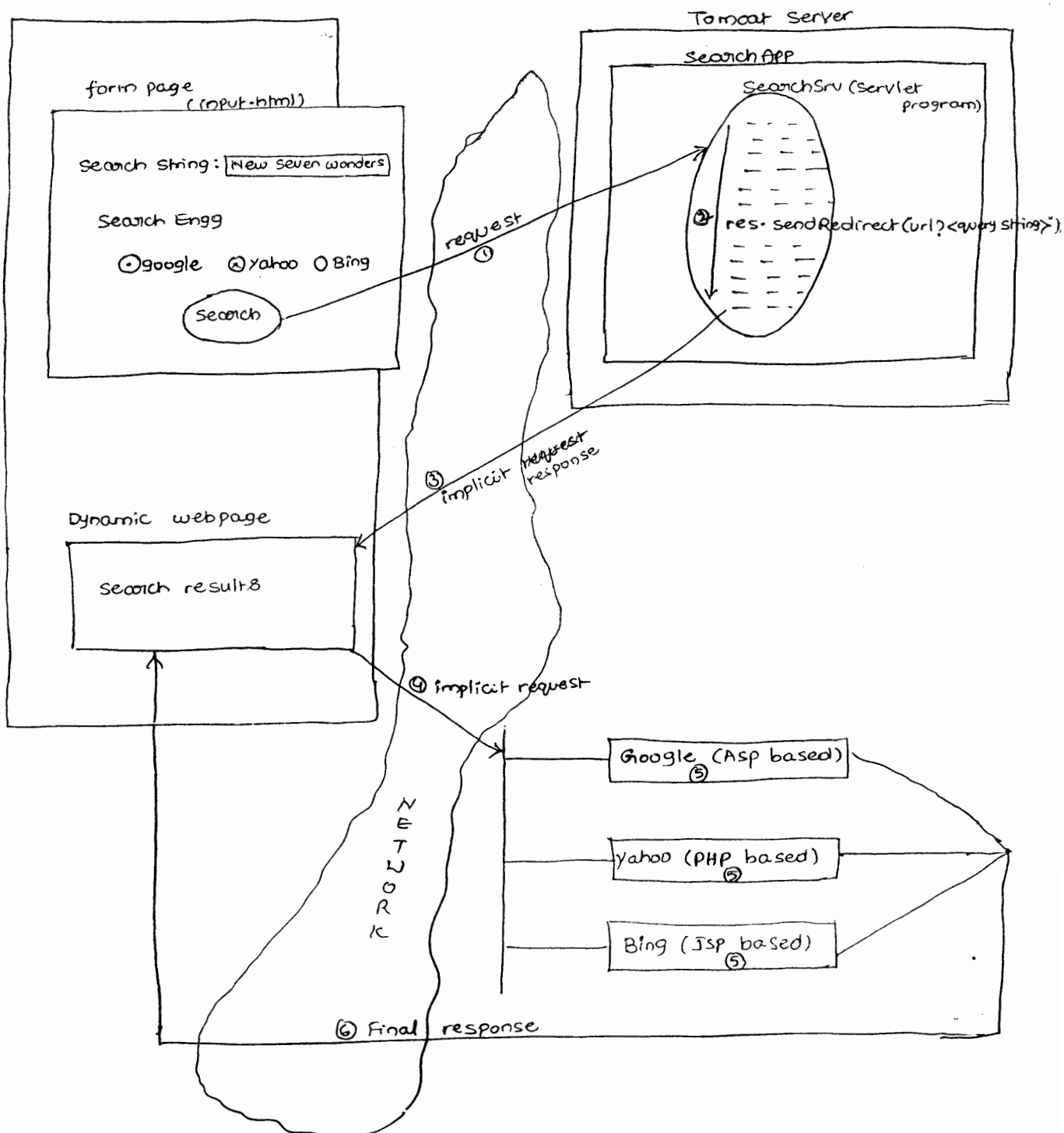
http://localhost:2020/TESTAPP1/form.html

what is the difference between rd.forward() and res.sendRedirect() ?

rd.forward(-,-)	res.sendRedirect(-,-)
* perform forward mode of servlet chaining.	* performs sendRedirection mode of communication.
* The source servlet program communicates with destination webresource program directly.	* The source servlet program communicates with destination servlet program by having network round trip with browser window.
* The source servlet program and destination web resource program uses same request and response objects so request data coming to source servlet program is visible and accessable in destination webresource program.	* The source servlet program and destination web resource program will not use same request and response object so the request data coming to source servlet program is not visible and accessable in destination program.
* Source servlet program can use request attributes to send additional data to destination program.	* Append query string to the url of response.sendRedirect() method to send additional data from source servlet program to destination program.
* source servlet and destination programs can be there in same web application (any server) (or) can be there in two different web applications of same server (only in few servers).	* The source servlet program and destination program can be there in same web application (or) in two different web applications of same server (or) different servers.
* Destination program can be a servlet (or) JSP (or) html program.	* Destination program can be a servlet (or) JSP (or) html (or) ASP (or) ASP.net (or) PHP programs etc...
* During forwarding request operation url in the browser address bar will not be changed.	* During sendRedirection operation the url in the browser address bar will be changed.
* Suitable when source servlet program and destination servlet program resides in the same web application.	* Suitable when source servlet program and destination program resides in the two different web applications.

Example on sendRedirection :

servlet program redirects the request to non java web resource programs of internet websites.



* The above application act as hub for working with multiple search engines.

* The above application the servlet SearchSrv redirects the request to search engine based on the search engine that is choosen in the form page.

* To work with above application gather the web resource program names and request parameter names of different search engines as shown below.

For google

http://www.google.co.in/search?q=new+seven+wonders

request parameter value

For yahoo

http://search.yahoo.com/search?p=new+seven+wonders

request parameter name

For bing

http://www.bing.com/search?q=new+seven+wonders

web resource program name

* For exam source code of above diagram refer application (9) of the page numbers 73-75.

How many ways are there to pass data between the source servlet program and destination web resource program?

(A) If source servlet program and destination web resource program resides in same web application then use

(1) request attributes

(2) session attributes

(3) servletContext attributes

If source servlet program and destination web resource program resides in two different web applications of same/different server(s).

(1) append query string to URL of res.sendRedirect(-) method

```
res.sendRedirect("url?<query string>");
```

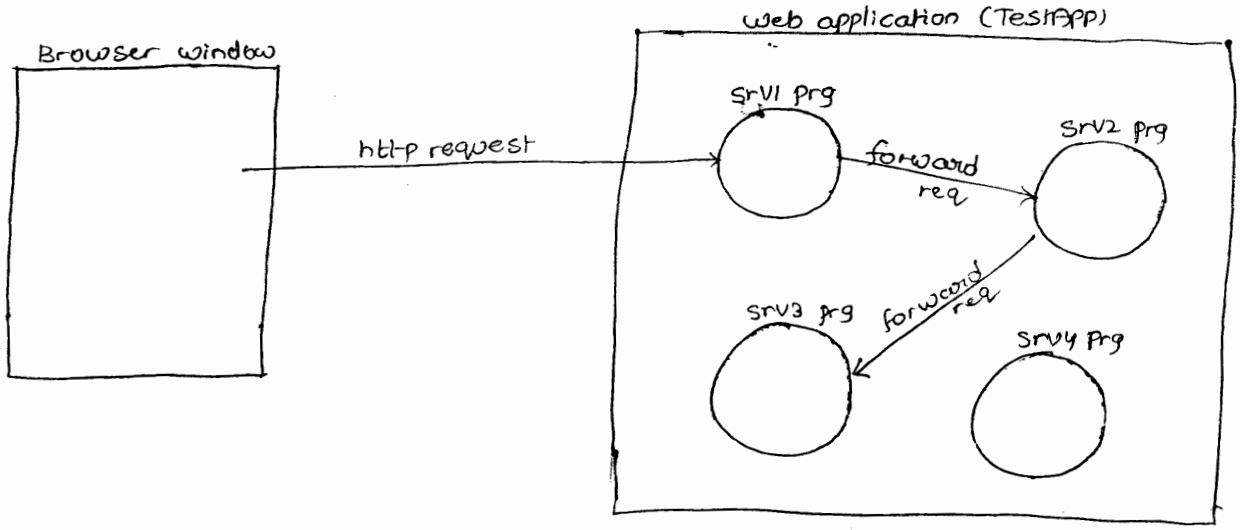
Ex.

```
res.sendRedirect("http://localhost:7001/TestApp2/S2Url?p1=10&p2=20");
```

* Attribute is a special logical name that can hold java object as an value.
* Attributes provide more visibility and accessability to the data of the application compare to the variables declared in servlet program.
* Attributes allows us to store only objects. If we give simple values to attributes then they will be converted into wrapper class objects and will be stored as attribute values.

Request attributes

* These attributes allocates memory in request object and they are visible through out request cycle.



* Request attributes that are created in one servlet program of servlet chaining are visible in other servlet programs of same servlet chaining because all the servlet programs of a servlet chaining will use same request and response objects.
* In the above diagram the request attribute created in srv1 program is visible and accessable in srv2, srv3 program but not visible in srv4 program. because srv1, srv2, srv3 programs are using same request and response objects by participating in servlet chaining and srv4 program is not participating in that servlet chaining.

To create request attribute

```
req.setAttribute("name", "raja");
                  attribute name → attribute value
req.setAttribute("age", new Integer(50));
req.setAttribute("total", 600);
```

attribute value will be converted into wrapper class object Integer through autoboxing.

To modify request attributes values

```
req.setAttribute("name", "raja1");
req.setAttribute("age", new Integer(45));
req.setAttribute("total", 550);
```

NOTE: The method setAttribute(-,-) can create new request attribute or can modify existing request attribute value.

To read values from request attributes

```
String S1= (String) req.getAttribute("name");

Integer S2 = (Integer) req.getAttribute("age");

int S3 = req.getAttribute("total");
```

↓

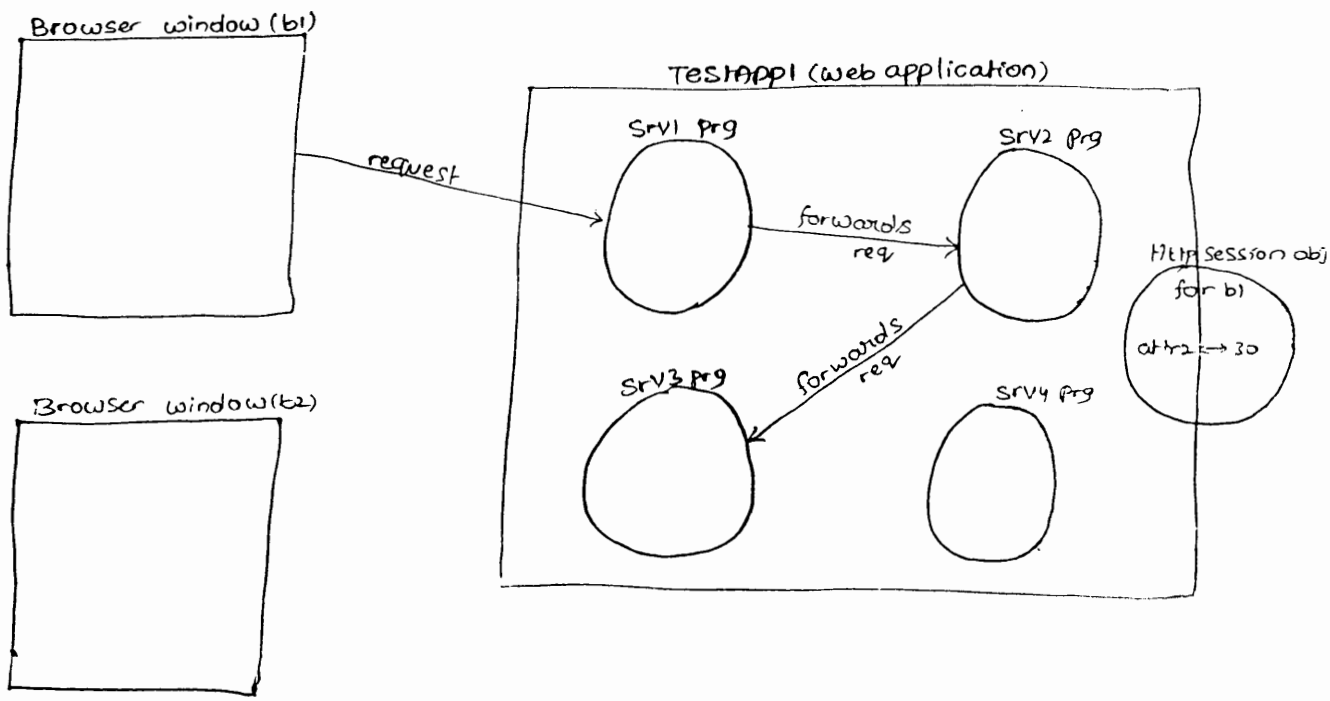
Converts wrapper class object Integer to simple int through auto unboxing concept.

To delete/remove request attribute

```
req.removeAttribute("name");
req.removeAttribute("age");
req.removeAttribute("total");
```

Session Attribute

* Session attributes allocates memory in http session object. The http session object allocates memory on the server and it is one per browser window so the session attributes of http session object are visible and accessable in all web resource programs of web application irrespective of their request and response objects but they must get request from that browser window for which http session object and its attributes are created.



* The session attribute created in srv1 servlet program by getting request from browser window b1 is visible and accessable in all other servlet programs of web application but they must get request from same browser window b1.

* session attributes are global attributes in the web application but they are specific to a browser window (client).

To create/access Httpsession object for browser window on servlet

```
HttpSession ses = req.getSession();
```

↓ creates or locates HttpSession object

NOTE: This req.getSession() method checks the availability of session object on server for browser window, if available it provides access to that session object otherwise (if not available) it creates new session object (HttpSession object) for browser window on the servlet.

Session attributes

To create ses attribute or to modify ses attribute value

```
ses.setAttribute("attr2", new Integer(30));
```

To read ses attribute value

```
Integer i1 = (Integer) ses.getAttribute("attr2");
```

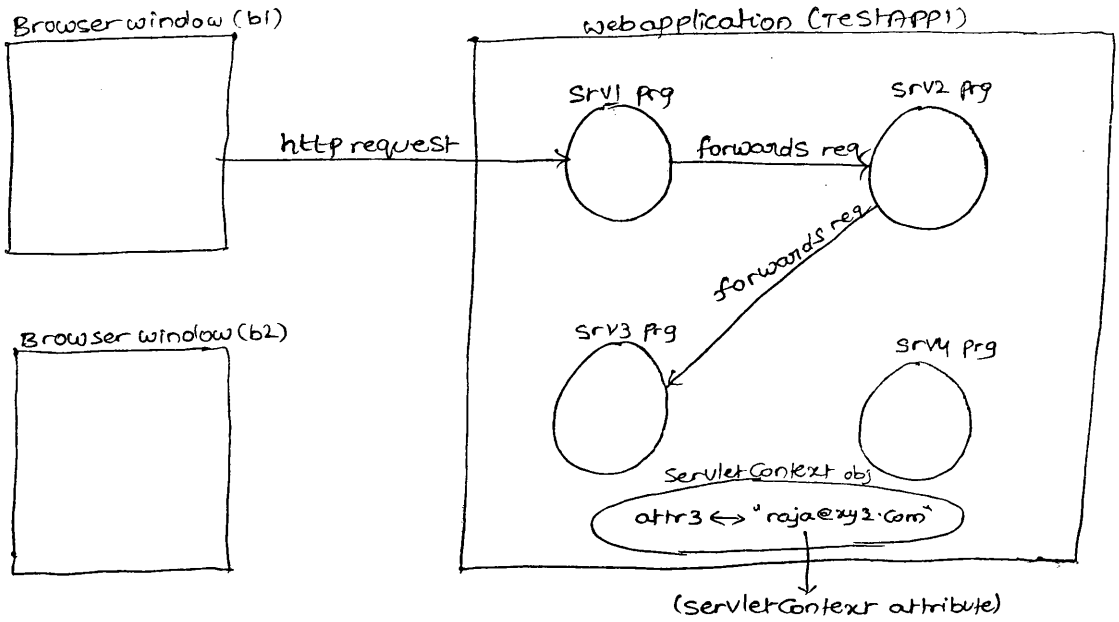
To remove ses attribute

```
ses.removeAttribute("attr2");
```

ServletContext attributes

* ServletContext attributes allocate memory in servletContext object Since servlet context object is one per web application and global memory of web application so the servletContext attributes are visible and accessable in all web-resource programs of web application irrespective of request and response objects they are using and irrespective of the browser window from which they are getting request.

* In the below diagram the servletContext attribute created in srv1 program is visible and accessable in all the remaining servlet programs of web application irrespective of the request and response objects they are using and irrespective of browser window from which they are getting request.



To create or modify servletContext attribute

```
ServletContext sc = getServletContext(); // gives access to servletContext object
sc.setAttribute("attr3", "raja@xyz.com");
```

To read servletContext attribute value

```
String s1 = (String) sc.getAttribute("attr3");
```

To remove servletContext attribute

```
sc.removeAttribute("attr3");
```

What is the difference between request parameters and request attributes?

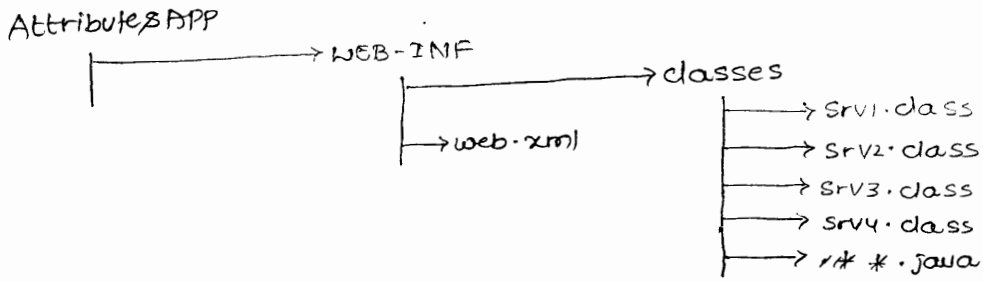
(A) Request parameters are client supplied input values to web resource program (form data). Request attributes are programmer created additional data to pass from one web resource program to another web resource program when they are participating in chaining (like servlet chaining).

Conclusion

* If source servlet program and destination program resides in the use same request and response objects then use request attributes to send the data.
* If source servlet program and destination program are getting request from same browser window then we use session attributes for sending data.

* If source servlet program and destination webresource program can not satisfy above two conditions then use ServletContext attributes for sending data.

Example application on to understand the scope of various attributes



Srv1.java

```
Public class Srv1 extends HttpServlet
{
   public void service (HttpServletRequest req, HttpServletResponse res) throws SE, IOE
    {
      //general settings
       res.SetContentType("text/html");
       PrintWriter pw = res.getWriter();
       //creating request attribute
        req.setAttribute("attr1","raja");
        //creating session attribute
        HttpSession ses = req.getSession(); // creates session obj
        ses.setAttribute("attr2", new Integer(30)); // creates session attribute
        //creating servletContext attribute
         ServletContext sc = getServletContext(); // creates session obj
         sc.setAttribute("attr3", "raja@xyz.com"); // creates servletContext attribute
         // forward req to Srv2 prg
         RequestDispatcher rd = req.getRequestDispatcher("/Srv2");
          rd.forward(req,res);
     } //service(-,-)
} // class
```

Srv2.java

```
public class Srv2 extends HttpServlet
{
    public void service(-,-) throws SE,IOE
    {
        //general settings
        res.setContentType("text/html");
        PrintWriter pw = res.getWriter();
        //reading req attribute value
        pw.println("Srv2: attr1(request) attribute value:" + req.getAttribute("attr1"));
        //reading session attribute value
        HttpSession ses=req.getSession(); //gives access to session obj
        pw.println("Srv2: attr2(session) attribute value:"+ ses.getAttribute("attr2"));
        //reading servletContext attribute value
        ServletContext sc = getServletContext();
        pw.println("Srv2: attr3 (servletContext) attribute value:"+ sc.getAttribute("attr3"));
        // forward req to Srv3 prg
        RequestDispatcher rd = req.getRequestDispatcher("/Srv3");
        rd.forward(req, res);
    } // service(-,-)
} // class
```

Srv3.java

same as Srv2.java and don't forward request to any other servlet.

Srv4.java

same as Srv3.java

To test the above application give first request to Srv1 program and give other than first request from to other than Srv1 program from same or different browser windows and check the visibility of attributes.

http://localhost:2020/AttributesApp/Srv1

http://localhost:2020/AttributesApp/Srv2

http://localhost:2020/AttributesApp/Srv3

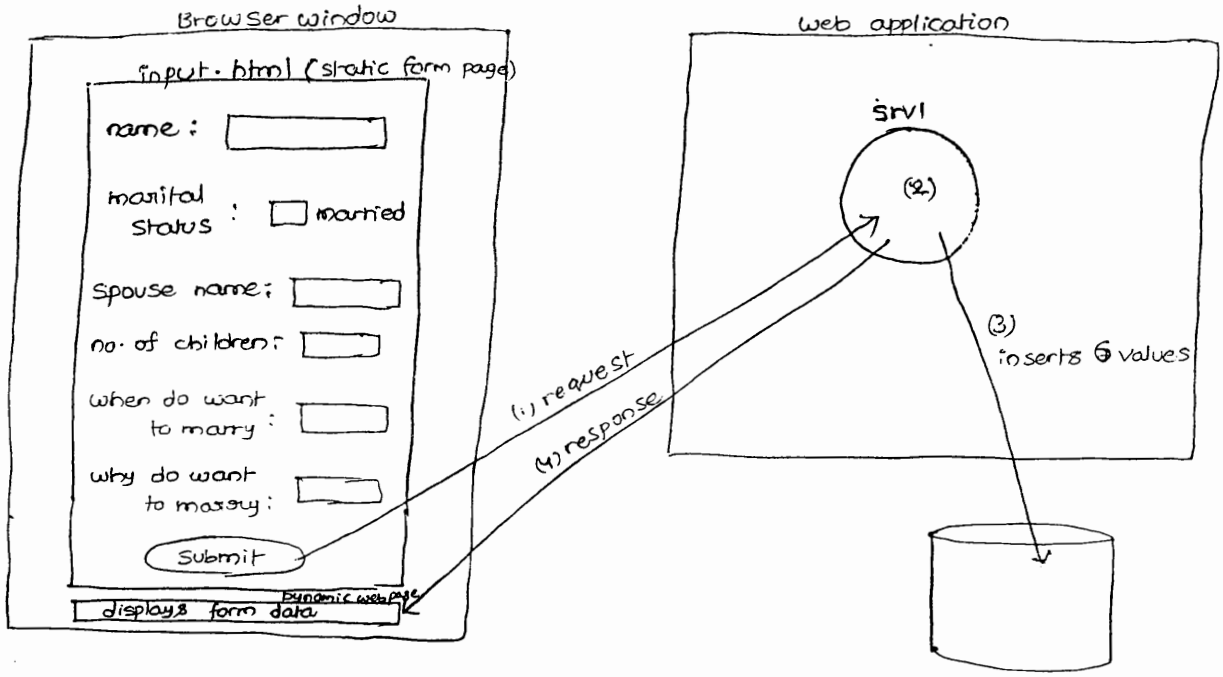
http://localhost:2020/AttributesApp/Srv4

Web.xml

Configure Srv1 servlet program with /Srv1 Program url pattern.

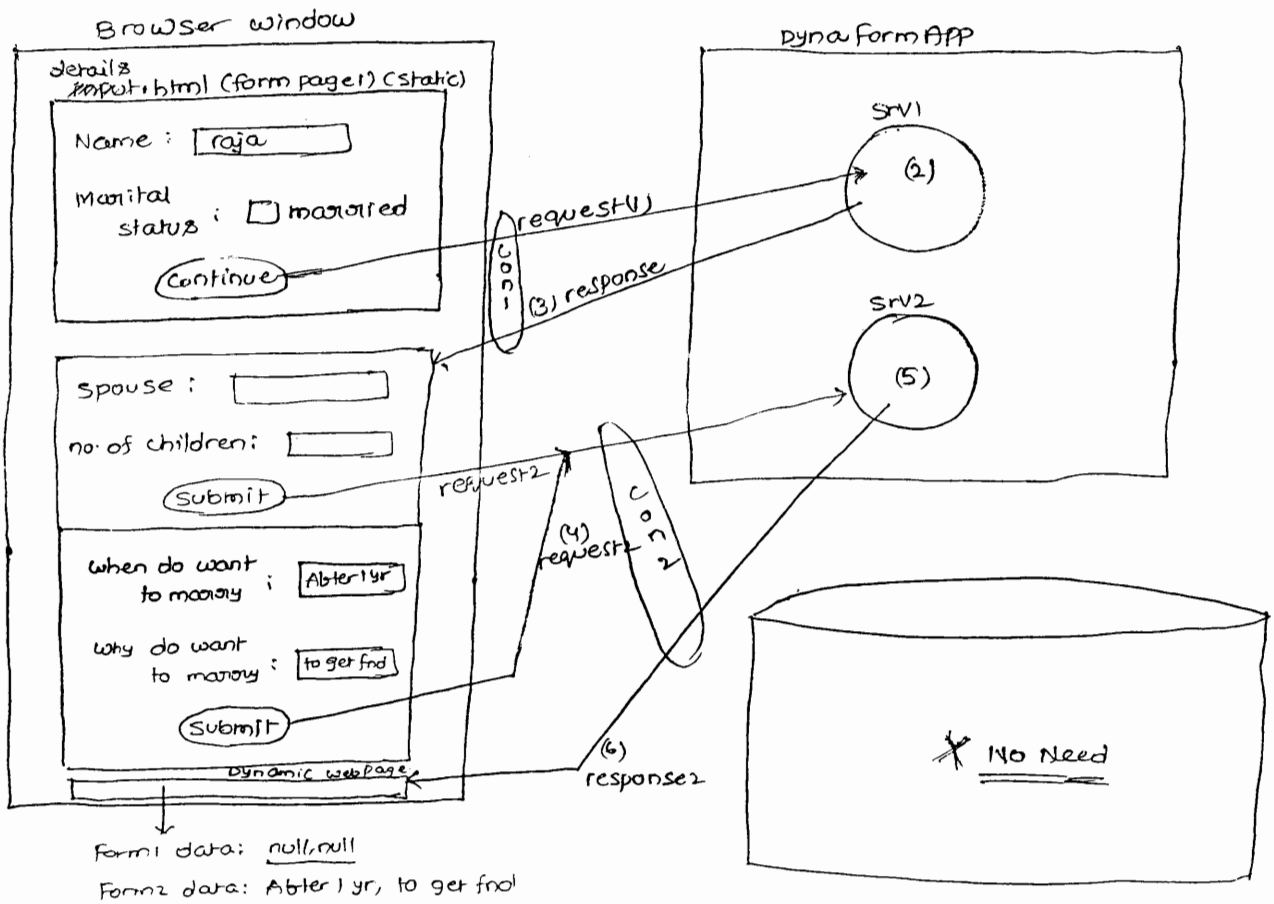
Srv2	/Srv2
Srv3	/Srv3
Srv4	/Srv4

* The form page that is prepared through .html file is called as static form page (fixed content).
* The form page that comes as response generated by servlet or JSP program is called as dynamic form page (dynamic content).
* Instead of asking all the details of end user in a single form page it is recommanded to ask those details in multiple form pages having the support of dynamic form pages.



Bad designing of form page

* In the above static form page the end user needs to read and answer some unnecessary questions when he selects or deselects the marital status checkbox. To overcome that problem work with dynamic form page as shown below.
* In the below diagram the content of form page 2 is dynamic and is dec: generated by Srv1 program based on the marital status value of form1 or request1 data.
* All web applications are stateless web applications by default that means they can not remember client data across the multiple request during a session.
* A session is a set of continuous and related operations (request) performed on the web application by a client (browser window / end user).
Ex: sign-in to sign-out in gmail.com, start of the class to end of a class on a particular day.



Good form Designing but web application is stateless here that means while processing request2 in srv2 program we can't use request1 data (form1 data).

* The stateless behaviour of web application is nothing but while processing current request in any web resource program we can't use previous request data that means while processing request2 we can't use request1 data. similarly while processing request3 we can't use request1 and request2 data as shown in the above diagram.

* web applications are stateless because the protocol http is given as stateless protocol. According to this one new connection will be created between browser window and web server for every request and this connection will be closed automatically once request related response goes to browser window.

ex: If browser window gives 10 requests to a web application then 10 connections will be created between browser window and webserver, due to this one connection related client's request data can not be used in other connection related client's request processing (that means we can't use previous request data in web resource program while processing current request).

* For the above diagram based application refer application(10) of page no:s 74 - 76

* By placing <form> tag, <input> tag in pw.println( ) statements of servlet program we can generate dynamic form page from that servlet program. (refer Srv1.java of page no. 75).

* In Naukari.com registration process multiple forms will be there. In that first form page is static form page and other form pages are dynamic form pages. In that based on the data given in form page1 the questions in form page2 will be rendered. similarly base on data in form page2 the question the questions in form page 3 will be rendered.

* web applications are stateless because they are using a stateless protocol called Http.

Why HTTP is designed as stateless protocol. what is the problem if http comes as statefull protocol ?

(A) If HTTP is statefull protocol for multiple requests given by client to web application single connection will be used between browser window and web server across the multiple requests. This may make clients to engage connections with web server for long time even though the connections are idle. Due to this the web server may reach to maximum connections even though most of its connections are idle.

To overcome that problem HTTP is given as stateless so no client can engage connection with webserver for long time. moreever the connection will be closed automatically at the end of each request related response generation. In internet environment since there is a chance of having huge amount of clients for each web site it is recommanded to have stateless behaviour for http.

---

* If web application is capable of remembering a client data during a session across the multiple requests then that web application is called as stateful web application.

* In state ful web applications the web resource programs can use the data of previous request while processing current request that means while processing

request2 it can use request1 data and etc...

* Even though http is stateless protocol we need to make our web applications as stateful web applications. For this we need to work with the following session tracking or session management techniques.

(1) Hidden Form Fields

(2) Http Cookies

(3) Http Session with Cookies

(4) Http session with URL rewriting (recommanded)

* Session tracking / session management is all about making web application as stateful web application by remembering client data across the multiple requests during a session.

Real Time implementations of this session tracking / session management

(1) Remembering email id and password during gmail.com email operations

(2) Remembering username and password during online shopping operations in websites.

(3) Remembering previous forms data until last form data arrives in online applications / registrations

(4) while customizing webpage look in website as required for enduser.

(5) while rendering direct advertisements in websites.

and etc...

* If advertisements are rendered based on the kind of content surfing / browsing done by end user then that advertisement is called as direct advertisement.

Ex: In google.com when you search for the results of seven wonders we will get advertisement related to tours and travels.

Working with Hidden Form fields based session tracking technique

* An invisible text box of the form page is called as hidden box.
* when form page is submitted the form data goes to the hidden box value goes to web resource program along with the request as request parameter value.

form page

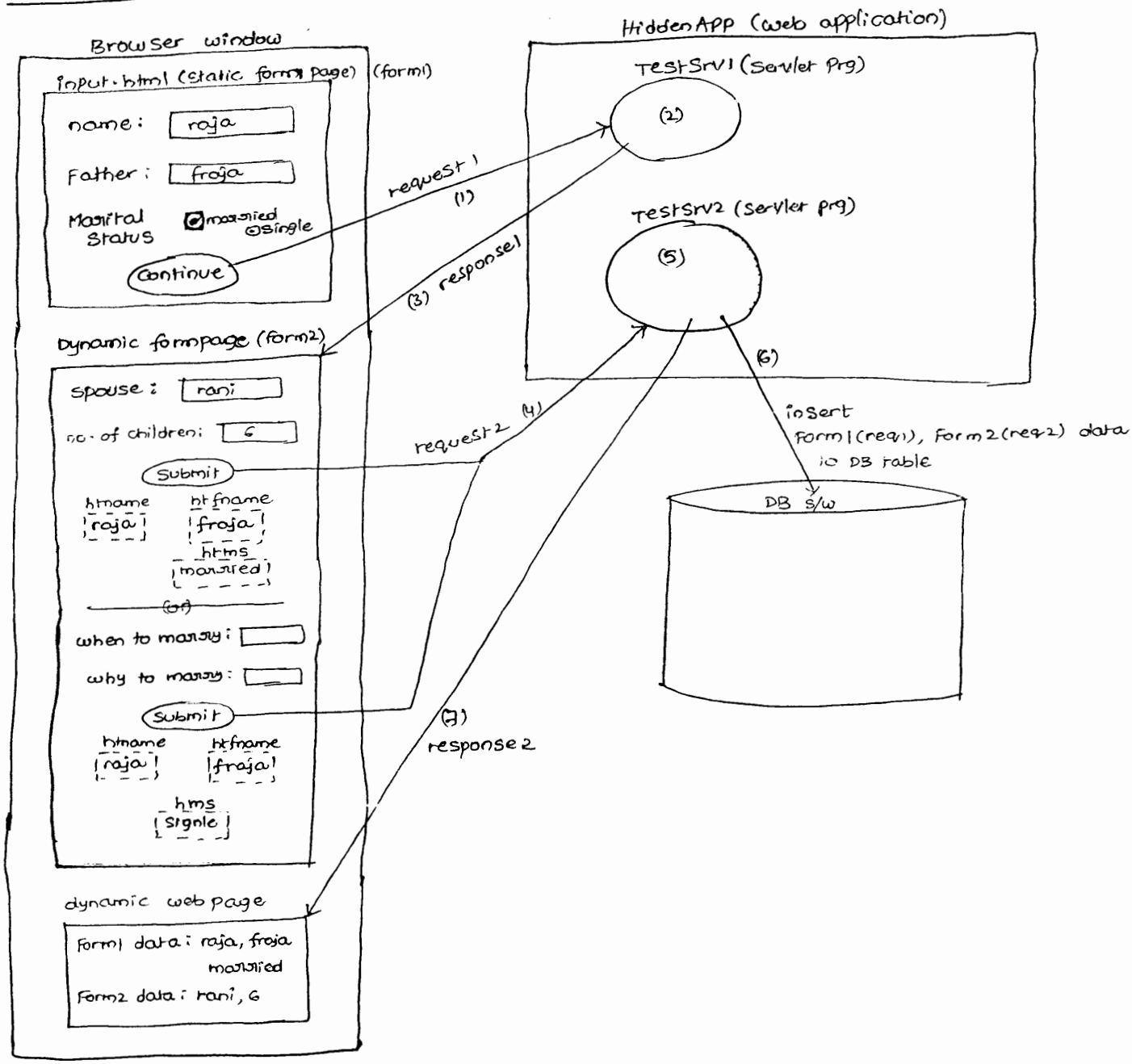
```
<input type="hidden" name="t1" value="hello"/>
```

In servlet program

String s1 = request.getParameter("ti");

gives "hello" to s1 variable

Making web application as stateful with the support of hidden boxes (hidden form fields)



* In the above diagram TestSrv1 program receiving form1/request1 data and generating form2 dynamically. In this form2, form1 data is added as hidden box values so when TestSrv2 program gets request from form2 it is able to read form1/request1 data along with form2/request2 data. This is nothing but session tracking based on hidden form fields.

* The above diagram based web application is stateful because TestSrv2 is able to use request1 data while processing request2.
* For above diagram based application refer application(11) of the page nos 76-78

Advantages of Hidden form fields based session tracking technique

* Basic knowledge of HTML is enough to work with this technique.
* Hidden boxes resides in web pages of browser window so they do not provide burden to the server.
* This technique can be used along with all kinds of server side technologies and all kinds of web servers and application servers.

Disadvantages

* Hidden form fields can not store java objects as values. They can just store text values.
* Hidden boxes doesn't provide data secracy because their data can be viewed through view source option
* Hidden boxes data travels over the network along with request and response. so we can say network traffic will be increased.
* while creating each dynamic form page we need to add the previous form pages data as hidden box values. This increases burden on the programmer.

Http Cookies

* Cookies are the small textual informations which allocate memory at client side by remembering client data across the multiple request during a session.
* The web resource programs of web application create cookies at server side but these cookies comes to client side along with the response and allocates memory at client side.
* Cookies of client side goes back to their web application along with the request given by clients (browser windows to web resource programs).
* there are two types of cookies

(1) In memory Cookies / per session Cookies

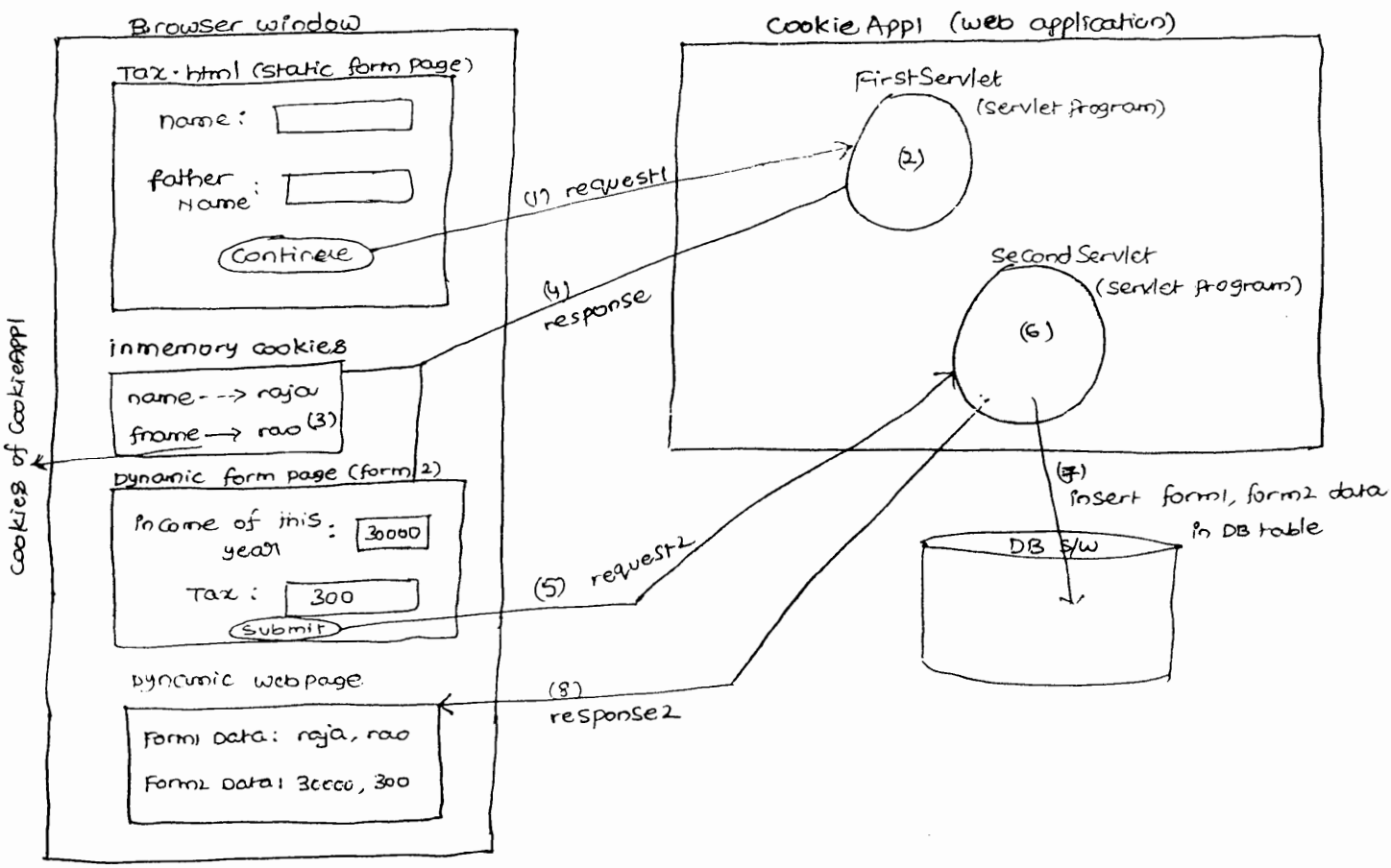
(2) persistent Cookies

(1) → allocates memory in browser window having "-1" as the expiry time (no expiry time). These cookies will be destroyed automatically once browser window is closed.

(2) → These cookies allocates memory in the files of client machine hard disk having positive number as expiry time. These cookies will not be destroyed when browser window is closed but they will be destroyed when their expiry time is completed / reached.

* The cookie that is created without expiry time is called as in memory cookie. The cookie that is created with the expiry time is called as persistent cookie.

* To work with cookies we must deal with pre defined http servlet class based servlet programs.



* Every cookie contains name and value as String information.

* At client side or browser window we can see multiple cookies belonging to different domains or web applications. Every cookie remembers the web application name or domain name for which it is created.

* Cookies of one web application that are there at client side will go to their web application along with the request then browser window gives request back to that web application.

with respect to the diagram

(1) The form page1 Tax.html gives request1 to FirstServlet program of CookieApp1 web application.

(2) FirstServlet program reads form1 data, creates two in memory cookies having that form1 data.

(3), (4) → FirstServlet program generates response to browser window adding the cookies & having dynamic form page. These cookies allocate memory on browser window as in memory cookies.

(5) The dynamic form page (form2) generates request to second servlet program of CookieApp1 application so the two cookies of CookieApp1 will go to SecondServlet program along with the request.

(6) SecondServlet program reads form2 data normally but reads form1/request1 data from the cookies of request2. This indicates secondServlet is able to use form1/request1 data while processing form2 request/request2. (which is nothing but Session tracking).

(7) secondservlet writes both form1, form2 data to db table.

(8) SecondServlet program generates dynamic web page having form1 and form2 data.

* The web resource programs of web application send cookies to client machine along with the response as 'set-cookie' response header values.

* The browser window sends the cookies of back to web application along with the request given to web resource programs as cookies request header value

Cookie API : (working with javax.servlet.http.Cookie class)

to create cookies and to add them to response

```
Cookie ck1 = new Cookie("ap", "hyd"); //creating new Cookie
```

("ap" → cookie name, "hyd" → cookie value) (must be String)

```
resp.addCookie(ck1); //adding Cookie to response
```

↓ Here ck1 Cookie is in memory cookie (because no expiry time is set)

```
Cookie ck2 = new Cookie("mh", "mumbai");
ck2.setMaxAge(1800); → setting expiry time for Cookie
resp.addCookie(ck2); → adding Cookie to response
```

Here ck2 Cookie is persistent Cookie having 1800 seconds as expiry time.

to know Max Age (Expiry time) of Cookie

```
int time = ck1.getMaxAge(); → gives -1 (for in memory Cookie)
int time = ck2.getMaxAge(); → give 1800 seconds
```

to modify Cookie value

```
ck1.setValue("new hyd");
ck2.setValue("navi mumbai");
```

To know domain name / web application name of the cookie

```
String d1 = ck1.getDomain();
String d2 = ck2.getDomain();
```

To set Comment to Cookie

```
ck1.setComment("holds ap's capital city");
ck2.setComment("holds mh's capital city");
```

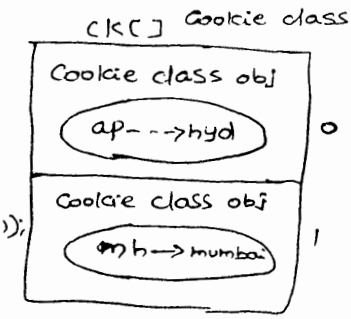
to get Comment of Cookie

```
String c1 = ck1.getComment();
String c2 = ck2.getComment();
```

to read Cookie values

```
String ck[] = req.getCookies();
```

↓ reads all the cookies from the current http request.




```
if (ck != null)
 {
   for (int i=0; i<ck.length; i++)
     {
       pw.println("Cookie name"+ck[i].getName());
       pw.println("Cookie value"+ck[i].getValue());
     }
 }
```

To delete Cookies

* Cookies can not be deleted manually programatically.
* In memory Cookies will be destroyed automatically once their browser window is closed. where as persistent cookies will be destroyed if their expiry time is reached / completed.
* for example application to know the basics and behaviour of Cookies refer application (12) of the page nos (48) and (49).
* In windows XP while working with internet explorer the persistant cookies of web application will be stored in the files created in c:\documents and settings\administrator\cookies folder and the notation of the file is :

<windows user> @ <web application name> [<cookies count>] . txt

Ex : administrator @ CookieApp [2] . txt

writing HTML code in servlet program with attribute values of html tags

```
pw.println("<table border=\"1\" align=\"center\">");
                    (or)
pw.println("<table border=1 align=center>");
                    (or)
pw.println("<table border='1' align='center'>");
```

* If we try to modify the content of the files where persistant cookies are stored then persistant cookies including that file will be destroyed.
* Internet explorer allocates the memory for in memory cookies on browser window where as the memory for persistant cookies will be allocated as files on the client machine hard disk.
* In netscape navigator both in memory and persistant cookies allocate memory on browser window but persistant cookies will be there with expiry time.

NOTE:

* Netscape navigator also internally uses client machine hard disk files for storing persistant cookies.
* for source code of example application based on the above diagram refer application (13) (13) of the booklet page numbers (79) to (81) (77) to (81)

* Every Cookie remembers the domain name or web application name to which it belongs to.
* Cookies belonging to yahoo.com website will go to yahoo.com website only when browser window or client gives request back to that yahoo.com web site.
* In yahoomail.com, gmail.com website related login page ☑ remember me on this computer option stores the given login details (username, password) as persistent cookies on client machine hard disk. That means by using persistent cookies we can remember client data during session and after session.

Advantages of Http Cookies :

(1) Cookies allocates memory at client side so they do not give any burden to server.
(2) All server side technologies and all web servers, application servers support Cookies.
(3) Persistent Cookies can remember client data during session and after session with expiry time.

Disadvantages

(1) Cookies can not store java objects as values. They can store only strings.
(2) Cookies data can be viewed through browser setting or through the files of file system. so they do not provide data secracy.
(3) Cookies can be deleted explicitly through browser settings it may fail session tracking
(4) There is a restriction on no. of Cookies that can be there in browser window. Per browser window and per web application max of 20 cookies. All together max of 300 Cookies per browser window.
(5) Cookies can be blocked/restricted coming to browser window. This fails session tracking.

IE ⟶ to delete Cookies

Tools ⟶ Internet options ⟶ delete Cookies

Netscape to delete

Tools ⟶ Cookie manager ⟶ remove all Cookies

IE → to block cookies

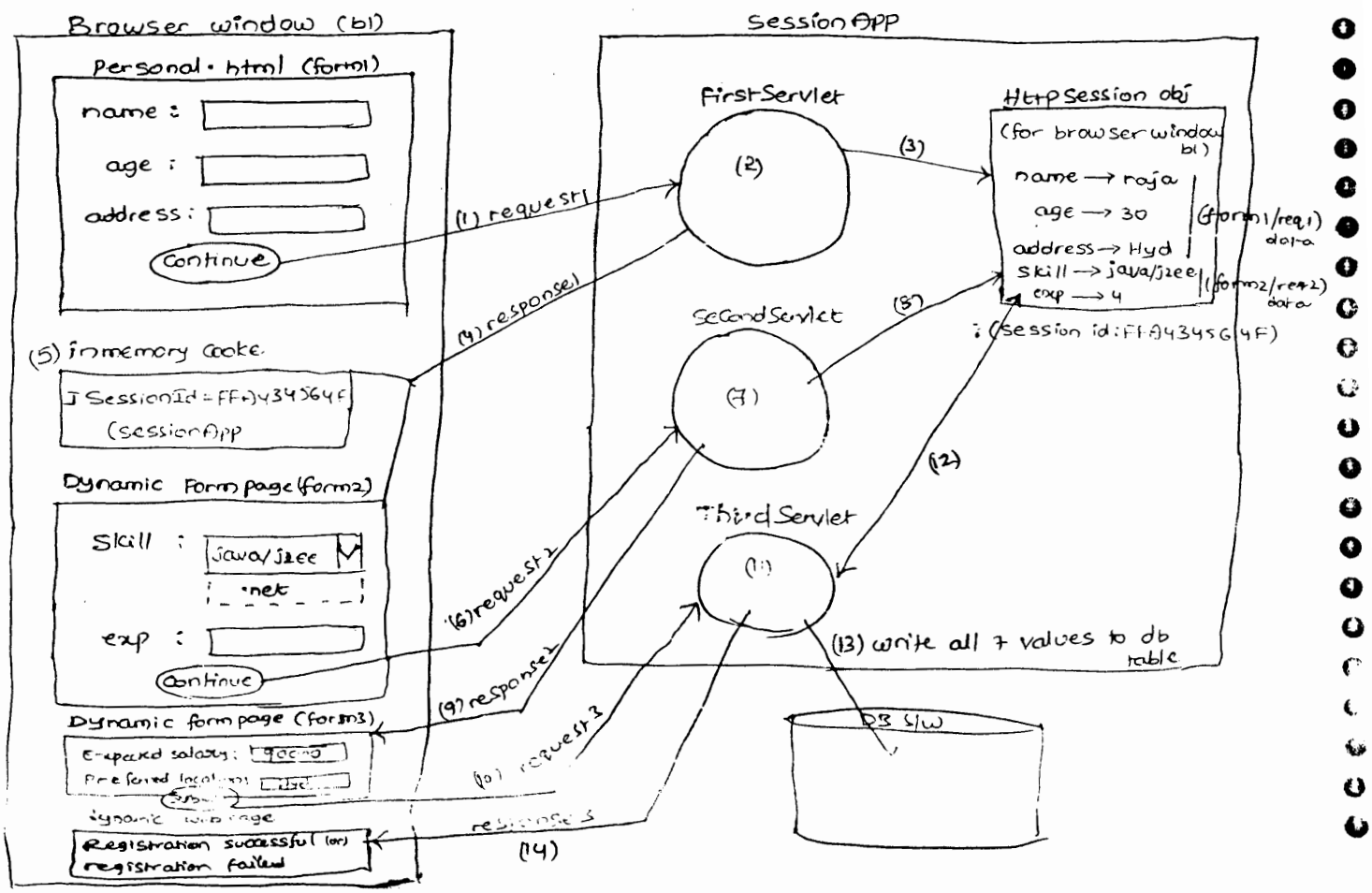
Tools → internet options → privacy → use slider

Netscape → to block cookies

Tools → Cookie manager → Block Cookies from this site

Http Session with Cookies :

* Http Session object allocates memory on the server and remembers client data across the multiple requests in the form of session attribute values.
* Http Session object is one per browser window (client) so each Http Session object can be used to remember client data during a session.
* Http Session object means it is the object of a servlet container supplied java class implementing javax.servlet.http.HttpSession interface.
* Every HttpSession object contains session id and this session id goes to browser window from web application and comes back to browser web application from browser window in the form of in memory cookie value so this technique is called HttpSession with Cookies technique.



with respect to the diagram

(1) Browser window b1 launches personal-html (form1) and sends the request to FirstServlet of SessionApp application.

(2) FirstServlet reads Form1 data / request1 data.

(3) FirstServlet creates HttpSession object on the server for browser window b1 and writes Form1/ request1 data to that session object as session attribute values.

(4) FirstServlet generates form2 as dynamic form page and sends the session id of HttpSession object to browser window as in memory Cookie value.

(5) The in memory cookie having session id of HttpSession object (b1) allocates memory on the browser window. That cookie also remembers its web application name SessionApp.

(6) End user fills up the form page (form2) and sends the request2 to Second Servlet of sessionApp. Along with the request the session id will go as Coolcie value.

(7) SecondServlet reads form2 data.

(8) SecondServlet uses the session id read from the Cookie to get access to the HttpSession object of browser window b1 and writes form2 /request2 data to that HttpSession object as session attribute values.

(9) SecondServlet generates form3 as dynamic form page.

(10) form3 generates request3 to third servlet of sessionApp application. Along with this request session id goes to third servlet as cookie value.

(11) ThirdServlet reads form3 data and session id from the Cookie

(12) ThirdServlet uses session id collected from the Coolcie to get access to HttpSession object of browser window b1 and reads form1 / request1, form2/ request2 data from the session attributes of that HttpSession attributes. This indicates ThirdServlet is able to use request1 / form1 data and request2/ form2 data while processing form3 request or request3. This is nothing but session tracking.

(13) Thirdservlet writes form1, form2, form3 values to the db table as a record.

(14) ThirdServlet generates dynamic web page having conformation message.

* when HttpSession object is created the session id of that session object automatically goes to browser window as in memory cookie value.
* In HttpSession with Cookies session tracking technique based web application the browser window (client) will be identified across the multiple request during a session based on the session id sent by the browser window along with the request.

Session API (working with javax.servlet.http.HttpSession interface)

To create/locate HttpSession object

a) HttpSession ses = req.getSession();

* This method creates new HttpSesion object on server per browser window if HttpSession object is not already available for browser window otherwise this method provides access to existing HttpSession object of that browser window.
* This method can create the new session between browserwindow and web application if session is not already between them otherwise this method makes current request to join in the existing session.
* If this method is called in FirstServlet program of above diagram then it makes request1 of browser window b1 begining new Session between browser window b1 and web application session App. If same method is called SecondServlet, ThirdServlet then it makes request2, requests of browser window b1 participating in existing session.

b) HttpSession ses = req.getSession(false);

* This method ~~creates~~ gives access to existing HttpSession object of browser window, if not available this method returns null (indicating new HttpSession object can not be created)
* when this method is called the request can always join in existing session but can not create new session between browser window and web application.
* In the above diagram it is recommended to call req.getSession() in FirstServlet and to start the session and it is recommended to call req.getSession(false) method in SecondServlet, ThirdServlet to make them participating in existing session.

c) HttpSession ses = req.getSession(true);

"same as (a)"

Conclusion:

to create new session/to locate existing session → use (a), (c) options only to locate existing session ⟶ use (b) option.

to know Session ID

```
String id = ses.getID();
```

TO know session object creation time / session started time

```
long ms = ses.getCreationTime();
Date d1 = new Date(ms);
```

* In the above code ms represents milli seconds that are elapsed from 1970 Jan 1st 00:00 hours to the date and time of HttpSession object creation.

Epoch standard

* TO know the last accessed time of HttpSession object

```
long ms = ses.getLastAccessedTime();
Date d2 = new Date(ms);
```

* The above code gives the last accessed date and time of HttpSession object.

TO get access to ServletContext object

```
ServletContext sc = ses.getServletContext();
```

TO know whether session is new or not

```
boolean b = ses.isNew();
```

* This method returns true when ses object is new object and just created object and its session id still yet to be delivered (sent) to client (browser window) otherwise this method returns false. (when session object is old objects and its session id is already there with client).

* If this method is called in FirstServlet program of diagram (which receives only first request of the browser window) then this method returns true. If this method is called in other Servlet programs of web application then this method returns false.

To invalidate the session

* Invalidating the session is nothing but closing the session between browser window and web application. In this process the entire data from session object will be removed and makes session object inactive object, ready for garbage collection.

a) when ses.invalidate() method is called

b) when browser window is closed

NOTE: since session id will be stored as in memory cookie value of browser window and that in memory cookie will be destroyed once browser window is closed so the session will be invalidated once browser window is closed.

c) when MaxInactiveInterval / SessionIdletimeout period is completed/reached

* If session object is continuously idle for certain amount of time then it will be invalidated automatically. The default session idle timeout period is 30 minutes (in most of the servers). But this can be changed explicitly either by using programatic approach or declarative approach.

(I) Programatic approach (Java code)

In servlet prg/ JSP prg

```
ses.setMaxInactiveInterval(1500);
                           ↓
                        seconds
```

(II) Declarative approach (xml code)

In web.xml

```
<web-app>
   <session-config>
        <session-timeout>20</session-timeout>
   </session-config>          ↓
                           minutes
</web-app>
```

* once session idle timeout period or maxInactiveInterval period is completed the underlaying webServer automatically expires the session (invalidates the session).

TO know current maxInactiveInterval period/ SessionIdletimeout period

```
int t1 = ses.getMaxInactiveInterval();
```

NOTE: The above method returns 30 if no value explicitly set as SessionIdle timeout period.

If you set sessionIdleTimeout period in both programatic and declarative approaches with two different values, can you tell me which value will be effected finally ?

(A) Since the code of servlet program executes after web.xml code so the time specified through programatic approach will be effected by overriding the time specified through declarative approach.

* In HttpSession object the client data will be preserved in the form of session attribute values.
* HttpSession object and its session attributes are visible and accessable in all web resource programs of web application but they must get request from that particular browser window (client) for which this session attribute, and session objects are created.

TO create / modify session attribute

```
ses.setSetAttribute("age", new Integer(30));
```

ses.putValue(-,-) is deprecated method of setAttribute(-,-)

TO read session attribute value

```
Integer i1 = (Integer)ses.getAttribute("age");
```

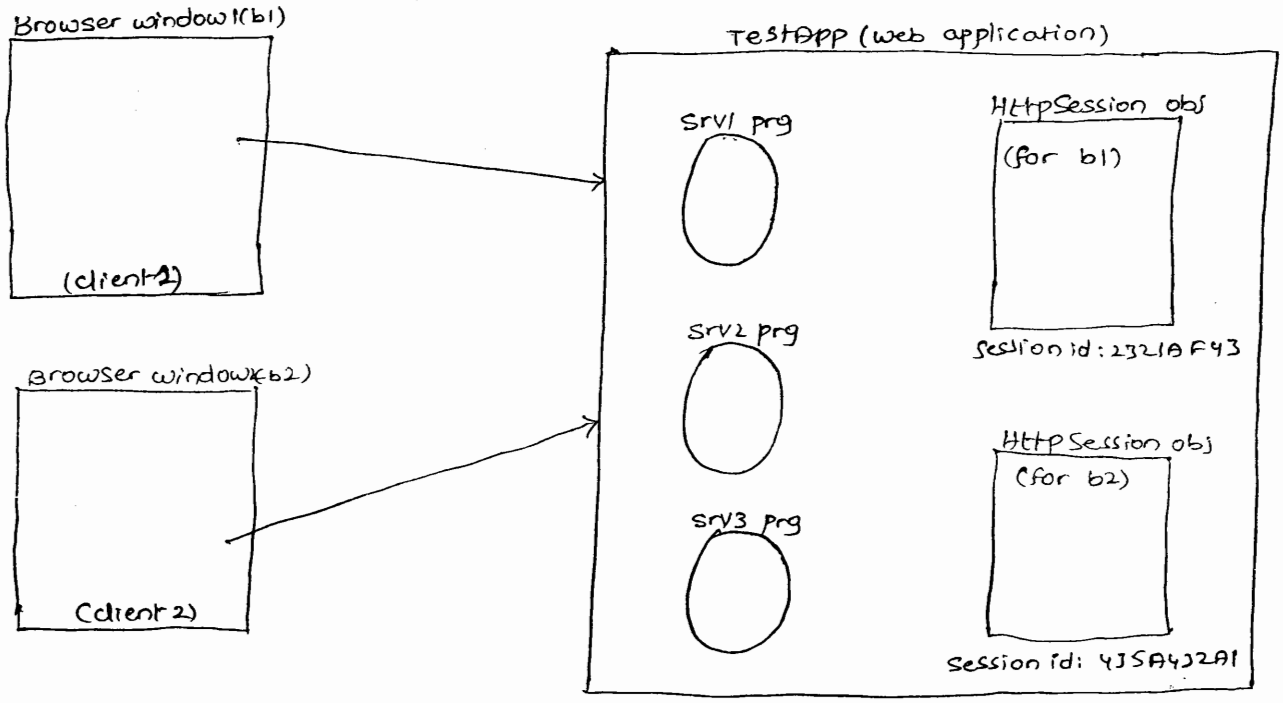
ses.getValue(-,-) is deprecated method of getAttribute(-,-)

TO remove session attribute

```
ses.removeAttribute("age");
```

ses.removeValue(-,-) is deprecated method of removeAttribute(-,-)

* For example application on HttpSession with Cookies based on previous diagram refer page nos ~~85~~ - ~~88~~ 81 - 85
+ If multiple clients (browser windows) are giving request to a webapplication in which HttpSession object based session tracking is enabled then for every browser window one HttpSession object will be created on the server having unique session id.



* In HttpSession with Cookies session tracking enabled web application the moment HttpSession object is created on server for browser window the servlet Container automatically creates one in memory cookie having sessionid and adds that Cookies to the response to send to the browser window (no need of performing manual work).

* while working HttpSession object based web application if browser window is closed in the middle of the session then the existing session will be invalidated and that session will not be continued with new browser window.

* while working with HttpSession object based application what happens if underlaying server is restarted in the middle of the session ?

=> The session will be continued.

Sever collects the data of Httpsession objects and writes to files through serialization process having session ids when programmer shutdown the server. When server is restarted HttpSession object will be created having old data and old session ids by reading data from the above said file through de-Serialization process. Due to this the session will be continued even though the server is restarted in the middle of session.

---

* Advantages of Httpsession with Cookies session tracking technique:

=> Http Session objects allocates memory on the server holding client data

during session as session attribute values. So there will be data secracy for client's session data.

(2) client data during session will not travel along with request and response over the network so this reduces network traffic between client and web server.

(3) The session attributes of Http session objects can take java objects as values.

(4) All java based web servers and application servers support this technique.

(5) This technique allows the programmer to specify session idle timeout period to invalidate inactive session objects.

DIS advantages:

(1) Http session objects allocates memory on the server. This increases burden on the server.

(2) If Cookies are restricted coming to browser window this technique fails to perform session tracking.

Http session with url rewriting

* The limitation with third technique is if Cookies are restricted to coming to browser window then third technique fails to perform session tracking because it uses in memory Cookie to send session id to browser from web application (along with response) and to bring session id back to web application from browser window along with request.

To overcome that problem work with Http session with URL rewriting which doesnot use Cookies to send and receive session id. Moreever this technique appends session id to a url that goes to browser window from web application along with the response and that comes back to web application from browser window along with the request. Generally we append session id to the action url (the action attribute of form tag) of dynamic form page generation code.

NOTE:

res.encodeURL("s2url"); → gives s2url;jsessionid:42435A1A3F723

This method use URL appended with current object's session id as shown above.

* By taking above kind of URL as action URL of dynamic form page we can perform http session with URL rewriting based session tracking.

In Servlet program

```
pw.println("<form action=" + res.encodeURL("surl") + " method=get>");
pw.println("- - - - - -");
pw.println("- - - - - -");
pw.println("</form>");
```

*** ↓ keeps the url appended with sessionid as the action attribute value of <form> tag

* For example application on Httpsession with URL rewriting refer application (15) of the page nos 85-88.

Advantages and disadvantages of Http session with URL rewriting

same as Http session with Cookies technique but this technique also works even though cookies are restricted coming to browser window.

Conclusion on Session tracking technique

* while developing large scale and commercial web site which contains huge customer base take the support of Http Cookies technique.

Ex: www.gmail.com, www.yahoomail.com

* while developing medium scale and certain organization based website which contains limited amount of customers use http session with URL rewriting technique.

Ex: www.citibank.com, www.satyatechnology.it.com

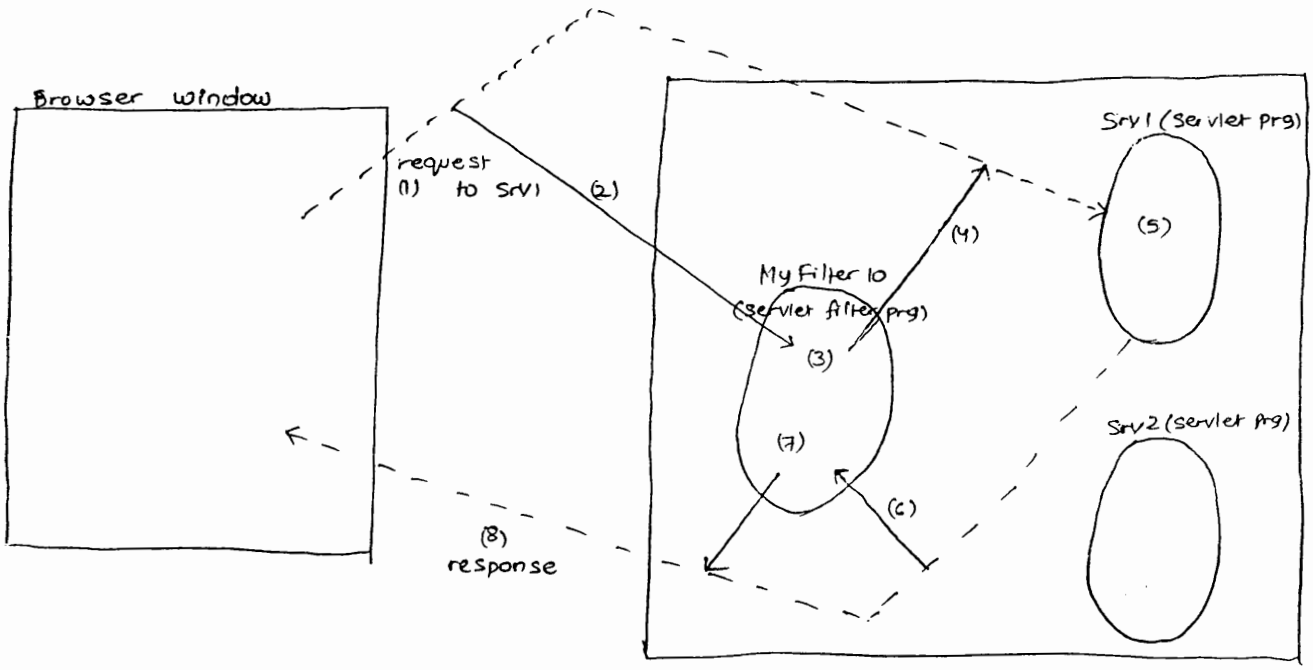
* we can use multiple session tracking techniques in a single web application development. For example the online shopping websites like www.yebhi.com uses both http Cookies, http session with URL rewriting techniques.

↓ for holding shopping cart information

↓ for holding login details (username, password)

Servlet Filters :

* A servlet filter is a special web resource program of java web application that is capable of trapping and taking request and response of other web resource programs of that web applications.
* In Servlet filter program we can keep the common and global pre request processing logic and post response generation logic
* To add new functionalities to web application without changing the source code of existing web resource programs we can take the support of servlet filter programs.



With respect to the above diagram

(1) Browser window generates request to Srv1 servlet program.

(2) The servlet filter program traps and takes that request.

(3) Servlet filter program executes the common and global pre requesting process logic (like authentication logic)

(4) Servlet filter program forwards the request to actual Srv1 servlet program.

(5) The main request processing logic of Srv1 program executes and generate response.

(6) Servlet filter program traps and takes the response.

(7) Filter program executes the common and global post response generation logic (like adding more content at the end of response).

(8) Servlet filter program sends response to browser window.

Servlet filter basics

* Servlet filter program is a java class that implements javax.servlet.Filter interface.
* Every servlet filter program must be configured in web.xml file using filter, filter mapping tags.
* To link servlet filter program with servlet program of web application the url pattern of servlet program must be taken as the url pattern of servlet filter program.
* Servlet container creates our servlet filter class object either during server start up (or) during the deployment of the web application.
* we can map servlet filter program with one servlet program, multiple servlet programs (or) all servlet programs of web application.

* There are 3 lifecycle methods in servlet filter program

(1) public void init (FilterConfig fg)

(2) public void doFilter (ServletRequest req, ServletResponse res, FilterChain fc)

(3) public void destroy()

FilterChain fc → This object points to the target mapped servlet programs of filter program.

(1) → executes during instantiation and initialization process of our servlet filter program.

(2) → Executes for every request trapping and response trapping.

(3) → Executes when ServletContainer is about to destroy our servlet filter class object

There are 3 types of filter programs.

(1) request Filter (Contains only pre request processing logics)

(2) response Filter (Contains only post response generation logics)

(3) request - response Filter (Contains both pre-request processing and post response generation logics)

Examples of request Filters

Authentication Filter

Authorization Filter

Examples of response Filters

Signature Filter

Logs Filter (removes html tags from response of the servlet program which are not supported by browser window).

Compression Filter (reduces the size of the response content)

Examples of request-response Filter

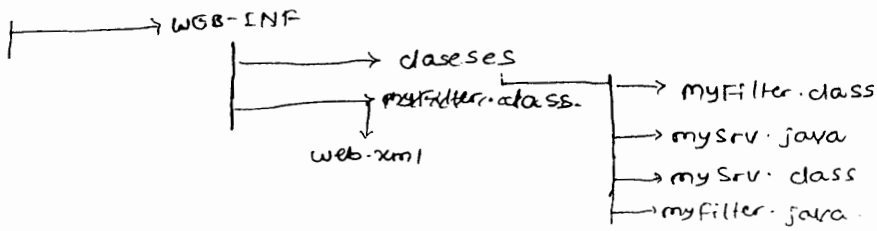
PerformanceTestFilter (This filter holds request traping time and response traping time of certain servlet program and calculates difference between both timings to decide the performance of the servlet program).



* FilterConfig object is right hand object of our servlet Filter class object and it can be used to read init parameter values of filter program in web.xml file.
* init(-) method of servlet Filter program contains initialization logic.
* doFilter(-,-,-) of ServletFilter program contains pre-request processing logic and post response generation logics.
* destroy() of ServletFilter program contains uninitialization logic.

understanding flow of execution for servlet filter program when it is mapped with a servlet program.

① TESTAP



MyFilter.java ②

```
Public class myFilter implements javax.servlet.Filter
                                  ↳ contains all three lifecycle methods of servlet Program Filter.
  {
    public void init (FilterConfig cg)
    { ③
      // initialization logic
    }
    public void doFilter (ServletRequest req, ServletResponse res, FilterChain fc)
    { ⑨
      } Pre-request processing logic
      fc.doFilter (req, res); ⑩
    } ⑯ } post-response generation logic
```

```
    public void destroy()
      {
        ==== //uninitialization logic
      }
  }
```

MySrv.java (servlet program)

```
public class MySrv extends HttpServlet
 {
   public void service(-,-) throws SE, IOE
    {
      ====== (15)
      ======
      ======
    }
 }
```

web.xml

```
<servlet>
   <servlet-name> abc </servlet-name>          (13)
   <servlet-class> MySrv </servlet-class>      (14)
</servlet>

<servlet-mapping>
   <servlet-name> abc </servlet-name>          (12)
   <url-pattern> /test1 </url-pattern>         (11)
</servlet-mapping>

<filter>
   <filter-name> xyz </filter-name>            (7)
   <filter-class> MyFilter </filter-class>     (8)
</filter>

<filter-mapping>
     <filter-name> xyz </filter-name>          (6)
     <url-pattern> /test1 </url-pattern>       (5)
</filter-mapping>
</web-app>.
```

↓ matching with servlet program (mySrv) url pattern

Browser window

http://localhost:2020/TestApp/test1 (4)

(17) response will be displayed on browser window.

With respect to

(1) Programmer deploys TestApp web application in web server/application server.

(2) Based on web.xml entries servlet container creates our filter class object (xyzFilter) either during server startup or during the deployment of the web application.

(3) Servlet container completes initialization process on filter by executing the logic of init(-) method.

(4) End user gives request to servlet program mySrv from browser window (having /test1 in the request url).

(5), (6), (7), (8) → since MyFilter program is configured with MySrv program the servlet Filter traps and takes the above request that is given to mySrv program. In this process servlet Container locates MyFilter class object.

(9) Servlet Container calls life cycle method doFilter(-,-,-) and executes pre request processing logic.

(10) The fc.doFilter(-,-) method call forwards the request to the target web resource program (mySrv).

(11), (12), (13), (14) → Servlet Container Completes instantiation and initialization of MySrv program based on its configuration done in web.xml file.

(15) Servlet Container calls service method of MySrv program to process the request and to generate the response.

(16) The filter program (MyFilter) traps the response of mySrv program (servlet program) and executes post response generation logic that is placed after fc.doFilter(-,-) method.

(17) The filter program MyFilter sends the response to browser window.

Sample web application

Srv1, srv2, srv3 are servlet programs

F1 is Servlet Filter program

to configure filter with all the servlet programs of web application

Configure F1 filter having url pattern /*

to configure F1 filter with srv1, srv2 programs of web application

Srv1 program url pattern : /x/y/s1url

Srv2 program url pattern : /x/y/s2url

F1 filter program url pattern : /x/y/*

to configure F1 filter with srv1 program of web application

Srv1 program url pattern : /s1url

F1 filter program url pattern : /s1url

What is the difference between doFilter(-,-,-) of javax.servlet.Filter interface and doFilter(-,-) method of javax.servlet.FilterChain interface?

(A) doFilter(-,-,-) method of Filter interface is life cycle method of ServletFilter program so programmer uses this method to place prerequest processing and post response generation logics.

doFilter(-,-) method of FilterChain is not life cycle method so programmer uses this method to invoke next filter in the chain or to invoke the mapped servlet program or JSP program of current servlet filter program.

Sample web application

Srv1, Srv2, Srv3 are Servlet Programs

F1, F2 are Servlet Filter Programs

To Configure F1, F2 filter programs with Srv1 Program

Url pattern of Srv1 program : /s1url

url pattern of F1 filter program : /s1url

Url pattern of F2 filter program : /s1url

To Configure F1, F2 filter programs with Srv1, srv2 programs

Url pattern of Srv1 Program : /x/y/s1url

Url pattern of Srv2 program : /x/y/s2url

Url pattern of F1 filter program : /x/y/*

Url pattern of F2 filter program : /x/y/*

To Configure F1, F2 filter programs with all servlet programs of webapplication

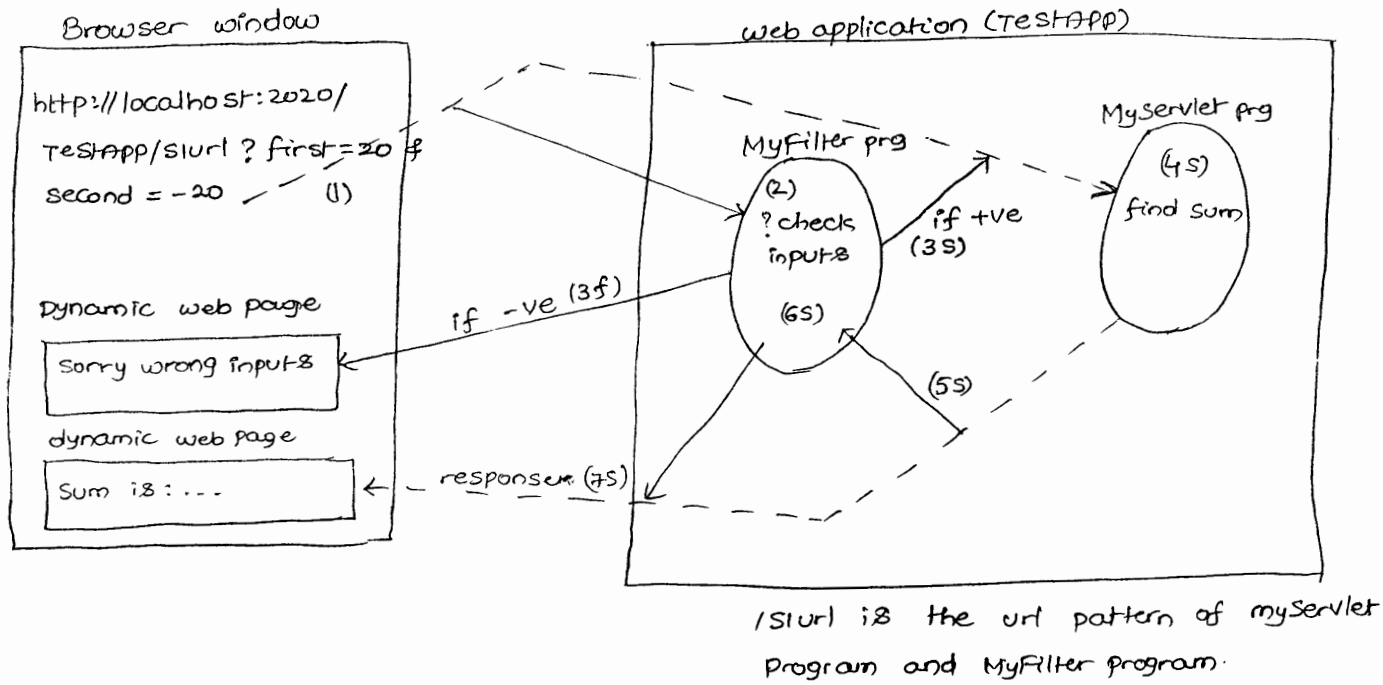
Url pattern of Srv F1 filter program : /*

Url pattern of F2 filter program : /*

* When filter is configured with servlet program then filter can trap the request and response of Servlet program.

Example application

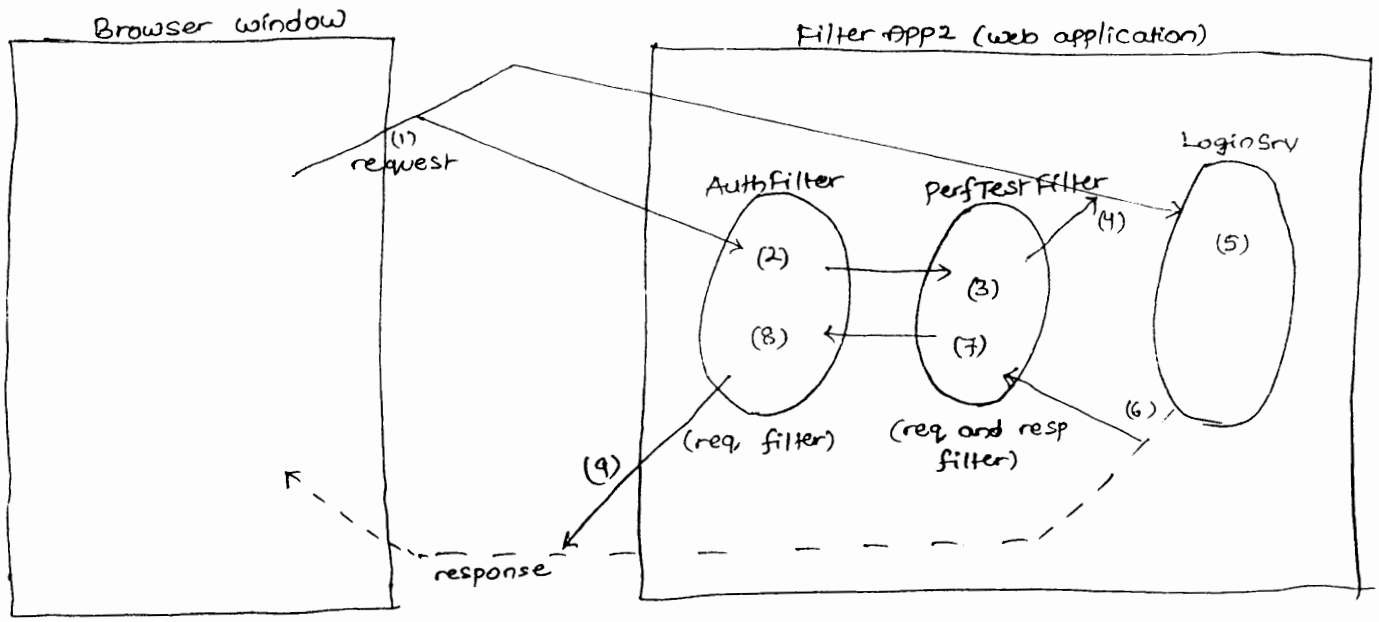
* In the below diagram MyFilter Program traps the request going to myServlet program and checks the input values (request parameter values). If those values are positive then it forwards the request to MyServlet Program (3S) otherwise it blocks the request in Filter Program and sends error response to browser window (3f)



/s1url is the url pattern of myServlet Program and MyFilter program.

* In the above diagram MyFilter also traps the response of MyServlet program but does not contain post response generation logic so the above filter is called as request filter.
* For above diagram based application refer application (16) of page nos (89 & 90)

Example application on filter



* After taking request from client the amount of time taken by servlet program to process the request is technically called as response time of servlet.

* The tomcat server maintains log files in Tomcat_home\logs folder on one per day basis having date in the file name.

* Instead of using S.O.P(-) statements which writes log messages to server console it is recommanded to use log(-) method of ServletContext object which writes the log messages to current day's log file of <tomcat_home>\log folder.

```
ServletContext sc = getServletContext();
sc.log("hello");
```

* JDBC code to perform username and password based authentication

```
PreparedStatement ps = Con.prepareStatement("select * from userlist where
                                              username=? and password=?");
ps.setString(1,"raja");
ps.setString(2,"hyd");
ResultSet rs = ps.executeQuery();
if(rs.next())
  {
    ==========  //Authentication successful
  }
else
  {
    ==========  //Authentication failed
  }
```

Userlist (db table)

username	password
raja	hyd
ravi	Vizag
ramesh	Delhi

* For above dragram based web application's source code refer application(17) of the page no's 90-93.

* The process of checking the identity of a user through his username and password is called as authentication.

* The process of verifying the access permissions of a user on particular resource is called as authorization.

To count no. of requests (hits) coming to web application when webapplication is in active mode we can develop filter program as shown below.

step(1): Develop the servlet filter program as shown below in WEB-INF\classes folder of web application (like URLAPP) having the logic of counting request.

```
//CountFilter.java
import javax.servlet.*;
import javax.servlet.http.*;
public class CountFilter implements Filter
   { FilterConfig fg;
     public void init(FilterConfig fg)
      {this.fg=fg;
      } int cnt=0;
     public void doFilter(ServletRequest req, ServletResponse res, Filterchain fc)
                                                  throws ServletException, IOException
      {
         cnt++;
         ServletContext sc=fg.getServletContext();
          sc.setAttribute("hitcnt",cnt);
          fc.doFilter(req,res);
       }
      public void destroy()
        {
        }
  }//class
```

step(2): Configure above filter in web.xml file with url pattern "/*"

step(3): Read and display servletContext attribute value in every web resource program of web application.

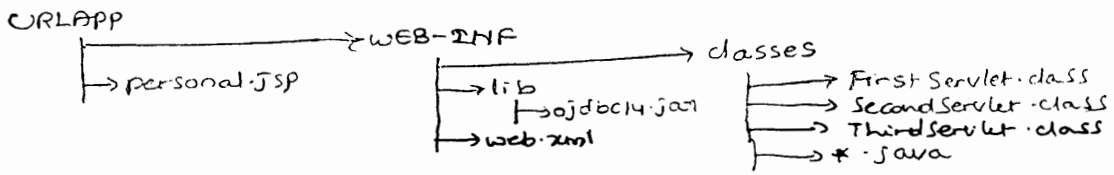
In personal.jsp

```
<% out.println("Request Count:"+application.getAttribute("hitcnt")); %>
```

In FirstServlet.java, SecondServlet.java, ThirdServlet.java

```
ServletContext sc=getServletContext();
pw.println("<br>Request Count"+sc.getAttribute("hitcnt"));
```

Deployment directory structure



Request url

http://localhost:2020/URLAPP/personal.jsp

* use application (IS) as base application to develop this application.

Applet to servlet communication

* In web applications we need form pages to submit the request to web resource programs by having enduser supplied data. These form pages provide Graphical user interface to end user to submit the request to web resource programs having data. There are two ways to develop form pages.
  1) As html form page (using <form>, <input> and etc... tags)
  2) As java applet based form page.
* there are two types of applets
  (a) Trusted applet
  (b) untrusted applet
* An applet is a compiled java class that can be sent over the network as web page.
* when untrusted applet is launched in the browser window by gathering it from network it can't interact with the files of file system so untrusted applets can not write virus to the computer (that means provides security)
* Trusted applets that are launched in the browser window can interact with file system and can write virus to file system (that means no security).
* while designing form page if performance is important (should be launched very fastly) then use html form pages. If security is important that take untrusted applet as form page.

What are the differences between Applet and servlet?

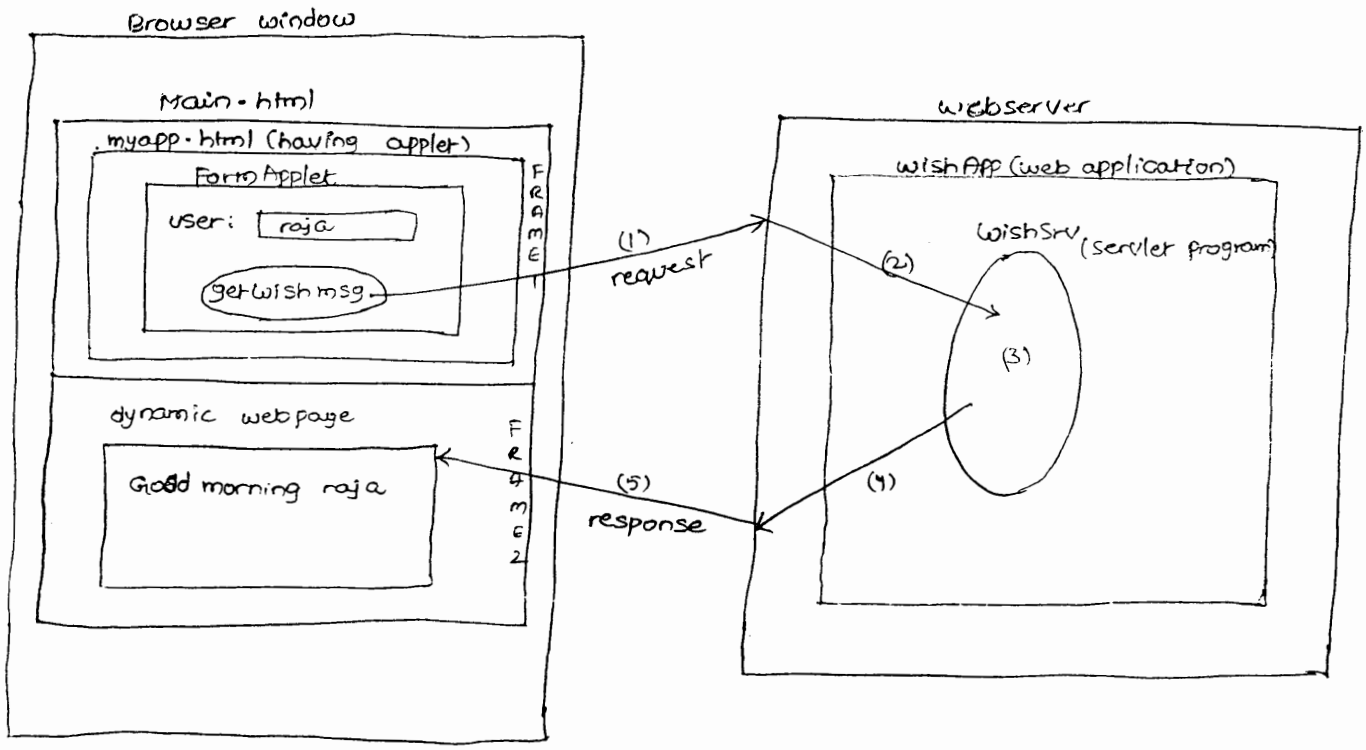
Applet	Servlet
* client side technology to develop client side web resource programs	* server side technology to develop server side web resource programs
* Comes to browser window for execution.	* uses servletContainer of webserver for execution.
* Generate static form pages.	* Generate dynamic web pages & form pages.
* need not be configured in web.xml file.	* must be configured in web.xml file.

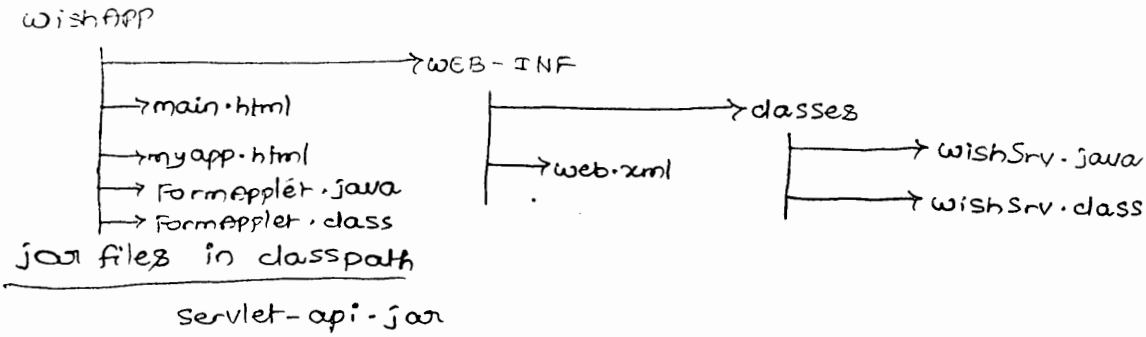
* Uses life cycle methods for execution. They are init(), start(), stop(), destroy(), paint()

* Executes through life cycle methods. They are init(-), service(-,-), destroy().

* Applet to servlet communication means making the applet based form page sending request to servlet program having data.

* In html form page to servlet communication the html tags automatically prepares request URL by having form data as query string values where as in applet to servlet communication all these operations should be performed by the programmer manually





wishSrv.java

```
import javax.servlet.*;
import javax.servlet.http.*;
import java.io.*;
import java.util.*;
public class wishSrv extends HttpServlet
 {
    public void service(HttpServletRequest req, HttpServletResponse res) throws
                                                    ServletException, IOException.
     {
         PrintWriter pw = res.getWriter();
          res.setContentType("text/html");
         //read form data (applet data)
          String uname = req.getParameter("user");
         //write b.logic
          Calendar cl = Calendar.getInstance();  //gives current date and time.
          int h = cl.get(Calendar.HOUR_OF_DAY); //gives current day hour of day
          if (h<12)
             pw.println("Good morning"+uname);
          else if (h<=16)
              pw.println("Good afternoon"+uname);
          else if (h<=20)
               pw.println("Good evening"+uname);
          else
               pw.println("Good night"+uname);
         //close the stream object
          pw.close();
        }
    }
```

main.html

```
<frameset rows="40%,*">
   <frame name="f1" src="myapp.html"/>
   <frame name="f2"/>
</frameset>
```

* frame with name is called as named frame.

myapp.html

```
<applet code="FormApplet.class" width="200" height="200">
</applet>
```

FormApplet.java

```
import java.applet.*;
import java.awt.*;
import java.awt.event.*;
import java.net.*;
public class FormApplet& extends Applet implements ActionListener
  {
      Label l1;
      TextField tf1;
      Button b1;
      public void init()
       {
          l1=new Label("user:");
          add(l1);
          tf1=new TextField(10);
          add(tf1);
          b1=new Button("Get wish");
          b1.addActionListener(this);
          add(b1);
       }
      public void actionPerformed(ActionEvent ae)
        {
          try{
              //example req url : http://localhost:2020/wishApp/wurl?user=raja+++rao
              String qrystr="?user="+tf1.getText().replace(' ','+'); //gives ?user=raja+++rao
              String requrl="http://localhost:2020/wisApp/wurl"+qrystr;
              //prepare req url as URL class obj.
              URL url=new URL(requrl);
              //get Access to AppletContext obj
              AppletContext apc=getAppletContext(); //public method of Applet class
              //send request to servlet program
              apc.showDocument(url,"f2"); //sends the req to wishSrv prg and
                                          //displays the generated webpage in
                                          //a frame window whose name is "f2"
          }
          catch(Exception e)
            {
              e.printStackTrace();
            }
        }
  }
```

web.xml

```
<web-app>
    <servlet>
        <s-n> xyz </s-n>
        <s-c> wishSrv </s-c>
    </servlet>
```

```
    <s-m>
        <s-n> xyz </s-n>
        <u-p> /wurl </u-p>
    </s-m>
</web-app>
```

* use following request url to test the application

  http://localhost:2020/wisApp/main.html

* AppletContext object represents the current executing environment of applet programming so it can be used to make current applet talking with another applet or another web resource program of web application. To get this AppletContext object we can use getAppletContext() predefined java.applet.Applet class

File downloading :

* selecting file from the client machine file system and sending that file to server machine file system through network is called as file uploading and reverse is called as file downloading.
* All the files of a computer managed by operating system together is called as file system of computer.
* Servlet api gives built-in support for file uploading. since it is complex we generally prefer working with third party api called java zoom api perform this file uploading.
* java zoom api comes in the form of jar files and we can download them from www.javazoom.net website. The jar files are

  uploadbean.jar (main jar file)
  struts.jar, cos.jar } dependent jar files to uploadbean.jar

* The classes and interfaces of uploadbean.jar files uses the classes and interfaces of struts.jar file, cos.jar file.
* If java web application uses third party api (other than jdk apis, jee apis) then add 3rd party api related main jar files to classpath and add the third party api related main and dependent jar files to WEB-INF\lib folder of web application.

Ex: If servlet, JSP programs of java web application uses third party api called java 3oom api then add the java 3oom api related main jar file uploadbean.jar file to classpath. similarly the java 3oom api related main and dependent jar files uploadbean.jar, struts.jar, cos.jar to WEB-INF\lib folder of web application.

* To select a file from client machine file system the form page of client machine web application should have file uploading component. For that we take <input> tag as shown below.

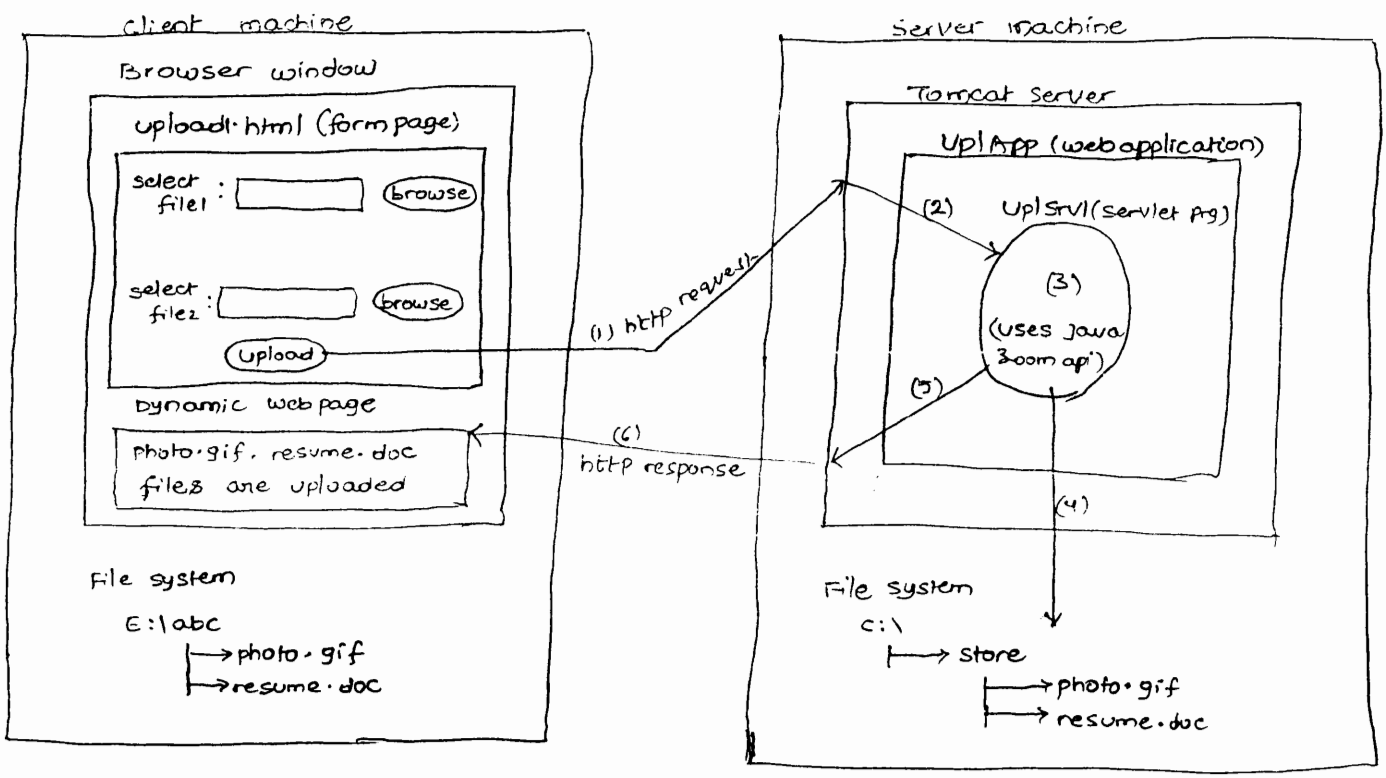
In form page

```
select file: <input type="file" name="f1"/>
```

select file: [ ] Browse

file uploading Component

* The form page that contains file uploading component is recommanded to generate "post" method based http request.



* For the source code of above application refer application (18) that is given in page nos 93 and 94 of booklet.

* we can see file uploading and downloading applications in email account applications (to send & receive attachments), job portal websites like naukari.com (job seeker uploads the resume, HR dept. of company downloads the resume) and matrymoni website

* we can perform file downloading operations in two modes.

(1) making the response of web resource program as downloadable file

(2) making downloading resources of the web application (img file, audio file, video file and etc...) as downloadable file through hyperlinks (or) buttons.

* write the following two lines of content in web resource program like servlet, jsp to make that response of that web resource program as downloadable file.

```
res.addHeader("Content-Disposition", "attachment; filename=abc.html");
res.setContentType("text/html");
```

"Content-Disposition" → response header

"attachment; filename=abc.html" → name of the downloadable file that holds response of the web resource program

"text/html" → response content type

* An action performed on java object or component is called as an Event.

* Executing some logics when event is raised is called as Event handling.

* To perform event handling we use event Listeners.

* The reusable java object is called as Component. we use AWT, Swing concepts of JSE module to perform event handling.

* To perform event handling we need four important details.

(1) Source object on which the event will be raised (like Button Component)

(2) Event class name (like java.awt.event.ActionEvent)

(3) Event Listener (like java.awt.event.ActionListener)

(4) Event handling method (like public void actionPerformed(-) {...})

* From Servlet api 2.4 onwards we can perform event handling on request, session and ServletContext object through servlet listeners.

* Event handling on request object allows us to keep track of the creation and destruction of request object and we can use these timings to calculate request processing time of each request. This event handling also allows us to keep track of when each request attribute is created, modified and removed.

Details that are required to perform event handling on request object.

Source object : request object
Event class : javax.servlet.ServletRequestEvent
Event listener : java.servlet.ServletRequestListener
Event handling methods : (1) requestInitialized(-)
(2) requestDestroyed(-)

} For Event handling on request object

Source object : request object
Event class : javax.servlet.servletRequestAttributeEvent
Event Listener : java.servlet.ServletRequestAttributeListener
Event handling methods : (1) attributeAdded(-)
(2) attributeRemoved(-)
(3) attributeReplaced(-)

} For event handling on request attributes.

* Event handling on ServletContext object helps us to keep track of when ServletContext object is created and destroyed. Based on this we can keep track of timings of web application deployment and undeployment or reloading or stop operations.
* This Eventhandling can also be used to keep track of creation, modification, destruction of ServletContext attributes.

Details that are required to perform eventhandling on ServletContext object

Source object : ServletContext object
Event class : javax.servlet.ServletContextEvent
Listeners : javax.servlet.ServletContextListener (I)
Event handling methods : (1) ContextInitialized(-)
(2) ContextDestroyed(-)

} For event handling on ServletContext object.

Source object : ServletContext object
Event class : java.servlet.ServletContextAttributeEvent
Listener : javax.servlet.ServletContextAttributeListener (I)
methods : (1) attributeAdded(-)
(2) attributeRemoved(-)
(3) attributeReplaced(-)

} For event handling on ServletContext attributes

* Event handling on session object can keep track of when HttpSession object is created and destroyed. Using this we can know how much time there the

user is there in the session. we can also use this event handling to keep track of when each session attribute is created or modified or destroyed.

Details that are required to perform event handling on HttpSession object

source object : HttpSession object
Event class : javax.servlet.http.HttpSessionEvent
Listener : javax.servlet.http.HttpSessionListener
event handling methods : (1) sessionCreated(-)
(2) sessionDestroyed(-)

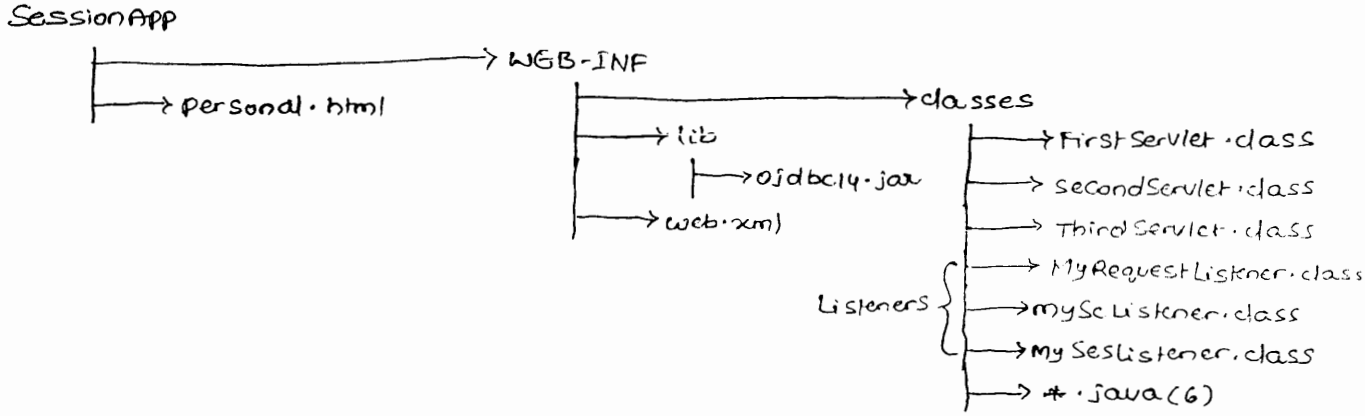
} For event handling on HttpSession object

source object : HttpSession object
Event class : javax.servlet.http.HttpSessionBindingEvent
Listener : javax.servlet.http.HttpSessionAttributeListener
event handling methods : attributeAdded(-)
attributeRemoved(-)
attributeReplaced(-)

} For event handling on HttpSession attribute

* Through servlet listeners we can perform event handling on request, servlet Context, session objects being from outside the servlet, JSP programs.

Example application (with respect to SessionApp application)



Every ServletListener class must be configured in web.xml file using <listener>, <listener-class> tags.

Step(1): Add Listener classes to WEB-INF\classes folder of web application as shown above.

```
// MyReqListener.java
import javax.servlet.*;
import javax.servlet.http.*;
```

```
public class MyReqListener implements ServletRequestListener
{  Long sttime, endtime;
   public void requestInitialized(ServletRequestEvent sre) //executes when
                                                            //req obj is created
     {
        sttime = System.currentTimeMillis();
     }
     //executes when req obj is destroyed
     public void requestDestroyed(ServletRequestEvent sre)
     {
        endtime = System.currentTimeMillis();
        ServletContext sc = sre.getServletContext();
        sc.log("Request is processed for "+(endtime-sttime)+"ms");
        System.out.println("request is processed for"+(endtime-sttime)+"ms");
     }
}
```

MyScListener.java

```
import javax.servlet.*;
import javax.servlet.http.*;
import java.io.*;
public class MyScListener implements ServletContextListener
{
   Long sttime=0;
   long endtime=0;
   public void contextInitialized(ServletContextEvent sce)
    {
       sttime=System.currentTimeMillis(); ServletContext sc=sce.getServletContext();
       sc.log("web application is deployed at"+new Date());
    }
   public void contextDestroyed(ServletContextEvent sce)
    {
       endtime = System.currentTimeMillis();
       sc.log("web application is undeployed/reloaded/stopped at"+new Date());
       sc.log("web application is there in running continuosly for"+(endtime-sttime));
       s.o.p("web application is there in running continuously for"+(endtime-sttime));
    }
}
```

MySesListener.java

```
import javax.servlet.*;
import javax.servlet.http.*;
import java.util.*;
import java.io.*;
public class MySesListener implements HttpSessionListener.
 { long sttime, endtime;
   public void sessionCreated(HttpSessionEvent se)
    {
        sttime = System.currentTimeMillis();
    }
    //execute when HttpSession obj is destroyed
     public void sessionDestroyed (HttpSessionEvent se)
     {
        endtime = System.currentTimeMillis();
        HttpSession ses = se.getServletContext(); getSession();
        //get access to servletContext obj
         ServletContext sc=ses.getServletContext();
         sc.log(ses.getAttribute("name")+"is there in the session for"+
                                                (endtime-sttime)+"ms");
        s.o.p(ses.getAttribute("name")+" is there in the session for"+
                                                (endtime-sttime)+"ms");
     }
}
```

step (2): compile all the listener classes.

```
>javac *.java
```

step (3): Configure all the three listener classes in web.xml file

In web.xml

```
<listener>
    <listener-class> MyReqListener </listener-class>
</listener>
<listener>
    <listener-class> MySesListener </listener-class>
</listener>
<listener>
    <listener-class> myScListener </listener-class>
</listener>
```

step(4): Test the application and also observe the log file (D:\Tomcat6.0\logs\localhost.2011-12-04 file)
current Date

* In our java web applications no servlet listener acts as default listener. That means every listener configuration is mandatory in web.xml file.
* Servlet listeners are useful to keep track of various operations related to request, session, ServletContext objects without disturbing the existing code of web application.
* To make the java class as servlet listener class the class must implement Xxx Listener interface.

Procedure to Configure Jboss 5.x servers with NetBeans IDE

step(1):

Tools → Servers → add server → Jboss application server → next →

Server location: E:\Jboss5.x Soft\jboss-5.1.0.GA → next →

Domain: default → finish → close

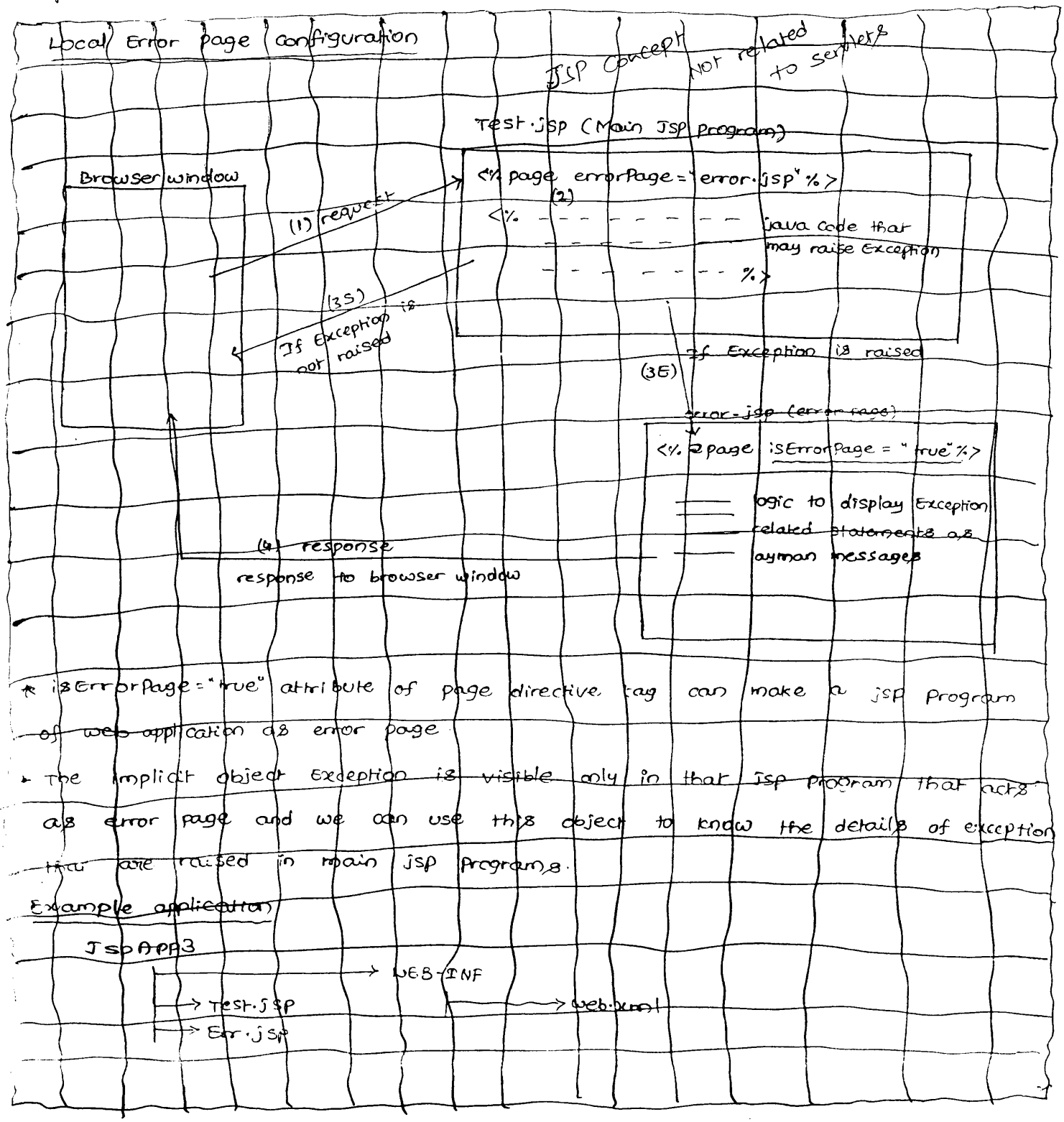
* In servlet programming we can use servletOutputStream class object pointing to response object to write the o/p of the servlet program to browser window as web page through webserver and it is alternate for printWriter stream object.

Ex:
```
res.SetContentType("text/html");
ServletOutputStream sos = res.getOutputStream();
sos.println("welcome to servlets");
sos.println("hello");
```

* In one servlet program we can not use both stream objects (ServletOutputStream, PrintWriter) at a time pointing to res object.
* PrintWriter is a character stream available in java.io package and this stream is good when servlet program generates more character data and less binary data to write to browser window.
* ServletOutputStream is byte stream supplied by servlet api and this stream is good when servlet program generates more binary data and

less text data to write to browser window.

* for better performance of servlet and to reduce one method call use pw.print() method in servlet program instead of of pw.println() method because this pw.println() method internally calls pw.print() method to complete the requirement.
* we can also apply this recommendation while working with ServletOutputStream.



* isErrorPage="true" attribute of page directive tag can make a jsp program of web application as error page
* The implicit object exception is visible only in that jsp program that acts as error page and we can use this object to know the details of exception that are raised in main jsp programs.

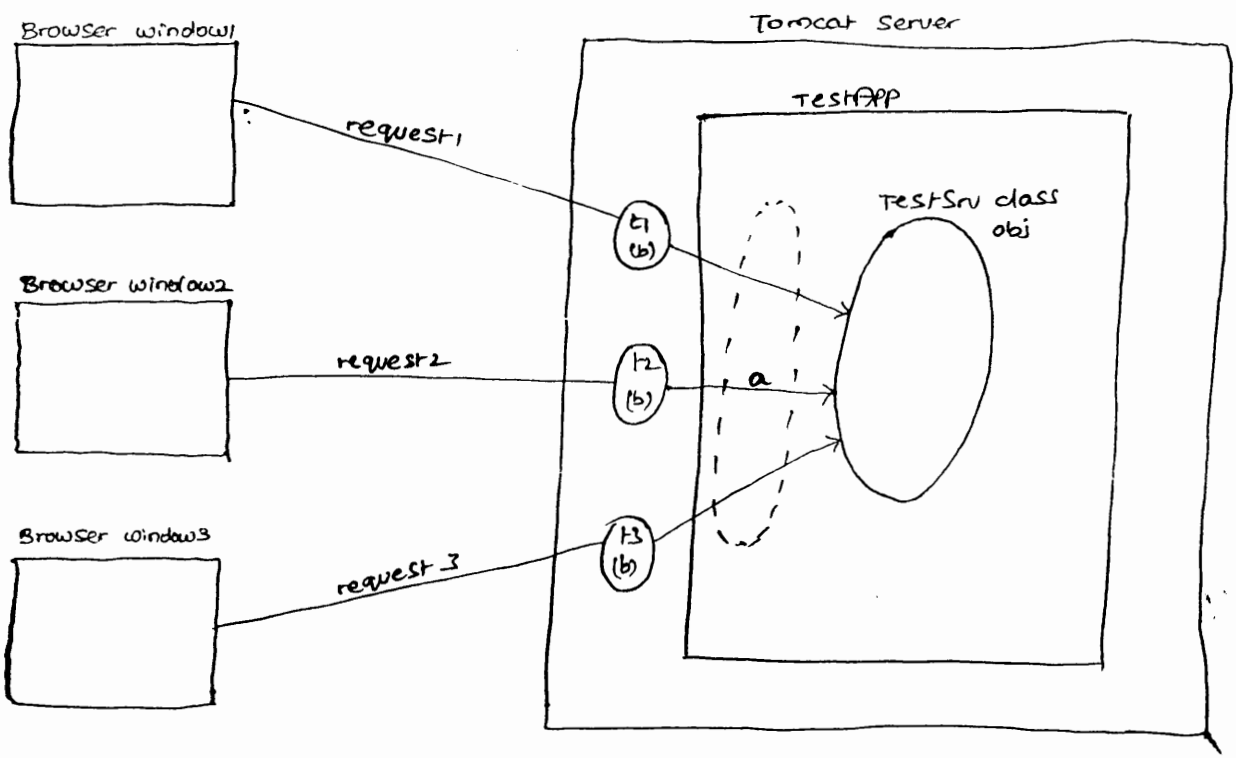
Example application)

JspApp3
- WEB-INF
  - web.xml
- Test.jsp
- Err.jsp

Thread safety

* If multiple threads are running on single object or variable simultaneously or concurrently then data of the object may corrupt. This says our object is not thread safe object.
* To make object/variable as thread safe apply lock on object or variable dto through synchronization and allow only one thread at a time to manipulate object/variable data. The instance variables of our servlet class are not thread safe by default where as the local variables declared in service(-,-) or doXxx(-,-) methods of our servlet class are thread safe by default.

```
public class TestSrv extends HS/GS
{
    int a;
    public void service(-,-)/doXxx(-,-) throws SE,IOE
    {
       int b;
       ----
    }
}
```

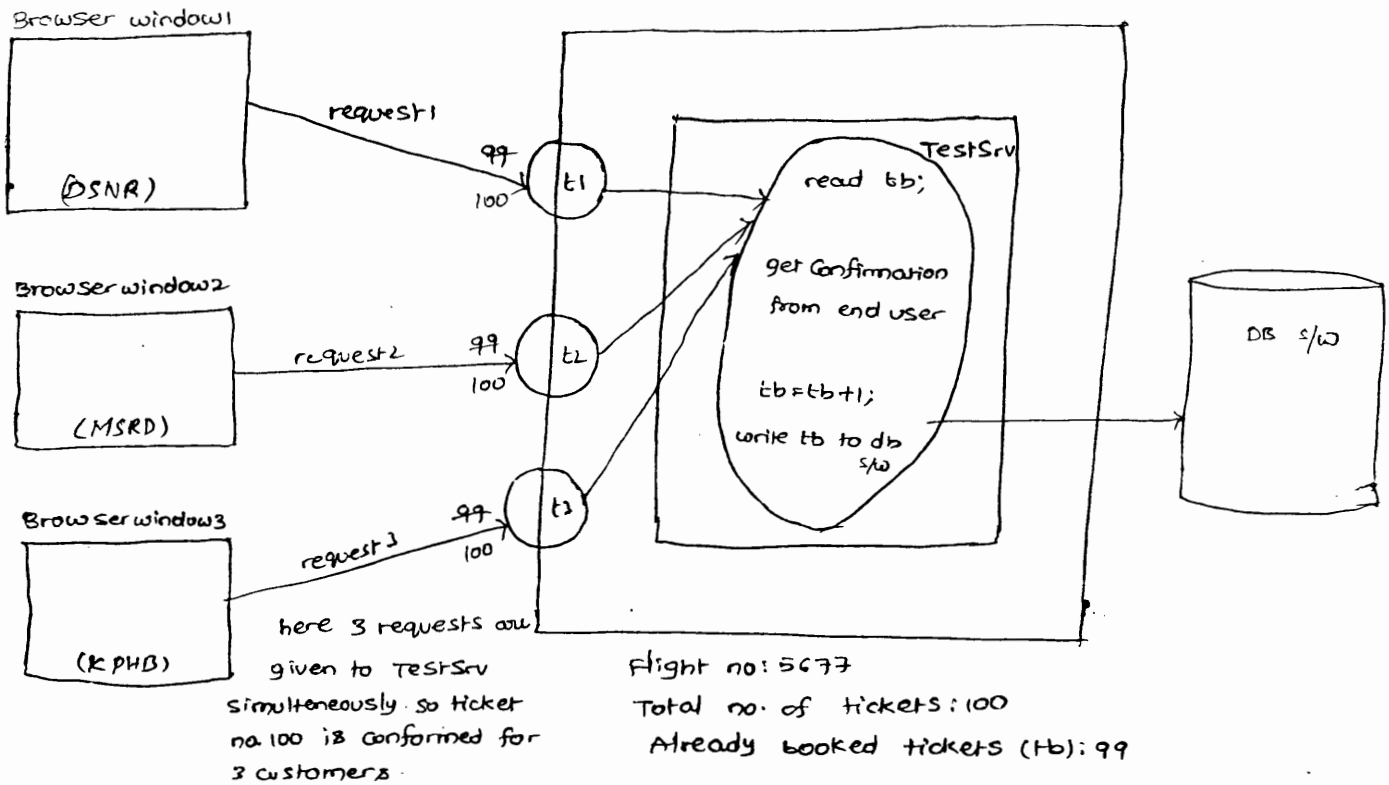


* In the above diagram all the three threads started on TestSrv class object will act on single copy of instance variable 'a' but every thread gets its own copy of local variable 'b'.

* By default every servlet program is a single instance multiple threads component so by default no servlet program is thread safe.

* Making servlet program as thread safe program is nothing but making the instance variables of servlet class as thread safe through synchronizations and other concepts.

understanding the problems related to working with non thread safe servlets (regular servlets)



* Simultaneous request or concurrent request in servlet execution is nothing but servlet program is allowing another request from clients to execute the logic before completition of existing request processing. In such situations servlet program may generate wrong results like booking same ticket number for multiple pessangers as show in the above diagram.

* TO make our servlet program as thread safe servlet program we can use one of the following four techniques.

(1) Work with only local variables of service(-,-)/doXxx(-,-) methods

2) Work with synchronized service(-,-)/doXxx(-,-) methods

(3) Work with synchronized blocks in service(-,-)/doXxx(-,-) methods

(4) make your servlet class implementing javax.servlet.SingleThreadModel (I).

(1) Working with only local variables

```
public class TestSrv extends HS/GS
{
   ---   no instance variables
  public void service(-,-)/doXxx(-,-) throws SE,IOE
   {
     int a,b,c;
     Connection con;
     ------  //write entire using local variables
   }
}
```

* Local variables of service(-,-)/doXxx(-,-) methods are thread safe variables by default. So the above servlet becomes thread safe servlet. Because it does not have instance variables and working with just local variables.

NOTE: This technique is not recommended to use because it is practically impossible to develop servlet classes without instance variables.

(2) Working with synchronized service(-,-)/doXxx(-,-)

```
public class TestSrv extends HS/GS
{
    Connection con;
    int a,b,c;        } Instance variables
    public synchronized void service(-,-)/doXxx(-,-)
    {
      ------
      ------  //write logic using instance variables
      ------
    }
}
```

* All requests that are given to servlet starts threads on our servlet class object and executes service(-,-)/doXxx(-,-).
* In the above code multiple threads started on our servlet class object will act on single copy of instance variables but only one thread is allowed at a time to act on those instance variables because the service(-,-) method is taken as synchronized method.
* Instead of making whole service(-,-) as synchronized method it is recommended to take the support of synchronized blocks on special objects on which you are looking for thread safety.

(3) Working with synchronized blocks in service(-,-)/doXxx(-,-)

```
public class TestSrv extends GS/HS
 {
    Connection con;
    int a,b,c;
    public void service(-,-)/doXxx(-,-)
     {
       ____
       ____
       ____
       synchronized(con obj)
        {
          ____
          ____  logic related to con obj
          ____
        }
       synchronized(session obj)
        {
          ____
          ____  logic related to HttpSession obj
          ____  (creating, modifying, reading and writing, moving attribute values)
        }
       synchronized(ServletContext obj)
        {
          ____
          ____  logic related to ServletContext obj
          ____  (creating, modifying, reading and moving attribute values)
        }
     }//
 }
```

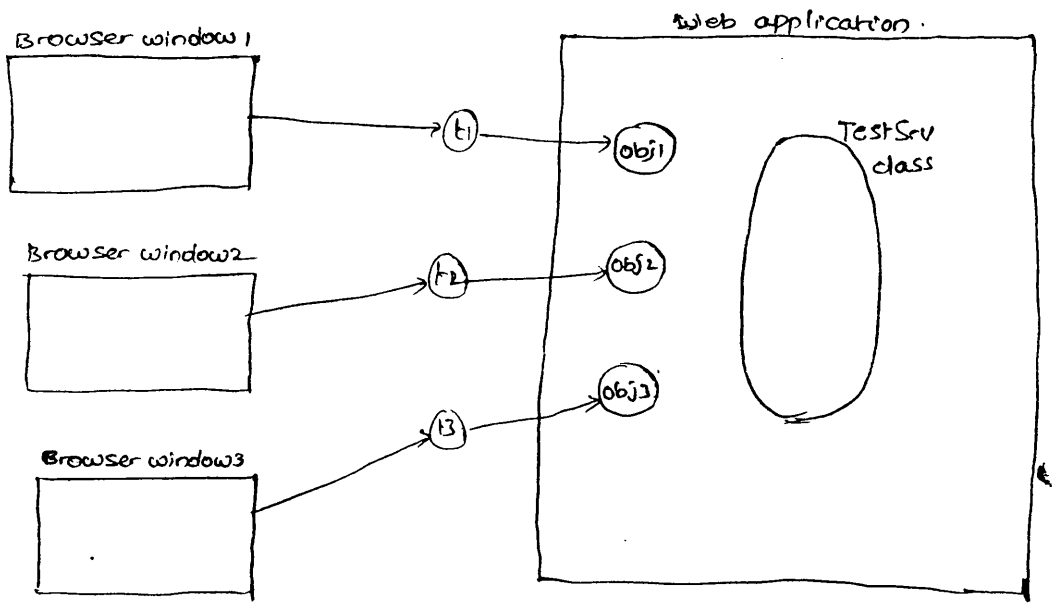
* Working with these synchronized blocks is the most popular technique of real world to make servlet programs as thread safe.
* Since HttpSession object and ServletContext objects are sharable objects in multiple web resource programs of web application it is recommended to use them in synchronized blocks as shown above.

(4) Making our servlet class implementing javax.servlet.SingleThreadModel (I)

```
public class TestSrv extends HttpServlet implements SingleThreadModel
 {
    int a,b;
    Connection con;
    public void service(-,-)/doXxx(-,-)
     {
       ____
       ____
       ____
     }
 }
```

* By seeing the implementation of SingleThreadModel interface the underlaying Server automatically makes our servlet as thread safe. But the technique of thread safety that is used by server will vary server to server.

* This interface has been depricated from servlet api 2.4 because different servers are using different mechanisms while implementing this interface and to make the servlet as thread safe some servers are even creating multiple objects for servlet class for multiple requests on one per request basis. This is violation of servlet specification that says a servlet is a single instance multiple threads component.



obj1, obj2, obj3 are the objects of TestSrv class

* Every Jsp program is not thread safe by default because the Jsp equalent servlet is not automatically implements SingleThreadModel interface and not automatically generates the necessary synchronized blocks in _jspService(--) method.

* TO make Jsp program as thread safe program use <%@ page isThreadSafe = "false" %> tag in Jsp program. The default value of isThreadSafe attribute is "true".

* isThreadSafe = "true" makes Jsp equalent servlet as non thread safe servlet. where as isThreadSafe = "false" makes Jsp and its equalent servlet as thread safe.

File downloading

1. Making the output of web resource program as downloadable file (approach 1)

2. Making the resource (file) of server machine file system as downloadable file (approach 2)

* In approach1 web resource program will not be downloaded. The output of the web resource program as downloadible file through browser window. place the following the two lines of code in servlet/Jsp program to make its response as downloadable file according to approach (1).

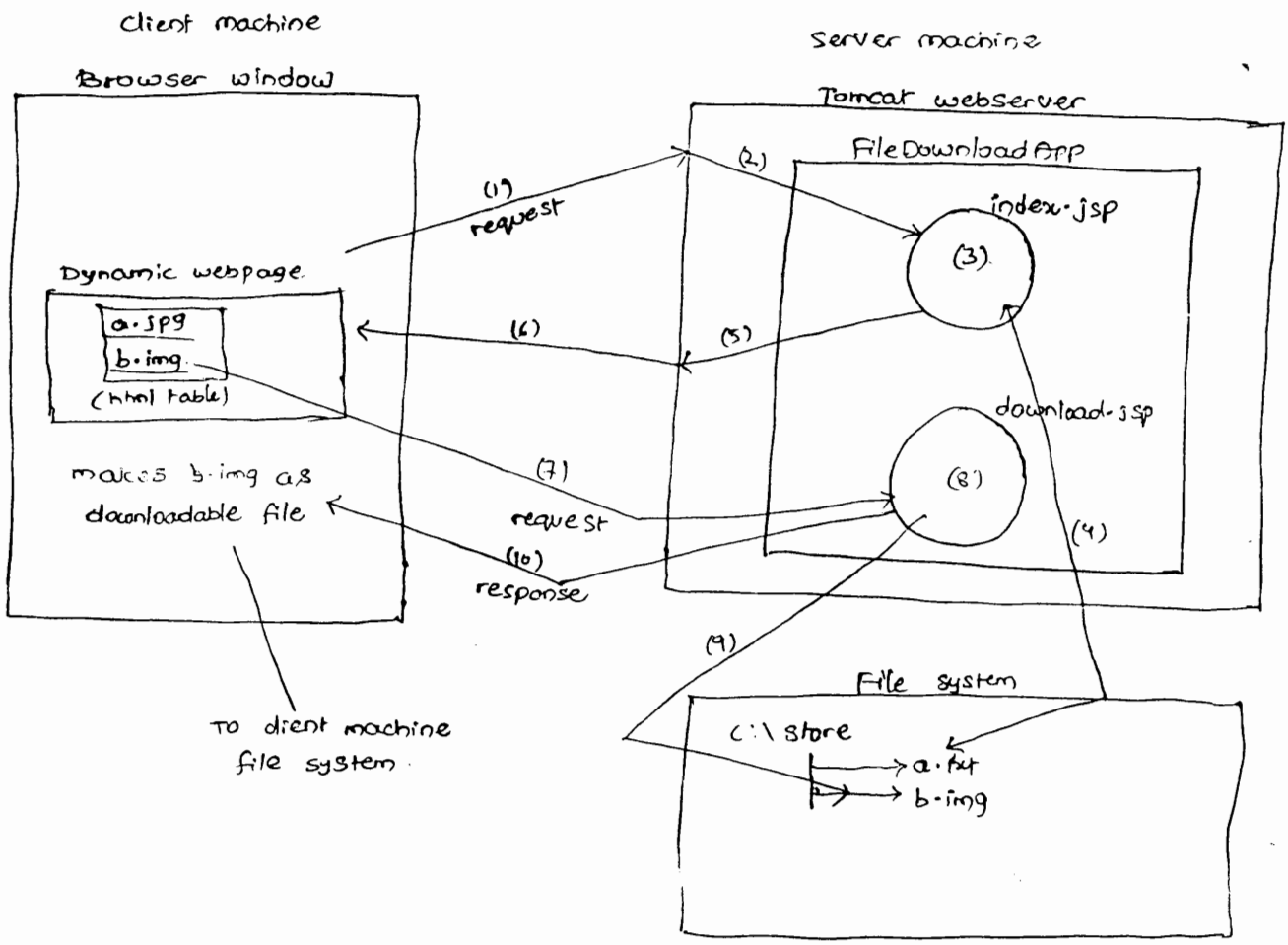
```
response.addHeader("Content-Disposition", "attachment; filename=abc.html");   *1
response.setContentType("text/html")
```

*mime type

filename that holds response of current web resource program.

*1 → special response header to guide webserver towards sending the response of the web resource program to client.

approach (2) based file downloading application

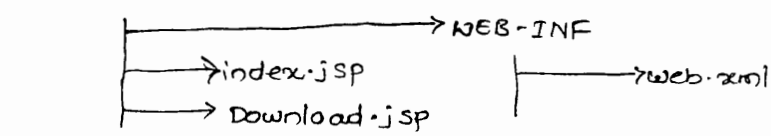


* The above application makes all the files of c:/store folder as downloadable files. Here index.jsp responsibility is to render all the files of c:/store folder as hyperlinks to client and download.jsp responsibility is to download specific file of c:/store folder to client machine file system.

* java.io.File class object can represent a file or directory of file system.

* The listFiles() method of java.io.File class gives all the files and sub directories of crertain folder in the form of java.io.File class object array. [File[]]

Deployement directory structure



Request URL to test the application

http://localhost:2020/FileDownloadApp/index.jsp

Source code

index.jsp

```
<%@ page import="java.io.File, java.util.*" %>
<H1> List of All files under c:\store </H1>
<%
    //locate folder on server machine file system
     File dir = new File ("c:\\store");
     File[] lf = dir.listFiles(); //gives all files and subdirectories of c:\store
     //add all the file names of c:\store to al obj (ArrayList obj)
     ArrayList al = new ArrayList();
     for(int i=0; i<lf.length; i++)
      {
        if (lf[i].isFile())
         {
           al.add(lf[i].getName());
         }//if
      }//for
```

```
//logic to display the file names of c:\\store folder as downloadable
                                                    files (hyperlinks based)
if(al.size()!=0)
 {
  %>
     <table border=1><tr><th><b>Filename</b></th></tr>
  <%
     String fname=null;
     for(int i=0; i<al.size(); i++)
      {
        fname=(String)al.get(i);
      %>
       <tr>
        <td><a href="Download.jsp?filename=<%=fname%>">
                          <%=fname%></a></td></tr>
     <%} //for
   }//if
  %>
</table>
```

web.xml

```
<web-app/>
```

Download.jsp

```
<%@page import="java.io.*"%>
<%
    //reading req param value (file name)
    String filename=request.getParameter("filename");
    //locate the file to be downloaded
    File f=new File("c:\\store\\"+filename);
    //get servletOutputStream object
    ServletOutputStream op=response.getOutputStream();
    //get mime type of the file to be downloaded dynamically
    String mimetype=application.getMimeType("c:\\store\\"+fileName);
    //set the MIME type as response content type
    response.setContentType((mimetype!=null)?mimetype:"application/octet-
                                                        Stream");
```

```
//set response content length
response.setContentLength((int)f.length());
//work with special header to make file as downloadable file.
response.setHeader("Content-Disposition", "attachement; filename=" +
                                                    fileName);
//Stream and buffer based logic to download the file
byte[] buf = new byte[1024];
int length=0;
DataInputStream dis=new DataInputStream(new FileInputStream(f));
while((dis!=null) && (length=dis.read(buf))!=-1)
 {
   op.write(buf,0,length);
 }//while
 dis.close();
 op.flush();
 op.close();
%>
```

Security in servlets :

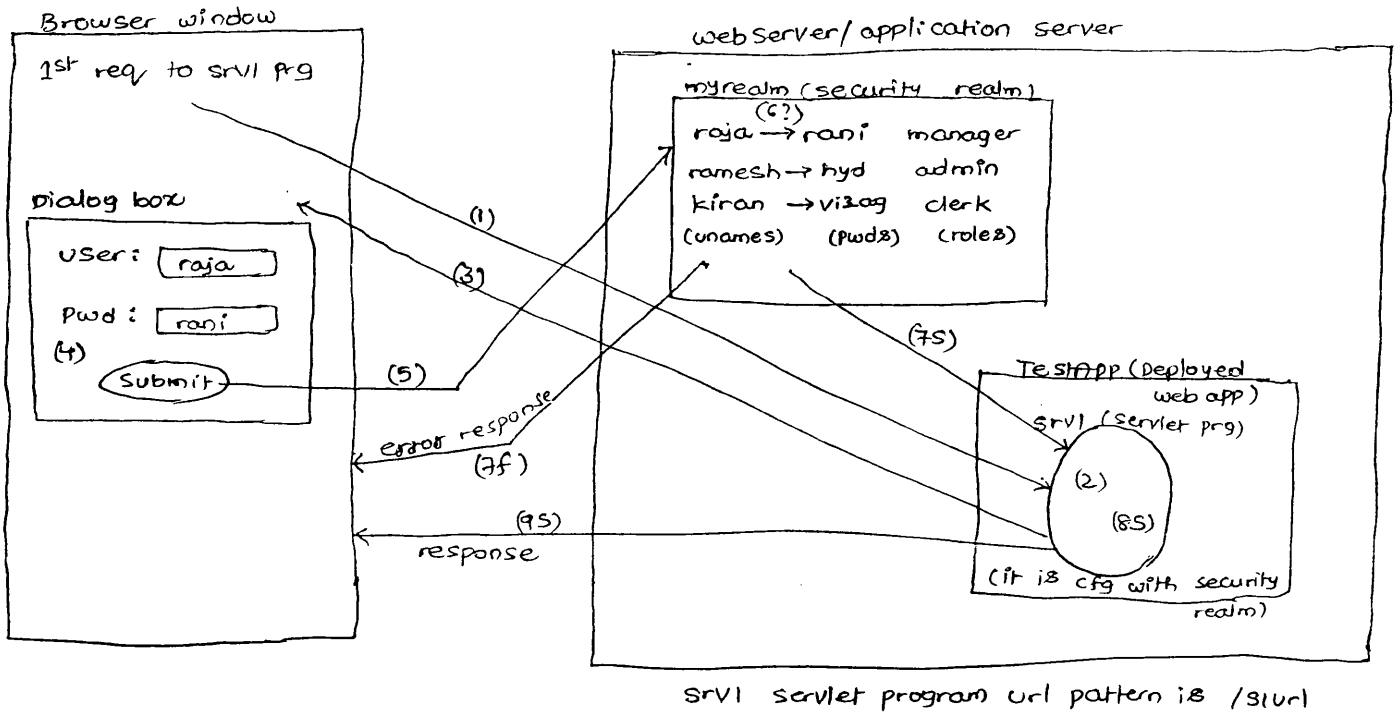
* security = Authentication + Authorization
* security means authentication + authorization.
* checking the identity of a user is called as authentication.
* checking the access permissions of a user to use certain resources of the application is called as authorization.

Ex : Every employee must be authenticated to use banking project. But only cashier is authorized to use cash module. Adminstrator is authorized to use every module.

* Programmers are not responsible to perform network security. These operations will be taken care by network adminstrator through firewalls.
* Programmers are responsible for application managed security and server/container managed security in web applications.
* Application managed security means it is the security that is developed by programmer manually inside the application.
* Server managed security means it is the authentication and authorization performed by container on web resource programs by using security middleware

service.

* In server managed security the logical context where users, passwords and roles are defined is called as security realm and every server comes with one default security realm called myrealm.
* In tomcat server this default realm related configurations can be done in <Tomcat_home>\conf\tomcat-users.xml file.



srv1 servlet program url pattern is /s1url

* A role defines access permissions (set of access permissions) instead of specifying access permission for each user seperately define a role specifying access permissions and assign that role to users.

with respect to diagram

(1) End user gives direct request to srv1 program.

(2),(8)Since no user name and password is coming along with the request and since srv1 is configured with security realm the srv1 sends 401 response (Status code based response having one dialog box.

(4),(9)End user submits dialog box having username and password and that request will be traped by security realm.

(6?) security realm verifies whether given username, password correct or not against the username and password of realm.

(7s) If correct, request goes to srv1 program.

(7f) If not correct renders same dialog box for two more times then shows 403 status code based error response.

(8S)(9S) Srvl program executes and generated response goes to browser window.

* We can configure server managed security authentication on our webresource program in four modes.

(1) BASIC
(2) DIGEST
(3) FORM
(4) CLIENT-CERT

BASIC → uses base 64 algorithm internally.

* does not encrypt given username and password.
* Generates built-in dialog box for end user to submit username and password.

DIGEST :

* uses MD5 hashing algorithm internally.
* It is stronger than base-64 algorithm.
* Remaining all are same as BASIC, but it encrypts given username and password.

FORM :

* same as BASIC but allows programmer to design his own form page, error page instead of ready made dialog box, ready made errors message.

CLIENT-CERT :

* works with digital certificates using some algorithms like RSA
* use https and SSL concepts to implement this.
* To configure security realm and authentication models in the webresource programs of java web application use various configurations in web.xml file.
* For example code on, BASIC, DIGEST, FORM based authentication refer supplymentary hand out given on 25-12-2011.

Procedure to create users and roles in default security realm (myrealm) of Tomcat server

Go to <tomcat_home>\conf\tomcat-users.xml → add these entries.

```
<role rolename="
<user username="raja1" password="raja1" roles="manager"/>
<user username="raja2" password="raja2" roles="admin"/>
<user username="raja3" password="raja3" roles="manager,admin,tomcat"/>
```

* Uncomment existing roles and users
* This server managed security can be used as outer layer security for web application even though web application contains application level security. This is very useful in milatory based, NASA based, ISRO based webapplications.
* The client-cert authentication model sends digital certificates to browser window as the response of initial request and that digital certificate will be installed at client side with users permission. Now onwards browser window and server communicates with each other having encrypted data based that is generated the algorithm that is used to generate digital certificate. In this browser window (client) and server should communicate with each other by using HTTPS protocol (HTTP over SSL).
* In CLIENT-CERT authentication model server allows only those clients which send request having digital certificate.
* To make server sending digital certificate to browser as the response of initial request, the programmers must develop and configure digital certificate with server
* In java on built-in tool keytool is given to generate digital certificates using different algorithms.

Procedure to create digital certificate, to concate digital certificate with server and to work with CLIENT-CERT authentication model using HTTPS protocol

step(1): Generate digital certificate using "keytool" by specifying RSA algorithm

```
>keytool -genkey -alias tomcat1 -keyalg RSA
                        ↓
                  any name (logical name of digital certificate
```

```
Enter key store password: sathya1
Re-enter new password : sathya1
        :
        :
        :
        correct?
      yes
Enter key password for <tomcat1>: (press Enter key)
```

step(2): The above steps based digital certificate will be generated in C:\Documents and setting\ welcome folder as .keystore file.
                                ↓
                  currently logged in windows username

Step(3): Configure the above generated digital certificate with Tomcat Server by enabling HTTPS protocol.

Go to <tomcat-home>\conf\server.xml file → un comment that <Connector> tag that points to SSL HTTP/1.1 protocol (line no. 66) → make sure that the <Connector> tag at line 66 (3rd <Connector> tag) is having following content

```
<Connector port="8443" protocol="HTTP/1.1" SSLEnabled="true"
    maxThreads="150" scheme="https" secure="true" clientAuth="false"
    sslProtocol="TLS" keystoreFile="c:\Documents and Settings\welcome\.keystore"
    keystorePass="satya1" />
```

keystoreFile: The above generated digital certificate file

keystorePass: The above choosen password.

Step(4): Restart the server.

Step(5): Give request to tomcat server from browser window.

https://localhost:8443

https://localhost:8443/waone/input.html

https: Protocol

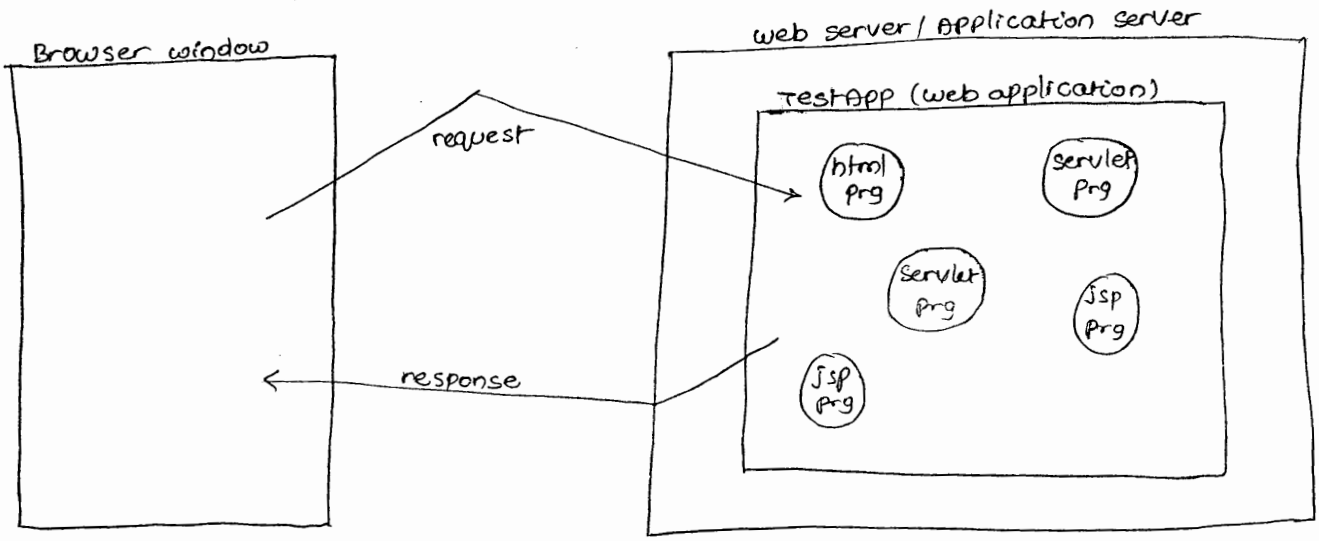
8443: Protocol port no. of Http with SSL environment

*For related information on SSL refer the SSL chapter of tomcat documentation.

NOTE: Industry prefers working with either form based or CLIENT-CERT models for securing web applications.

When CLIENT-CERT is enabled we can even also enable Basic or DIGEST or FORM authentication models.

JSP



* A web application is a collection of web resource programs having capability to generate web pages. There are two types of web resource programs
  (1) Server side programs
  (2) client side programs
* server side programs resides and executes from the server.
  Ex: servlet program, JSP program and etc..
* clientside programs resides in web server but they come to browser window for execution.
  Ex: html program, JavaScript program, VB script program and etc...

NOTE: Decide wheather web resource program is client side or server side program based on the place where it executes not based on the place where it resides.

* Server side programs generate dynamic web pages.
* client side programs generate static web pages.
* Every web application is a collection of both client side and server side web re source programs.

client side technologies (useful to develop clientside web resource programs)

html
JavaScript
VB script
Ajax

Server side technologies (useful to develop serverside web resource programs)

Servlets ——→ from Sun ms (2) } (1)
JSP ——→ from Sun ms (3) }
ASP ——→ from microsoft
asp.net ——→ from microsoft (4)
PHP ——→ from ~~Adobe~~ & Apache foundation (5)
SSJS ——→ from Nestcape
Coldfusion ——→ From Adobe

Why Sun microsystem has given JSP as server side technology when they have already got servlets as server side technology?

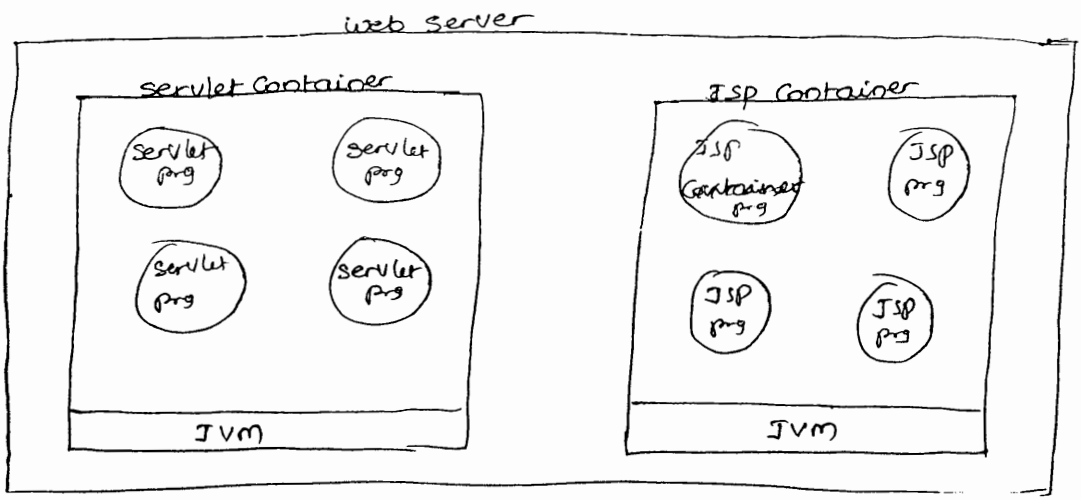
(A) While working with asp, PHP technologies we can go for tags based programming. To work with servlets strong java knowledge is required and tags based programming is not possible so no programmer had liked servlets in its initial days. To attract non-java programmers like asp programmers the Sun microsystem has given a tag based server side technology called JSP having all the features of servlets.

Drawbacks of servlets Programming

(1) Strong java knowledge is required so not suitable for non-java programmers.
(2) The Presentation logic (html code) will be mixed up with java code (pw.println()) so modification done in one code may effect another code.
(3) writing HTML code in servlet programming is a complex process.
(4) In servlet programming implicit objects are there but we need to write some additional code to get access to those objects.
(5) Modification done in source code of servlet program will be effected only after recompilation of servlet program and reloading of web application in most of the servers.
(6) Programmer should explicitly take care of exception handling.
(7) The session object required for session tracking should be created by the programmer manually.
(8) Learning curves quite high in servlets & servlet programming is not thread safe by default so we need to write some additional code to make our servlet program as thread safe.

Features of JSP

(1) Allows tag based programming so strong java knowledge is not required.
(2) suitable for both java and non-java programmers.
(3) use 9 implicit objects and we can use them directly in our JSP program.
(4) Modifications done in JSP program will be recognised by underlaying server automatically without reloading of web application.
(5) Takes care of exception handling.
(6) ~~JSP program is thread safe by default.~~
(7) Allows us to seperate presentation logic from (html code) java code (business logic)
(8) Easy to learn and utilize.
(9) Allows us to work with JSP supplied built-in tags and third party supplied JSP tags and even allows to develop custom JSP tags.
(10) use all the features of servlet.



* JSP, asp, asp.net, PHP programs are there to execute based on page compilation process.
* The process of converting one form of program source code to another form of source code is called as page compilation.
* In JSP page compilation the source code of JSP program will be converted into an equivalent servlet program source code. For this JSP page compiler is required.
* Every web server/application server supplies one JSP page compiler.
* Servlet Container executes servlet programs and takes care of the life cycle of servlet programs.

* JSP Container executes JSP programs by executing the JSP equivalent servlet programs and also takes care of the life cycle of JSP programs.
* Every JSP Container is enhancement of servlet Container.
* Every web server and application server supplies servlet Container and JSP container.
* The servlet Container is developed based on servlet api specification. In tomcat server servlet Container name is catalina.jar.
* JSP Container will be developed based on servlet and JSP api specification. In Tomcat server JSP Container name is Jasper (Tomcat_home\lib\jasper.jar)

The 9 implicit objects of JSP programming

out
config (It is ServletConfig)
application (It is ServletContext obj)
request
response
page
pageContext
exception
session

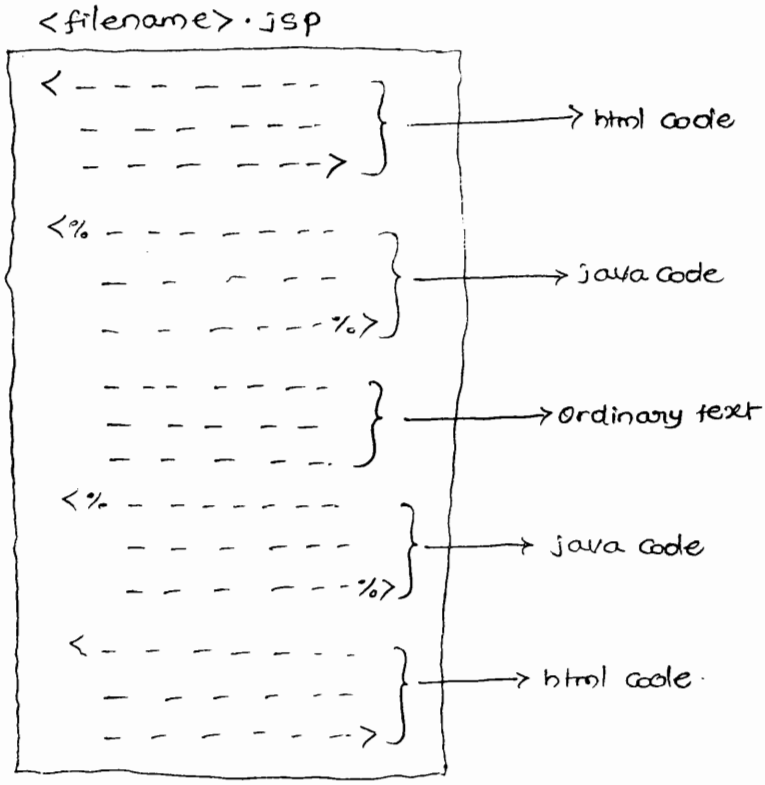
* Once the JSP equalent servlet is generated all these objects will be created in that servlet automatically. So we can call them as implicit objects or) built-in objects of JSP.

What is the difference between HTML and JSP ?

HTML	JSP
* client side technology	* server side technology
* Needs html parser interpreter for execution.	* Needs JSP page Compiler and JSP Container for execution.
* HTML program generates static web pages.	* JSP programs can generate static & dynamic web pages.
* Given by W3C	* Given by Sun micro systems
* Can be used in any kind of web application development (Java & non-Java web applications).	* Should be used only in java based web applications.

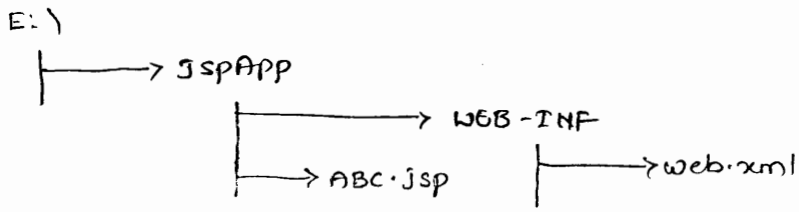
* Does not allow to create custom tags. * Allows us to create custom JSP tags.

Structure of JSP program


<filename>.jsp
<- - - -

Procedure to deploy the Jsp program based web application

Step(1): create deployment directory structure by taking the JSP program as web resource program.



ABC.JSP

same as above code.

web.xml

<?web-app/>

NOTE: Configuration of JSP program in web.xml file is optional.

Step(2): Deploy the above web application in tomcat server.

Copy E:\JspApp folder to <tomcat_home>\webapps folder.

Step(3): start the tomcat server.

Step(4): Test the webapplication. Give request to ABC.jsp

open browser window → Type this url

http://localhost:2020/JspApp/ABC.jsp

* while working with only JSP programs based web applications the web.xml file is optional in most of the servers but it is recommended to place web.xml file.
* we can hide the JSP program file name & extension from end users to hide technologies details from end users by providing url pattern to JSP program in web.xml file.
* we can provide url-pattern to JSP program by containing JSP program in web.xml file as shown below.

<web-app>

<servlet>

```
        <servlet-name> xyz </servlet-name>
        <jsp-file> /ABC.jsp </jsp-file>
    </servlet>
    <servlet-mapping>
        <servlet-name> xyz </servlet-name>
        <url-pattern> /test1 </url-pattern>
    </servlet-mapping>
</web-app>
```

* Using following request urls to generate request to Jsp program.

  http://localhost:2020/JspApp/ABC.jsp

  http://localhost:2020/JspApp/test1

* WEB-INF is a private directory of java web application so only web server can recognize that directory.

* The web resource programs that are placed inside WEB-INF (or) its sub folders will be recognized by the web server only when they are configured in "web.xml" file.

  Ex: servlet programs.

* The web resource programs that resides outside the WEB-INF folder can be recognized by the webserver directly so there is no need of configuring those programs in web.xml file.

  Ex: html programs, Jsp programs.

* The JSP program can have two types of objects.

  (1) Implicit objects:

  The 9 implicit objects like out, request, response and etc.

  (2) Explicit objects

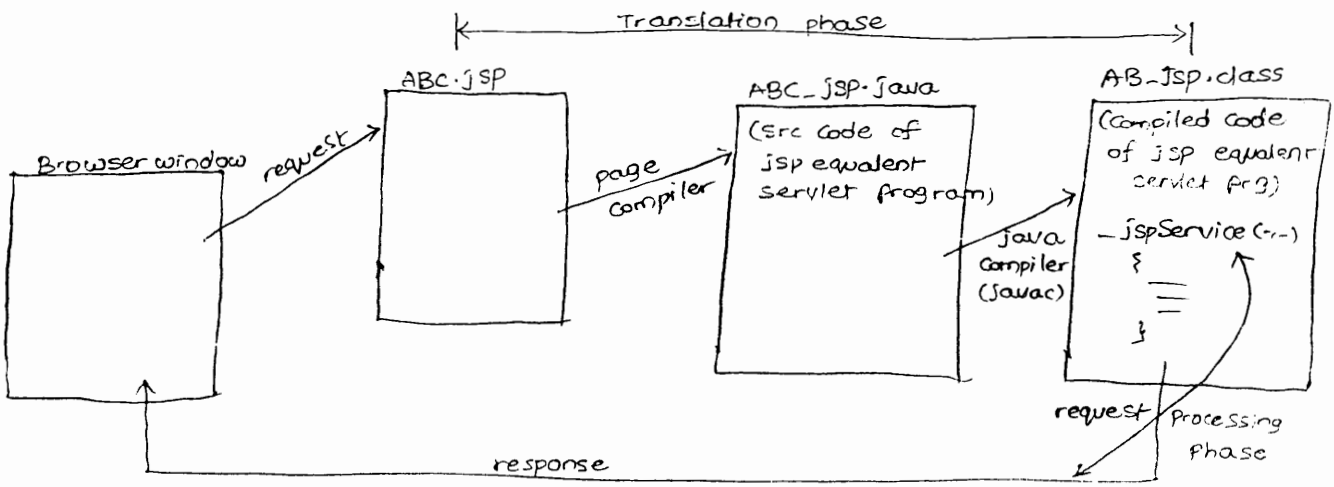
  Objects created by programmer manually.

Ex:

```
<%. java.util.Date d = new java.util.Date(); %>
```

d ↓ Explicit object

* this, super key words are implicit objects of stand alone java programs.
* request, response, ServletConfig & ServletContext objects are implicit objects of Servlet Programming and we need to write code to get access to those objects.
* In the execution process of JSP program two phases are there.
  1) Translation phase
  2) Request Processing phase
* In translation page the JSP program will be converted into an equaient servlet program.
* In RequestProcessing phase the Compile code of JSP ~~<~~ servlet (equalent) executes and the generated response goes to browser window.



* In Request Processing phase the _jspService(--) of JSP equalent servlet executes and generated response goes to browser window as web page.

The life cycle methods of Servlet program

init(ServletConfig cg)

service(ServletRequest req, ServletResponse res)

destroy()

The life cycle method of JSP program

jspInit() / _jspInit() ①
_jspService(HttpServletRequest req, HttpServletResponse res)
jspDestroy() / _jspDestroy() ②

① & ② are given from Tomcat 6 onwards and not there in other servers as life cycle methods.

* _jspInit(), _jspDestroy() methods are introduced as life cycle methods from recent versions of JSP so only latest web server s/w and application server s/ws are supporting them.
* jspInit() / _jspInit() life cycle methods execute when JSP equalent Servletclass is instantiated.
* _jspService(-,-) life cycle method executes when JSP program gets request from clients.
* jspDestroy() life cycle method executes when the servletContainer is about to destroy object of JSP equalent servlet Program (servlet class).
* In Tomcat server the JSP equalent servlet Program of ABC.JSP belonging to JSPApp web application comes in <tomcat_home>\work\Catalina\localhost\JSPApp\org\Apache\JSP folder as ABC_JSP.java, ABC_JSP.class.
  - org\Apache\JSP: package name of JSP equalent servlet class
  - ABC_JSP.java: Source Code of JSP equalent servlet
  - ABC_JSP.class: Compiled Code of JSP equalent servlet.
* The file name and class name of JSP equalent servlet program changes Server to server.
* The generated JSP equalent servlet is HttpServlet.
* JSP Container internally internally takes the support of servlet Container and servlet life cycle methods call JSP life cycle method and to execute JSP equalent servlet program.
* Every JSP equalent servlet extense from one JSP Container supplied Java class and this java class extent from HttpServlet class and overrides servlet life cycle methods calling JSP life cycle methods internally.
* In Tomcat server JSP equalent servlet class exten from the Jasper (JSP container) supplied org.apache.jasper.runtime.HttpJspBase class and this HttpJspBase class extends from HttpServlet class, overrides servlet life cycle methods calling JSP life cycle methods internally.
* init(-) ⟶ internally calls jspInit() / _jspInit()
* service(-,-) ⟶ internally calls _jspService(-,-)
* destroy() ⟶ internally calls _jspDestroy() / jspDestroy()

* refer source code of JSP equalent servlet and HttpJspBase class once.
* In Tomcat server jasper.jar represents JSP container.
* In weblogic server weblogic.jar represents JSP container, EJB Container, servlet Container and etc....
* To deploy above given JSPAPP web application in Adv.javaBatchDomain of weblogic 10.3

  copy JSPAPP folder to <weblogic_home>\middleware\user_projects\domains\Adv.javaBatchDomain\autodeploy folder.

* To generate request to JSPABC.jsp of above JSPAPP webapplication use the following request URL.

  http://localhost:7001/JSPAPP/ABC.JSP

* JSP page Compiler name in weblogic is jspc and it generates ABC.JSP related JSP equalent servlet as shown below.

  --abc.java (source code jsp equalent servlet program)
  --abc.class (Compiled code of JSP equalent servlet program) } For location Search in subfolders of Adv.javaBatchDomain

NOTE: this jspc pageCompiler deletes source code of jsp equalent Servlet program once Compiled code is generated.

* The request given to JSP program participates only in request processing phase when JSP program source code is not modified Compare to previous request and .class file of JSP equalent servlet program is not deleted otherwise the request given to JSP program participates in both translation phase, request processing phase.
* JSP Container preserves the source code of JSP program internally related to current request until next request comes to that JSP program for comparision purpose. In this process JSP container internally uses ARAXIS, WDIFF kind of comparision tools internally.

  There is no necessary of Configuring JSP equalent servlet program generated by JSP page Compiler in web.xml file.

- All Implicit objects of JSP will be created automatically in _jspService(-,-) method of JSP equalent servlet program.

Tomcat server gives servlet-api.jar, jsp-api.jar files representing the sun micro system supplied Servlet, JSP api specifications. The same Tomcat server gives catalina.jar (as servlet Container), jasper.jar (as JSP Container)

based on that are developed based on servlet, JSP api specifications.

What happens if we modify the source code of JSP equalent servlet program manually?

(A) The modifications will be effected in the output of JSP program when

(i) JSP equalent servlet program is recompiled.

(ii) When the web application is reloaded.

Min If request is given to JSP program without modifying source code. otherwise the modifications done in source code of JSP equalent servlet program will not be effected.

NOTE: Compile the JSP equalent servlet program generated by page compiler of Tomcat server add the following four jar files to classpath.

jsp-api.jar, servlet-api.jar, jasper.jar, el-api.jar

Available in <tomcat_home>\lib folder.

What is the use of enabling <load-on-startup> on JSP program (or) How to make first request given to JSP program directly participating in request processing phase?

(A) Enable <load-on-startup> on JSP program Configuration done in web.xml file which makes JSP Container and servlet Container complete translation phase on JSP program & and to perform pre instantiation and pre initialization on JSP equalent servlet program either during server startup or during deployment of web application

* If you modify source code of JSP program before first request then the entire effect of <load-on-startup> will be wasted.

To enable <load-on-startup> on JSP Program

```
<web-app>
    <servlet>
        <servlet-name> x </servlet-name>
        <jsp-file> /ABC.jsp </jsp-file>
        <load-on-startup> 4 </load-on-startup>
    </servlet>
    <servlet-mapping>
        <servlet-name> x </servlet-name>
        <url-pattern> /test1 </url-pattern>
    </servlet-mapping>
</web-app>
```

Can you generate JSP equalent Servlet program outside the web server or application server before actually deploying the web application and requesting the web application?

(A) yes we can do by running the jsppage Compiler tool outside the server at Command prompt. This process is useful to know the syntatical errors of JSP program out side the server.

The weblogic server supplies "weblogic.jspc" tool as JSP page Compiler tool.

Example on weblogic.jspc tool

Step(1): Add <welogic-home>\middleware\wlserver_10.3\server\lib\weblogic.jar file to classpath.

Step(2): Arrange .jsp program

E:\temp
⟶ ABC.jsp

Step(3): use weblogic.jspc tool

e:\temp> java weblogic.jspc -Compiler javac ABC.jsp

It deletes --abc.java after generating --abc.class (JSP equalent Servlet Prg).

The compiler that has to be used to compile the source file jsp equalent servlet program.

e:\temp>java weblogic.jspc -keepgenerated -Compiler javac AB.jsp

It preserves --abc.java even after generating --abc.class file.

* Tomcat server gives org.apache.jasper.JspC class as jsp page Compiler.

Example to work with org.apache.jasper.JspC Tool

1) Keep your web application ready as shown below

E:\Temp
⟶ JSPAPP
⟶ WEB-INF ⟶ web.xml
⟶ ABC.jsp

2) Make sure that the following jar files are added to classpath.

servlet-api.jar, jsp-api.jar, jasper.jar, jasper-el.jar, tomcat-juli.jar

jsp-api.jar, jasper.jar, jasper-el.jar available in <Tomcat_home>\lib folder

tomcat-juli.jar <Tomcat_home>\bin folder

3) Run the Tool.

E:\Temp\JSPAPP> java org.apache.jasper.JspC -d . ABC.jsp

this tool is capable of just generating source code of JSP equalent servlet program

destination folder to genarate JSP equalent servlet program.

JSP tags

* The element name that is surrounded with "<", ">" symbol is called tag.
  <B> ---> Bold tag name
  B ---> Bold element name.
* Java code placed in JSP is called as script code. Javascript code, VB script code, place in HTML program is called as script code.
* The code that can't be executed independently and should be embeded with other technologies based programs for execution is called as script code.

(Tags) Built-in tags/ elements of JSP programming

Scripting elements / tags

① scriptlet (<% ---- %>)
② Declaration (<%! ---- %>)
③ Expression (<%= ---- %>)

* Scripting tags are there to allowing the programmer to place Java code.

Directive tags/elements

① Page Directive (<%@ page attributes %>)
② Include Directive (<%@ include attributes %>)
③ taglib Directive (<%@ taglib attributes %>)

* Directive tags are given to provide global information to JSP page.

JSP comments

<%-------- %> (for Commenting JSP tags)

Standard Action tags

<jsp : forward>
<jsp : include>
<jsp : useBean>
<jsp : setProperty>
<jsp : getProperty>
<jsp : param>
<jsp : plugin>
<jsp : fallback>
:
:
:

* Standard Action tags are given to perform dynamic operations in JSP programming at request processing phase.

scriptlets

* Every JSP tag can be used with two types of syntaxes.
  (1) Standard syntax
  (2) xml syntax

Standard syntax of scriptlet

```
<%
   - - -
   - - -   } java code
   - - - -
%>
```

xml syntax of scriptlet

```
<jsp:scriptlet>
   - - - -
   - - - -   } java code
   - - - -
</jsp:scriptlet>
```

* The java code placed in scriptlet goes to _jspService(-,-) method of jsp equalent servlet program as it is.
* Programmers can place business logic or requesting processing logic code in the scriptlets of JSP program.
* Implicit objects of JSP are visible in the scriptlet. In one JSP program we can keep multiple scriptlets.
* variable declared in JSP comes as local variables of _jspService(-,-) method in JSP equalent servlet so they must be initialized.

Ex: In ABC.jsp (with standard syntax)

```
<%  int a=10;
    int b=20.
    int c= a+b;
    out.println("Result is:" +c);
%>
```

In ABC.jsp (with the support of XML standard)

```
<jsp:scriptlet>
    java.util.Date d= new java.util.Date();
    out.println(d.toString());
</jsp:scriptlet>
```

Ex(2):

```
<jsp:scriptlet>
    int a=10;
    int b=20;
    if (a<b)
        out.println("a is small");
    else
        out.println("b is small");
</jsp:scriptlet>
```

* The above code execution gives Exception because in (a<b) statement "<" symbol we bee be taken as begining of subtag having XML (or) JSP meaning. That means it will not be taken as the conditional operator of java code.

Solution(1): (work with standard syntax)

```
<% int a=10;
   int b=20;
   if (a<b)
       out.println("a is small");
   else
       out.println("b is small");
%>
```

Solution(2): (work with XML syntax along with <![CDAT..]]>)

```
<jsp:scriptlet>
    <![CDATA[   int a=10;
     *1         int b=20;
                if (a<b)
                    out.println("a is small");
                else
                    out.println("b is small");
</jsp:scriptlet>
```

(*1) This "CDATA" makes page compiler to take the body of the tag as text information (java code) and supresses the meaning of xml and JSP for that code.

NOTE: while working with XML syntax based JSP tags use "<" symbol carefully but there are no problems with ">" symbol.

```
<%. public int Sum (int x, int y)
                {
                return x+y;
                } %>
```

The above java code is invalid as java does not support nested method definition. But we can place method calls code in scriptlets.

```
<% String s="hello";
   int leng = s.length();
   out.println("Length is:" +leng);
%>.
```

* while developing JSP based webapplication, while deploying and executing those web applications there is no necessity of adding any jar files to classpath because there is no need of compiling JSP programs from command prompt.

Declaration

standard syntax

```
<%!
    Decl. of instance variables,
    definitions of user defined methods,
    definitions of jspInit() & jspDestroy() life cycle methods
%>
```

xml syntax

```
<jsp:declaration>
    ____
    ____
</jsp:declaration>
```

* The java code placed in declaration tag comes out side the _jspService() of jsp equalent servlet. So implicit objects are not visible in this declaration tag because all implicit objects are local variables in _jspService() method.

Ex(1): `<%! int a=10; int b=20; %>`

* variables declared in declaration tag will come as the instance variables of the jsp equalent servlet class.

How to differentiate variables in the scriptlet when declaration tag variables and scriptlet tag variable are taken having same name?

(a) use "this" key word (or) implicit object "page" to differentiate declaration tag variable from the scriptlet tag variable.

```
<%! int a=10; %>
<% int a=20;
    out.println("local variable (scriptlet) a value is:"+a);
    out.println("instance variable (declaration) a value is:"+this.a); %>
```

Ex(2): keeping user defined java method definitions

```
<%! public int sum(int x, int y)
    {
    return x+y;
    } %>
```

```
<%= out.println("Result is:" + sum(10,20)); %>
```

* while working with xml syntax of declaration tag take care of < symbol problem with the support of CDATA.

```
<jsp:declaration>
<![CDATA[
    public int findBig(int a, int b)
      {
        if (a<b)
            return b;
        else
            return a;
      } ]]>
</jsp:declaration>
<%= out.println("Big value is:" + findBig(10,20));%>
```

* we can also use declaration tag to place JSP init, JSPDestroy() life cycle method definitions in our JSP program. By default the JSP equalent servlet class does not provide these JSPInit(-), jspDestroy() life cycle methods.

Ex (3):

```
<%! public void jspInit()
    {
      s.o.p("JspInit(): can contain initialization");
    } %>
<%! public void jspDestroy
    {
      s.o.p("jsp Destroy(): can contain some uninitialization");
    } %>
```

* JSP Container calls jspInit(-), on _jspInit life cycle methods through init(-) life cycle method when container instantiate JSP equalent servlet class
* since JSP equalent servlet automatically supplies _jspInit(-) method the programmer in JSP equalent servlet the programmer should use only jspInit() method to place his initialization logic.
* Container calls jspDestroy(), _jspDestroy() methods through destroy() life cyde methods when Container is about to destroy the object of JSP equalent servlet class.
* since JSP equalent servlet automatically supply _jspDestroy() method the programmer should use jspDestroy() in his JSP program to place on initialization logic.

NOTE: only Tomcat 6.0 onwards _jspInit(), _jspDestroy() methods are introduced in earlier versions of tomcat and in other servers _jspInit(), _jspDestroy() methods are not there.

The overall conclusion is _jspInit(), _jspDestroy() methods are useless methods for programmer.

* we can not define _jspService(-,-) method in declaration tag of JSP program because jsp equalent servlet automatically gives that method so our method becomes duplicate method to it and java does not support to have duplicate methods in a java class.

In Jsp program

```
<%! public void _jspService(HttpServletRequest req, HttpServletResponse res)
    {
    } %>
```

In valid code

Expression tag :

Standard syntax

```
<%= java expression %>
```

xml syntax

```
<jsp:expression>
    java expression
</jsp:expression>
```

* This tag evaluates given expression and writes generated result to browser window.
* Code placed in expression tag goes to _jspService(-,-) of jsp equalent servlet so implicit objects are visible in expression tag.
* Expressions are nothing but arithmetic operations, logical operations and method calls.

```
<%  int a=10;
    int b=20;
    out.println(a+b); %>
```

(equals)

```
<% int a=10;
   int b=20; %>
<%= a+b %>
```

Evaluates a+b expression and writes the result to browser window.

* we can also use the expression tag the values of the variables.

```
<% int a = 10;
   int b = 20; %>
A value is: <%= a %>
B value is: <%= b %> Result is:
<%= a+b %>
```

* we can use expression tag to call java methods (both pre defined and user defined).

```
<%! public int sum (int x, int y)
    {
      return x+y;
    } %>
The result is: <%= sum (10,20) %> <br>
<% string s = "hello how r u"; %>
The substring of <%= s %> is : <%= s.substring (0,5) %>
```

* If expression tag is used to instantiate a java class then the default data of the object will be written to browser window as response.

Ex: Date and Time is : <%= new java.util.Date() %>

Example on xml based expression tag

```
<jsp:scriptlet>
    int a=10;
    int b=20;
</jsp:scriptlet>
Result is: <jsp:expression> a+b </jsp:expression>
```

* By using xml syntax of expression tag we can not pass expressions that are using "<" symbol, Even though CDATA option is used

```
<jsp:scriptlet>
    int a=10;
    int b=20;
</jsp:scriptlet>                          Invalid code
Result is: <jsp:expression>
             <![CDATA[a<b]]>
           </jsp:expression>
```

* The above problem can be solved with standard syntax of expression tag.

Ex: <jsp:scriptlet>
int a=10;
int b=20;
</jsp:scriptlet>
Result is: <%=a<b%>

* It is always recommanded to use scripting elements of JSP by following standard syntax.

* The JSP expression tag can evaluate only one expression at a time. It can not be used to evaluate multiple expressions as shown below.

<% int a=10;
int b=20; %>
Results are: <%=a+b, b+a, a+a%> — wrong statement

* we can not write one scripting tag of JSP inside another scripting tag that means we must use these tags as independent tags.

<% int a=10;
int b=20;
<%=a+b%> ——— Invalid statement
%>

Procedure to develop JSP based web application by using netbeans IDE

Step(1): create web Project having name JspApp2

File menu → new project → Java web → web application → next →
Project name: JspApp2 → next → server: GlassFish 2.1 → next → Finish

Step(2): Add JSP program to the web pages folder of JspApp2 web application

Right click on webpages folder → new → JSP → Jsp file name: First →
Finish.

First.jsp

```
<%!
    public String getWishMsg(String uname)
    {
        //current date and time
        java.util.Calendar cl=java.util.Calendar.getInstance();
        //get current hour of the day
        int h=cl.get(java.util.Calendar.HOUR_OF_DAY);
        //generate wish msg
        if(h<=12)
```

```
            return "Good morning "+uname
        else if (h<=16)
            return "Good afternoon "+uname
        else
            return "Good night "+uname
        }%>
    Date and time is : <%= new java.util.Date() %> <br>

    <% String uname = "raja";

    %> <br> the wish msg is : <%=getwishmsg(uname)%>
```

Step(3): Run the project

Right click on project → Run

Re writing the above First.jsp with xml syntax

```
<jsp:declaration>
    <![CDATA[
    public String getwishmsg(String uname)
    {
        ==
    } ]]>
</jsp:declaration>

Date and time is : <jsp:expression> new java.util.Date() </jsp:expression>

<jsp:scriptlet> String uname = "raja"; </jsp:scriptlet> <br>

The wish msg is : <jsp:expression> getwishmsg(uname) </jsp:expression>
```

* we can install tomcat server in two ways

(1) through setup file (to installer) (exe)

(2) through zip file

* The Tomcat server that installed through zip file can only configured with netbeans IDE

Procedure to Configure external tomcat with netbeans IDE

Step (1): Install tomcat that is there in zip file mode (apache-tomcat-6.0.33-binary.zip file) (Extract this file to a folder)

Step (2): Configure the above tomcat server with netbeans IDE.

Tools → Servers → Add Server → Tomcat 6.0 → next →

Server location: <Tomcat_home>\apache-tomcat-6.0.33

The folder that comes after zip file extraction.

Base private configuration folder

catalina Base : <Tomcat_home>\apache-tomcat-6.0.33

username : admin

password : admin → Finish → close

Step(3): Link Tomcat server to webproject of netbeans IDE

Right click on web project → properties → run → server : Tomcat 6.0 → ok

Step(4): Run the web project

Procedure to configure Adv. JavaBatch Domain server of weblogic 10.3 with netbeans IDE

Tools → servers → Add server → oracle weblogic server → next

Server location: D:\oracle\middleware\wlserver_10.3 → next

Local instances: Admin server (localhost:7001)

username : javaboss

password : javaboss1 → Finish → close

* Link weblogic server with your web project

same as above

* Run the web project.

Implicit objects

request : javax.servlet.HttpServletRequest → request scope

response : javax.Servlet.HttpServletResponse → response scope

out : java.servlet.jsp.JspWriter (AC) → Page Scope

Session : javax.servlet.http.HttpSession (I) → session scope

page : this (Currently invoking jsp equalent servlet class obj ref) → page scope

PageContext : javax.servlet.jsp.PageContext (C) → page scope

application : javax.servlet.ServletContext (I) → web application scope

Config : javax.servlet.ServletConfig (I) → Page Scope

Exception : java.lang.Throwable (C) → page scope

* All the above given 9 implicit objects of Jsp are the objects of Servlet Container and Jsp Container supplied java classes implementing the above given interfaces or extending from the [illegible] interfaces of Servlet, Jsp apis given by Sun micro systems.

* Jsp programmers never worries and remembers the class names of these implicit objects because they will be changed based on the server we utilize.

Ex: The implicit object out is not the object of javax.servlet.jsp.JspWriter class. It is the object of underlaying Jsp Container supplied java class that extends from javax.servlet.jsp.JspWriter class.

In Tomcat server that class name is : org.apache.jasper.runtime.JspWriterImpl

In weblogic server that class name is : weblogic.servlet.jsp.JspWriterImpl.

* In jsp program we can use 3 types of Comments to comment the code

(1) Scripting Comments (Java Comments) (// and /* .... */)

given to Comment the java code of scripting elements (<% ... %>, <%! ... %>, <%= ... %>)

(2) Html Comments / Template Text Comments (<!-- ..... -->)

given to Comment the html code and template text code of jsp program.

(3) Hidden Comments / Jsp Comments (<%-- ..... --%>)

given to Comment the JSP tags of JSP programs

* Every Compiler or interpreter generates white spaces for Commented code so we get the feeling the Commented code is not participating in execution.

* JSP page compiler recognizes JSP Comments and generates white spaces in the source code of JSP equalent servlet program.

* Java Comments will be recognized by javac Compiler and keeps white spaces in .class file.

+ HTML Comments will be recognized by HTML interpreter.

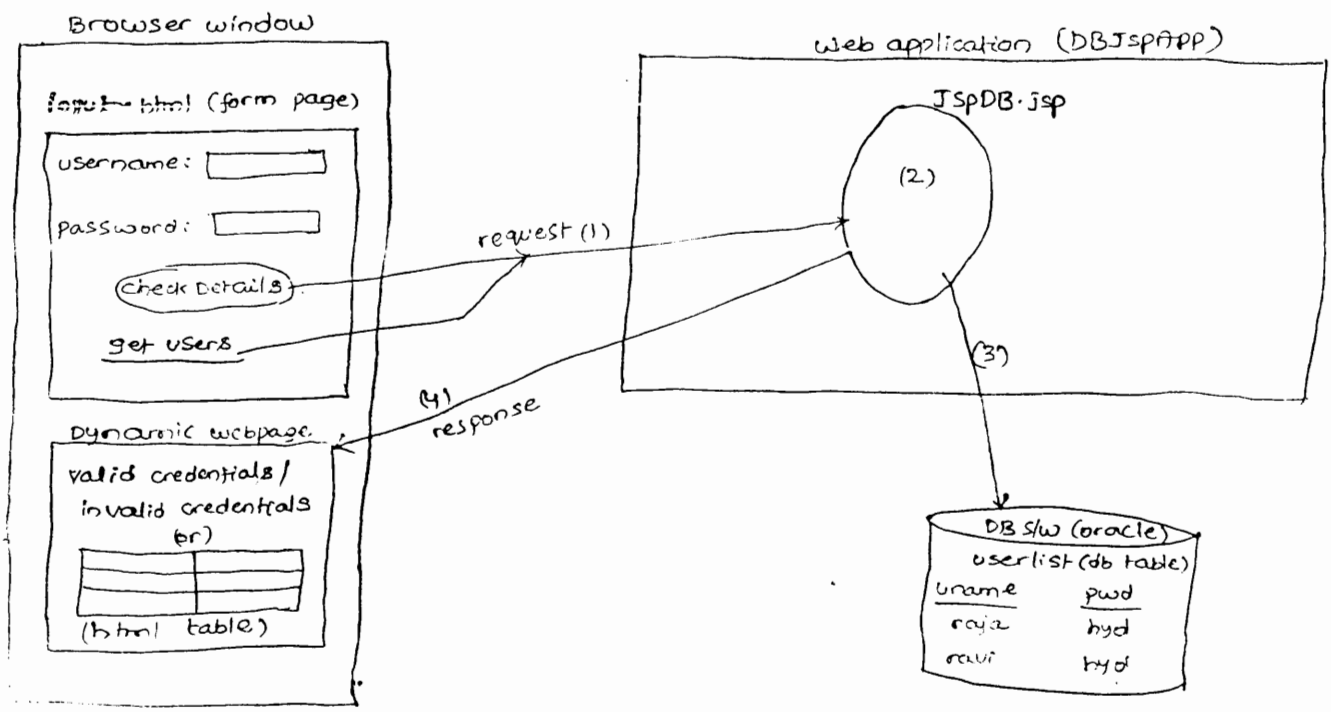
* The JSP program related JSP Comments are not visible in any phase of JSP programs execution so they are technically called as hidden Comments.

* we can comment html code of JSP program with JSP comments but not recommended. we can comment jsp tags with html comments but it is also not recommanded.

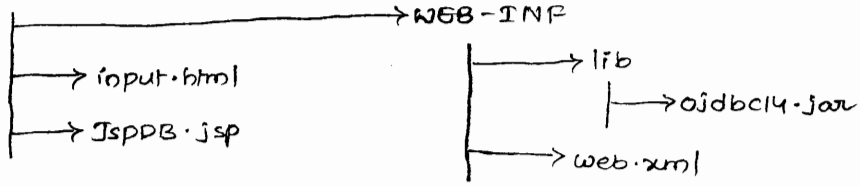
comment type	visibility			
	In src code of jsp equalent servlet	In Compiled code of jsp equalent servlet	In the html code that goes to browser window	in the output/ response
jsp Comment <%-- ..... --%>	no	no	no	no
html Comment <!-- ... -->	yes	yes	yes	no
b/s scripting Comment (// or /* ... */)	yes	no	no	no

* The HTML program related web page can have form or hyperlink to generate request to JSP or servlet program. This provides GUI ness to end user to Communicate with servlet/ JSP program from browser window.
* For HTML to JSP Communication you can use the JSP file name or its URL pattern as action url of form page (as action attribute value of form tag) or as href of anchor tag.
* For JSP to DB Communication place JdbC code in JSP program.

Example application on html to JSP and JSP to DB s/w Communication



DBJspAPP



* The code that comes to _JspService(-,-) method of a JSP equalent servlet will be automatically handling the exceptions because that _jspService(-,-) method code will be there surrounded with try and catch blocks.
* The code placed in declaration tag of JSP program must handle the exception explicitly where as the code placed in scriptlet, expression tag automatically handles the exception.

source code of the above diagram based application

input.html

```
<form action="JspDB.jsp" method="get">
    Login username: <input type="text" name="tname" /> <br>
    Login password: <input type="password" name="tpwd" /> <br>
    <input type="submit" value="check Details" name="s1" />
</form> <br><br><br>
<a href="JspDB.jsp?s1=link"> Get All User Details </a>
```

JspDB.jsp

```
<%@ page import="java.sql.*;" %>  //taken to import the java packages
<%!
    Connection con;
    PreparedStatement ps1, ps2;
    public void JspInit()
    {
        try
        {
            Class.forName("oracle.jdbc.driver.OracleDriver");
            Con = DriverManager.getConnection("jdbc:oracle:thin:@localhost:1521:
                                              sathya", "scott", "tiger");
            PS1 = Con.prepareStatement("select count(*) from userlist where
                                       uname=? and pwd=?");
```

```
        PS2 = Con.prepareStatement("select * from userlist");

      }//try
      catch(Exception e)
       {
         e.printStackTrace();
       }
    }//jspInit()

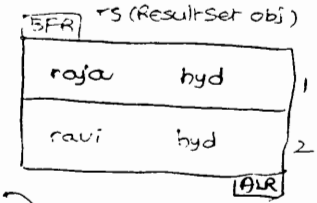
%>

<% //read form data from form page
     String user = request.getParameter("tname");
     String pwd = request.getParameter("tpwd"); //read additional req param value
     String param = request.getParameter("s1");

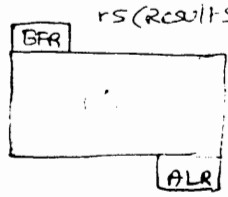
     if(param.equals("link")) //when hyperlink is clicked
     {
       //write jdbc code to get all users details

        ResultSet rs = PS2.executeQuery();
        out.println("<table>");
        While(rs.next())
        {
          out.println("<tr>");
          out.println("<td>"+rs.getString(1)+"</td>");
          out.println("<td>"+rs.getString(2)+"</td>");
          out.println("</tr>");
        }//while
        out.println("</table>");
        rs.close();
     }//if
     else //when submit button is clicked
     {
       //write jdbc code for Authentication

        //read form data
         String user = request.getParameter("tname");
         String pass = request.getParameter("tpwd");
        //set form data as param values of query
         PS1.setString(1,user);
         PS1.setString(2,pass);
        //execute the query
        ResultSet rs = PS1.executeQuery();
```



logic to display ResultSet obj data as html content



```
                int cnt=0;
                if (rs.next())
                    cnt = rs.getInt(1);
                if (cnt==0)
                   out.println ("<b><i><font color=red>Invalid credentials </font>
                                                                    </i></b>");
                else
                   out.println ("<b><i><font color=red>valid credentials </font></i></b>
              }//else
          %>
          <%! public void jspDestroy()
                {
                  try
                    {
                      ps1.close();
                      ps2.close();
                      con.close();
                    }
                  catch(Exception e)
                    {
                      e.printStackTrace();
                    }
                }//jspDestroy()
```

web.xml

```
<web-app/>
```

* keep ojdbc14.jar file in WEB-INF\lib folder.
* Deploy the DBJspApp in Tomcat server.
* use the following request URL to test the application.

http://localhost:2020/DBJspApp/input.html

userlist (DB table)

uname	pwd
raja	hyd
ravi	hyd

Select * from userlist where uname='raja' and pwd='hyd'

count(*)
1

——> valid credentials

Select * from userlist where uname='raja' and pwd='hyd'

count(*)
0

——> Invalid credentials.

Page Directive Tag

* This tag is given to provide global information to JSP program like buffer size, package import statements, content type and etc...

standard syntax

```
<%@ page attributes %>
```

xml syntax

```
<jsp:directive.page attributes />
```

attributes

language = "java" default → java // java is the default and only one language that you can pass here as a value.

* The above attribute specifies the language that we need to use to write code in scripting tags. (only java is allow).

import = "java.util.*, java.net.*, ..." //default : java.lang.*

* useful to import the java packages that are required for java application jsp program.

extends = "class name"

* Not recommended to use, no default value.

* Generally the jsp equalent servlet class extends from one or other jsp container supplied predefined java class. If you want to see that jsp equalent servlet class extending from your own or your choice java class then use this extends attribute. Since programmer can not develop that full fledged java class should act as super class of jsp equalent servlet class so this extends attribute is not recomended to use.

contentType = "text/html" //resp content type, // no default value

* this is useful to specify content type of the response that should be generated by JSP program.

Ex: <%@ page language="java" import="java.net.*, java.sql.*" contentType = "text/html" %>

buffer = "18kb" or more //default → 8kb

- specifies the size of the buffer at server side.

* Even though the code of JSP program is executing part by part, because of underlaying CPU algorithms (like round Robin algorithm) the server preserves output of the executed statements in buffer of the JSP program where until last statement of JSP program gets executed.

Ex: <%@ page buffer = "10kb"%>
hello
↳ The buffer size of JSP program at server side

Ex: <%@ page buffer = "none" %>
↳ makes the JSP program running without buffer.

NOTE: Every servlet program also contains one default server side buffer. But there is no default size for this buffer.

* The server side buffers servlet/JSP program is no way related with client side (browser window) buffer.

* Buffer is a temporary memory that can hold data for temporary period.

autoFlush = "true" (false) //default → true

* It cleans the buffer of JSP program and sends to browser window as web page

Ex: <%@ page buffer = "8kb" autoFlush= "true" %>

session = "true" (false) //default → true

* Enables JSP equalent servlet create or not to create implicit object session

Ex(1): <%@ page session= "false" %>
↳ implicit object session will not be created.

Ex(2): <%@ page session = "true" %>
↳ Implicit object session will be created.

* use when we don't enable session tracking we don't need session object in JSP program. To disable that session object creation use above statements.

errorPage = "url" //no default
isErrorPage = "false" (true) //default → false

} Required to configure error pages for JSP program to handle the exceptions raised in JSP program.

isThreadSafe = "true" (false) //default → true

* Useful to enable or disable thread safty on JSP equalent servlet

Ex: <%@ page isThreadSafe = "false" %>
make JSP equalent servlet as Thread Safe servlet by implementing javax.servlet.SingleThreadModel (i)

info = " String"

* The short desc about JSP page - no default value.

Ex: <%@ page info="basic program" %>

isELIgnored = "true" (false) //default → false

* To perform arithmetic and logical operations in JSP program without using java code, we can use EL (expression Language).
* The above attribute can make page compiler to ignore or recognize that EL.

Rules to remember while working with page directive tag (<%@ page. %>)

(1) Page directive tag name and attribute names are case sensitive.

(2) unknown attributes will not be recognized

(3) Except import attribute no other attribute can have multiple values seperated with ',' symbol

<%@ page import="java.net.*, java.awt.*" buffer="8kb, 10kb" %>

It is allowed

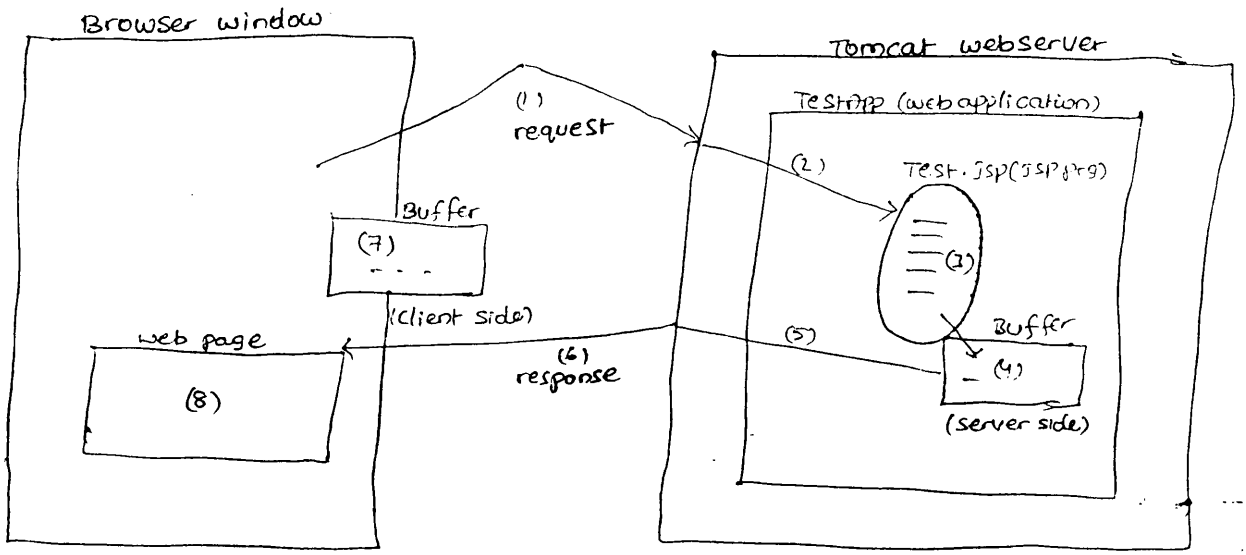
It is not allowed

(4) Except import attribute no other attribute can not occur for multiple times with different values either in the same page directive tag or in the multiple page directive tags of same JSP program.

<%@ page import="java.net.*" import="java.net.*" buffer="8kb" buffer="10kb" %>

allowed

In valid



* The JSP program buffer size is taken as 1kb but that JSP program is generating more than 1KB output then what happens?

(A) The JSP program dynamically expandes its size as needed.

NOTE: Even though buffer = none is taken the JSP program is capable of expanding Buffer size as needed at runtime so we can enable autoFlush = "true" eventhough buffer is none.

Ex: <%@ page buffer = "none" autoFlush = "true" %>

What is the difference between printWriter() and JSPWriter()?

* The process of story result/output in a buffer before devlivering to the orignal destination is technically called as buffering operation.
* PrintWriter can not perform buffering while sending its output to browser window where as JSPwriter can perform buffering.
* When JSP program is taken with out buffer then JSPwriter internally uses Printwriter to write the output to browser window.
* JSP Writer is the type of the implicit object called out in JSP Program.
* Printwriter is useful in servlet program.
* Printwriter belongs to java.io package and JSPwriter belongs to javax.servlet.jsp package

—•—

* Servlet api is given as

javax.servlet package, javax.servlet.http package

Latest version servlet api is : 3.0 (supports Annotations based programming)

running version of servlet api is : 2.4/2.5

* JSP api is given as

javax.servlet.jsp package

javax.servlet.jsp.el package

javax.servlet.jsp.tagext package

Latest version of JSP api is 2.1

running version of JSP api is 2.0

* JSP equalent servlet automatically handles the exceptions but displays the exceptions related error messages quite ugly on browser window when exception is raised. Since these are technical messages the non-technical end users can not understand them. To overcome this problem Configure error pages for JSP program.

* Error page is a JSP or HTML program that are capable of executing only when exception is raised in other JSP programs. These are also useful to display the exception related messages non technical messages guiding the end users.
* Always develop webapplications by keeping non-technical end users in mind. For that error pages configuration is important.

Error page Configuration in Jsp Programs

1. Local Configuration

→ This error page cfg is specific to one Jsp prg.

→ use errorPage, isErrorPage attributes of <%@page...%>

2. Global Configuration

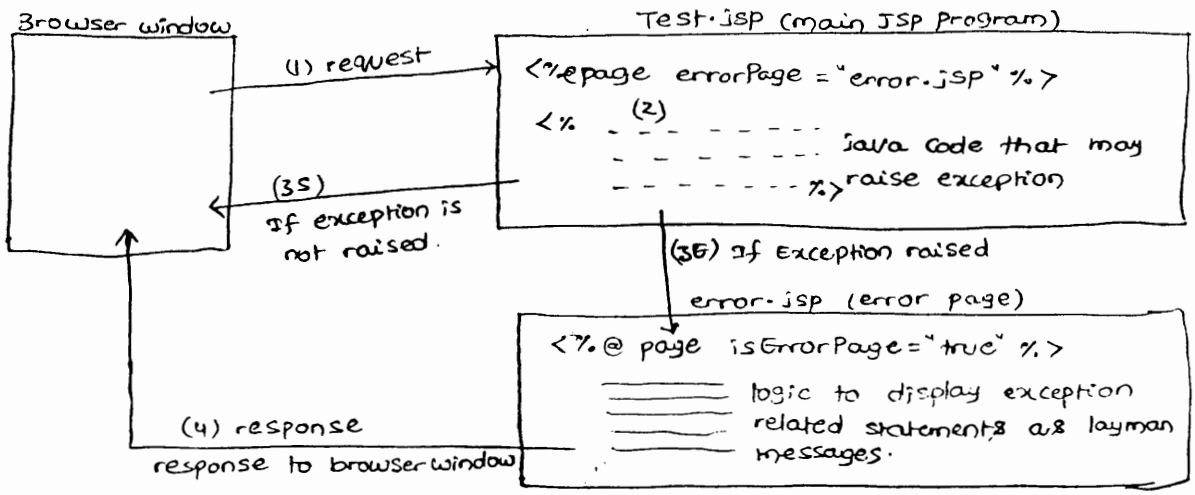
→ this error page cfg is common for all the jsp prgs of a web application

→ use <error-page> of web.xml file

In servlet program we take support of rd.forward(-,-) for Error servlet/ Error page configuration.

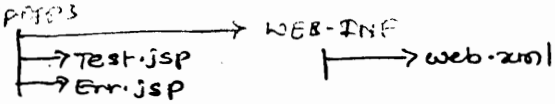
* The ErrorPage Configuration done in web.xml file for jsp programs (global Configuration) will not work for servlet programs of web application.

Local Error page Configuration



- isErrorPage = "true" attribute of page directive tag can make a jsp program of web application as error page.
- The Implicit object Exception is visible only in that JSP program that acts as error page and we can use this object to know the details of exceptions that are raised in main JSP programs.

Example application



Test.jsp

```
<%@ page errorPage="Err.jsp" %>
<b> from Test.jsp </b> <br>
<% int x=Integer.ParseInt("a123");
    out.println("x value is:"+x); %>
```

Err.jsp

```
<%@ page isErrorPage="true" %>
<b> from err.jsp </b> <br>
<font color=red> Internel problem </font> <br>
The exception that is raised is: <%= exception.toString() %>
```

Implicit object

web.xml

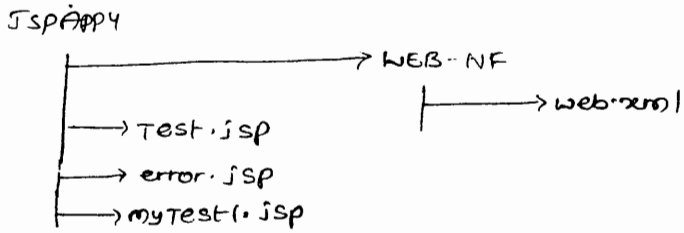
```
<web-app/>
```

Request URL to test the application

http://localhost:2020/Struts App3/Test.jsp

* The above exception handling configuration also allows to take .html file as error page but we can not choose to work with implicit object exception in that error page.

Example application on global error page Configuration



Test.jsp

```
<b> from Test.jsp </b><br>
<% int x = Integer.paseInt("a123");
    out.println("x value is:"+x); %>
```

My Test1.jsp

```
<b> from myTest.jsp </b><br>
<% Class.forName("java.lang.System1");
%>
```

Error.jsp

* Same as the above Err.jsp

web.xml :

```
<web-app>
   <error-page>
      <exception-type> java.lang.Exception </exception-type>
      <location> /err.jsp (or .html) </location>
   </error-page>
</web-app>
```

NOTE: Because of the above configuration done in web.xml file for any exception raised in any jsp program of web application the control goes to error.jsp page.

* Global error page configuration also allows to take .html program as error page.
* If both local error page, global error page configurations are done in same page to handle same exception pointing to two different error pages can you tell me which settings will be effected ?

(A) The configurations done in local error page configuration will be effected.

Directive include tag

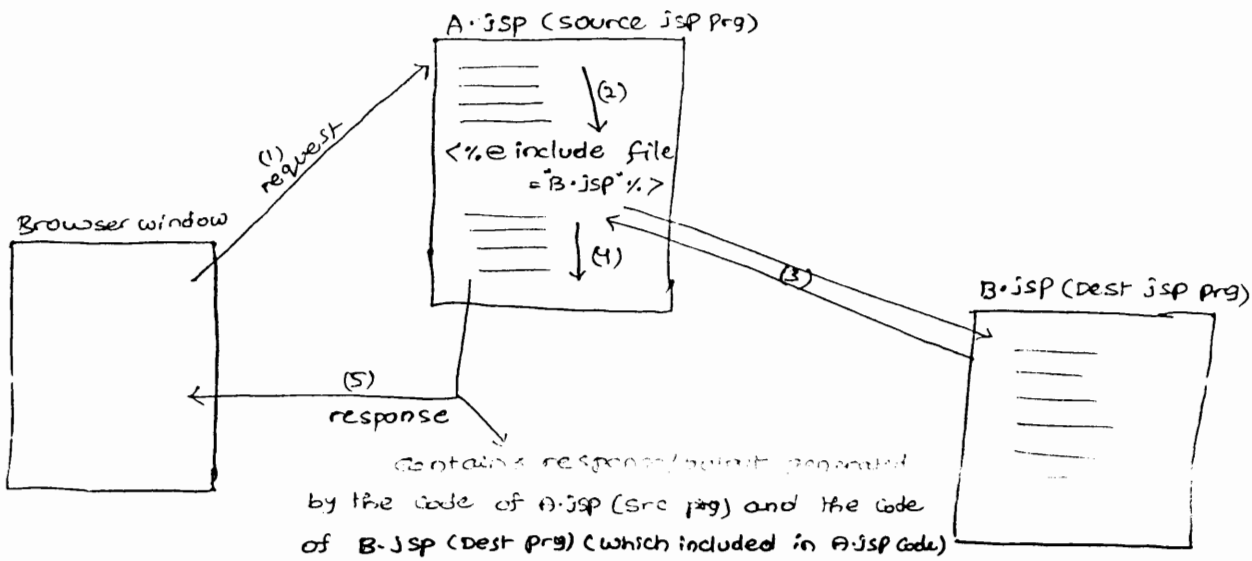
standard syntax :

```
<%@ include file="dest url" %>
```

xml syntax

```
<jsp:directive.include file="dest url" />
```

* This tag is given to include the code or content of destination web resource program to the source code of jsp equalent servlet belonging to the source jsp program.

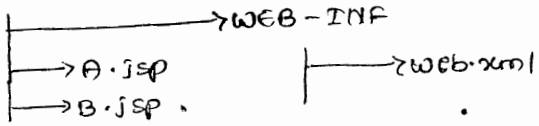


here B.jsp will not be executed seperately but its code will be included to the code of A.jsp and executes along with A.jsp. (Refer step (3) of diagram).

* Directive Include (<%@ include ... %>) does not perform output indusion but it performs code/content indusion.

Example:

JspApp5



A.jsp

```
<b> from A.jsp </b> <br>
<%@ include file="B.jsp" %>
<b> from end of A.jsp </b>
```

B.jsp

```
<b> from B.jsp </b><br>
<%= new java.util.Date() %>
```

web.xml

```
<web-app/>
```

Request URL to test the application

http://localhost:2020/JspApp5/A.jsp

* when the above A.jsp is requested

(a) JSP page compiler generates JSP equalent servlet for A.jsp

(b) Because of <%@ include file="B.jsp" %> the code of B.jsp will be included to A.jsp related jsp equalent servlet program.

(c) The java compiler compiles JSP equalent servlet source code of A.jsp and executes that code then sends response to browser window.

* In the above execution B.jsp program will not be executed seperately so its JSP equalent servlet will not be generated seperately.

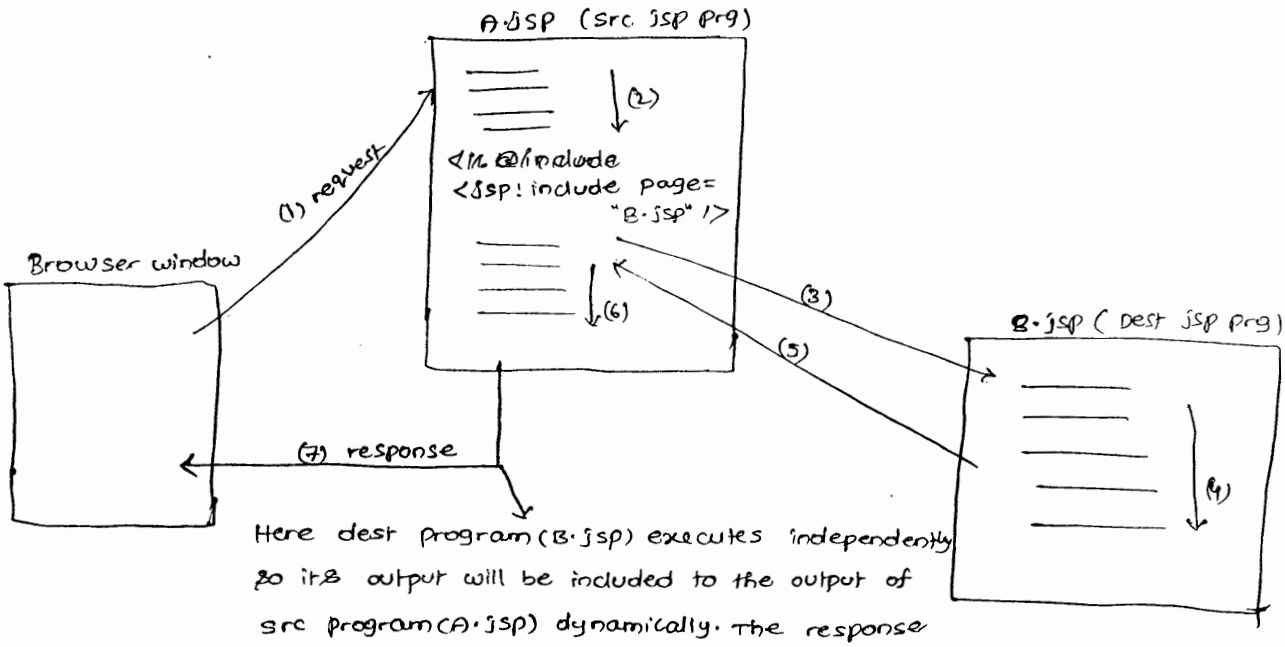
Action Include (<jsp:include>)

syntax

```
<jsp:include page="dest url"/>
```

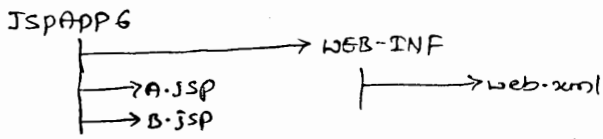
NOTE: All standard Action tags of JSP like <jsp:include> are having only xml syntax and there is no standard syntax for them.

* <jsp:include> is given to include the output of destination web resource program to the output of to the source JSP program.



Here dest program (B.jsp) executes independently so its output will be included to the output of src program (A.jsp) dynamically. The response that comes to browser window contains the output of A.jsp (src prg) and B.jsp (dest prg) programs.

Example application



A.jsp

```
<b> from A.jsp </b><br>
<jsp:include page="B.jsp"/>
<b> from end of A.jsp </b>
```

B.jsp

same as B.jsp in JspApp5

web.xml

```
<web-app/>
```

URL to test

http://localhost:2020/JspApp6/A.jsp

* <jsp:include> performs output inclusion but not the code inclusion.
* <jsp:include> is tag based alternate for rd.include(--) method of servlet programming.
* When the above A.jsp is requested two seperate jsp equalent servlets will be generated for both A.jsp, B.jsp and the jsp equalent servlet of A.jsp internally uses some rd.include(--) method to include the output of B.jsp.

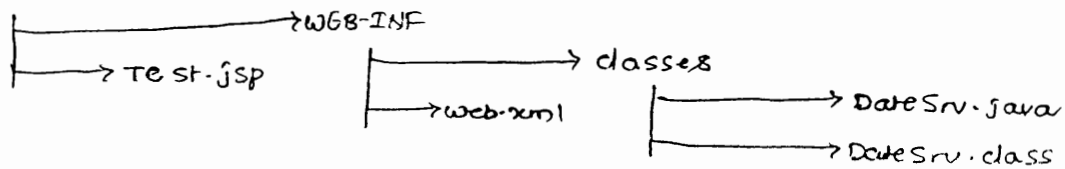
* The action tags of JSP can be used only with xml syntax. Compare to other tags of JSP (other tags can be having both xml and standard syntax).
* Standard Action tag of JSP are given to perform more dynamic operations as runtime/request processing phase operation.

What is the difference between "include Directive" and "action include tags?

Include Directive <%@ include ....%>	Action Include <jsp:include>
* given to perform code inclusion.	* given to perform output inclusion
* performs code inclusion at translation phase so this work is called as compile time binding or static binding.	* performs output inclusion at run time/request processing phase so this work is called as dynamic/runtime binding.
* If destination program is JSP the seperate JSP equalent servlet will not be generated for that program.	* will be generated.
* can not include code and output of servlet program.	* can include (includes the output)
* Suitable to take static web resource programs as destination programs. (like html programs)	* Suitable to take dynamic web resource programs as destination program (like servlet program, JSP program)
* Does not use rd.include(-,-) internally.	* uses rd.include(-,-) internally.
<%@ include file="B.jsp" %>	<jsp:include page="B.jsp" />

Example application on JSP program including the output of servlet program

JspApp7



Request URL

http://localhost:2020/JspApp7/Test.jsp.

DateSrv.java

```
import javax.servlet.*;
import javax.servlet.http.*;
import java.io.*;
public class DateSrv extends HttpServlet
{
  public void service(HttpServletRequest req, HttpServletResponse res) throws
                                                ServletException, IOException
  {
    //general settings
     PrintWriter pw=res.getWriter();
     res.setContentType("text/html");
     //write b.logic
     java.util.Date d=new java.util.Date();
     pw.println(d.toString());
  }//service(-,-)
}//class
```

web.xml

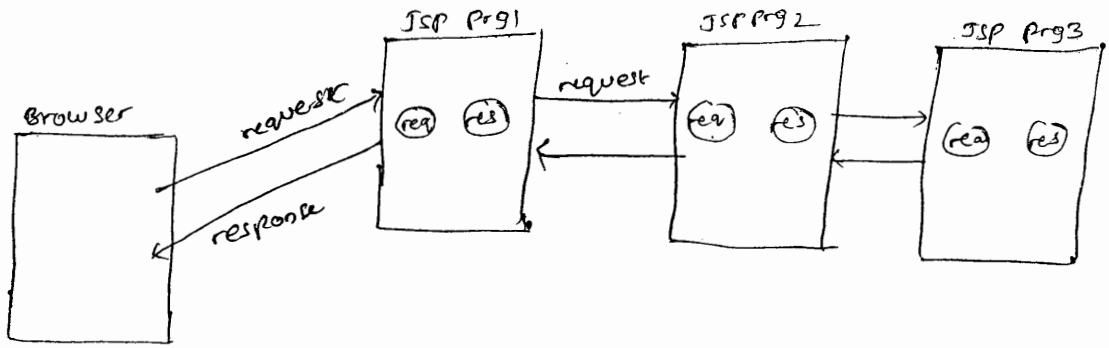
Configure DateSrv Servlet program with test1 url pattern.

Test.jsp

```
<b> from begining of Test.jsp </b> <br>
Date and Time is <jsp:include page="test1" /> <br>
<br>
<i> end of Test.jsp </i>
```

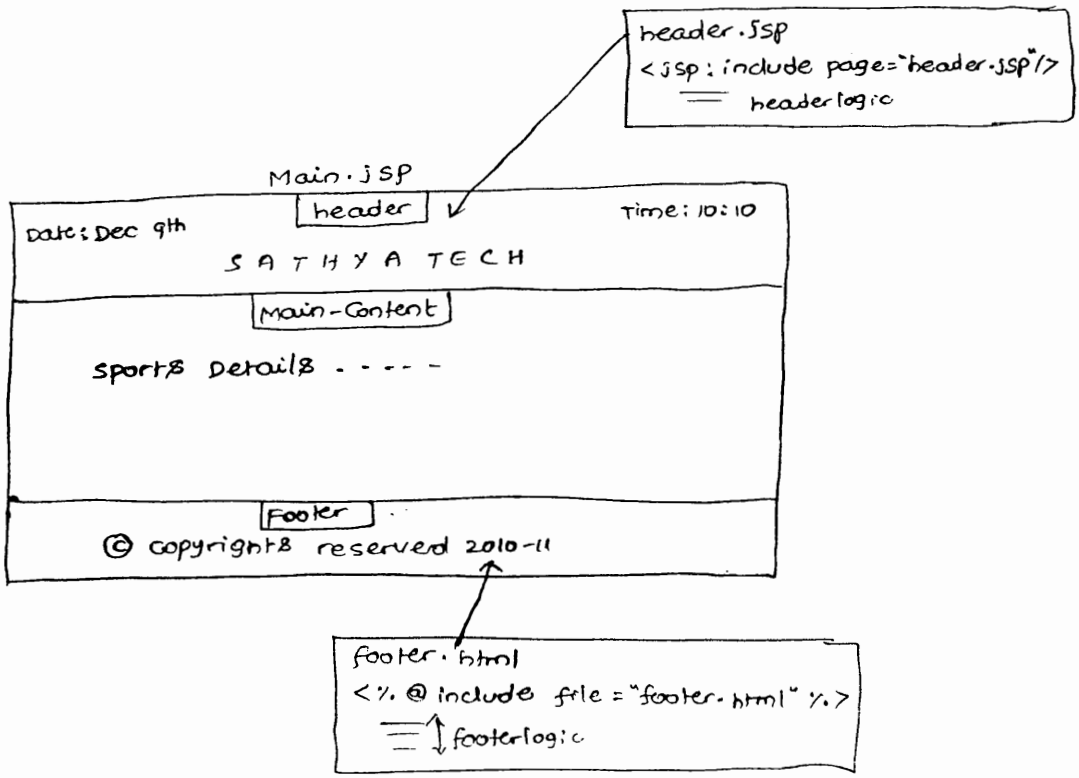
We can not replace this tag with include directive tag

* while working with directive include and action include don't commit the response in destination web resource program by using out.close() (or pw.close() methods because it does not allow to add further output of source jsp program to the response.
* Taking a request from a browser window and process the request by using multiple jsp programs as chaining. For this we can use <jsp:include> tag or <jsp:forward/> tag.
* All the JSP programs which participate in a JSP chaining will use same request and response objects.



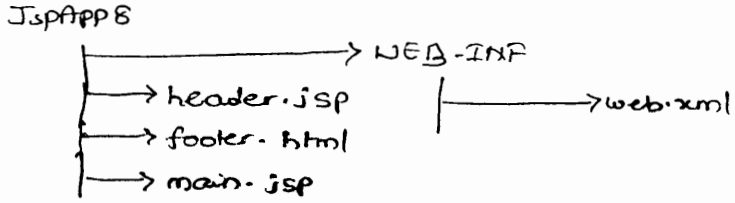
Java program participating in JSP chaining, all the steps JSP programs of jsp chaining will use same request and response objects.

* If the content to include is static content use directive include tag. If the content to include is dynamic content use action include tag.



* In one JSP program we can see the utilization of both directive include, action include tags for multiple times.

Example application based on the above diagram



header.jsp

```
<%@ page import="java.util.*" %>
<% Calendar cl = Calendar.getInstance();
   int date = cl.get(Calendar.DAY_OF_MONTH);
   int month = cl.get(Calendar.MONTH); %>
<%=date%>/<%=month%>      <%=cl.getTime()%>
<br><br>
<marquee><font color=red size=5> S A T H Y A  T E C H </font></marquee>
<br>
<hr>
```

footer.html

```
<hr>
<br><br> <center>
&copy; copy rights reserved 2010-11 </center>
```

main.jsp

```
<jsp:include page="header.jsp"/>
<br><br>
Sports news of the day <%=new java.util.Date()%>;
   <font size=2> Sehwag has scored 219 runs in an innings of one day
      cricket match beating sachin's record of 200. </font> <br><br><br>
<%@ include file="footer.html" %>
```

web.xml

```
<web-app>
```

Request URL to test the application

http://localhost:2020/~~Struts~~Jsp App 8/main.jsp

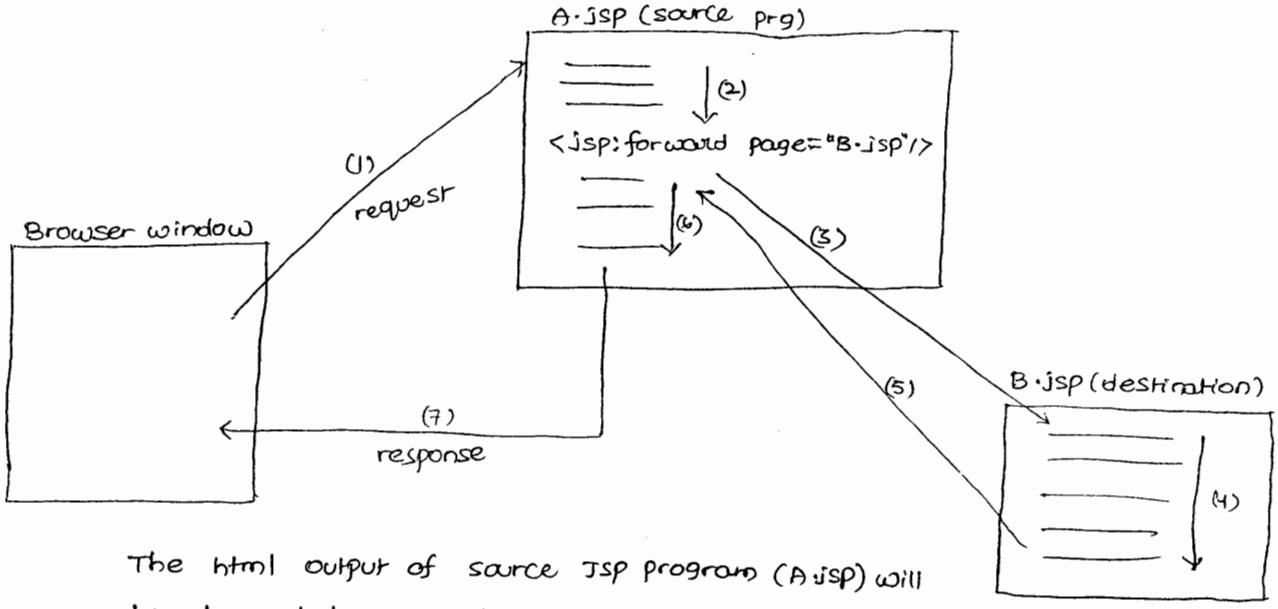
* for summary table that talks about all JSP tags refer page nos 112 & 113.

<jsp:forward>

Syntax

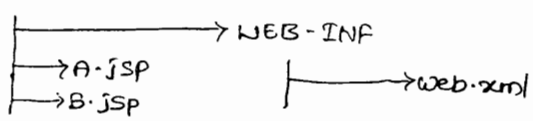
```
<jsp:forward page="dsturl"/>
```

* Given to perform forwarding request mode of JSP chaining. It internally uses rd.forward(-,-) method.



The html output of source JSP program (A.jsp) will be discarded and only the html output of (B.jsp) (destination prg) goes to browser window as response through A.jsp (src jsp prg). Here two seperate and Independent equalent servlet programs will be generated for A.jsp, B.jsp programs.

Example application



A.jsp

```
<b> start of A.jsp </b> <br>
<jsp:forward page="B.jsp"/>
<b> end of A.jsp </b>
```

B.jsp

```
<br><%= new java.util.Date() %> <br>
<i> from B.jsp </i>
```

* When source jsp program is interacting with destination jsp program either by using <jsp:forward> or <jsp:include> then source jsp program can send additional request parameter values to destination JSP program by using <jsp:param> tag.

↓

This tag is allowed to use only as sub tag of <jsp:include>, <jsp:forward> tags.

Example

In A.jsp of JspApp9

```
<b> start of A.jsp </b><br>
<jsp:forward page="B.jsp">
    <jsp:param name="bookname" value="CRJ" />
    <jsp:param name="price" value="300" />
</jsp:forward>
<b> end of jsp </b>
```

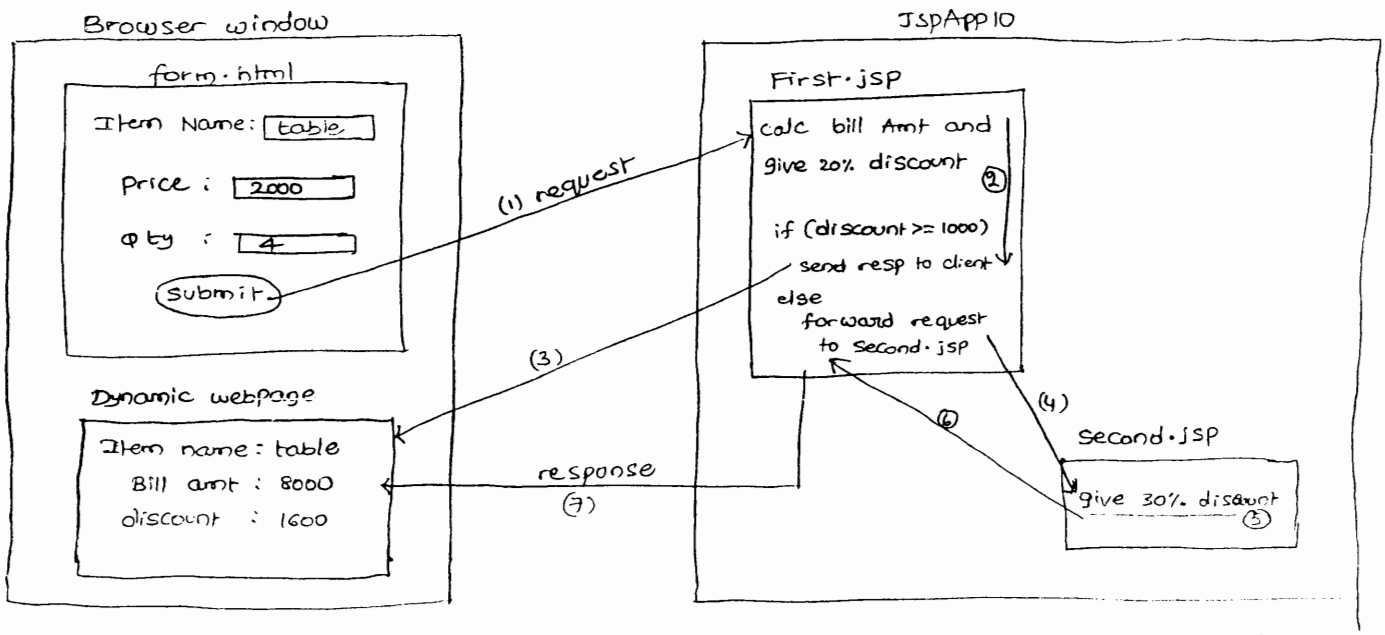
} Additional request parameters name, values

In B.jsp of JspApp9

```
<br><i> from B.jsp </i>
<br> Additional request parameters given by A.jsp are
<% = request.getParameter("bookname") %>
<% = request.getParameter("price") %>
```

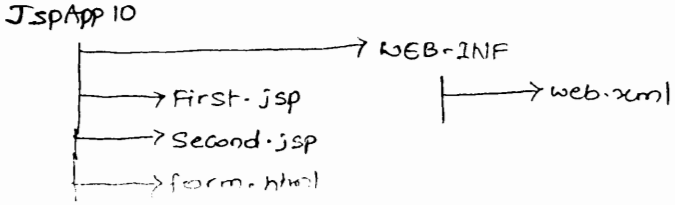
} logic to read additional request parameter values

Example application on <jsp:forward>, <jsp:param> tags



If step(3) executes then step (4), (5), (6), (7) will not execute

Deployment directory structure



Procedure to develop above application by using MyEclipse IDE

NOTE: It is always recommended to execute the <jsp:forward> tag in source jsp program by enabling the Condition...

Step(1): Launch MyEclipse IDE by choosing workspace folder.

Step(2): create web project in MyEclipse IDE.

File menu → new → web Project → JspApp10 → Finish

Step(3): Add form.html to webroot folder of the project.

Right click on webroot folder → new → JSP html → file name: form.html → finish

form.html

```
<form action="First.jsp" method="get">
  Item name: <input type="text" name="tname" /> <br>
  Item price: <input type="text" name="tprice" /> <br>
  Item qty: <input type="text" name="tqty" /> <br>
  <input type="submit" value="submit" />
</form>
```

Step(4): Add First.jsp to the webroot folder of the project.

Right click on webroot folder → new → jsp → File name: First.jsp → Finish

First.jsp

```
<% //read form data
   String iname = request.getParameter("tname");
   int price = Integer.parseInt("tprice");
   int qty = Integer.parseInt(request.getParameter("tqty"));
   // calculate bill amount
   int bamt = price * qty;
   // give 20% discount
   float discount = bamt * 0.2f;
   if (discount >= 1000)
   {
      out.println("Item name:" + iname);
      out.println("Bill Amt:" + bamt);
      out.println("Discount:" + discount);
   }
   else
   {
      // forward the request to second.jsp %>
```

```
<jsp:forward page="second.jsp">
    <jsp:param name="p1" value="<%= iname %>" />
    <jsp:param name="p2" value="<%= bamt %>" />
</jsp:forward>
<% } %>
```

Sending iname, bamt details to second.jsp from first.jsp as additional request parameter values.

Step(5): Add second.jsp to webroot folder of project.

second.jsp

```
<b> from second.JSP </b><br>
<% // read additional request parameters sent by First.jsp
    String name = request.getParameter("p1");
    int bamt = Integer.parseInt(request.getParameter("p2"));
    //calculate 30% discount amount
    float discount = bamt * 0.3f;
    //display item details
    out.println("Item name:<br>" +name);
    out.println("Bill amt:<br>" + bamt);
    out.println("Discount:<br>" +discount); %>
```

Step(6): Configure Tomcat Server with MyEclipse IDE.

window menu → preferences → MyEclipse → Servers → Tomcat → Tomcat 6.x →

Enable → <Tomcat_home> directory → apply → ok

Step(7): Start Tomcat server

Goto Servers icon in the tool bar → Tomcat 6.x → Start

Step(8): Deploy the project in Tomcat server.

Go to deploy icon in the tool bar → project → JspApp10 → add →

Server : Tomcat 6.x → Finish → ok

Step(9): Test the application

Open browserwindow & type this URL

http://localhost:2020/JspApp10/form.html

There are directive include, Action include tags. can you tell me why there is no directive forward tag? Only Action forward tag is available in JSP

(b) The basic work of forwarding request operation is discarding the output of source JSP program.

The Directive tags perform their activity at translation phase by including the code of destination program to the code of JSP equalent servlet generated for source JSP program.

In this process discarding the output of source JSP program is impossible. so there is no directive forward tag in JSP.

What is the difference between <jsp:forward> tag and response.sendRedirect(-) method?

(A)

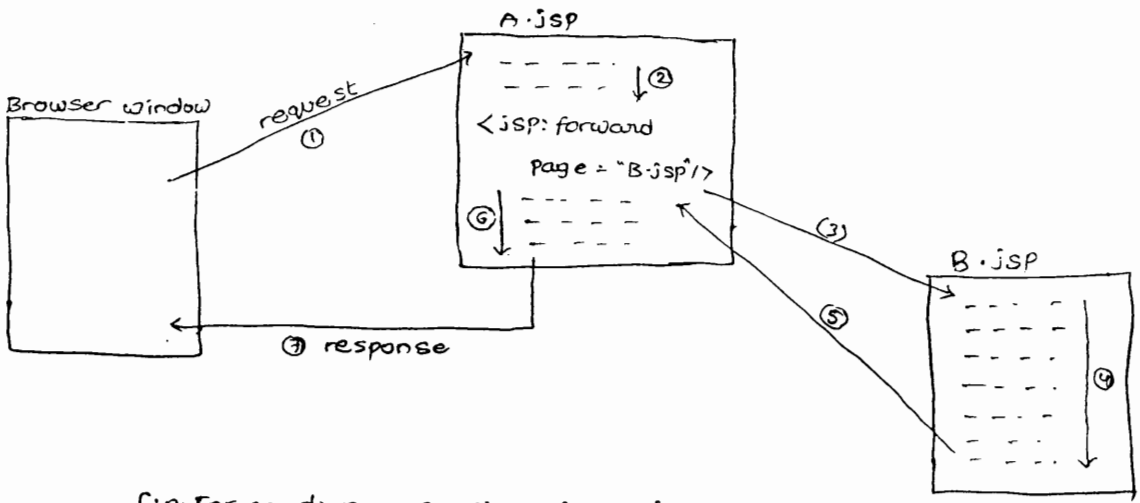


fig: Forwarding request using <jsp:forward> tag

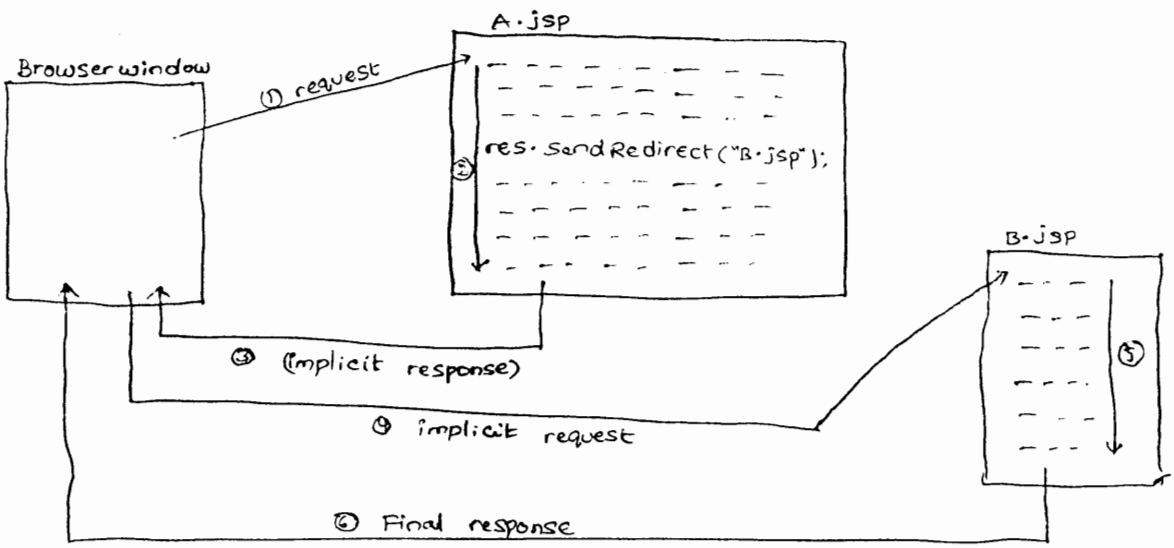


fig: sendRedirection using res.sendRedirect(-) method.

<jsp:forward>	response.sendRedirect(-)
* source JSP program communicates with destination program directly.	* Source Jsp program communicates with destination program by having one network round trip with browser window.
* Both source, destination programs use same request and response objects so request data coming to source JSP program can be used in destination program.	* source JSP program and destination program will not use same request and response objects so the request data coming to source destination program is not visible and accessable in destination program.
* Both source and destination programs must be there in same web application.	* Both source and destination programs can be there in same web application or can be there in two different web applications of same server or different servers.
* A destination program must be a html program or JSP program or Servlet program	* Destination program can be html program or java based web resource program or non-java based web resource program.
* while forwarding request url in the address bar will not be changed.	* while redirecting the request the URL in the address bar will be changed into destination program related URL.

---

* <jsp:forward>, <jsp:include> tags are the tag based alternates for rd.forward(-,-), rd.include(-,-) methods. and there is no JSP tag providing the functionality of response.sendRedirect(-) method. That means java code should be written directly in JSP program.

What is the difference between the implicit objects page & pageContext?

(A) The implicit object page holds "this" key word which is nothing but reference of currently invoking JSP equalent servlet class object.

→ pageContext object holds multiple details of JSP page like this, request, response objects, errorPage name, buffer size, auto flush mode, session objects availability status.

→ JSP equalent servlet creates / gets Access to application, Config, Session and out implicit objects through this PageContext object.

Code in JSP equalent servlet program

for implicit object page

```
Object page = this;
```

for implicit object pageContext

```
pageContext = _jspxFactory.getPageContext(this, request, response,
                "Abc.jsp", true, 10240, true);
```

"Abc.jsp" → error page; true → availability of status of session object; 10240 → Buffer size; true → auto flush mode

How to pass data between the JSP programs of web application?

(A) (1) If source and destination programs are using same req, res objects

(a) use request attributes

(b) use additional request parameters (<jsp:param>)

(2) If source and destination JSP programs are getting request from same browser window and not using same req and res objects

use session attributes

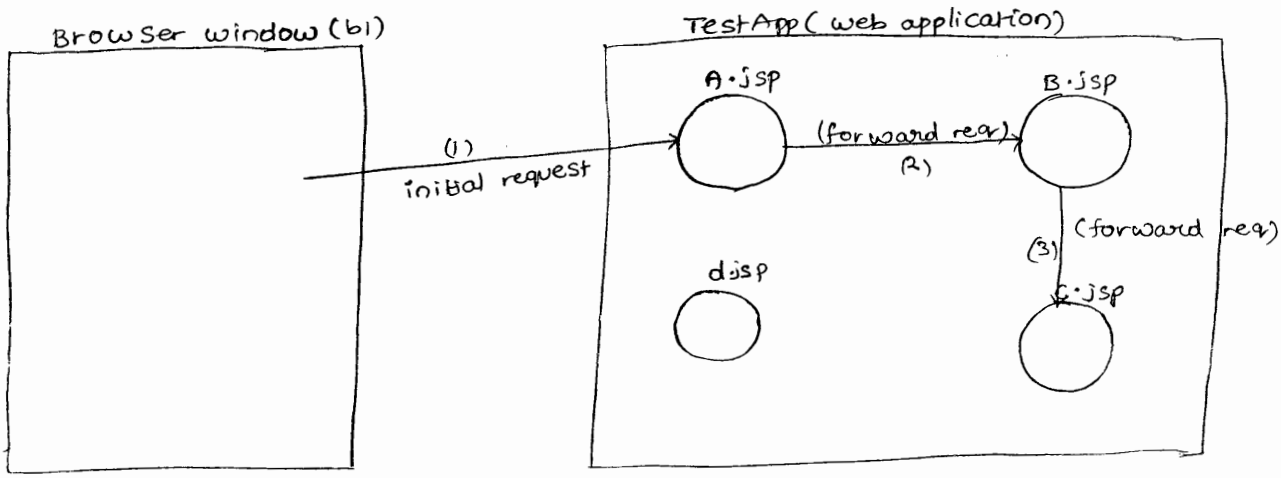
(3) If source and destination programs are not getting request from same browser window and not using same request and response objects

use application attributes.

(4) Use pageContext attributes

we can create pageContext attribute having pageScope (or) requestScope (or) sessionScope (or) applicationScope.

— o —

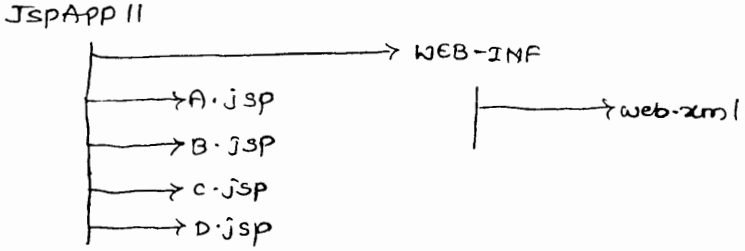


* In the above diagram the request attribute created in A.jsp is visible and accessable in B.jsp, C.jsp programs because A.jsp, B.jsp, and C.jsp programs use same request, response objects.
* Request attributes are visible through out request cycle.
* The session attribute created in A.jsp by getting request from browserwindow b1 is visible and accessable in all jsp programs of web application irrespective their request, response objects but all these web resource programs must get request from same browser window b1.

NOTE: session attributes are visible in all web resource programs of web application but they are specific to a browser window.

* Application attribute created in A.jsp is visible and accessable in all other web resource programs of web application irrespective of the browser window from which they are getting request and irrespective of request and response objects they are using.

Example application



Attributes are logical names holding values the servlet programming, JSP programming uses attributes to pass data from one web resource program to another web resource program

setAttribute(-,-) → To create new attribute or to modify existing attribute values.

getAttribute(-,-) → To read existing attribute

removeAttribute(-) → To remove existing attribute

A.jsp

```
<% //create attributes
    request.setAttribute("attr1", "val1");
    request.setAttribute("attr2", "val2");
    application.setAttribute("attr3", "val3");

   <!-- forward the request to B.jsp -->
   <jsp:forward page="B.jsp" />
```

B.jsp

```
<!-- read attribute values --> <b> from B.jsp </b> <br>
attr1 (req) attribute value is = <%=request.getAttribute("attr1") %> <br>
attr2 (session) attribute value is = <%=session.getAttribute("attr2") %>
attr3 (application) attribute value is = <%=application.getAttribute("attr3") %>
<!-- forward the request to C.jsp -->
<jsp:forward page="C.jsp" />
```

C.jsp

same as B.jsp but don't forward request any where

D.jsp

same as C.jsp

URL to test the application

give first request to A.jsp and give other request to other jsp programs either from same browser window or from different browser windows.

* Request object can create only request scope attributes.
* session object can create only session scope attributes.
* application object can create only application scope attributes. where as pageContext object can create page scope or request scope or session scope or application scope attributes.

TO create pageContext attribute

```
pageContext.setAttribute("attr1","val1"); → create pageContext attribute having page scope
pageContext.setAttribute("attr2","val2", pageContext.SESSION_SCOPE);
   → create pageContext attribute having page/scope: session scope.
```

other possible scopes

pageContext.PAGE_SCOPE

pageContext.SESSION_SCOPE

PageContext.APPLICATION_SCOPE

pageContext.REQUEST_SCOPE

All these scopes constants of javax.servlet.jsp.PageContext class.

To modmodify pageContext attribute values

PageContext.setAttribute("attr1", "val1");

→modifies attr1 value of page scope.

pageContext.setAttribute("attr2", "val122", PageContext.SESSION_SCOPE);

→modifies attr2 attribute value of session scope.

To read pageContext attribute values

String s1 = (String) pageContext.getAttribute("attr1");

→Reads attr1 attribute value from page scope.

String s2 = (String) pageContext.getAttribute("attr2", PageContext.SESSION_SCOPE);

→Reads attr2 attribute value from session scope.

To remove pageContext attributes

pageContext.removeAttribute("attr1");

→remove attr1 value attribute from page

pageContext.removeAttribute("attr2", pageContext.SESSION_SCOPE);

→removes attr2 attribute from session scope.

To find attribute

String s1 = (String) pageContext.findAttribute("attr2");

→searches and reads attr2 attribute value in multiple scopes in the following order...

a) page scope b) request scope c) application scope d) session scope.

What is the difference between getAttribute() and findAttribute() invoked on the pageContext object?

getAttribute() searches for the given attribute only in the specified scope (no scope is specified page scope) where as findAttribute() searches for the given attribute if multiple scopes in a particular order like page scope, request scope, session scope, application scope.

Revised JspApp11 application with the support of pageContext attribute

A.jsp

```
<% // create attributes
    pageContext.setAttribute("attr1", "val1", pageContext.REQUEST_SCOPE);
    pageContext.setAttribute("attr2", "val2", pageContext.SESSION_SCOPE);
    pageContext.setAttribute("attr3", "val3", pageContext.APPLICATION_SCOPE);
    <!-- forward the request to B.jsp -->
     <jsp:forward page="B.jsp" />
```

B.jsp

```
<!-- //read attribute values -->
  <b> from B.jsp </b>
  attr1 (req) attribute value is = <%=pageContext.getAttribute("attr1", pageContext.
                                                    REQUEST_SCOPE) %> <br>
  attr2 (session) attribute value is = <%=pageContext.findAttribute("attr2") %><br>
  attr3 (application) attribute value is = <%=pageContext.findAttribute("attr3") %><br>
  <!-- forward the request to C.jsp -->
   <jsp:forward page="C.jsp" />
```

C.jsp

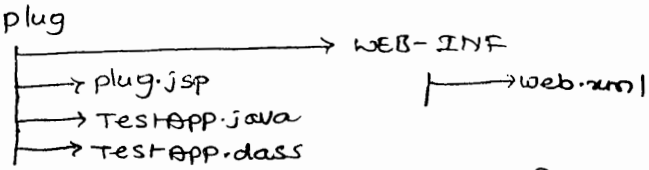
Same as B.jsp but don't forward request any where

D.jsp

Same as C.jsp

* <jsp:plugin> tag is given to display applets on the browser window. This tag internally uses <applet> tag or <object> tag of html to display applets.
* <jsp:fallback> tag should be used as sub tag of <jsp:plugin> tag. This tag executes only when underlaying browser window does not support applets.
* The Tomcat web server internally uses <embed>, <noembed> tags for <jsp:plugin>, <jsp:fallback> tags.

Example application



Request URL to test the application

http://localhost:2020/jsp-plug/plug.jsp

TestApp.java

```
import java.applet.Applet;
import java.awt.*;
public class TestApp extends Applet
 {
     public void paint (Graphics g)
      {
         setBackground(color.red);
         g.drawString ("this is from applet to test", 50, 50);
      }
 }
```

plug.jsp

```
<html>
<body bgcolor="pink">
   <jsp:plugin type="applet" code="TestApp.class" codebase="/jsp-plug" width="300"
                                                    (*1)
       height="300" >
      <jsp:fallback> browser does not support applets </jsp:fallback>
   </jsp:plugin>
</body>
</html>
```

web.xml

```
   <web-app/>
```

(*1) ⟶ specifies the directory where given applet class is available.
(TestApp.class)

Java Bean :

* A java bean is a java class that contains getter and setter methods. There is no special key word to develop the java class as java bean.
* The member variables of java bean class are technically called as bean properties and every bean property contains one getter and one setter method.
* Getter methods are useful to get or read the data from bean properties. where as setter methods are useful to write the data to bean properties.

Ex:

```
public class StudentBean
  {
```

```
//bean properties
  private int sno;
  private String sname;
  private float avg;
// write getXxx() and setXxx(-) methods
  public void setSno(int no)
     {
       this.no=no;
     }
  public int getNo()
     {
       return no;
     }
  public void setSname(String sname)
     {
       this.sname=sname;
     }
  public String getSname()
     {
       return sname;
     }
  public void setAvg(float avg)
     {
       this.avg=avg;
     }
  public float getAvg()
     {
       return avg;
     }
}//class
```

* If java programmers follow java bean standards while developing their java classes then it becomes easy for the programmers to exchange the java classes.
* Java bean is a standard to develope java classes having getXxx() and setXxx(-) methods.
* the best way to develop re-usuable simple java class is developing that class as java bean class.

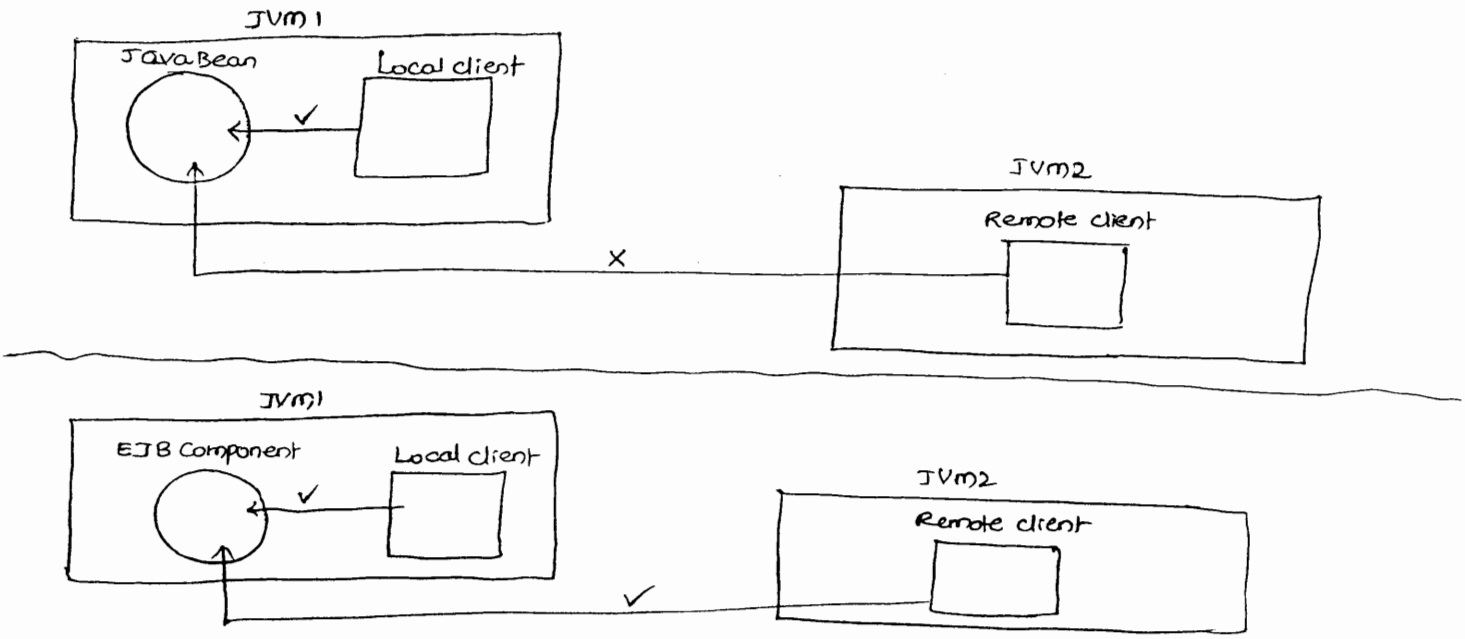
what is the difference between java bean and EJB (Enterprise Java Bean)?

Java bean	EJB
* An ordinary java class containing getter and setter methods.	* Distributed technology develop distributed applications.
* Just needs JVM for execution	* Needs JVM and EJB Container for execution.

* No life cycle methods.
* useful as a reusable helper class in projects.
* Allows only local clients.

* Life cycle methods are there.
* useful as technology to develop business components of the project.
* Allows both local and remote clients.

* In one computer multiple JVMs can be there to run multiple java applications simultaneously or parallelly. If application and its client resides in the same JVM then that client is called local client to application. If application and its client resides on two different JVMs of same computer or different computers then that client is called remote client to application.



* A Java Bean class can have 3 types of Bean properties.

(1) Simple Bean properties
(2) boolean Bean properties
(3) Indexed Bean properties

Simple Bean properties

* Allows to store only one value at a time.

Ex:
```
int sno;
public void setSno(int sno)
    { this.sno=sno;
    }
public get getSno()
    { return sno;
    }
```

Boolean Bean properties

* Allows to store only true or false value.

Ex: boolean married;

```
public void setMarried(boolean married)
        {
          this.married = married;
        }
public boolean getMarried()/isMarried()
        {
          return married;
        }
```

Indexed Bean properties

* Allows to store multiple values. This bean property type is an array.

Ex: String[] colors;

```
public void setColors(String[] colors)
    {
      this.colors = colors;
    }
public String[] getColors()
    {
      return colors;
    }
```

* For servlet program to javaBean communication the servlet program should be an object of javabean class and should call methods on that object.

* For jsp program to javabean communication we can use the following tags.

```
<jsp:useBean>
<jsp:setProperty>
<jsp:getProperty>
```

java based web application development models

1. Model 1 Architecture

→ use only servlets or only JSP programs as server side web resource programs of web application

2. Model 2 architecture (MVC Architecture)

1. MVC1 architecture
2. MVC2 architecture

→ In MVC1 and MVC2 architectures multiple technologies support will be taken to develop web applications.

Logics that can be there in our java web applications

(1) Request Data gathering logic

(2) Form validation logic

(3) Business logic

(4) Session management logic

(5) persistance logic

(6) middlewoore services logic

(7) Presentation logic and etc...

* Request Data gathering logic is responsible to gather various details given by client in its httprequest like form data, header values and etc. For this, use getXxx() methods on request object.
* In form validation logic checks the format and pattern of the form data. Ex: checking wheter email id is having @, . symbols or not.
* Business logic processes the request and generates the results.
* Business logic is main logic of web application.
* Session management logic remembers the client data across the multiple requests during a session through session tracking techniques like hidden form fields, cookies, httpsession with cookies and Httpsession with URL rewriting and etc...

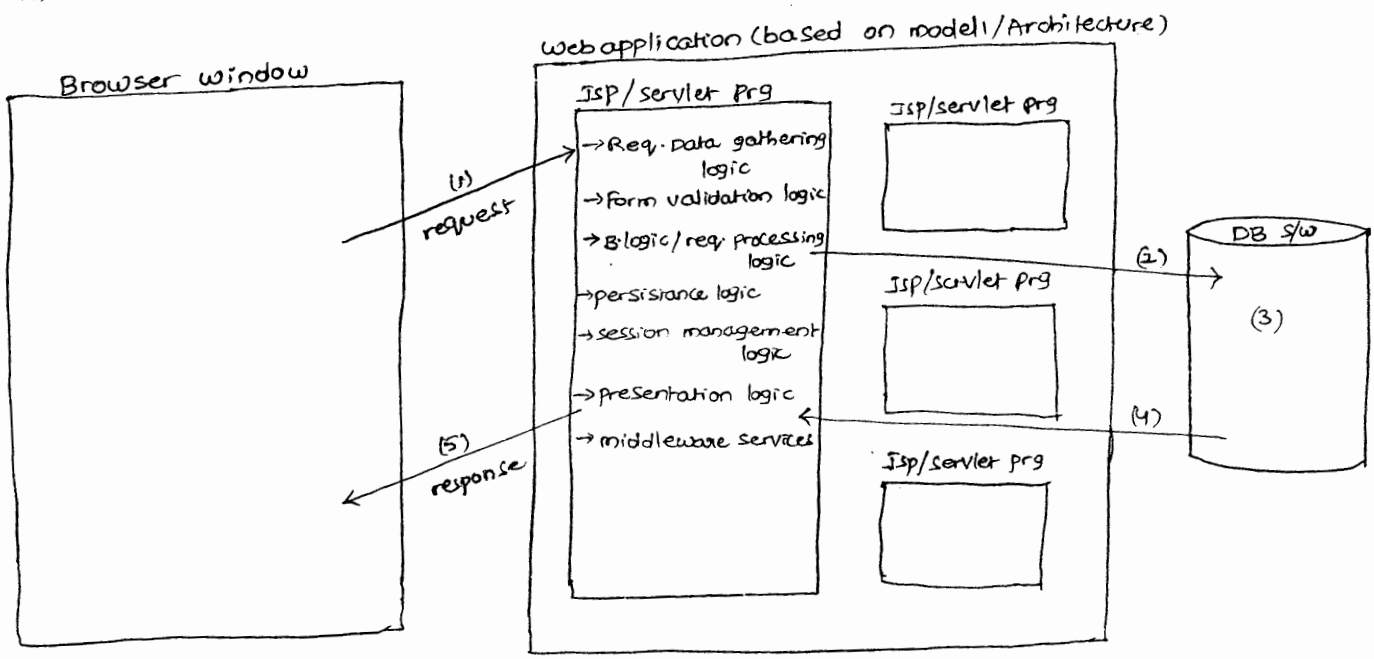
NOTE: There are no special tracking techniques in JSP. JSP should also use the above said four session tracking techniques.

* persistance logic is responsible to manipulate database data by performing "curd" operations. (Ex: JDBC code)
* middleware services are the additional services which can be applied on the applications like JDBC Connection pooling, security and etc...
* Presentation logic generates user interface required for end user. It is also responsible to format the results and to display the results. (html code).

Model - 1 architecture

* In this model either servlet programs or JSP programs will be used as server side web resource programs. more ever if servlet programs are used the JSP programs will not be used and viceversa.

* In every server side program multiple logics will be mixed up.



* In model1 architecture web application, the client side web resource programs can be there but either servlets or jsp must be used as server side programs.
* Most of the programmers prefer working with JSPs (compare to servlets) while developing model1 architecture base web application.
* Disadvantages of model1 architecture
  * multiple logics will be mixed up in every server side web resource program. This indicates there is no clean seperation of logics and this also indicates modification done in one logic may disturb other logic.
  * Parallel development is not possible so the productivity is very poor.

NOTE: Doing more work in less time with good accuracy gives good productivity.

  * middleware services (some) must be implemented by the programmer manually. This improves burden on the programmer.

Advantages
* knowledge on either servlet or JSP is sufficient to develop web resource programs
* Since parallel development is not possible multiple programmers are not required to develop the web applications.

---

* TO overcome the drawbacks of model-1 architecture use model-2 (or) MVC architectures to develop the web applications.

MVC / model-2 architecture

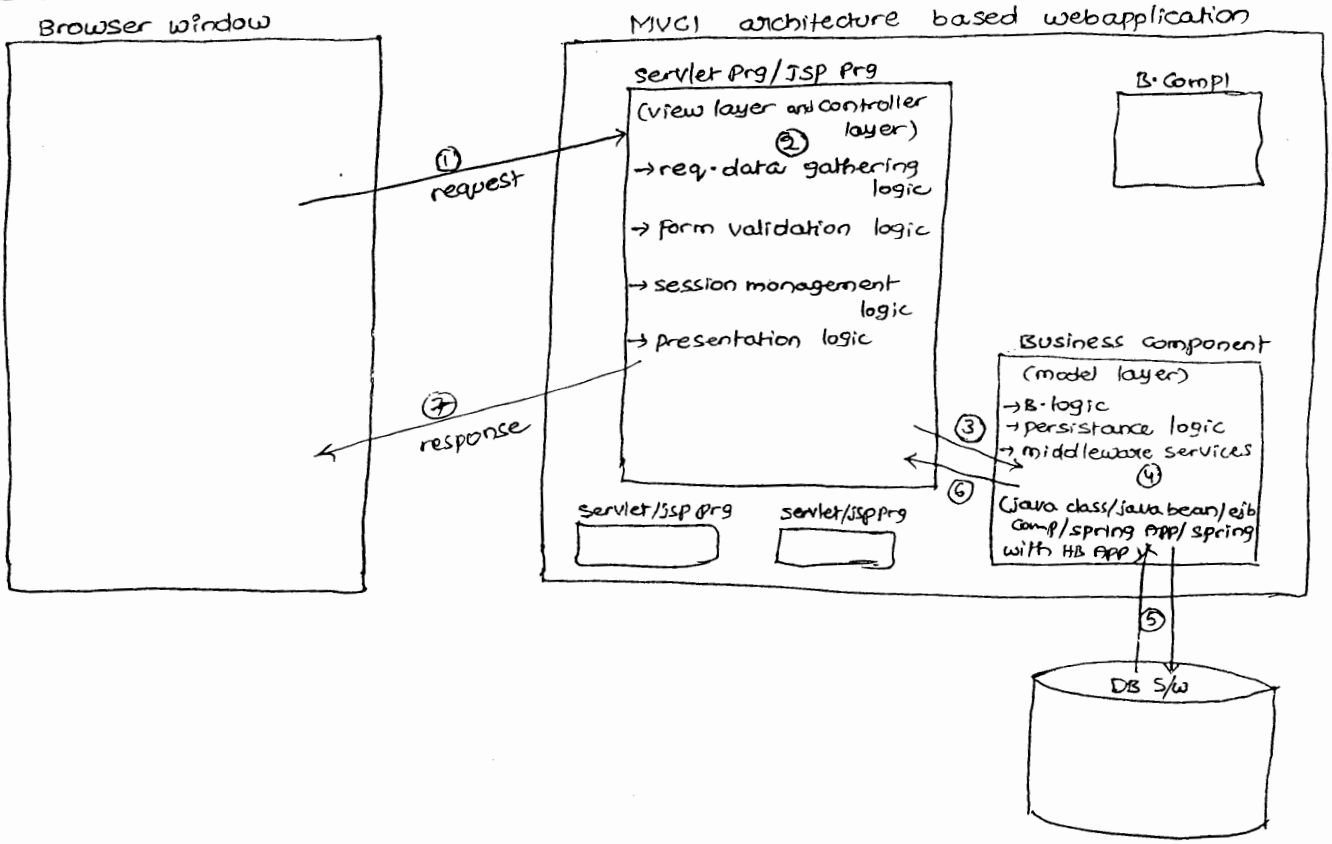
M ⟶ model Layer → Represents b·logic + persistance logic
→ use java class/ java bean / ejb components / spring App / spring with HB App in model layer to develop logics
→ It is like Accounts officer.

V ⟶ View Layer ⟶ Represents presentation logic
→ use html / JSP programs to develop this presentation logic.
→ It is like Beautician.

C ⟶ Controller Layer → Represents Integration logic / Connectivity logic
→ use servlet program or servlet filter program to develop these logics.
→ It is like Traffic police / supervisor

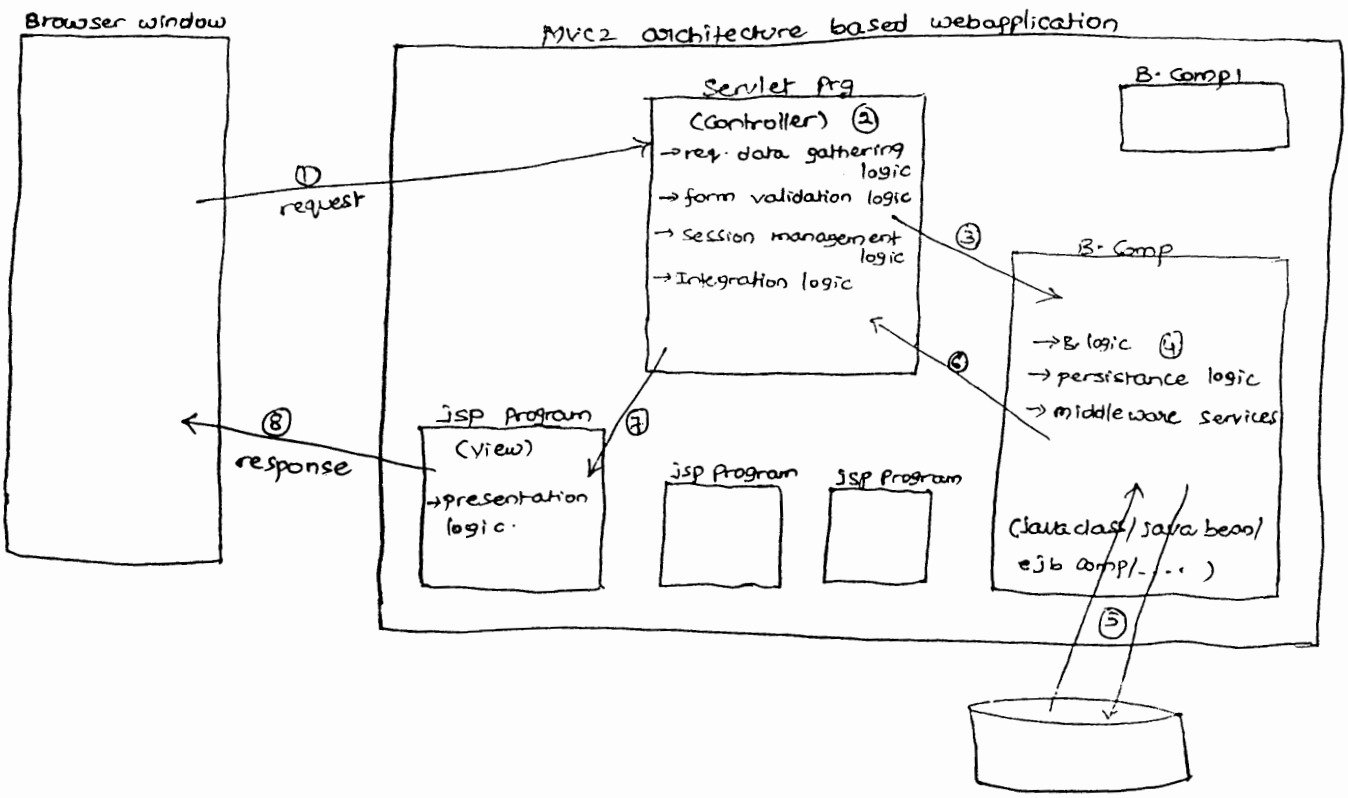
* Integration logic is responsible to take the request from browser window, to pass that request to model layer resource, to gather result from model layer resource and to pass the result to view layer resource.
* Integration logic is responsible to control and monitor every operation of the web applications execution.
* In model 2 / mvc architecture based web application development multiple technologies support will be taken by programmers.
* In model MVC1 architecture the same servlet programs / JSP programs will act as view layer, controller layer resources representing presentation, business logics and seperate resources will taken for model layer.
* In mvc2 architecture three different resources will be taken in 3 different layers.
  JSP programs ⟶ In view layer
  servlet program ⟶ In controller layer
  java class / javabean / ejb component / ... ⟶ In model layer
* In mvc1 architecture multiple resources can be there in each layer.
- In mvc2 architecture multiple resources can be there in view, model layers but only one servlet program will be in controller layer.

MVC-1 architecture



* MVC1 gives quite better clean seperation of logics compare to model-1 because it is seperating business logic and persistance logic from servlet program/ JSP program. But for more clean seperation of logics it is recommended to use MVC2 architecture.

MVC-2 architecture



with respect to the diagram

① Browser window gives request to web application.

② A/S a Controller servlet program traps and takes the request

③ Servlet program passes the request to model layer business component by using integration logic.

④,⑤ → The business logic of business component process the request and generates the results. Also ~~talk~~ talks with DB s/w if necessary.

⑥ → Business component generated results goes back to servlet program (Controller)

⑦ The Controller servlet program passes the results to the view layer Jsp program.

⑧ The Jsp program uses the presentation logic to format the results and sends formatted result to browser window as response.

* use model-1 architecture to develop small scale web application.

* use mvc-1 architecture to develop medium scale web application.

* use mvc-2 architecture to develop large scale web application.

Advantages of MVC-2 architecture

(1) There are multiple layers in web application development so we can acheive clean separation of logics.

(2) Modifications done in logics of one layer does not effect logics of another layer.

(3) Parallel development is possible so productivity will be good.

(4) Provides easyness in enhancement and maintanance of the web application

(5) MVC-2 is a & industry standard to develop the web applications.

(6) When spring, hibernate, EJB technologies are used in the model layer we can get the benifit from middleware services (built-in). This reduces burden on the programmer.

(7) MVC-2 is industry standard to develop the web applications.

disadvantages

* For parallel development multiple programmers are required.

* Knowledge in multiple technologies is required to develop the web applications.

While developing MVC2 architecture based web applications along with taking support from multiple technologies we should also implement set of rules which are called MVC-2 rules or principles.

(1) Every layer is given to have certain logics. Just place only those logics and don't place any excess logics.

(2) It is recommended to use JSPs use in view layer, servlets in Controller layer, use javaBean or java class in model layer when business logic is less.

(3) use EJB or spring with HB in model layer when business logic is huge.

(4) Every operation in the web application must take place under the Controller of Controller servlet.

(5) There can be multiple resources in the view layer and multiple resources in model layer but there must be only one controller servlet or controller servletFilter program in Controller layer.

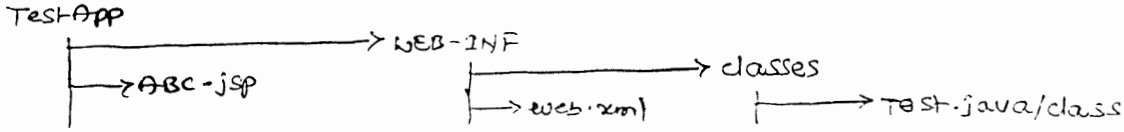
(6) The view layer resources must not talk with model layer resources directly and vice versa. That means they must Communicate with each other through Controller servlet.

(7) Two resaures of view layer (Jsp programs) must not talk with each other directly. That mean they must Communicate with each other through Controller layer.

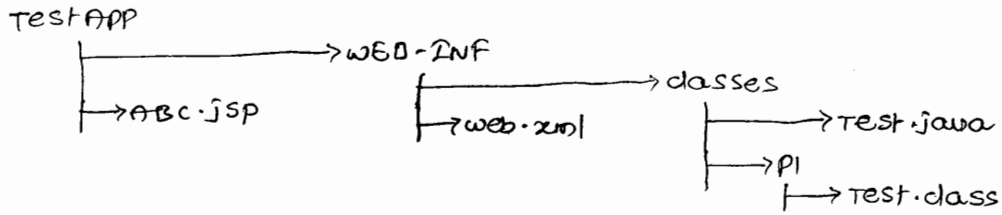
* In a web application where jsp to java bean communication is taking place with the support of <jsp:useBean>, <jsp:setProperty>, <jsp:getProperty> tags comes under MVC-1 architecture based web application development.

* A java bean class can also contain business methods along with getter and setter methods.

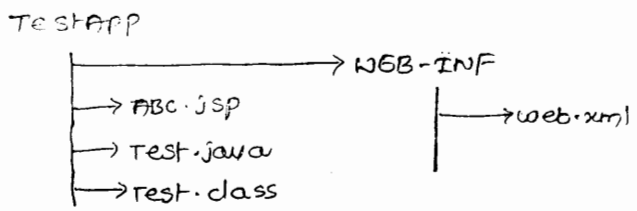
* If you want to use java class or java bean in Jsp program then it must be placed in a package defined in WEB-INF\classes folder. Otherwise it must be placed along with Jsp program. If that java class is directly there in WEB-INF\classes folder then it can not be used in Jsp program.



Test class can not be used in ABC.jsp



Test class can't be used in ABC.jsp.



Test class can be used in AB.jsp

<jsp:useBean>

Syntax : <jsp:useBean attributes />

used for creating/locating an object of javabean class

attributes

id = " " // instance name (object name)

class = " " // a fully qualified javabean class name.

Scope = "page/session/request/application" (default scope is "page")

// specifies the scope of the bean class obj

page (default)

bean class obj is available through out the request page.

session

bean class obj is available through out the request session

request

throughout the request

application

available for all pages within the context (web application)

beanName = "logical name of java bean"

type = "reference class type of java bean class object".

Ex: <jsp:useBean id="st" class="p1.StudentBean" scope="session" />

creates p1.StudentBean class obj having name "st" and keeps in session scope

If this object is already available in session scope then it locates that object from session scope. For this it internally uses pageContext.setAttribute(--) pageContext.getAttribute(--) methods.

The above statement creates object like this:

pl.StudenBean st = new pl.StudentBean();

"st" object type is pl.StudentBean

"st" reference object type is pl.StudentBean

Ex:

<jsp:useBean id="st" type="ABC" class="pl.StudentBean" scope="session" />

creates or locates pl.StudentBean class object (st) in/from request scope.

Bean class object creation statement

ABC st = new pl.StudentBean();

"st" → reference type is ABC class

"st" → object type is pl.StudentBean class

→ Here pl.StudentBean class extends from ABC class.

* <jsp:useBean>, <jsp:setProperty>, <jsp:getProperty> are 3 independent tags

<jsp:setProperty>

calls setXxx methods on java class object to set given values to bean properties.

Syntax: <jsp:setProperty attributes />

attributes

property = " " // bean property name

name = " " // (bean class obj name) id attribute value given in the </jsp:useBean>

value = " " // value to be set for bean property

param = " " // input (request) parameter name

NOTE: value or param → any one attribute has to be used at a time.

Ex: <jsp:setProperty name="st" property="sno" value="567" />

// this internally calls setSno() methods on st obj and assigns value "567" to the bean property "sno".

Ex:

<jsp:setProperty name="st" property="sname" param="stname"/>

this internally calls setSname(-) method on st obj to assign the value of "stname" req param to sname bean property.

<jsp:getProperty>

calls getXxx() methods to read values from bean properties.

syntax: <jsp:getProperty attributes/>

attributes

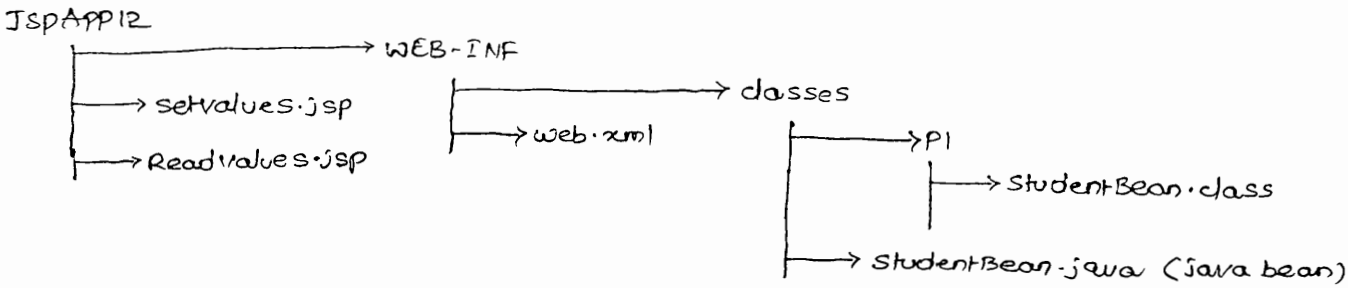
name="" // (bean obj name) id attr value given in <jsp:useBean>

Property="bean property name"

ex: <jsp:getProperty name="st" Property="sno"/>

gives "sno" bean property value by calling getSno() method on bean class object ("st") and also displays that value on browser window.

Example application on Jsp to Java bean communication (MVC-1 architecture)



StudentBean.java

```
package p1;
public class StudentBean
 {
   // bean properties
    Private int sno;
    Private String sname;
    Private float avg;
    // Constructor
    Public StudentBean()
     {
       System.out.println("StudentBean o-param constructor");
     }
      // write getXxx() and setXxx() methods
```

```
    public void setSno(int sno)
    {
      s.o.p("StudentBean: setSno(-) method");
      this.sno = sno;
    }
    ========
    }
```

> javac -d . StudentBean.java

SetValues.jsp

```
<%@ page import="p1.StudentBean" %>
<!-- create or locates java bean class obj -->
<jsp:useBean id="st" class="p1.StudentBean" scope="session" />
<!-- set values to bean properties -->
<jsp:setProperty name="st" Property="sno" value="101" />
<jsp:setProperty name="st" Property="sname" value="rajesh" />
<jsp:setProperty name="st" Property="avg" value="67.89" /> <br> values are set to bean properties
```

ReadValues.jsp

```
<%@ page import="p1.StudentBean" %>
<!-- create or locate javabean class obj -->
<jsp:useBean id="st" class="p1.StudentBean" scope="session" />
<!-- read and display bean properties values on browser window -->
  sno value is : <jsp:getProperty name="st" property="sno" /> <br>
  sname value is : <jsp:getProperty name="st" property="sname" /> <br>
  avg value is : <jsp:getProperty name="st" property="avg" />
```

web.xml

```
<web-app/>
```

* Irrespective of whether Thread safety is enabled or not enabled (isThread Safe="true"/"false") the jsp equalent servlet manages session scoped, application scoped javabean class objects having thread safety with the support of synchronised blocks.

URL to test the application

http://localhost:2020/JSPApp12/setValues.jsp

http://localhost:2020/ReadValues.jsp

* we can use the param attribute of <jsp:setProperty> tag to set the received request parameter value as bean property value.

```
<jsp:setProperty name="st" property="sno" param="stno"/>
<jsp:setProperty name="st" Property="sname" param="stname"/>
<jsp:setProperty name="st" property="avg" param="stavg"/>
<br> values are set to bean properties
```

avg → bean property name; stavg → req. param name

Request URL is

http://localhost:2020/JSPApp12/SetValues.jsp?stno=3456&stname=ramesh&stavg=37.5

query string representing req. params

* If request parameter names are matching with bean property names then we can use property = "*" in <jsp:setProperty> tag to set request parameter values as bean property values.

SetValues.jsp

```
<%@ page import="p1.StudentBean" %>
<jsp:useBean id="st" class="p1.StudentBean" scope="application"/>
<jsp:setProperty name="st" Property="*"/>
<br> value are set to bean properties.
```

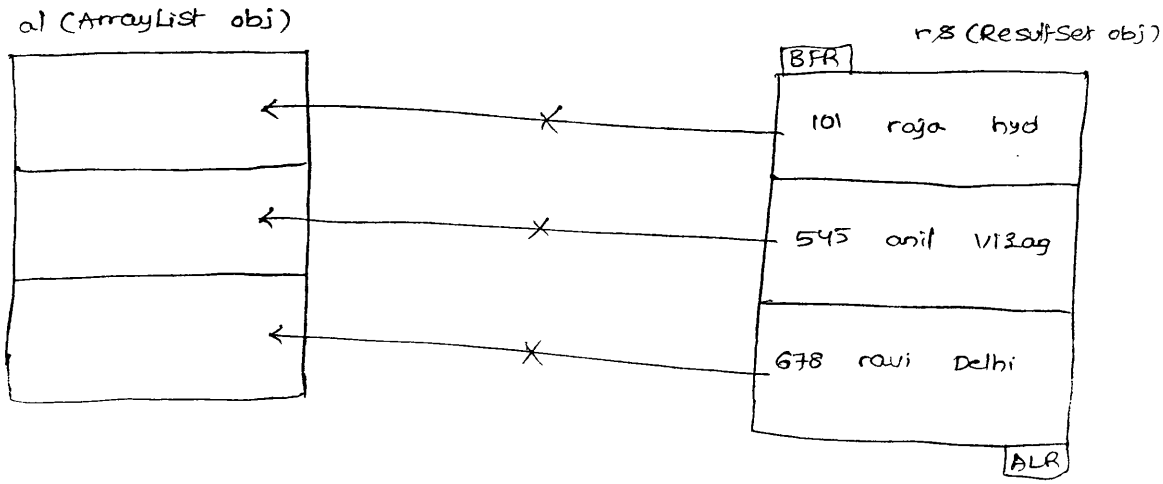
* The application that can display and rotate advertisements / Ads in web pages is called as AdRotator.
* Our AdRotator application is having the following features.
  (1) Should Render the advertisments as graphical hyperlinks pointing to client's related web sites.
  (2) The advertisments must be changed automatically at regular intervals.
  (3) Advertisment should be rendered randomly in the web page
  (4) Develop application based on MVC1 architecture.
* For example application on advertisment rotator refer application - 6 of page nos 110 and 111.

Mini Projects Discussion

* In order to send java objects over the network it must be developed as serializable object.
* Java object becomes serializable object only when it's class implements java.io.Serializable interface directly or indirectly.
* All Collection framework data structure objects are serializable objects by default.
* ResultSet object is not serializable object so we can not send that object over the network. But in multilayer environment (like mvc2) we need to send the ResultSet object data over the network.
* To solve the above said ResultSet problem there are two solutions.

  solution(1): use RowSets as alternate for ResultSets

  solution(2): Copy the data of ResultSet obj to Collection Framework datastructures and send them over the network.
* RowSets are serializable objects so we can send them over the network but very few jdbc drivers are support for RowSets. Due to this industry prefers working with solution(2) to send ResultSet object data over the network.
* If you are performing only read operations on data structures after receiving them then it is recommended to work with non synchronized data structures like ArrayList, HashMap for better performance.
* If you want to perform read write and other operations on the received Collection Framework data structure then it is recommended to work with synchronized data structure like Vector, Hashtable to avoid multi threading related concurrency issues.
* we can not move ResultSet object records directly to ArrayList object elements on each record to each element basis because in one element of ArrayList object we can store only one java object at a time. But in each record of ResultSet there is a possibility of having multiple objects and other values.



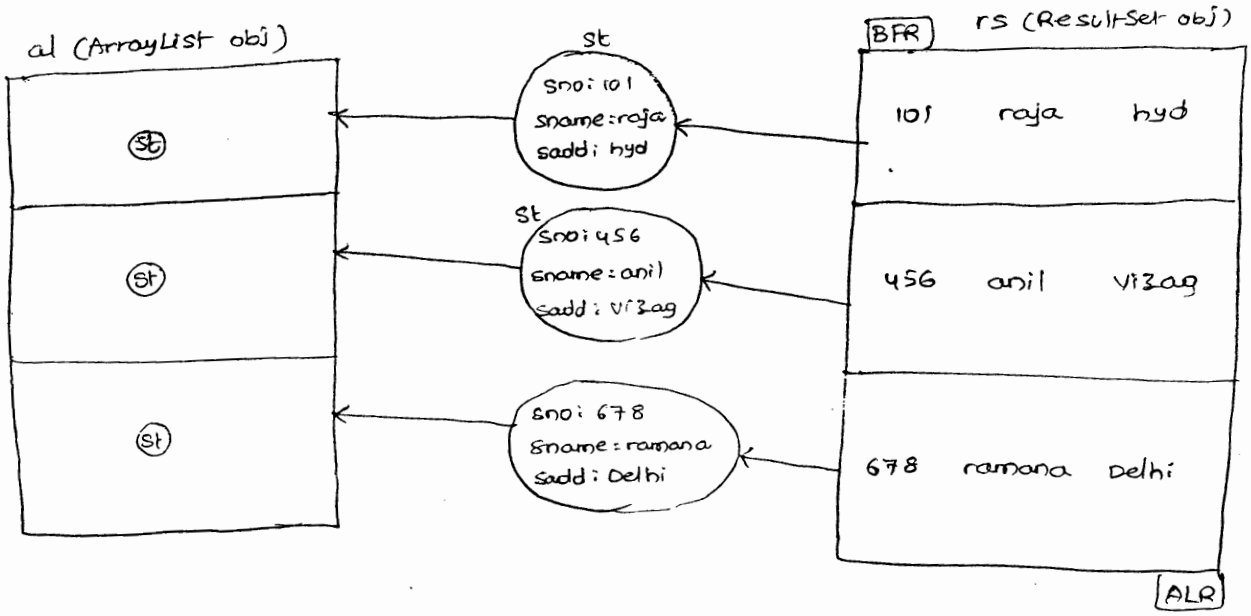
* To solve the above problem store each record values of ResultSet in a user defined java class object and add that object to each element of ArrayList object. Here this user defined java class is a java Bean and it is technically called as DTO class or VO class (value Object).

D.T.O class or V.O class

```
public class StudentBean implements java.io.Serializable
{
    Private int Sno;
    Private String Sname;
    private String Sadd;
    //write setXxx(-) and getXxx() methods
    ==========
}
```

Logic to transfer ResultSet object records to ArrayList object elements D.T.O class Support

```
ResultSet rs=st.executeQuery("select * from student");
ArrayList al = new ArrayList();
while(rs.next())
 {
    StudentBean st=new StudentBean();
    //add each record values to D.T.O class obj
     St.setSno(rs.getInt(1));
     st.setSname(rs.getString(2));
     St.setSadd(rs.getString(3));
     //add D.T.O class obj to ArrayList
      al.add(st);
 }
```



Where did you use java bean in your project ?

(A) (i) As model layer resource in MVC-1, MVC-2 architecture based projects.

(ii) As D.T.O class or V.O class while transfering ResultSet object to Collection Frame work data structure as shown above.

(iii) As HB perisistance class in HB Programming.

(iv) As SpringBean in spring Programming and etc.

---

* Can you tell me where exactly you have utilized the Collection frame work data structures in your project?

(A) * java.util.Properties is required to maintain while maintaing JDBC driver and database details outside the JDBC application.

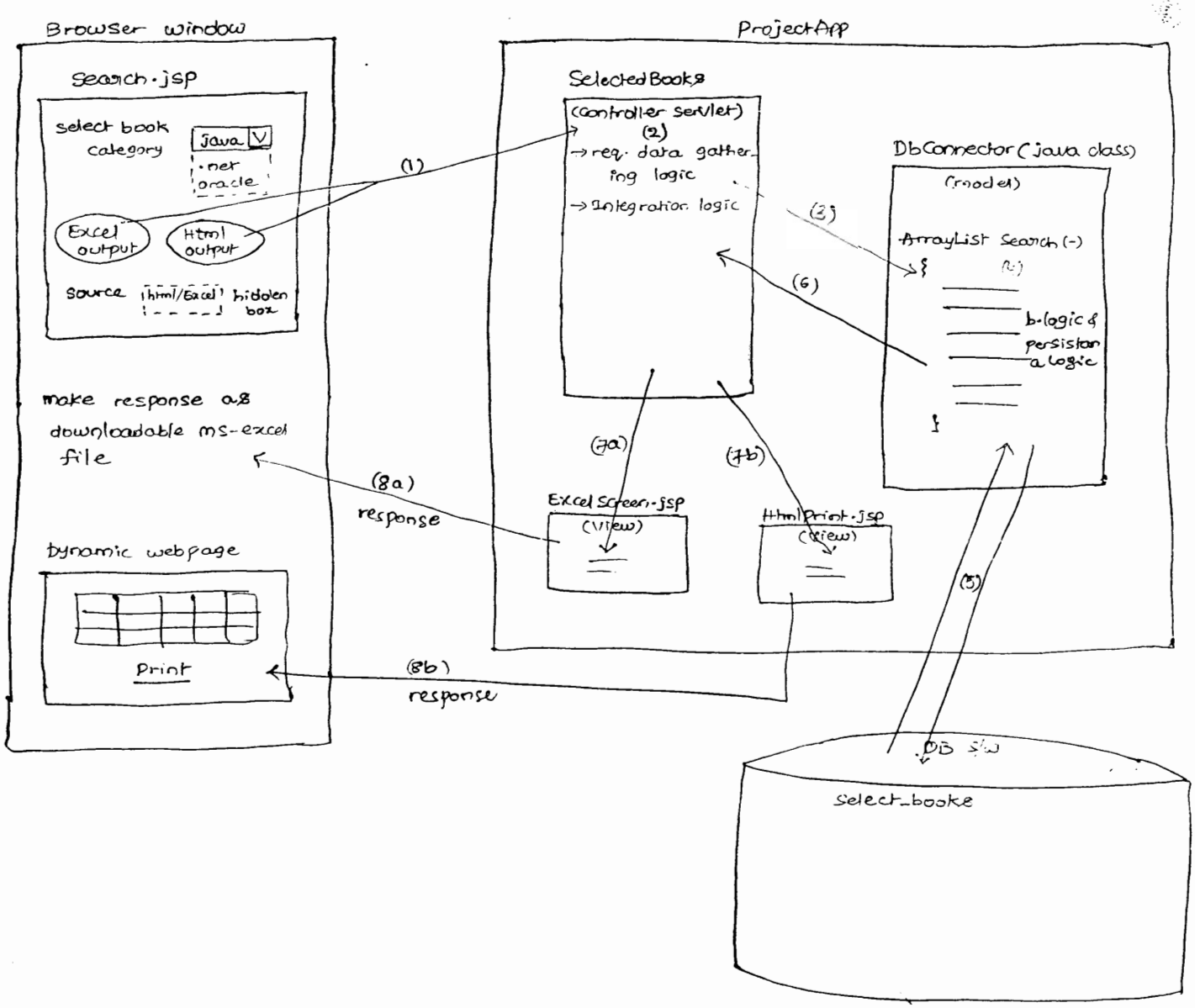
* ArrayList or Vector support will be taken while to send ResultSet object data over the network.

* To maintain JNDI properties, mail properties of Java mail api Map data structures are required.

* To maintain application data (huge amount) with in the application Collection Frame work data structures are required.

* To maintain same form data page data across the multiple requests as a single value during the session Collection frame work data structures are required.

NOTE: To send collection framework data structure over the network the objects that added to datastructures as elements must be serialisable objects.



(1) Enduser selects the book category and submits the request to web application.

(2) The Controller Servlet traps and takes the request and reads form data from the form page.

(3) Controller servlet uses integration logic to call seach business method of DbConnector class having the selected book category as argument value

(4) the search business method uses business logic and persistance logic to gather book details from db table into ResultSet object and moves

that ResultSet object data to ArrayList by taking the support of D.T.O class.

(6) Search business method sends the ArrayList object back to the Controller Servlet.

(7a), (7b) The Controller Servlet sends ArrayList to result pages (ExcelScreen.jsp or HtmlPrint.jsp) as request attribute value.

(8a), (8b) Result pages sends response to browser window.

NOTE(1): ExcelScreen.jsp sends response to client as downloadable MS-Excel file where as HtmlPrint.jsp sends response to browser window as printable html table content.

The following observations can be done in the above project.

(i) web application is developed based on MVC2 architecture.

where

M → Model Layer → java class (DbConnector)

V → view Controller → jsp programs (3 prgs)

C → Controller Layer → Servlet Prg (SelectedBooks)

(ii) Form Page is submitting request to web application through ordinary buttons taking the support of javaScript.

(iii) Multiple button generated request related logics are differentiate in web application.

(iv) JavaScript also used for form validations (selecting item from in selectbox is mandatory.

(v) D.T.O class support is taken to transfer ResultSet object data to ArrayList.

(vi) Servlet chaining support is taken by Controller Servlet Communicating with result pages and request attributes are used to pass data from Controller servlet to result pages.

(vii) Response is made as downloadable ms-Excel file and also made as the printable web page.

* For the source code of above project refer page nos 145-150.

* For other mini projects refer page nos 132, 133.

Custom Jsp Tag Library:

* Jsp tag library is a library that contains set of Jsp tags.
* Programmers can work with three types of tag libraries in their Jsp programs.
  (1) Built-in tag libraries of Jsp technologies.
      (All Standard Action tags of Jsp)
  (2) Third party supplied Jsp tag libraries
      Ex: Struts tag libraries, JSF tag libraries, JSTL and etc...
  (3) Custom Jsp tag libraries
      (Developed by programmers)
* According to industry standard it is recommended to develop Jsp programs as java codeless Jsp programs to increase readability and easyness in Jsp program. That means we should not use scriptlet, declaration, expression tags in our jsp programming.
* The built-in tags of Jsp are not at all sufficient to develop Jsp programs as Java codeless jsp programs so we need to think above developing custom tag libraries or using third party tag libraries.
* In custom jsp tag libraries all jsp tags are custom Jsp tags.
* The java class that defines the functionality of a Jsp tag is called as tag handler class. For this the class must extend from javax.servlet.jsp.tagext.TagSupport class.
* Every Jsp tag library contains one tld file (Tag Library Discriptor) having the configuration details of JSP tag like Jsp tag name, attribute names, tag handler class name, list of possible values for attributes and etc..

Procedure to develop custom Jsp tag library

Step(1): Design the tag library and its Jsp tags.

<ABC> → should print 1-10 numbers

<XYZ> → should print 10-1 numbers

Step(2): Develop tag handler classes defining the functionalities of JSP tags in WEB-INF\classes folder

ABCTag.java

```
public class ABCTag extends TagSupport
{
   public int doStartTag()
    {
      ======
      ======  (g)
      ======
    }
    public int doEndTag()
    {
      ======
      ======  (g)
      ======
    }
}
```

XyzTag.java

```
public class XyzTag extends TagSupport
  {
     public int doStartTag()
       {
       =====
       }
      public int doEndTag()
       {
       =====
       }
  }
```

NOTE: The logic of doStartTag() executes when open JSP tag [<name>] encounters so in this method definition we can use method definitions and body of JSP tags to define the functionality of JSP tag. where as the doEndTag() executes when [</name>] encounters. In this method definition we can define the finishing functionalities

* doStartTag(), doEndTag() methods are container call back methods are(or) life cycle methods of Tag handler classes

Step(3): Configure all the JSP tags of custom JSP tag library in tld file.

WEB-INF\sathya.tld (xml file) (Blue print)

<ABC> (f) → ABCTag.class

<Xyz> → XyzTag.class

* Step(1) to step(3) talks about custom JSP tag library.

Step(4): configure JSP taglibrary in web.xml file of web application.

```
<web-app>
    <taglib>
        <taglib-uri>demo</taglib-uri>   (d)
```

```
<taglib-location> /WEB-INF/sathya.tld </taglib-location>
                  (e) name & location of tld file
</taglib>
</web-app>
```

NOTE: Every JSP Taglibrary that is configured in web.xml file will be identified through its <taglib-uri> (like demo).

step(5): use the jsp tags of tag library in JSP programs of web application.

Test.jsp

```
<%@taglib uri="demo" Prefix="st" %>
          (c)
          collect from web.xml
                      User defined
<st:ABC/>   (b)
   (a)
<st:Xyz/>
```

→ (b) : output of <st:ABC> tag goes to browser window.

NOTE: Prefix of the tag is useful to identify the tag library of JSP tag. when two jsp tags of two different JSP tag libraries contains same name and different functionalities in order to use both JSP tags in a single JSP program we can use prefixes to differentiate those tags.

* All custom JSP tags must be developed as action tags so we must use prefixes while working with those tags.
* In the above code the alphabets (a) to (h) indicates flow of execution related to <st:ABC> tag.

(a) → The <st:ABC/> tag encounters in the execution of JSP program.

(b) → The prefix st will be gathered from the tag.

(c) → Based on the prefix the <taglib-uri> demo will be gathered from <%@ taglib ... > tag (This tag is useful to specify the <taglib-uri> of tag library whose tags are going to be used in current JSP program)

(d), (e) → From web.xml file the tld file name and location will be gathered based on <taglib-uri>.

(f) → From tld file the tag handler class of <ABC> tag will be gathered (ABCTag).

(g) → The code placed in Tag handler class executes through container call back methods.

(h) → The output generated by Taghandler class goes to browser window as the output of <st:ABC> tag.

* In every tag handler class the pageContext object is visible because it is protected member variable of pre defined tag support class.
* Programmer uses this inherited pageContext object in Tag handler class to create/ to get access through other implicit objects of jsp like out, request, response and etc..
* For example application on custom Jsp tag library development refer application given in page nos 129-132.

JSTL

* The built-in jsp tags of Jsp technology are not sufficient to make JSP programs as java code less Jsp programs.
* Developing custom JSP tags and tag libraries is complex and time consuming process.
* To overcome above two limitations and to make Jsp programs as java codeless JSP programs use third party supplied ready made custom JSP tag libraries like JSTL, Display tags, struts tag Libraries and etc.

JSTL
- → Given by sun micro systems
- → is not part of JSP technology
- → should configure and should be used with JSP programs seperately.
- → Gives 4 no. of JSP tag libraries. They are
  - (1) Core JSP tag library
  - (2) SQL JSP tag library
  - (3) XML JSP tag library
  - (4) Formatting JSP tag library

* By using JSP tags of these four JSP tag libraries we can perform all operations using tags like variable declaration, write data to browser window, Conditional operations, Iterational operations, Arithmetic and logical operations (also take the support of EL), DB interactions, xml Processing, Formatting data and etc..
* Since JSTL gives readly available custom Jsp tag libraries there is no need of designing JSP tags, developing tag handler classes and tld files.
* Core tag library is given for basic operations like variable declarations, flow of control, Conditions and iterations.
* Sql tag library is given for DB interactions.
* xml tag library is given for xml processing.

Tag Library	tld file name	recommended Prefix	jar files
core	c.tld	c	jstl.jar, standard.jar, saxpath.jar, jaxen-full.jar
SQL	sql.tld	sql	"
xml	x.tld	x	"
fmt	fmt.tld	fmt	"

* Fmt tag library is given for specifying Date formats, Time formats, currency symbols, NumberFormats and etc...
* For tags of various JSTL tag libraries refer page nos 116 to 119.
* we can gather the above given four jar files JSTL.jar, Standard.jar, saxpath.jar, jaxen-full.jar from java.sun.com or oracle.com website.

Procedure to work with JSTL tags in java web application

(1) Gather the above said four jar files, keep them in WEB-INF\lib folder
(2) Extract the tld file from standard.jar file and place in webapplication like WEB-INF folder.
(3) Read the documentation about JSTL tags
(4) Configure JSTL supplied tag libraries in web.xml file having taglib uris.
(5) use JSTL tags the jsp programs of web application.

* while working with JSTL we can use more implicit object along with the regular 9 implicit objects. They are
(1) requestScope
(2) param
(3) paramValues
(4) header
(5) headerValues
(6) Cookie
(7) sessionScope and etc...

* Tags are not there in tags can not be given to perform arithmetic operations in our Jsp programs. For this purpose we can take the support of EL which can be used with tags or without tags.

Syntax

${------<ELCode>.....}

* For example application on JSTL tags refer to examples given in 120-122 of Booklet.
* The implicit object param can be used to gather single request parameter value. where as the implicit object paramValues can be used to gather all request parameter values.

Web Sphere server

Type: Application server software
version: 6.x (Compatable with jdk 1.4)
vendor: IBM
default port no: 9080 for web applications
Commercial software
jar file that represents jee apis: j2ee.jar
download s/w as ZIP file from www.IBM.com website.
Allows to create domains.
If websphere s/w is installed as windows service then each domain must be started services of Control panel.

* To install websphere 6.0 use install.exe file of WAS folder that comes by extracting IBM WebSphere Application Server V6.0 win rar/ zip file.
* To create domain in web sphere server

Start → programs → IBM web Sphere → Application Server V6 → Profile creation wizard.....

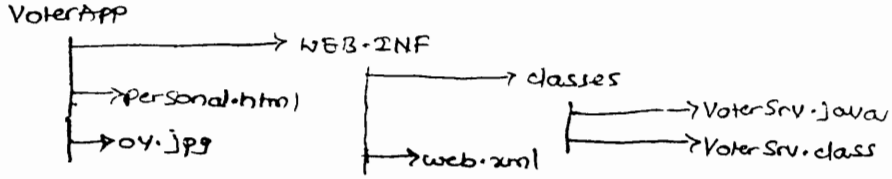
* In websphere server war file based console deployment is possible.

Procedure to deploy java web application in websphere 6.0

Step(1): set path to jdk 1.4 that comes with websphere s/w installation.

> PATH = D:\Program Files\IBM\Appserver\java\bin

Step(2): Prepare war file on deployment directory structure of web application.



NOTE: Compile VoterSrv.java in jdk1.4 environment.

<!DOCTYPE ... > statement in web.xml file is mandatory for websphere server.

>jar cf VoterApp.war .

step(3): start the domain server of websphere from services of Control panel.

Control panel → performance and maintainance → Administrative tools → Services → IBM Websphere Server → start.

step(4): open adminstrative console of domain server.

start → All Programs → IBM websphere → application server V6 → profiles → mydomain → Adminstrative console → userID: xyz

↓ user defined

step(5): Deploy the web application

Admin console → Install new application → Browse and select VoterApp.war file → Context root: VoterAPP → next → next → Continue → next → next → next → Finish → Save to master Configuration → save

step(6): keep the deployed VoterApp application in running mode.

Admin console → Enterprise applications → ☑ VoterApp.war → start

step(7): Test the web application.

open browser window → http://localhost:9080/VoterApp/person.html

What is the difference between encodeURL(-), encodeRedirectURL(-) methods of response object ?

(A) encodeURL(-) is useful to append session id to the given URL. This is useful while working with HttpSession with URL rewriting technique.

encodeRedirectURL(-) is useful to send session id from one web application to another application then they are interact with each other with help of sendRedirect(-). This method is useful to make session obj of one web application to another webapplication.

Example on encodeRedirectURL(-) (In Srv1 prg of WA1 web application)

```
String enurl = res.encodeRedirectURL("http://machine20:7001/WA2/srv2");
res.sendRedirect(enurl);
```

The above code sends session id from WA1 web app to WA2 web app

Annotations

* Data about data is called as meta data.
* Gathering more details about already given details is called meta data operation.
* In JDBC level we can perform meta data operations in 3 modes.
  (1) Database meta data
  (2) Parameter meta data
  (3) ResultSet meta data
* Configuration of resource programs in xml files comes under meta data operations.
  Ex: By configuring servlet, JSP programs in web.xml we are passing more details about those programs to underlaying container or server software. This is nothing but meta data operations.
* While working with advanced technologies we generally use xml files to perform resources configuration based meta data operations.
* Annotations are the java statements and they are alternate for the xml files based meta data, resources configuration operations.

Syntax:

@ <annotation name> (param1 = val1 , param2 = val2, - - - -)

* To read and process xml documents we need xml parser which is a heavy weight software so it may kill the performance of the application but xml files give good flexibility of modification without disturbing the java source code.
* Annotations will be recognized by java tools or underlaying containers automatically so they give good performance but they give bad flexibility of modification.

Two types of annotations

(1) Documentation related Annotations (should be used in /** - - - - */ Comments)
(2) Programming related Annotations (can be used in java source code level)

(1) → ex: @author, @param, @see, @since, @return, ...

(2) → ex: @ override, @webServlet, @ ServletFilter, @ InitParam and etc..

* Annotations for documentation are given from jdk 1.1 onwards (refer src.zip file of Jdk s/w).
* Annotations for programming are given from jdk 1.5 so all technologies that are compatable with jdk 1.5 gives support for annotations.
Ex: EJB 3.x, Struts 2.x, Spring 2.5, 3.x, GJB 3.3+, Servlet 2.5, servlet 3.x and etc..
* Each annotation is a special java interface (with @interface keyword) and parameters of annotation will be declared as java methods
* Parameters of annotations are like attributes of xml tags that means they can be used to configure additional details about resource.
* Recognizing annotations and recognizing annotation related resources and applying functionality on them is the responsibility of underlaying jdk or jre or Container software.
* we can apply annotations at 3 levels.
(1) Resource level (class level / Interface level)
(2) Method level
(3) Field level (Member variable level)
* Annotations helps the programmer to configure the resources as required for underlaying Container software or framework software.

Setup required to servlet 3.0 based servlet programs (no web.xml file)

jdk 1.6 + (jdk 1.6)

Tomcat 7.0 + (The server that is compatable with servlet 3.x specification)

GlassFish 3.x

Important annotations of Servlet 3.x programming

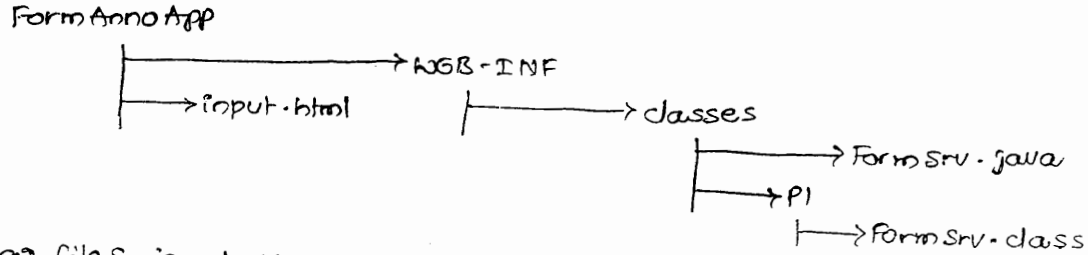
@ WebServlet → to Configure servlet program with logical name and url pattern.
@ InitParam → to configure init parameters of servlet program
@ ServletFilter → to configure servlet filter program
@ WebServletContextListener → to configure ServletContextListeners.
:

* All these annotations are part of Servlet 3.x api and can be collected from <tomcat_home (7)>\lib\servlet-api.jar.

* The main feature of servlet 3.x is good support for annotations and ability to develop servlet programs based web application without web.xml file.
* From jdk 1.5 onwards each java package contains classes, interfaces, annotations and enums.
* The servlet api 3.0 is given having 3 packages.

  javax.servlet, javax.servlet.http and javax.servlet.annotation.
* To get servlet 3.x api documentation refer javaees api docs...

Example application (using servlet 3.x annotations)



jar files in class path

<Tomcat 7_home>\lib\servlet-api.jar file

Deploy above application in Tomcat 7.0 or GlassFish 3.x server

input.html

```
<form action="test1">
    Name: <input type="text" name="username"><br>
    <input type="submit" value="check">
</form>
```

FormSrv.java

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
import javax.servlet.annotation.WebServlet
@WebServlet(name="abc", urlPatterns={"/test1"})
Public class FormSrv extends HttpServlet
  {
    public void doGet(-,-) throws SE, IOE
      {
        String username = request.getParameter("username");
        response.setContentType("text/html");
```

```
        PrintWriter pw = response.getWriter();
        pw.println("Hello Mr." +username+ " Today's Date :: " +new java.util.Date());
        pw.close();
    }
    public void doPost(-,-) throws SE,IOE
    {
        doGet(-,-);
    }
}
```

Request URL

http://localhost:2525/FormAnnoApp/input.html

* For related info on servlet 3.0 annotation refer supplementary hand out given on 18-12-2011.
* NetBeans 6.9+ is giving support for annotations based servlet programming.
* Reflection means mirror image.
* Reflection api application is capable of gathering given java class or interface details programatically and dynamically.
* Being from one application if you want to gather certain java class details like member variables, methods and etc details programatically then develop the code of that application by using reflection api.
* To develop and use our own annotations we need to develop one simulator class called container on the top of JRE then only we can provide functionality to that annotation.

Procedure to develop user defined annotation having the following functionality

@ sum(num1=10, num2=20)
int result → should assign 30 to result variable

@ sum(num1=30, num2=40)
int res → should assign 60 to res variable

Step(1): make sure that jdk 1.5+ software is available.

Step(2): Design your annotation.

```
package P3;
import java.lang.annotation.*;
@Retention(RetentionPolicy.RUNTIME) makes annotation as runtime
                                    annotation
```

```
@Target(ElementType.FIELD)  -> This annotation is applicable only on fields.
public @interface Sum {
    int num1() default 0;
    int num2() default 0;
}                                   } parameters.
```

Step(3): Develop your own Container class.

```
package P3;
import java.lang.reflect.Field;
public class Container
 {  public static void addThis(Object theObject)
     {
       try {
            //get access to all the member variables of current obj (Test class obj)
            Field [] fields = Class.forName(theObject.getClass().getName()).getDeclaredFields();
            for(int i=0; i<fields.length; i++)
             {
               //get access to @Sum annotation
               Sum obj = fields[i].getAnnotation(Sum.class);
               if(obj!=null)
                 { //getting values from annotations parameter
                   int val1 = obj.num1();
                   int val2 = obj.num2(); int tot = val1+val2;
                   //setting it to our class variable (like result)
                   fields[i].setInt(theObject, tot);
                 }//if
             }//for
           }catch(Exception e) {e.printStackTrace();}
     }
 }
```

Step(4): use the annotation in your application

```
Package P3;
Public class Test
 {
    @Sum(num1=15, num2=10)
     Static int result;
     public static void main(String args[])
       {
         Test t1 = new Test();
         Container.addThis(t1); //activate user-defined Container class
         S.o.p("The sum is:" +result);
       }
 }
```

Step(5): Compile and execute the application.

>javac -d. *.java (Compilation)

>java P3.Test (execution)

technicalmumbo.wordpress.com

---

* There are multiple ways to know the details of java class or interface.

They are

(1) By working with javap tool

(2) By working with java api documentation.

(3) By opening source code of java resources.

javap :

* It is a built-in tool of jdk software.

Ex: javap java.lang.String

javap java.io.Serializable

javap MyTest

---

* In jdk software installation we can use java_home\src.zip file to get the source code of all the built-in classes and interfaces of java api.

* When all the above three techniques are used to gather details about certain java class or interface we can not use those details as input values in our java application.

* Reflection means mirror image.

* Reflection api based java applications can gather the internal details about given java class or interface programatically and dynamically and these details can be used in the application as input values for other requirements.

* Each reflection api based java application acts as mirror image having the ability to gather details about given class or interface dynamically.

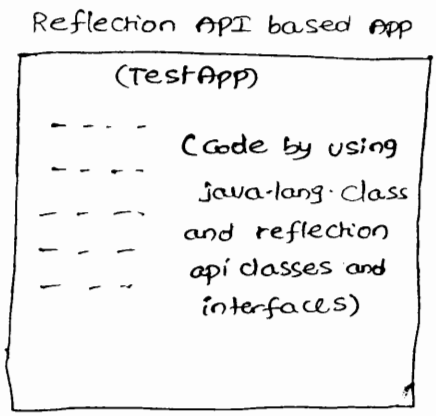
* Reflection api means working with classes and interfaces of java.lang.reflect package.

* java.lang is default package in java applications but its sub packages are not default packages.

```
public final class Test
{
    float static int a;
    private float b;
    Test()
    {
    }
```

```
    Test (int x)
    {
      ==
    }
    public int sum (int x, int y)
    {
      ==
      ==
    }
}
```

* In java environment member variables are technically called as fields and constructors are called as creation methods.



>java TestApp.java

>java TestApp java.lang.System → Command line argument value

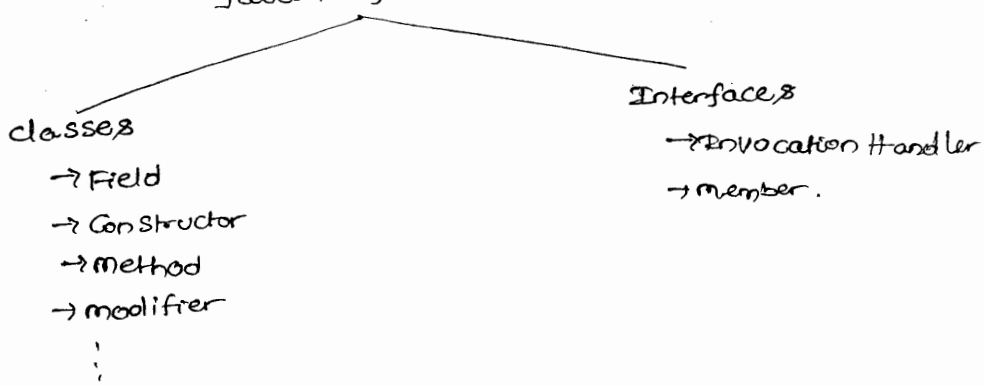
→ prints all the details of java.lang.System class as output

>java TestApp java.lang.Runnable

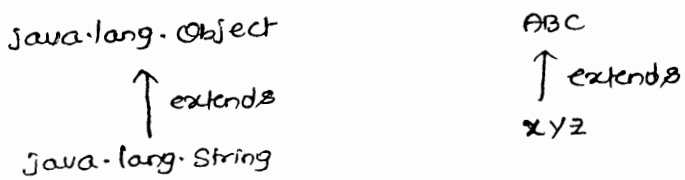
→ prints all the details of java.lang.Runnable (I) as output

* Reflection api is popularly used in the development of java based software products like debuggers, ClassBrowsers, GUI Builders and etc....
* Debugger allows us to trace flow of execution in the application.
* ClassBrowser is nothing but hint box that comes in IDE s/w
* GUI Builder allows the programmer to design the application through drag and drop operations.
* every IDE contains Debugger, classBrowser, GUI Builder products. This indicates the IDE s/w internally using reflection api support.
* we can gather the basic details about given class or interface by using various methods of java.lang.class but use classes and interfaces of java.lang.reflection api to gather advanced details.

java.lang.reflect package



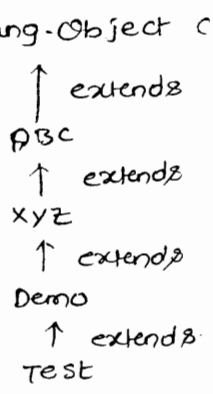
write a java application to gather super class name of given java class




```
// Appl.java
  Public class Appl
   {
     public static void main (String args[]) throws Exception
      {
         // load given java class into App
          Class c = Class.forName (args[0]);
          // get super class of given class
           Class sc = c.getSuperclass();
           // print given class, super class name
            System.out.println ("given class name:" + c.getName());
            System.out.println ("super class name:" + sc.getName());
      }//main
   }//class.
  // > javac Appl.java
  // > java Appl java.lang.String  → command line argument value
  // > java Appl java.lang.System
```

* when the object of java.lang.Class points to class or interface and to get that class name or interface name as string value use getName() method call on java.lang.Class object.

write a java application to get all the classes of inheritance hiearchy for a given java class




```
//App2.java
public class App2
{
   public static void main(String args[]) throws Exception
    {
      //load given java class into App
        class c = class.forName(args[0])
         //get Super class
         class sc = c.getSuperclass();
         //print all the classes of inheritance hierarchy
          system.out.println("classes in the inheritance hierarchy for "+args[0]+
                                                    " class are:");
          while(sc!=null)
           {
             System.out.println("\t\t"+sc.getName);
              c=sc;
              sc=c.getSuperclass();
           }//while
      }//main
  }//class
//>javac App2.java
//>java App2 java.lang.Integer
//>java App2 java.applet.Applet
```

* Object of java.lang.Class can represent a concrete class or interface or abstract class or data type.

write a java application to get list of java interfaces implemented by the java given java class

```
public class Test implements X, Y, Z
  {
  }
```

```
//App3.java
public class App3
 {
   public static void main(String args[]) throws Exception
    {
      //load the given java class into App
       class c = class.forName(args[0]);
       //call getInterfaces() of java.lang.class
       class inter[] = c.getInterfaces();
       //print interfaces names
        for(int i=0; i< inter.length; i++)
         {
           system.out.println("/t\t" + inter[i].getName());
         }//for
     }//main
  }//class
//>javac App3.java
//> java App3 java.lang.String
//> java App3 java.lang.Thread
```

* int[] means all elements in that array are int values.
* java.lang.class object[] means all elements in that array are the objects of java.lang.class.
* public, final, abstract are the modifiers of java class. But we can not apply final and abstract modifiers at a time on a java class because final class does not support inheritance and abstract class needs the support of inheritance (we can not write subclass to final class but we need to write subclass to abstract class to implement abstract method's).

write a java application to get the modifiers of given java class

```
//App4.java, import java.lang.reflect.*;
public class App4
 {
   public static void main(String args[]) throws Exception.
    {
      //load the given java class into App
       class c = class.forName(args[0]);
      //call getModifiers() of java.lang.class
        int x = c.getModifiers();
```

→ returns int value representing the modifiers of given java class.

```
        //print modifiers
        if (Modifier.isPublic(x))
            System.out.println("\t public");
        if (modifier.isAbstract(x))
            System.out.println("\t abstract");
        if (modifier.isFinal(x))
            System.out.println("\t final");
    }//main
}//class
//>javac App4.java
//>java App4 java.lang.String
//>java App4 java.lang.Thread
```

* isXxx(-) methods of Modifier class are boolean methods. Checking whether the given numeric value represents certain modifier or not.
* By default all interfaces are abstract.
* In java there is no terminology called access specifier.
* public, private, protected, final, static, synchronized, volatile, transient and etc.. keywords are called modifiers in java.
* The object of java.lang.reflect.Field class can represent one member variable of given java class.

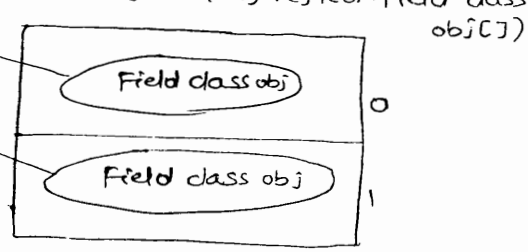
<u>write a java application to get the details of member variables/fields available in java class</u>

```
//App5.java
public class App5
{
    public static void main(String args[]) throws Exception
    {
        //load the given java class into App
        Class c = Class.forName(args[0]);
        //call getDeclareFields() of java.lang.class
        Field f[] = c.getDeclareFields();
        //Print Field details
        for(int i=0; i<f.length; ++i)
        {
            int x = f[i].getModifiers(); //gives modifiers of each field
            String type = f[i].getType().getName(); //gives datatype of each field.
```

```
String name = f[i].getName(); //gives name of each field
if (Modifier.isPublic(x))
    System.out.println("public");
  - - - - -
  - - - - -
System.out.println(type+"  "+name);
}//for
}//main
}//class
//> javac App5.java
//> java App5 java.lang.String
```

Ex:

```
public class Test
{
  private int a;
  static final float b;
  ----
  ---- //method 8
}
```



Write a java application to gather all the method details of given java class

```
//App6.java
import java.lang.reflect.*;
public class App6
{
  public static void main(String args[]) throws Exception
  {
    // load the given java class into App
    class c = class.forName(args[0]);
    Method m[] = c.getDeclareMethods();
    // print method details
    for(int i=0; i<m.length; ++i)
    {
      int x = m[i].getModifiers() //gives modifiers of each method
      String rtype = m[i].getReturnType.getName(); //gives return type
      String mname = m[i].getName(); // gives method name.
      class p[] = m[i].getParameterTypes(); //gives parameter types
      if(Modifier.isPublic(x))
        S.o.p("public");
```

```
if (modifier.isFinal(x))
    s.o.p("final");
- - - - -
- - - - -
s.o.p(rtype + "  " + mname + "(");
for(int k=0; k<p.length; ++k)
   {
     s.o.p(p[k].getName() + " ");
   }// for
   System.out.println(")");
}// for
}//main
}// class.
```

```
//> javac APPG.java
//> java APPG java.lang.String
//> java APPG java.lang.Runnable
```

Memory diagram

```
public class Test
{
  public void hello()
   {
   }
  public static final int Bin sum(int x, int y)
   {
     return x+y;
   }
}
```

